

THÈSE DE DOCTORAT

Inférence efficace en apprentissage automatique : sous-réseaux élagués et ordonnancement de tâches d'inférence compressibles

Davide FERRE'

Laboratoire d'Informatique, de Signaux et Systèmes de Sophia Antipolis (I3S)
UMR7271 UCA CNRS

**Présentée en vue de l'obtention
du grade de docteur en Informatique
d'Université Côte d'Azur**

Dirigée par : Frédéric GIROIRE, Directeur
de recherche, CNRS

Co-dirigée par : Emanuele NATALE, Chargé
de recherche, CNRS

Devant le jury, composé de :

Rebekka BURKHOLZ, Full professor,
CISPA Helmholtz Center for Information
Security

Vincent GRIPON, Professeur des universi-
tés, IMT Atlantique

Frederik MALLMANN-TRENN, Associate
Professor, King's College London

Giovanni NEGLIA, Directeur de recherche,
Inria

Laurent VIENNOT, Directeur de recherche,
Inria Paris

**INFÉRENCE EFFICACE EN APPRENTISSAGE AUTOMATIQUE :
SOUS-RÉSEAUX ÉLAGUÉS ET ORDONNANCEMENT DE TÂCHES
D'INFÉRENCE COMPRESSIBLES**

*Efficient machine learning inference: sparse subnetworks and
scheduling of compressible tasks*

Davide FERRE'



Jury :

Rapporteurs

Rebekka BURKHOLZ, Full professor, CISPA Helmholtz Center for Information Security
Vincent GRIPON, Professeur des universités, IMT Atlantique

Examineurs

Frederik MALLMANN-TRENN, Associate Professor, King's College London
Giovanni NEGLIA, Directeur de recherche, Inria
Laurent VIENNOT, Directeur de recherche, Inria Paris

Directeur de thèse

Frédéric GIROIRE, Directeur de recherche, CNRS

Co-directeur de thèse

Emanuele NATALE, Chargé de recherche, CNRS

Davide FERRE'

Inférence efficace en apprentissage automatique : sous-réseaux élagués et ordonnancement de tâches d'inférence compressibles

x+165 p.

Inférence efficace en apprentissage automatique : sous-réseaux élagués et ordonnancement de tâches d'inférence compressibles

Résumé

La croissance des réseaux de neurones et l'essor du Machine Learning as a Service entraînent d'importants coûts de calcul et d'énergie lors de l'inférence. Cette thèse étudie la réduction et l'allocation de ce calcul à deux échelles complémentaires : la structure des modèles et l'ordonnancement des ressources.

Nous étudions d'abord l'élagage dans le cadre de la Strong Lottery Ticket Hypothesis (SLTH), selon laquelle un grand réseau aléatoire contient, dès son initialisation, un sous-réseau approchant un modèle cible sans entraînement. Les preuves fondées sur le problème Random Subset Sum ne contrôlaient pas la taille du ticket obtenu. Grâce au problème Random Fixed-Size Subset Sum, nous établissons des garanties de parcimonie pour les réseaux denses et équivariants, avec des bornes inférieures presque correspondantes. Nous montrons ensuite une limite de l'élagage structuré : approcher un neurone ReLU par élagage de neurones requiert une largeur en $\Omega(d/\varepsilon)$, contre $O(d \log(1/\varepsilon))$ par élagage de poids, soit une séparation exponentielle en fonction de la précision.

Nous abordons ensuite l'ordonnancement de tâches d'inférence compressibles. Un premier modèle traite des échéances, de machines hétérogènes et d'un budget énergétique global par relaxation fractionnaire et approximation. Un second intègre les dates d'arrivée, la préemption et des fenêtres d'exécution dépendant des communications dans le continuum edge-cloud. Nous proposons une méthode de relaxation et d'arrondi, des formulations par flot de coût minimal et des garanties sur la charge conservée et la précision.

En complément, une contribution présentée en annexe permet le passage de messages des réseaux de neurones sur graphes par multiplication de matrices creuses dans l'écosystème Julia. Elle réduit la mémoire intermédiaire utilisée par les opérations gather/scatter sur GPU dans les configurations étudiées.

Mots-clés : Apprentissage automatique, réseaux de neurones, parcimonie, élagage, hypothèse du ticket gagnant, ordonnancement, efficacité énergétique, calcul creux.

Efficient machine learning inference: sparse subnetworks and scheduling of compressible tasks

Abstract

The growing scale of neural networks and the adoption of Machine Learning as a Service create substantial computational and energy costs during inference. This thesis studies how inference computation can be reduced and allocated at two complementary scales: model structure and resource scheduling.

First, we study neural-network pruning through the Strong Lottery Ticket Hypothesis (SLTH), according to which a sufficiently large random network contains, at initialization, a subnetwork that approximates a target model without training. Earlier proofs based on the Random Subset Sum problem did not control the size of the resulting ticket. Using the Random Fixed-Size Subset Sum problem, we establish sparsity guarantees for dense and equivariant networks, together with near-matching lower bounds. We then identify an intrinsic limitation of structured pruning: approximating a single ReLU neuron by neuron pruning requires width $\Omega(d/\varepsilon)$, whereas weight pruning needs only $O(d \log(1/\varepsilon))$, yielding an exponential separation in the accuracy dependence.

Second, we address the scheduling of compressible inference tasks. One setting combines deadlines, heterogeneous machines, and a global energy budget, for which we develop an exact fractional algorithm and an approximation algorithm. A second setting incorporates release times, preemption, and destination-dependent execution windows induced by communication delays across the edge–cloud continuum. We provide a relax-and-round method, exact min-cost-flow formulations in important cases, and workload-retention and accuracy guarantees.

An additional contribution, presented in an appendix, enables message passing based on sparse matrix multiplication for graph neural networks in the Julia ecosystem. In the tested configurations, it reduces the intermediate memory used by gather–scatter operations on GPUs.

Keywords: Machine learning, neural networks, sparsity, pruning, strong lottery ticket hypothesis, scheduling, energy efficiency, sparse computing.

Contents

1	Introduction	1
1.1	Motivation: From Capability to Efficiency	3
1.2	Efficiency at Two Complementary Scales	3
1.2.1	The model scale: which computation can be removed?	4
1.2.2	The system scale: how should computation be allocated?	6
1.2.3	From mathematical sparsity to executable computation	7
1.3	Methodological Overview	7
1.3.1	Probabilistic and approximation-theoretic analysis	7
1.3.2	Mathematical optimization and approximation algorithms	8
1.3.3	Computational evaluation and software development	9
1.4	Research Questions	9
1.5	Contributions	10
1.5.1	Model-level contributions	10
1.5.2	Scheduling contributions	10
1.5.3	Additional systems contribution	11
1.6	Organization of the Thesis	11
 Pruning at Initialization: Sparsity and Structure		
2	Foundations and Related Work on Pruning at Initialization	15
2.1	Background	17
2.1.1	Overparameterization and neural-network pruning	17
2.1.2	Notation and conventions	17
2.1.3	Unstructured and structured pruning	17
2.1.4	The Lottery Ticket Hypothesis	18
2.1.5	The Strong Lottery Ticket Hypothesis	18
2.1.6	Approximation, ticket size, and overparameterization	19
2.1.7	Random subset sums as a proof mechanism	20
2.2	Related Work	20
2.2.1	Empirical lottery tickets and training by pruning	21
2.2.2	Theoretical guarantees for the SLTH	21
2.2.3	Random subset sums and control of sparsity	22
2.2.4	Structured pruning and random-feature lower bounds	22
2.3	Perspective on the Two Contributions	23
3	On the Sparsity of the Strong Lottery Ticket Hypothesis	25
3.1	Introduction	27
3.1.1	Our Contribution	28
3.2	Fixed-Size Random Subset Sum	29

3.2.1	Proof of Theorem 3.2.1	31
3.3	Sparse Strong Lottery Ticket Hypothesis (SSLTH)	34
3.3.1	Sparsity-controlled dense networks	34
3.3.2	Proof of Theorem 3.3.1	35
3.3.3	Comparison with previous SLTH results	37
3.3.4	Extension to equivariant networks	39
3.3.5	Lower bound on the required overparameterization	40
3.4	Conclusion and Future Work	41
4	Structured vs. Unstructured Pruning: An Exponential Gap	43
4.1	Introduction	45
4.2	Problem Formulation and Preliminaries	47
4.3	Main Results and Proof Strategy	48
4.4	Proof of Theorem 4.3.1	50
4.4.1	Union bound over all pruned subnetworks	50
4.4.2	Restriction over simple input families	50
4.4.3	Breakpoints and necessary conditions for approximation	51
4.4.4	Construction of a dominating capped process	53
4.4.5	Construction of a dominating birth-death process	53
4.4.6	Back to the union bound	58
4.5	Conclusion and Future Work	59
Scheduling Compressible Machine-Learning Tasks		
5	Foundations for Scheduling Compressible Tasks	63
5.1	Background	65
5.1.1	Scheduling compressible inference tasks	65
5.1.2	From inference-time configurations to accuracy functions	65
5.1.3	Optimization as a language for scheduling	66
5.1.4	Continuous, integer, and mixed-integer optimization	67
5.1.5	Fractional relaxations and upper bounds	67
5.1.6	Rounding and approximation guarantees	68
5.1.7	Lagrangian multipliers and KKT conditions	69
5.2	Related Work	69
5.2.1	Model compression and inference-time configuration	70
5.2.2	Placement and scheduling of inference requests	70
5.2.3	Scheduling with controllable processing times	71
5.2.4	Energy-aware inference and scheduling	71
5.2.5	Communication- and deadline-aware edge scheduling	72
5.2.6	Optimization and approximation in inference scheduling	72
5.2.7	Positioning of the two contributions	73

6	Energy-Aware Scheduling of Compressible Inference Tasks	75
6.1	Introduction	77
6.2	Problem Formulation	78
6.2.1	Accuracy function	80
6.2.2	Fractional Relaxation	80
6.3	Optimal algorithm for DSCT-EA-FR	84
6.3.1	Algorithm for DSCT-EA-FR, one machine	84
6.3.2	Algorithm for DSCT-EA-FR, multiple machines	85
6.4	Approximation algorithm for DSCT-EA	89
6.5	Results	90
6.6	Conclusion	94
7	Communication-Aware Scheduling across the Edge-Cloud Continuum	95
7.1	Introduction	97
7.2	Problem Model and Exact Formulation	98
7.2.1	Communication-Aware Scheduling Model	98
7.2.2	Exact Integral Formulation	99
7.3	Fractional Relaxation and Rounding	100
7.3.1	Assignment-Free Fractional Relaxation	100
7.3.2	Eligibility-Aware Minimum-Discard Rounding	101
7.4	Min-Cost-Flow Formulations of the Fractional Relaxation	102
7.5	Theoretical Analysis	104
7.5.1	From the Rounding Rule to Workload Loss	105
7.5.2	Capacity Overflow in One Interval	106
7.5.3	General Destination-Dependent Certificates	111
7.5.4	From Retained Workload to Accuracy	113
7.5.5	Specialization to Common Windows	114
7.6	Experimental Evaluation	117
7.6.1	Methodology	117
7.6.2	Communication-Aware Scheduling	118
7.6.3	Quality Under Common Windows	120
7.6.4	Machine-Speed Heterogeneity	121
7.6.5	Running-Time Scalability	121
7.7	Conclusion	122
8	Conclusion and Perspectives	123
8.1	Summary of Contributions	123
8.2	Limitations and Open Questions	123
8.3	Research Perspectives	124
	References	127
	List of Figures	135

Appendix

A	Additional Results and Proofs for Chapter 3	139
A.1	Uniform sums and the Sum-Bounded property	139
A.2	Probability amplification	142
A.3	Simplified asymptotic condition	144
A.4	Details of the comparison with Malach et al.	145
A.5	Proof of Theorem 3.3.5	145
B	Additional Results and Proofs for Chapter 4	149
B.1	Extensions and Variants	149
B.1.1	Output weights bounded away from zero	149
B.1.2	Randomly initialized output weights	150
B.2	The Single-Neuron Regime	152
C	Sparse GPU Message Passing for Graph Neural Networks in Julia	155
C.1	Context and Motivation	157
C.2	From Gather–Scatter to Sparse Matrix Multiplication	157
C.2.1	The gather–scatter execution path	158
C.2.2	The SpMM execution path	158
C.3	Implementation in <code>GraphNeuralNetworks.jl</code>	158
C.3.1	Compressed sparse graphs (<code>:sparse</code>)	159
C.3.2	Coordinate graphs (<code>:coo</code>) and coalescing	159
C.3.3	Interaction with the CUDA sparse stack	160
C.4	Memory Evaluation	160
C.5	Discussion and Outlook	161

CHAPTER 1

Introduction

Machine learning is increasingly supported by large models and distributed computing infrastructures. Their scale brings remarkable capabilities, but it also makes computation, memory, energy, and communication central design concerns. This thesis studies a common question at two complementary levels: how can the computation required by machine-learning inference be reduced or allocated more intelligently while preserving useful performance guarantees?

At the model level, we ask how much of a large randomly initialized neural network must be retained to approximate a target model, and how the answer changes when pruning is restricted to entire neurons rather than individual weights. At the system level, we ask how the computation assigned to inference tasks should be adjusted and placed on heterogeneous machines when time, energy, and communication are limited.

1.1 Motivation: From Capability to Efficiency	3
1.2 Efficiency at Two Complementary Scales	3
1.2.1 The model scale: which computation can be removed?	4
1.2.2 The system scale: how should computation be allocated?	6
1.2.3 From mathematical sparsity to executable computation	7
1.3 Methodological Overview	7
1.3.1 Probabilistic and approximation-theoretic analysis	7
1.3.2 Mathematical optimization and approximation algorithms	8
1.3.3 Computational evaluation and software development	9
1.4 Research Questions	9
1.5 Contributions	10
1.5.1 Model-level contributions	10
1.5.2 Scheduling contributions	10
1.5.3 Additional systems contribution	11
1.6 Organization of the Thesis	11

1.1 Motivation: From Capability to Efficiency

More computation has become one of the most reliable ingredients in the progress of machine learning. Larger datasets, models, and computing platforms have enabled systems of steadily increasing capability, and the pace at which notable models are developed reflects the importance of scale in contemporary artificial intelligence (Epoch AI, 2024; Perrault and Clark, 2024). These gains come with growing resource requirements. Once a model is accurate enough to be useful, it is no longer sufficient to ask what the model can do. We must also ask what resources are needed to train it, deploy it, and execute it repeatedly.

This concern is especially visible during inference. Training a model can be expensive, but it is performed only periodically; the cost of inference is incurred every time a deployed model is queried. Measurements of Google’s machine-learning workloads for one week in each of 2019, 2020, and 2021 attributed roughly three-fifths of their energy use to inference and two-fifths to training (Patterson et al., 2022). The exact balance depends on the deployment, but repeated use can therefore make inference a dominant lifetime cost.

One way to reduce the cost of each prediction is model compression: making a network smaller or less expensive to run. This can come at the cost of some predictive accuracy, so the choice of how much to compress depends on the application. Pruning is one approach to compression. It removes selected weights or entire units from a network, leaving a smaller model to execute (Cheng et al., 2018; Hoefler et al., 2021). The trade-off between cost and accuracy matters especially when a model is queried repeatedly by many users.

A common example is Machine Learning as a Service (MLaaS). In this model, a provider hosts trained models and exposes their predictions through a service interface: clients submit data and obtain outputs without deploying or maintaining the model infrastructure themselves (Ribeiro et al., 2015). The provider must consequently provision the underlying resources and coordinate concurrent inference tasks with different workloads and timing requirements. A single task may be inexpensive, yet the accumulated cost of serving a large population can be substantial (Desislavov et al., 2023; McDonald et al., 2022).

Inference is also no longer confined to centralized data centers. It can take place on end devices such as mobile phones, on edge servers close to users, at access sites co-located with cellular base stations, at intermediate fog or regional nodes, or in a central cloud. Specialized accelerators such as GPUs and FPGAs may be available at several of these tiers (Drainakis et al., 2021). This edge–fog–cloud continuum can reduce response time and unnecessary data movement, but it also makes communication delay, machine heterogeneity, and placement part of the inference problem. Depending on the deployment, the cost of inference appears as latency, energy consumption, memory pressure, network traffic, or competition for shared machines.

1.2 Efficiency at Two Complementary Scales

The two main parts of this thesis study inference at the model and system scales. At the model scale, the first part asks when a smaller subnetwork of a large randomly initialized network can approximate a prescribed target model. We study how large the original network must be, how many parameters must remain, and how the answer changes if pruning can remove only whole neurons rather than individual weights. This theoretical study establishes when such subnetworks exist and identifies limits on their size and structure.

At the system scale, the second part studies how many inference tasks can share machines that differ in speed, energy efficiency, and communication delay. The amount of computation assigned to a task can be adjusted. For example, an image classifier may use a narrower network to respond sooner or a wider one to obtain higher accuracy. The scheduler must choose both the workload of each task and the machine that executes it. These choices must respect deadlines and, depending on the setting, a global energy budget or the communication delay to the selected machine. Given descriptions of the tasks and machines, our algorithms produce feasible allocations with guarantees relative to an optimal allocation.

Sections 1.2.1 and 1.2.2 introduce the two main parts in turn. Section 1.2.3 then considers what it takes to execute sparse computations efficiently in software, using graph-neural-network message passing as an example.

1.2.1 The model scale: which computation can be removed?

Modern neural networks are often highly overparameterized, containing many more parameters than appear necessary to perform their task (Frankle and Carbin, 2018; Hoefler et al., 2021). This excess capacity can be beneficial during training, but it also increases the memory and computation required to deploy the resulting model. As introduced above, pruning removes selected parameters to reduce these requirements while preserving useful behavior. Empirical results have shown that many trained networks can be reduced substantially while retaining much of their predictive performance, motivating the search for smaller subnetworks within larger models (Y. He and Xiao, 2023; Hoefler et al., 2021).

The Lottery Ticket Hypothesis made this idea more specific by suggesting that a large randomly initialized network contains a sparse subnetwork which, when trained from its original initialization, can attain performance comparable to that of the full network (Frankle and Carbin, 2018). This subnetwork is referred to as a winning "lottery ticket". The Strong Lottery Ticket Hypothesis then asked an even more stringent question: can a suitable subnetwork already approximate a prescribed target model at initialization, without training or otherwise modifying its retained weights? Empirical results showing that useful networks can be obtained through pruning alone motivated this question (Ramanujan et al., 2020; Zhou et al., 2019).

The setting is illustrated in Figure 1.1. A target model specifies the function to be approximated, and a larger source network is sampled at random. A binary mask is then applied to the weights of the source network: weights associated with a value of one are retained unchanged, whereas those associated with zero are removed from the network. Selecting a mask is thus simply a mathematical representation of pruning the network. The resulting subnetwork succeeds when its input–output map approximates that of the target model. Depending on the admissible family of masks, pruning may act independently on individual weights or only remove complete neurons together with all their incident weights.

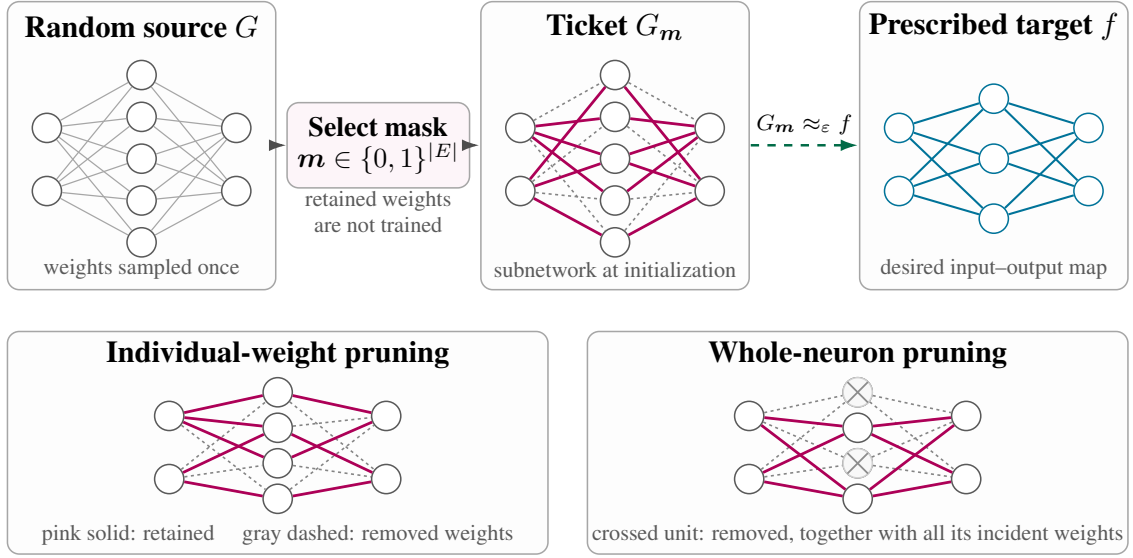


Figure 1.1 – The Strong Lottery Ticket setting. A larger randomly initialized source network G is pruned, without training its retained weights, to produce a ticket G_m that approximates a prescribed target model f . Solid magenta edges are retained and dashed gray edges are removed. Individual-weight pruning removes connections independently, whereas whole-neuron pruning removes a hidden unit together with all its incident weights. The two SLTH contributions in this thesis study how the required source size, the sparsity of the resulting ticket, and the granularity of the admissible mask affect approximation.

Two aspects of this approximation problem then need to be distinguished. The first is the size of the random source network required to contain an accurate approximation. The second is the size and form of the subnetwork that remains after pruning. A theorem may control the former without providing meaningful control of the latter. The number of retained parameters alone does not determine how easily a subnetwork can be executed: the structure of the remaining connections matters as well. Removing arbitrary individual weights offers great freedom, whereas removing complete neurons produces regular smaller matrices that are easier to execute on standard hardware. The first part of this thesis explores both directions: source size versus ticket size, and weight-pruning versus neuron-pruning.

1.2.2 The system scale: how should computation be allocated?

At deployment, an inference service may need to process many tasks that compete for a limited pool of computing resources. Tasks can become available at different times, have different deadlines, and call for different levels of prediction quality, while the available machines differ in speed, energy consumption, and proximity to the data source. Scheduling determines where and when each task is executed so that these requirements can be met with the available resources.

Architectures such as slimmable and Once-For-All networks expose a collection of subnetworks with different widths, depths, resolutions, or kernel sizes (Cai et al., 2020; Yu and T. Huang, 2019). They make it possible to adapt inference after training to a deployment condition. A larger configuration generally offers better prediction quality at a higher computational cost; a smaller configuration can reduce the prediction time and energy at the price of some accuracy. Rather than choosing one configuration permanently, a scheduler could select a configuration for each task in response to the current demand and available resources. It must therefore decide not only where and when the task should run, but also how much computation to allocate to it.

Figure 1.2 illustrates the resulting scheduling problem. Each task becomes available for processing at a given time, referred to as its release time, and must be completed by a deadline. Its workload is continuously adjustable. The scheduler must choose both the workload assigned to the task and the machine that executes it. These choices are coupled: a higher workload can improve prediction quality but requires more time and energy, while the selected machine determines processing speed, energy use, and, in a distributed platform, communication delay.

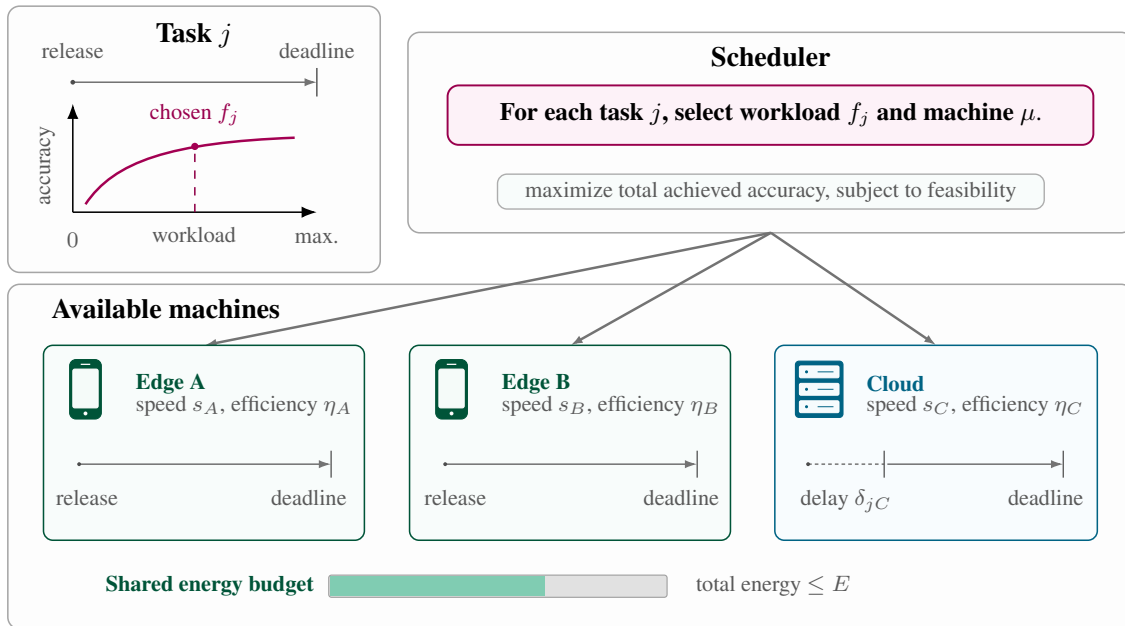


Figure 1.2 – Scheduling compressible inference tasks. Each task has a release time, a deadline, and a continuous workload–accuracy trade-off: extra computation improves accuracy, with diminishing returns. The scheduler jointly chooses the workload and execution machine for every task, maximizing aggregate accuracy subject to feasibility. The energy-aware setting adds a global processing-energy budget and heterogeneous machine energy efficiencies; the communication-aware setting adds destination-dependent communication delays and effective execution windows.

When demand exceeds the available capacity, the scheduler can respond by adjusting how much computation is assigned to each task. Rather than executing some tasks at their maximum configuration and rejecting the rest, it can reduce the workloads of several tasks and distribute the available resources among them. The objective is to find a feasible allocation that maximizes the aggregate utility achieved across all tasks.

The relevant constraints depend on the platform. In a cloud setting, heterogeneous machines can have different speeds and energy efficiencies, and the schedule may have to remain within a global energy budget. Across the edge–cloud continuum, tasks arrive over time and data must travel to the selected execution site. Communication delay is then part of the scheduling decision: it changes the interval during which computation can occur and may make a powerful remote machine less useful than a slower nearby resource. In both cases, adapting the model and scheduling its execution cannot be separated cleanly. The compression level determines processing time, while the schedule determines which compression levels are feasible.

1.2.3 From mathematical sparsity to executable computation

The path from a reduction on paper to a reduction in running time is not automatic. Modern hardware and numerical libraries are highly optimized for regular dense operations. An irregular pattern of zero weights can lower the parameter count without producing a proportional speedup, because indices must be stored and memory accesses become less predictable. Structured pruning addresses this gap by retaining groups that can still be represented as smaller dense arrays, but the stronger constraint can reduce approximation power.

Sparse execution offers another route: instead of forcing the remaining computation to become dense, the software stack can provide kernels designed for sparse data. This issue appears prominently in Graph Neural Networks (GNNs), where message passing follows a graph that is often sparse. The additional systems contribution reported in Appendix C enables a sparse-matrix-multiplication formulation of message passing in the Julia ecosystem. It is a smaller contribution than the work developed in the two main parts and is therefore presented separately, but it illustrates how useful mathematical structure must be exposed in a form that the implementation and hardware can exploit.

1.3 Methodological Overview

The contributions of this thesis span theoretical analysis, algorithm design, computational evaluation, and software development. The two main parts consequently rely on different families of methods. The study of sparse subnetworks uses tools from probability, combinatorics, and approximation theory, whereas the scheduling work draws primarily on mathematical optimization and the analysis of approximation algorithms. These analytical contributions are complemented by numerical experiments and by an implementation-oriented contribution to sparse graph computation. This section gives a high-level account of the principal methods and the role they play; the relevant chapters introduce their formal details.

1.3.1 Probabilistic and approximation-theoretic analysis

The SLTH chapters formulate pruning as an approximation problem inside a randomly initialized neural network. The analysis must show that, with sufficiently high probability over the

random initialization, at least one admissible pruning mask produces a subnetwork close to a prescribed target. A recurring proof strategy is therefore to translate the requirement that a pruned network approximate the target network’s input–output map into probabilistic events involving its randomly initialized parameters, and then to control how the resulting local approximation errors propagate through the network.

For weight pruning, random subset-sum constructions provide the main link between random parameters and target coefficients. Chapter 3 introduces a fixed-cardinality variant of this problem so that the number of selected terms, and hence the density of the resulting subnetwork, can be controlled explicitly. Its analysis uses the second-moment method to estimate the probability that a suitable subset exists, together with distributional estimates for sums of independent random variables, concentration inequalities, probability amplification, and union bounds. These scalar approximation results are then inserted into layerwise neural-network constructions. Packing arguments provide complementary lower bounds and make it possible to assess how closely the upper bounds capture the intrinsic dependence on source size, ticket density, and approximation accuracy.

The neuron-pruning analysis in Chapter 4 relies on lower-bound techniques rather than constructive approximation arguments. It studies the restrictions of ReLU networks to carefully chosen one-dimensional input families, where their behavior can be analyzed by tracking activation breakpoints. The resulting necessary conditions for approximation are studied probabilistically by modeling neuron selection as a stochastic process and comparing it, through coupling arguments, with simpler birth–death chains. Hitting-probability estimates, independence across input families, and union bounds are then used to obtain statements that apply uniformly over the possible pruning masks.

1.3.2 Mathematical optimization and approximation algorithms

The scheduling chapters begin by expressing the deployment setting as an optimization problem. Tasks are described through deadlines and through concave utility functions that relate allocated computation to prediction quality; machines are described through their speeds and, depending on the setting, their energy efficiencies, availability, and communication delays. Decision variables represent both the workload assigned to each task and its execution machine. Mixed-integer formulations capture these decisions exactly, but their discrete assignment variables also reflect the computational difficulty of the problems.

Fractional relaxation is the principal algorithmic tool used to move from these exact formulations to tractable solution methods. In the energy-aware setting of Chapter 6, allowing a task to be divided among machines produces a convex relaxation. For piecewise-linear utility functions, marginal-utility arguments and the Karush–Kuhn–Tucker optimality conditions lead to a dedicated polynomial-time solution method. A rounding procedure then converts the fractional allocation into a schedule in which each task is assigned to one machine, while controlling energy feasibility and the loss relative to the fractional upper bound.

Chapter 7 follows a related relax-and-round methodology but must also preserve the time intervals in which each task is eligible to execute on each machine. An exact mixed-integer formulation is relaxed to an assignment-free linear program, after which an eligibility-aware rounding algorithm selects one destination for each task while limiting discarded workload. In several structured cases, the fractional problem is represented exactly as a minimum-cost-flow problem, revealing additional combinatorial structure and enabling specialized polynomial-time algorithms.

The analysis combines feasibility proofs, workload-retention certificates, utility bounds, approximation guarantees under additional assumptions, and explicit complexity analysis.

1.3.3 Computational evaluation and software development

The scheduling algorithms are evaluated through controlled experiments designed to complement their analytical guarantees. Synthetic and application-informed workloads make it possible to vary the number of tasks and machines, resource pressure, machine heterogeneity, energy budgets, and communication delays independently. Exact mixed-integer solutions and fractional upper bounds, computed with mathematical-programming solvers including MOSEK and Gurobi, provide reference points for small and medium instances. Comparisons with scheduling baselines and measurements of solution quality and running time are then used to assess both the practical loss introduced by rounding and the scalability gained over exact optimization.

In Appendix C, we describe an alternative GPU path for graph message passing: when messages reduce to a weighted sum of neighboring features, sparse matrix multiplication replaces gather-scatter. This execution path is integrated into `GraphNeuralNetworks.jl` through Julia’s sparse graph representations and dispatch mechanisms, with GPU execution provided by `CUDA.jl` and NVIDIA’s `cuSPARSE` library. Supporting the new path required work on sparse-format conversion, coordinate coalescing, regression tests, and an upstream CUDA issue, while memory benchmarks quantify the effect of avoiding edge-indexed intermediate arrays.

1.4 Research Questions

The thesis is organized around four research questions, grouped into its two main parts.

- RQ1.** *What sparsity guarantees can be established for strong lottery tickets?* More precisely, how does prescribing the number of retained parameters change the overparameterization required for a random network to contain an accurate approximation of a target network?
- RQ2.** *How does pruning entire neurons affect the approximation guarantees of strong lottery tickets?* More precisely, can neuron pruning match the dependence on approximation accuracy obtained with individual-weight pruning, or does it inherently require a much larger source network?
- RQ3.** *How should compressible inference tasks be scheduled under a limited energy budget?* The scheduler must jointly choose task workloads and heterogeneous machines while meeting deadlines and balancing accuracy against energy consumption.
- RQ4.** *How can compressible inference tasks be scheduled across the edge–cloud continuum when their execution windows depend on both their arrival times and selected destinations?* Assigning a task to a given machine determines how quickly its workload can be processed, while destination-dependent communication delay determines how much time remains before its deadline. Workload allocation, placement, and execution over time must therefore be considered jointly.

1.5 Contributions

1.5.1 Model-level contributions

Controlling the sparsity of strong lottery tickets (RQ1). Existing SLTH constructions largely focused on the size of the random source network. Their central Random Subset Sum argument ensures that an appropriate subset of random contributions exists, but does not prescribe the cardinality of that subset. As a result, a theorem can guarantee an approximating subnetwork without controlling how many parameters it retains.

We introduce the Random Fixed-Size Subset Sum problem to make cardinality part of the guarantee. We establish essentially tight probabilistic bounds for this problem and use them to derive SLTH results in which the density of the winning ticket is an explicit parameter. The analysis covers dense and equivariant neural networks, relates the required source-network overparameterization to the desired ticket sparsity, and provides near-matching lower bounds. This contribution was published at NeurIPS 2024 (Natale et al., 2024).

An exponential gap between weight and neuron pruning (RQ2). Parameter count alone does not determine computational efficiency. Unstructured weight pruning produces irregular masks, while neuron pruning removes complete hidden units and therefore leads directly to smaller dense layers. We examine whether this practical regularity can preserve the strong approximation guarantees known for weight pruning.

We study the elementary but revealing task of approximating a single bias-free ReLU neuron by pruning a randomly initialized two-layer network. Even in this minimal setting, neuron pruning requires a number of hidden units that grows linearly with the inverse approximation error, while weight pruning needs only logarithmic growth. This yields an exponential separation in their dependence on accuracy. The result identifies a genuine cost of pruning granularity: the flexibility of choosing individual weights is not merely convenient for existing proofs, but can be fundamentally more powerful than selecting whole neurons. This contribution is based on joint work currently under submission (Ferré, Giroire, et al., 2026).

1.5.2 Scheduling contributions

Energy-aware scheduling of compressible inference tasks (RQ3). We consider inference tasks whose workloads can be reduced at a corresponding loss of accuracy. The tasks must meet deadlines and execute on heterogeneous machines with different speeds and energy efficiencies, while their total energy consumption is limited by a global budget. The resulting problem jointly combines discrete placement, continuous workload allocation, and an accuracy objective.

We formulate the problem mathematically, characterize its computational difficulty, and develop a fractional solution method together with an approximation algorithm for the implementable scheduling problem. The algorithm uses the fractional allocation as an upper bound and converts it into a feasible assignment while controlling the loss in value. Experiments with image-classification workloads show that adapting the computation assigned to each task can substantially reduce energy use while preserving most of the achievable accuracy. This contribution was published at ICPP 2024 (Silva Barros et al., 2024).

Communication-aware scheduling across the edge–cloud continuum (RQ4). We extend the scheduling perspective to a dynamic distributed setting. Tasks have release times and end-to-end deadlines, preemption is allowed, and the delay required to send a task to a machine produces a destination-dependent execution window. Placement must therefore account for communication and computation simultaneously.

We provide an exact integral formulation and an eligibility-aware relax-and-round algorithm. The fractional relaxation can be solved through min-cost-flow formulations in several important cases, while the rounding analysis yields general certificates on retained workload and accuracy and stronger approximation guarantees under additional structure. The experimental evaluation shows that the method remains close to the fractional upper bound and scales to instances for which the exact formulation becomes impractical. This contribution is based on joint work currently under submission (Ferré, Da Silva Barros, et al., 2026).

The two scheduling contributions share the same algorithmic backbone but address different sources of scarcity. The first introduces a global resource constraint through energy. The second introduces time-varying availability and machine-dependent communication. Together, they show how inference-time configurability can be incorporated into scheduling without treating model choice and resource allocation as separate decisions.

1.5.3 Additional systems contribution

In addition to the work associated with the four main research questions, Appendix C reports a smaller implementation-oriented contribution on message passing for GNNs in Julia. Conventional gather–scatter implementations can incur considerable intermediate-memory and execution overhead on GPUs. We enable an alternative formulation based on sparse matrix multiplication so that graph structure can be handled through optimized sparse linear-algebra operations. This work complements the main contributions by emphasizing that sparsity becomes operationally useful only when it is supported by the software stack.

1.6 Organization of the Thesis

The remainder of the thesis comprises two main parts, a conclusion, and appendices. The first part, *Pruning at Initialization: Sparsity and Structure*, studies model-level selection. Chapter 2 introduces the background and related work underlying RQ1 and RQ2. Chapter 3 addresses RQ1 by developing fixed-size subset-sum bounds and applying them to control the sparsity of strong lottery tickets. Chapter 4 addresses RQ2 by establishing the separation between weight and neuron pruning.

The second part, *Scheduling Compressible Machine-Learning Tasks*, studies system-level allocation. Chapter 5 introduces the concepts and related work underlying RQ3 and RQ4. Chapter 6 addresses RQ3 through the scheduling of compressible tasks under a global energy budget. Chapter 7 addresses RQ4 by incorporating task arrivals and destination-dependent communication delays across edge–cloud resources.

Chapter 8 summarizes the results, limitations, and future directions. The appendices collect additional proofs and technical details for the model-level chapters, as well as the sparse-GNN Julia contribution.

Pruning at Initialization: Sparsity and Structure

CHAPTER 2

Foundations and Related Work on Pruning at Initialization

This chapter introduces the concepts shared by the two contributions in this part of the thesis. We distinguish weight pruning from structured forms of pruning, recall the Lottery Ticket and Strong Lottery Ticket Hypotheses, and identify the quantities that a theoretical pruning guarantee must control: approximation error, overparameterization, and ticket size. We then explain how random subset sums enter proofs of the Strong Lottery Ticket Hypothesis and why controlling the cardinality of the selected subset, or imposing structure on the pruning mask, changes the problem substantially. Finally, we review the empirical and theoretical literature on lottery tickets, subset-sum constructions, structured pruning, and random-feature lower bounds, and use it to position the two contributions developed in the following chapters.

2.1 Background	17
2.1.1 Overparameterization and neural-network pruning	17
2.1.2 Notation and conventions	17
2.1.3 Unstructured and structured pruning	17
2.1.4 The Lottery Ticket Hypothesis	18
2.1.5 The Strong Lottery Ticket Hypothesis	18
2.1.6 Approximation, ticket size, and overparameterization	19
2.1.7 Random subset sums as a proof mechanism	20
2.2 Related Work	20
2.2.1 Empirical lottery tickets and training by pruning	21
2.2.2 Theoretical guarantees for the SLTH	21
2.2.3 Random subset sums and control of sparsity	22
2.2.4 Structured pruning and random-feature lower bounds	22
2.3 Perspective on the Two Contributions	23

2.1 Background

2.1.1 Overparameterization and neural-network pruning

Modern neural networks are commonly trained in an *overparameterized* regime: the number of trainable parameters can be much larger than what appears necessary to represent the function ultimately learned by the network. Overparameterization is closely associated with the empirical success of deep learning, but it also increases memory requirements and the cost of training and inference. This tension motivates *neural-network pruning*, which seeks to remove parameters or larger architectural components while preserving the behavior of the original model. Broad surveys of pruning methods and sparse deep learning are provided by [Hoefer et al. \(2021\)](#) and [Y. He and Xiao \(2023\)](#).

Consider a feed-forward network with weight matrices $\mathbf{W}_1, \dots, \mathbf{W}_L$ and activation function σ . Ignoring biases for simplicity, its successive representations take the form

$$\mathbf{h}_0(\mathbf{x}) = \mathbf{x}, \quad \mathbf{h}_\ell(\mathbf{x}) = \sigma(\mathbf{W}_\ell \mathbf{h}_{\ell-1}(\mathbf{x})), \quad \ell \in \{1, \dots, L-1\}, \quad (2.1)$$

with a final linear or nonlinear output layer. A weight-pruned network is obtained by associating each $\mathbf{W}_\ell$ with a binary mask $\mathbf{M}_\ell$ of the same shape and replacing it by

$$\mathbf{W}_\ell^{\text{pruned}} = \mathbf{M}_\ell \odot \mathbf{W}_\ell, \quad (2.2)$$

where $\odot$ denotes entrywise multiplication. Entries masked by zero are removed, whereas all retained weights keep their original values unless an additional retraining stage is explicitly allowed. The *density* of a mask is the fraction of weights it retains; its *sparsity* is the fraction it removes.

Pruning may be performed after training, during training, or at initialization. These regimes answer different questions. Post-training pruning asks how much redundancy remains in an already optimized network. Pruning during training treats the mask as part of the optimization process. Pruning at initialization asks the more restrictive question of what can be achieved by selecting from the parameters produced by a random initialization, before those parameters are modified. The two chapters in this part focus on this last regime.

2.1.2 Notation and conventions

For a positive integer n , we write $[n] = \{1, \dots, n\}$. We use bold lowercase letters for vectors, such as $\mathbf{x}$ and $\mathbf{w}$, and bold uppercase letters for matrices, such as $\mathbf{W}$ and $\mathbf{M}$; scalars are not bold. The ReLU activation is $\sigma(t) = \max\{t, 0\}$, applied coordinatewise to vectors. We use ε for approximation error and δ for failure probability when the latter is a separate parameter. The domain and norm of approximation are specified in each result.

2.1.3 Unstructured and structured pruning

The mask above can select every weight independently. This is usually called *unstructured pruning* or *weight pruning*. It offers the finest possible control: a network with m parameters admits as many as 2^m distinct binary weight masks. This flexibility is central to the strongest known approximation guarantees for pruning at initialization.

Unstructured sparsity, however, need not yield a proportionate reduction in execution time. Irregular sparsity requires storing indices and performing non-contiguous memory accesses, while

common accelerators and numerical libraries are optimized for regular dense operations. Parameter count is therefore an imperfect proxy for practical computational cost.

Structured pruning removes groups of parameters together. Depending on the architecture, one may prune blocks, channels, filters, attention heads, or complete neurons. For a two-layer network with N_h hidden neurons,

$$g(\mathbf{x}) = \sum_{i=1}^{N_h} a_i \sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle), \quad (2.3)$$

neuron pruning chooses a subset $S \subseteq [N_h]$ and produces

$$g_S(\mathbf{x}) = \sum_{i \in S} a_i \sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle). \quad (2.4)$$

All weights incident to a discarded neuron are removed together. The remaining network can consequently be represented by smaller dense matrices, making its computational effect more predictable.

The regularity of structured pruning comes at the price of a smaller search space and stronger dependencies among pruning decisions. Weight pruning can retain one incoming edge of a neuron while discarding another; neuron pruning must retain or discard the whole unit. The distinction is therefore not merely implementational. It changes the family of functions that can be obtained from a given randomly initialized network and, as Chapter 4 will show, can fundamentally change the amount of overparameterization required for approximation.

2.1.4 The Lottery Ticket Hypothesis

The Lottery Ticket Hypothesis (LTH) arose from the observation that a dense network may contain a sparse subnetwork that can be trained successfully from the weights it received at initialization. In the experiments of [Frankle and Carbin \(2018\)](#), a dense network is trained, low-magnitude weights are pruned, and the surviving weights are reset to their original values at initialization. Repeating this procedure can reveal sparse subnetworks that match the accuracy of the original network when trained in isolation. Such subnetworks are called *winning tickets*.

The important point is that the LTH is not simply a statement about post-training compression. The final sparse architecture is discovered using information from training, but its retained parameters are then returned to their initial values and trained again. The hypothesis is therefore concerned with the interaction between initialization, architecture, and trainability. It suggests that the success of a large network may partly result from its high probability of containing a favorable smaller architecture at initialization.

The original LTH still uses training to identify the ticket. It therefore motivates a stronger question: can a useful subnetwork be found by selecting a mask without changing its inherited random weights?

2.1.5 The Strong Lottery Ticket Hypothesis

The *Strong Lottery Ticket Hypothesis* (SLTH) concerns subnetworks that already perform the desired computation at initialization. No subsequent optimization of the retained weights is allowed. In theoretical work, this idea is commonly expressed as an approximation problem rather than directly in terms of prediction accuracy.

Let $\mathcal{F}$ be a family of target functions on an input domain $\mathcal{X} \subseteq \mathbb{R}^d$, and let G be a randomly initialized source network. One form of the guarantee starts with a chosen target $f \in \mathcal{F}$. It says that, with probability at least $1 - \delta$ over the initialization of G , some pruned subnetwork $g \in \text{Prune}(G)$ satisfies

$$\sup_{\mathbf{x} \in \mathcal{X}} \|f(\mathbf{x}) - g(\mathbf{x})\| \leq \varepsilon. \quad (2.5)$$

This is the form of the first rigorous SLTH result for ReLU networks, due to [Malach et al. \(2020\)](#): the target is chosen before the random source network is sampled. A stronger result says that, with high probability, a single sampled source network contains an approximation for *every* target in $\mathcal{F}$. Each target may need a different pruning mask. [Pensia et al. \(2020\)](#) proved a result of this stronger kind, as does Chapter 3. The two statements share the approximation goal in Equation (2.5), but differ in whether the same sampled source network must work for all targets. Their precise domains, norms, and initialization distributions also depend on the result.

These are existence guarantees: they show that an appropriate mask is present with high probability, but do not by themselves give an efficient way to find it. The existence of a winning ticket and the problem of identifying it should therefore be kept separate.

The term *strong* also marks a genuine difference from the original LTH. In the LTH, the selected subnetwork must still be trainable. In the SLTH, it must already approximate the target with its inherited random weights. Removing training makes rigorous analysis more tractable, because one can study a random function class without modeling optimization dynamics. At the same time, it imposes a considerably stronger requirement on the source network.

2.1.6 Approximation, ticket size, and overparameterization

Let m_t be the number of parameters in the target network, m the number of parameters in the random source network, and m_S the number of source parameters retained by a pruning mask. An SLTH statement involves several distinct notions of size. The ratio m/m_t measures the overparameterization of the source network relative to the target, whereas m_S/m measures the actual density of the ticket relative to the source; its sparsity is $1 - m_S/m$. Neither quantity determines the other. A theorem may show that a moderately overparameterized source contains an accurate subnetwork while allowing that subnetwork to retain a constant fraction of all source parameters. Conversely, an extremely large source might contain a very sparse ticket whose absolute size is still large compared with the target.

It is therefore useful to distinguish three questions:

1. How large must the random source network be relative to the target?
2. How many source parameters must the winning ticket retain?
3. How do both quantities depend on the approximation error ε and failure probability δ ?

The distinction becomes particularly important in deep networks. Approximation errors introduced in individual layers propagate through the subsequent layers, so the local accuracy required by a construction generally becomes more stringent as the target network grows in width or depth. A logarithmic dependence on $1/\varepsilon$ can remain manageable under this composition, whereas a polynomial dependence may quickly dominate the total network size.

Classical SLTH results primarily optimize the first quantity: they reduce the size of the random source network needed to contain an accurate ticket. This does not automatically yield a nontrivial guarantee on the third network size in the problem, namely the ticket itself. Chapter 3 addresses

precisely this distinction by treating ticket density as an explicit parameter. Chapter 4, in turn, studies how the dependence on ε changes when pruning decisions are constrained to complete neurons.

2.1.7 Random subset sums as a proof mechanism

The connection between pruning and random subset sums explains both the strength of weight-pruning constructions and the difficulty of controlling their sparsity. In its basic approximation form, the Random Subset Sum problem considers independent random variables $X_1, \dots, X_n$ and asks whether the realized sample has the following covering property:

$$\text{for every } z \in [-1, 1], \quad \text{there exists } S \subseteq [n] \text{ such that } \left| \sum_{i \in S} X_i - z \right| \leq \varepsilon. \quad (2.6)$$

Results such as that of [Lueker \(1998\)](#) show that, for suitable distributions, a sample whose size is logarithmic in $1/\varepsilon$ suffices with high probability.

To see why this is relevant to neural networks, consider the task of reproducing a single target coefficient inside a larger random construction. Pruning individual paths through the source network selects some random contributions and removes the others. The effective coefficient of the pruned construction is consequently a sum indexed by the retained paths. If the random contributions satisfy a property of the form in Equation (2.6), a suitable pruning mask can make their sum approximate the target coefficient. Repeating this local construction for all target weights, while controlling the accumulation of errors, yields an approximation of the complete target network.

The standard subset-sum property guarantees that some suitable subset exists, but it says nothing about its cardinality. Two masks can approximate the same coefficient while retaining very different numbers of random contributions. When the construction is replicated across a network, this missing cardinality information prevents the theorem from prescribing the number of retained weights in the resulting ticket.

A fixed-size version strengthens Equation (2.6) by additionally requiring $|S| = k$, for a prescribed value of k . This separates the accuracy of the approximation from the number of retained contributions. Chapter 3 develops this idea and uses it to express the overparameterization required by the SLTH as an explicit function of ticket density.

The same perspective also clarifies why structured pruning cannot be treated as a cosmetic modification of the argument. Under weight pruning, different paths can be selected almost independently, producing the combinatorial freedom needed for subset-sum approximation. Under neuron pruning, many incident weights are retained or removed together. The available random contributions are coupled, and the scalar subset-sum abstraction no longer describes the full approximation problem. Chapter 4 studies the resulting limitation directly.

2.2 Related Work

The literature most directly related to this part of the thesis developed along four connected lines. Empirical work asks whether useful subnetworks can be identified inside initialized networks; theoretical work asks when such subnetworks must exist; random subset-sum results provide a central probabilistic tool for proving their existence; and research on structured pruning investigates what happens when the mask is required to respect computationally meaningful groups

of parameters. The following review separates these questions because their guarantees are not directly interchangeable.

2.2.1 Empirical lottery tickets and training by pruning

The move from tickets identified through dense-network training (Frankle and Carbin, 2018) to masks learned over fixed random weights is the key empirical step towards the SLTH.

Zhou et al. (2019) showed that masks can give an untrained random network above-chance accuracy and also learned improved masks by optimizing a sampling probability for each weight while keeping the weights fixed. Ramanujan et al. (2020) instead assigned a trainable score to each fixed random weight and kept the highest-scoring fraction, reporting subnetworks competitive with conventionally trained networks. Related methods optimize binary masks or continuous mask parameters while leaving the underlying weights unchanged (Yulong Wang et al., 2020). Thus, leaving the weights untrained does not mean that no optimization is needed: the mask is learned from task data. These experiments provide algorithmic evidence for networks that learn through selection rather than weight adaptation, and form the empirical motivation for the SLTH.

Mask-based representations have also been used beyond the original image-classification setting. For example, a common random network and task- or client-specific masks can reduce communication in multi-task and federated learning (Isik, Pase, Gunduz, Koyejo, et al., 2024; Isik, Pase, Gunduz, Weissman, et al., 2022). Other work reduces the mask-search space by randomly pruning the source network before optimization (A. H. Gadhikar et al., 2023), or by searching only over part of a network while freezing the remaining parameters (Otsuka et al., 2024). These methods show that useful masks can be found in the settings tested. An SLTH existence theorem makes a different claim: under its stated assumptions, a suitable mask is present with high probability, even if the proof does not give an efficient way to find it.

An experiment can report how many parameters its learned mask retains. This does not, by itself, establish a probability guarantee that a ticket with a specified density exists in a randomly initialized network.

2.2.2 Theoretical guarantees for the SLTH

For a prescribed ReLU target, Malach et al. (2020) proved that a random source network contains an approximating subnetwork with a number of active weights comparable to the target’s parameter count. The trade-off was that the required source-network size depended polynomially on the inverse approximation error.

Pensia et al. (2020) used random subset sums to obtain logarithmic dependence on the inverse approximation error for the source-network size, but did not allow a desired ticket density to be specified. Independently, Orseau et al. (2020) obtained a logarithmic-overparameterization guarantee through a different construction. These results improved the size of the source network, leaving open how that size changes when the density of the ticket is prescribed.

The basic theory has since been extended beyond fully connected ReLU networks. Strong lottery-ticket guarantees are available for convolutional and residual architectures (Burkholz, 2022a; da Cunha et al., 2022), and a general framework covers equivariant networks (Ferbach et al., 2022). Other results broaden the admissible activation functions and reduce depth requirements (Burkholz, 2022b), construct a single pruned ticket that can be adapted to a family of functions by fitting only its output layer (Burkholz et al., 2022), and analyze the effect of nonzero random

biases (Fischer, A. Gadhikar, et al., 2022). Constructive procedures for planting and recovering tickets have also been investigated (Fischer and Burkholz, 2022).

Although these results differ in architecture, initialization, and approximation norm, most exploit the fine-grained freedom of weight pruning. Chapter 3 adds ticket density as an explicit parameter of the guarantee.

2.2.3 Random subset sums and control of sparsity

Random subset-sum approximation predates its use in neural-network pruning. Early probabilistic results established that random samples can approximate all targets in an interval by suitable subset sums (Lueker, 1982, 1998). More recent work sharpened and generalized these covering arguments, including multidimensional variants (Borst et al., 2023; A. C. W. d. Cunha et al., 2023).

Those covering results control how many random values must be available, but not how many are selected. Chapter 3 instead studies the Random Fixed-Size Subset Sum problem and turns its cardinality guarantee into an upper bound on ticket density. The chapter also gives lower bounds on the source-network size required for this control.

Multidimensional subset sums also provide positive guarantees for some forms of structured pruning. In particular, A. d. Cunha et al. (2023) use a multidimensional construction to prove structured lottery-ticket results for convolutional networks. Such results are compatible with the limitation studied in Chapter 4: the term structured pruning covers different grouping constraints, architectures, target classes, and approximation regimes, and a positive result for one combination does not imply that neuron pruning can reproduce the efficiency of weight pruning in another.

2.2.4 Structured pruning and random-feature lower bounds

Structured pruning has a long empirical history: early work already considered measuring the relevance of hidden units and removing unnecessary neurons (Mozer and Smolensky, 1988, 1989). Broad surveys cover modern methods and their hardware considerations (Y. He and Xiao, 2023; Hoefler et al., 2021). Compared with this extensive algorithmic literature, theoretical SLTH results under structured masks remain relatively limited.

Neuron pruning of a two-layer network is closely related to the random-feature model. Both begin with randomly chosen hidden features and approximate a target using a subset or linear combination of them. Standard random-feature approximation is less restrictive, however, because the output coefficients are normally learned. Under strong lottery-ticket neuron pruning, the output coefficients are inherited from the random initialization as well; the only permitted operation is to keep or discard a complete neuron. A lower bound for random features can therefore inform the pruning problem, but the two models should not be identified without accounting for this difference.

Several lower bounds show that random features can be inefficient for particular target functions and input distributions. Yehudai and Shamir (2019) exhibit a single-neuron ReLU target for which approximation under a Gaussian input requires exponentially many random features or very large coefficients. Kamath et al. (2020) strengthen this obstruction by allowing unrestricted output coefficients. These results rely on a target with a large bias, whereas the networks studied in Chapter 4 are bias-free. Hsu et al. (2021) derive lower bounds for approximating smooth functions with random features, addressing a broader function class under a different set of assumptions.

The lower bound in Chapter 4 isolates the effect of the pruning granularity in a minimal bias-free setting. It compares weight and neuron pruning for approximation of a single ReLU neuron and makes the dependence on the error tolerance explicit. The purpose is not to rule out useful structured tickets in general, but to show that the logarithmic dependence achievable through independent weight selection cannot, in this setting, be recovered by merely selecting complete random neurons.

2.3 Perspective on the Two Contributions

The next two chapters examine complementary aspects of pruning at initialization. Chapter 3 is constructive: it asks how the size of a strong lottery ticket can be controlled while preserving the favorable approximation guarantees of weight pruning. Its central technical object is the Random Fixed-Size Subset Sum problem.

Chapter 4 is instead a limitation result. It asks what is lost when the pruning mask must follow the hardware-friendly structure of complete neurons. By comparing its lower bounds with the logarithmic dependence available to weight pruning, it identifies an intrinsic approximation-theoretic cost of that structural restriction.

Together, the chapters separate two notions that are easily conflated. A ticket may be sparse in parameter count without having a structure that standard hardware can exploit efficiently; conversely, enforcing a computationally convenient structure can substantially increase the source-network size required to obtain an accurate ticket. The positive and negative results in this part describe these two sides of the same trade-off.

On the Sparsity of the Strong Lottery Ticket Hypothesis

Considerable research efforts have recently been made to show that a random neural network N contains subnetworks capable of accurately approximating any given neural network that is sufficiently smaller than N , without any training. This line of research, known as the Strong Lottery Ticket Hypothesis (SLTH), was originally motivated by the weaker Lottery Ticket Hypothesis, which states that a sufficiently large random neural network N contains sparse subnetworks that can be trained efficiently to achieve performance comparable to that of training the entire network N . Despite its original motivation, results on the SLTH have so far not provided any guarantee on the size of subnetworks. Such limitation is due to the nature of the main technical tool leveraged by these results, the Random Subset Sum (RSS) Problem. Informally, the RSS Problem asks how large a random i.i.d. sample Ω should be so that we are able to approximate any number in $[-1, 1]$, up to an error of ε , as the sum of a suitable subset of Ω .

We provide the first proof of the SLTH in classical settings, such as dense and equivariant networks, with guarantees on the sparsity of the subnetworks. Central to our results, is the proof of an essentially tight bound on the Random Fixed-Size Subset Sum Problem (RFSS), a variant of the RSS Problem in which we only ask for subsets of a given size, which is of independent interest.

3.1	Introduction	27
3.1.1	Our Contribution	28
3.2	Fixed-Size Random Subset Sum	29
3.2.1	Proof of Theorem 3.2.1	31
3.3	Sparse Strong Lottery Ticket Hypothesis (SSLTH)	34
3.3.1	Sparsity-controlled dense networks	34
3.3.2	Proof of Theorem 3.3.1	35
3.3.3	Comparison with previous SLTH results	37
3.3.3.1	Why near-optimal tickets cannot be highly sparse	39
3.3.4	Extension to equivariant networks	39
3.3.5	Lower bound on the required overparameterization	40
3.4	Conclusion and Future Work	41

3.1 Introduction

The Lottery Ticket Hypothesis (LTH) is a research direction that has attracted considerable attention over the years, stemming from the empirical contrast between the fact that, while large neural networks can be successfully trained to achieve good performance on a given task and successively pruned to a great level of sparsity without compromising their performance, researchers have struggled to train sparse neural networks from scratch. Frankle and Carbin (2018) observed that, using a simple pruning strategy (namely Iterative Magnitude Pruning while rewinding the original weights of the remaining edges to their value at initialization), *starting from a sufficiently large random neural network, it is possible to identify sparse subnetworks that can be trained to achieve the performance achievable by the starting network* (see Figure 3.1 for an illustration). The LTH soon gave rise to an even stronger claim. Empirical studies of “training-by-pruning” algorithms (Ramanujan et al., 2020; Zhou et al., 2019) provided evidence that *starting from a sufficiently large random neural network, it is possible to identify sparse subnetworks that exhibit good performance as they are, without changing the original weights* (see Figure 3.2 for an illustration).

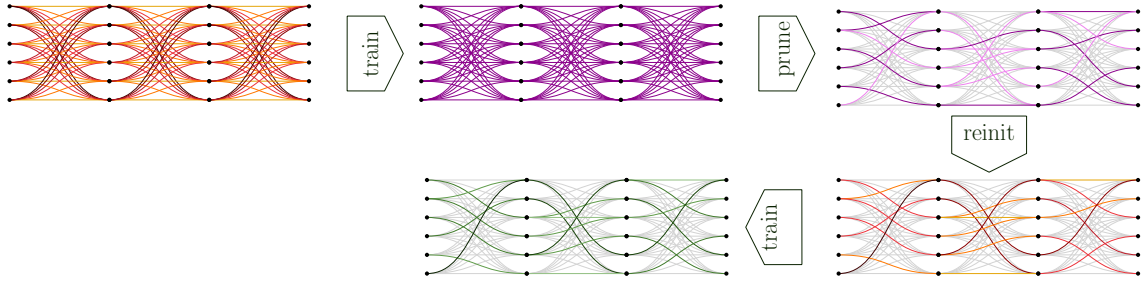


Figure 3.1 – **Simplified representation of the procedure for finding Lottery Tickets (LTH).** A large random neural network (step 1) is trained by iterative pruning with rewind: when the loss reaches a local minimum (step 2), some weights with smallest absolute value are pruned (step 3) and the value of the remaining edges is then reset to that of the initialization (step 4); finally, training is resumed and the final network is obtained (step 5). Remarkably, the sparser subnetwork is consistently able to reach a loss not larger than that right after pruning.



Figure 3.2 – **Simplified representation of the procedure for finding Strong Lottery Tickets (SLTH), or training by pruning.** Previous work has shown that it is possible to sparsify a large random neural network in order to obtain subnetworks that achieve good performance for a task under consideration, motivating the *Strong Lottery Ticket Hypothesis*. No training is required.

By removing the need to analyze the dynamics of training, the last statement, namely the Strong Lottery Ticket Hypothesis (SLTH), allowed a fruitful series of rigorous proofs for increasingly general architectures. The broader development of this theory is reviewed in Section 2.2.2; the guarantees most relevant to this chapter can be summarized as follows:

Theorem 3.1.1 (Informal statement of previous SLTH results). *With high probability, a random artificial neural network N_Ω with m parameters can be pruned so that the resulting subnetwork N_S ε -approximates (i.e., approximates up to an error ε) any target artificial neural network N_t with $O(m/\log_2(1/\varepsilon))$ parameters.*

It is important to note that, to this day, we only have proofs on the existence of such subnetworks, also called winning tickets, but it remains an open question how to find them reliably.

All theoretical results on the SLTH however have so far not investigated the interplay between the sparsity of the winning ticket N_S and the size of the random neural network N_Ω . This is in contrast to the original motivation of the LTH and to the practical application of the aforementioned training-by-pruning algorithms that motivated the SLTH, such as (Isik, Pase, Gunduz, Koyejo, et al., 2024; Isik, Pase, Gunduz, Weissman, et al., 2022). In fact, to approximate target networks with $O(m/\log_2(1/\varepsilon))$ parameters, essentially all winning tickets N_S have $\Theta(m)$ parameters (see Section 3.3.3.1), thus being roughly of the same size of the original network N_Ω . We thus ask the following natural question:

If we want to ε -approximate a family of target artificial neural networks with m_t parameters by pruning a fraction α , called sparsity, of the m parameters of a random artificial neural network N_Ω , how big should m be?

We are particularly interested in the regime in which the density parameter $\gamma = 1 - \alpha$ vanishes as the size of the network increases, so that the size of the winning ticket N_S is $\gamma m = o(m)$.

The above question has so far remained unanswered as a consequence of the limitation inherited from the core technical tool that has been leveraged so far to prove SLTH results, namely the Random Subset Sum (RSS) Problem (Lueker, 1998). Informally, the RSS asks how large a random i.i.d. sample Ω should be so that we are able to approximate any number in $[-1, 1]$ as the sum of a suitable subset of Ω . The applicability of RSS to the SLTH was first recognized by Pensia et al. (2020) within the proof strategy previously developed by Malach et al. (2020). Our analysis of the RFSS Problem follows the second-moment approach introduced for random subset sums by Lueker (1982) and subsequently refined in multidimensional settings (Borst et al., 2023; A. C. W. d. Cunha et al., 2023). Those refinements control the simultaneous approximation of multidimensional targets, whereas the constraint central to this chapter is different: we prescribe the cardinality of the selected subset. This distinction is precisely what permits the subset-sum construction to control the density of the resulting winning ticket.

3.1.1 Our Contribution

We answer the aforementioned question by introducing and proving a refined variant of the RSS Problem, namely the Random Fixed-Size Subset Sum Problem (RFSS), in which the approximation of the target values should be achieved by only considering subsets of fixed size k from a set of n samples (Theorem 3.2.1). We focus on subsets of fixed size k rather than subsets of size up to k for two main reasons. From a theoretical point of view, it is a stronger requirement, and practically speaking, using fixed-size subsets enables us to achieve SLTH results where the layers of the lottery ticket exhibit a uniform structure, potentially offering a computational advantage in their implementation.

In Section 3.3, we show how the density γ impacts the *overparameterization*, i.e., the ratio (m/m_t) between the number of parameters of the original network N_Ω and that of the class of target networks N_t that can be ε -approximated by pruning N_Ω down to a subnetwork N_S with γm

parameters. In our analysis, we also compare and recover as special cases previous SLTH results such as (Burkholz, 2022a; da Cunha et al., 2022; Ferbach et al., 2022; Malach et al., 2020; Pensia et al., 2020). For instance, when $\gamma m = \Theta(m)$, we recover up to a logarithmic factor the result of Pensia et al. (2020), which states that the overparameterization needed is $O(\log_2(m_t^2/\varepsilon^2))$. In the case of Dense Neural Networks, Theorem 3.3.1 thus bridges the gap between the two extreme cases of $\gamma m = \Theta(m_t)$ and $\gamma m = \Theta(m)$ considered by Malach et al. (2020) and Pensia et al. (2020), respectively. It is worth noting that the result of Pensia et al. (2020) is often considered an improvement over that of Malach et al. (2020), as it exponentially reduces the overparameterization, albeit at the cost of a trivial sparsity level. Finally, we prove that our bounds on the overparameterization as a function of the subnetwork sparsity are essentially tight.

Organization of the chapter. Section 3.2 develops the Random Fixed-Size Subset Sum Problem, including the proof of its main theorem and the amplification results needed later in the chapter. Section 3.3 applies these results to dense and equivariant neural networks, compares the resulting guarantees with previous SLTH bounds, and establishes matching lower bounds. Section 3.4 concludes the chapter by discussing its limitations and directions for future work. Missing proofs and details are collected in Appendix A.

3.2 Fixed-Size Random Subset Sum

In this section we present our technical contributions on the RFSS, which are the foundation of our proofs regarding the sparsity of the SLTH.

Given a sample $\Omega = \{X_1, \dots, X_n\}$ and a set of indices $S \subseteq [n]$, using the notation of Section 2.1.2, we define $\Sigma_S^\Omega = \sum_{i \in S} X_i$, and we omit Ω when clear from the context. We now define a class of distributions for which our RFSS result holds.

Definition 3.2.1 (sum-bounded). We say that a probability density function f is *sum-bounded* if there exist positive constants c_l and c_u such that, for all $k \in \mathbb{N}$, given k independent samples $X_1, \dots, X_k$ with density f , the density of their sum $f_{\Sigma_{[k]}}$ satisfies

$$\frac{c_l}{\sqrt{k}} \leq f_{\Sigma_{[k]}}(x) \leq \frac{c_u}{\sqrt{k}},$$

with the lower bound holding for all $x \in [-\sqrt{k}, \sqrt{k}]$ and the upper bound holding for all $x \in \mathbb{R}$.

At first, our definition of sum-bounded could look as a weaker version of a classical local limit theorem on the sum of random variables (e.g., see (Petrov, 1975, Chapter VII, Theorem 7)). However, that is not the case, since we require a lower bound on the sum for any k , which is needed to prove our main result.

Denote, for all $x \in [0, 1]$, the binary entropy as

$$H_2(x) = -x \log_2 x - (1 - x) \log_2(1 - x).$$

Our main technical result is the following proof of a fixed-size subset variant of the RSS Problem.

Theorem 3.2.1. *Let $0 < \varepsilon < 1$, $c_{\text{hyp}} \geq 1$, k, n be integers with $1 \leq k \leq \frac{n}{2}$, and let $\Omega = \{X_1, \dots, X_n\}$ where the X_i 's are i.i.d. random variables with sum-bounded density. There exists a constant c_{thm} such that, if*

$$n \geq c_{\text{hyp}} \frac{\log_2 \frac{k}{\varepsilon}}{H_2\left(\frac{k}{n}\right)}, \quad (3.1)$$

then for every fixed $z \in [-\sqrt{k}, \sqrt{k}]$ it holds that

$$\Pr(\exists S \subset [n], |S| = k : |\Sigma_S - z| < \varepsilon) \geq c_{\text{thm}}.$$

Remark 3.2.1 – The proof of Theorem 3.2.1 is given in Section 3.2.1, and it actually holds for any $1 \leq k \leq \lambda n$, for an arbitrary $\lambda \in [1/n, 1)$. We state the theorem this way for readability and because we are primarily interested in high-sparsity settings (i.e., small size k of the subsets), so considering values of $k \geq \frac{n}{2}$ does not add much to our analysis. The same remark also holds for Corollary 3.2.3.

The sum-bounded condition of Definition 3.2.1 is easily verified for distributions such as the Gaussian distribution. Previous SLTH results rely on a classical resampling argument by (Lueker, 1998, Corollary 3.3), which shows how RSS results for $\text{Uniform}[-1, 1]$ independent random variables naturally extend to independent random variables that *contains* a uniform distribution, in the sense that they can be expressed as the mixture of distributions one of which is $\text{Uniform}[-1, 1]$ with constant probability.* The next lemma thus proves that the $\text{Uniform}[-1, 1]$ distribution is sum-bounded†. Its detailed proof is given in Appendix A.1.

Lemma 3.2.2. *The $\text{Uniform}[-1, 1]$ probability density function is sum-bounded, i.e., given a set $\mathcal{U}_n = \{U_i\}_{i \in [n]}$ of i.i.d. variables U_i with $\text{Uniform}[-1, 1]$ probability density function, there exist constants c_l and c_u such that the probability density function $f(x, n)$ of the sum $\Sigma_{[n]}^{\mathcal{U}_n}$ of these variables, for all $n \in \mathbb{N}$,*

$$\frac{c_l}{\sqrt{n}} \leq f(x, n) \leq \frac{c_u}{\sqrt{n}}, \quad (3.2)$$

with the lower bound holding for all $x \in [-\sqrt{n}, \sqrt{n}]$, and the upper bound holding for all $x \in \mathbb{R}$.

Finally, in our proofs on the Sparse SLTH in Section 3.3, we make use of the following corollary of Theorem 3.2.1, which ensures a uniform high probability of hitting any target $z \in [-\sqrt{k}, \sqrt{k}]$, considering independent random variables that contain a uniform distribution.

Corollary 3.2.3. *Let $0 < p \leq 1$ and $\varepsilon \in (0, 1/2)$ be constants, k, n with $1 \leq k \leq \frac{n}{2}$, and let $\Omega = \{X_1, \dots, X_n\}$ be i.i.d. random variables whose density is a mixture of a $\text{Uniform}([-1, 1])$ with probability p , and some other density otherwise. There exists a positive constant c_{amp} that only depends on p such that, if*

$$n \geq c_{\text{amp}} \frac{\log_2^2 \frac{k}{\varepsilon}}{H_2\left(\frac{k}{n}\right)}, \quad (3.3)$$

then

$$\Pr\left(\forall z \in [-\sqrt{k}, \sqrt{k}], \exists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon\right) \geq 1 - \varepsilon.$$

*. The definition in (Lueker, 1998, Corollary 3.3) is actually more general, since it concerns a different problem.

†. We believe that Lemma 3.2.2 is known, but we could not find a reference.

Proof Idea.

The corollary follows from three arguments. First, by a standard sampling argument, we can assume that a constant fraction of the sample follows a $\text{Uniform}[-1, 1]$ distribution. Secondly, by Lemma 3.2.2, the uniform probability density function is sum-bounded. We can thus apply Theorem 3.2.1, which guarantees a success probability of c_{thm} for approximating a given target. Finally, by a standard probability amplification argument and a union bound applied to Theorem 3.2.1, by paying an extra factor $\log_2(k/\varepsilon)$ in Eq. 3.1, the constant c_{thm} can be amplified to $1 - \varepsilon$, and the existence of a suitable subset S_z holds simultaneously for all $z \in [-\sqrt{k}, \sqrt{k}]$. The complete argument is given in Appendix A.2. $\square$

For sufficiently large k , the squared logarithmic dependence on k in the right-hand side of Equation 3.3 can be removed, as shown by the following corollary. Its proof is given in Appendix A.3.

Corollary 3.2.4. *Let $0 < p \leq 1$ and $\varepsilon \in (0, 1/2)$ be constants, k, n be integers with $1 \leq k \leq \frac{n}{2}$ and $k \geq 2c_{\text{amp}} \left(\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon} \right)$. Let $\Omega = \{X_1, \dots, X_n\}$ be i.i.d. random variables whose density is a mixture of a $\text{Uniform}([-1, 1])$ with probability p , and some other density otherwise. There exists a positive constant c_{amp} that only depends on p such that, if*

$$n \geq 2c_{\text{amp}} \frac{\log_2^2 \frac{1}{\varepsilon}}{H_2\left(\frac{k}{n}\right)}, \quad (3.4)$$

then

$$\Pr\left(\forall z \in [-\sqrt{k}, \sqrt{k}], \exists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon\right) \geq 1 - \varepsilon.$$

Throughout the proofs, ceilings and floors are taken as appropriate whenever a displayed quantity is not an integer. This convention simplifies the notation without affecting the asymptotic results.

3.2.1 Proof of Theorem 3.2.1

For simplicity, throughout the proof we will often use c to denote any positive constant. Let $\mathcal{S}_k = \{S \subset [n] \mid |S| = k\}$ and define, for a fixed $z \in [-\sqrt{k}, \sqrt{k}]$,

$$Y = Y(z) = \sum_{S \in \mathcal{S}_k} Z_S$$

where $Z_S = Z_S(z) = \mathbf{1}_{\{|\Sigma_S - z| < \varepsilon\}}$. Following (Lueker, 1982), we exploit the second moment method for RFSS, generalizing it to arbitrary k .

$$\Pr(Y > 0) \geq \frac{(\mathbb{E}[Y])^2}{\mathbb{E}[Y^2]}, \quad (3.5)$$

it thus suffices to prove that

$$\mathbb{E}[Y^2] \leq c(\mathbb{E}[Y])^2. \quad (3.6)$$

We first rewrite Eq. 3.5 in a more convenient form. Let $\tilde{S}$ and $\tilde{S}'$ be two independently and uniformly at random chosen subsets of $[n]$ of size k , and denote $H_S(z)$ as the event that Σ_S ε -approximates z , namely

$$H_S = H_S(z) = \{|\Sigma_S - z| < \varepsilon\}.$$

We have

$$\mathbb{E}[Y] = \sum_{S \in \mathcal{S}_k} \mathbb{E}[Z_S] = \sum_{S \in \mathcal{S}_k} \Pr(H_S) = \binom{n}{k} \Pr(H_{\tilde{S}}) \quad (3.7)$$

and

$$\begin{aligned} \mathbb{E}[Y^2] &= \mathbb{E} \left[\left(\sum_{S \in \mathcal{S}_k} Z_S \right) \left(\sum_{S' \in \mathcal{S}_k} Z_{S'} \right) \right] = \sum_{S, S' \in \mathcal{S}_k} \mathbb{E}[Z_S Z_{S'}] \\ &= \sum_{S, S' \in \mathcal{S}_k} \Pr(H_S \wedge H_{S'}) = \binom{n}{k}^2 \Pr(H_{\tilde{S}} \wedge H_{\tilde{S}'}). \end{aligned} \quad (3.8)$$

Using Eqs. 3.7 and 3.8 we can rewrite the r.h.s. of Eq. 3.5 as follows

$$\frac{(\mathbb{E}[Y])^2}{\mathbb{E}[Y^2]} = \frac{[\Pr(H_{\tilde{S}})]^2}{\Pr(H_{\tilde{S}} \wedge H_{\tilde{S}'})} = \frac{\Pr(H_{\tilde{S}})}{\Pr(H_{\tilde{S}'} | H_{\tilde{S}})}.$$

Eq. 3.6 thus becomes

$$\Pr(H_{\tilde{S}'} | H_{\tilde{S}}) \leq c \Pr(H_{\tilde{S}}). \quad (3.9)$$

Let I_i denote the event $\{|\tilde{S} \cap \tilde{S}'| = i\}$ and $I_{a,b}$ the event $\bigcup_{a \leq i \leq b} I_i$. Fix $\mu \in (\lambda, 1)$. By the law of total probability and independence of I_i and $H_{\tilde{S}}$, we rewrite the l.h.s. of Eq. 3.9 as follows:

$$\begin{aligned} &\Pr(H_{\tilde{S}'} | H_{\tilde{S}}) \\ &= \Pr(H_{\tilde{S}'} \wedge I_k | H_{\tilde{S}}) + \Pr(H_{\tilde{S}'} \wedge I_{\mu k, k-1} | H_{\tilde{S}}) + \Pr(H_{\tilde{S}'} \wedge I_{0, \mu k-1} | H_{\tilde{S}}) \\ &= \Pr(I_k) \cdot \Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_k) \end{aligned} \quad (3.10)$$

$$+ \Pr(I_{\mu k, k-1}) \cdot \Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_{\mu k, k-1}) \quad (3.11)$$

$$+ \sum_{i=0}^{\mu k-1} \left(\Pr(I_i) \cdot \Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_i) \right). \quad (3.12)$$

To conclude the proof, it suffices to show that each addendum in Eqs. 3.10, 3.11 and 3.12 are upper-bounded by some constant multiple of $\varepsilon/\sqrt{k}$, since the lower bound in Definition 3.2.1 ensures that

$$\frac{\varepsilon}{\sqrt{k}} \leq c \Pr(H_{\tilde{S}}). \quad (3.13)$$

As for the first addendum (Eq. 3.10), since $\Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_k) = 1$, then

$$\begin{aligned} \Pr(I_k) \cdot \Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_k) &= \Pr(I_k) = \frac{1}{\binom{n}{k}} \stackrel{(a)}{\leq} \sqrt{\frac{8k(n-k)}{n}} 2^{-nH_2(\frac{k}{n})} \\ &\stackrel{(b)}{\leq} \sqrt{\frac{8k(n-k)}{n}} 2^{-c_{\text{hyp}} \log_2 \frac{k}{\varepsilon}} \stackrel{(c)}{\leq} 2\sqrt{2} \frac{\varepsilon}{\sqrt{k}}, \end{aligned} \quad (3.14)$$

where inequality (a) in Eq. 3.14 is a standard lower bound on $\binom{n}{k}$ holding for all $k \leq n-1$; in inequality (b) in Eq. 3.14 we used Eq. 3.1, namely $nH_2(\frac{k}{n}) \geq c_{\text{hyp}} \log_2 \frac{k}{\varepsilon}$; in inequality (c) in Eq. 3.14 we used that $c_{\text{hyp}} \geq 1$.

As for the second addendum (Eq. 3.11), we next show that

$$\Pr(I_{\mu k, k-1}) \Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_{\mu k, k-1}) \leq c \frac{\varepsilon}{\sqrt{k}} \quad (3.15)$$

by proving that

$$\Pr(I_{\mu k, k-1}) \leq \frac{c}{\sqrt{k}} \quad (3.16)$$

and

$$\Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_{\mu k, k-1}) \leq c\varepsilon. \quad (3.17)$$

First, observe that $I = |\tilde{S} \cap \tilde{S}'|$ follows a Hypergeometric(n, k, k) distribution, thus by Chebyshev's inequality

$$\begin{aligned} \Pr(I_{\mu k, k-1}) &\leq \Pr(I \geq \mu k) = \Pr\left(I - \frac{k^2}{n} \geq \mu k - \frac{k^2}{n}\right) \leq \frac{\text{Var}[I]}{\mu^2 k^2 \left(1 - \frac{k}{\mu n}\right)^2} \\ &\leq c' \frac{\frac{k^2}{n} \frac{n-k}{n} \frac{n-k}{n-1}}{k^2} \leq \frac{c}{\sqrt{k}}, \end{aligned}$$

having set $c' = \mu^2(1 - \lambda/\mu)^2 > 0$, thus proving Eq. 3.16. We now establish the two remaining estimates.

Proof of Eq. 3.17 Define $A = \tilde{S}' \setminus \tilde{S}$, and observe that

$$\Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_{\mu k, k-1}) \quad (3.18)$$

$$= \sum_{i=\mu k}^{k-1} \Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_i) \Pr(I_i | H_{\tilde{S}}, I_{\mu k, k-1}) \quad (3.19)$$

$$\begin{aligned} &= \sum_{i=\mu k}^{k-1} \int_{-\infty}^{\infty} \Pr(|\Sigma_A - (z - y)| < \varepsilon | \Sigma_I = y, I_i, H_{\tilde{S}}) \Pr(\Sigma_I = y | H_{\tilde{S}}, I_i) dy \\ &\quad \cdot \Pr(I_i | H_{\tilde{S}}, I_{\mu k, k-1}) \end{aligned} \quad (3.20)$$

$$\begin{aligned} &= \sum_{i=\mu k}^{k-1} \int_{-\infty}^{\infty} \Pr(|\Sigma_A - (z - y)| < \varepsilon | \Sigma_I = y, I_i) \Pr(\Sigma_I = y | H_{\tilde{S}}, I_i) dy \\ &\quad \cdot \Pr(I_i | H_{\tilde{S}}, I_{\mu k, k-1}) \end{aligned} \quad (3.21)$$

$$\begin{aligned} &\leq c\varepsilon \sum_{i=\mu k}^{k-1} \int_{-\infty}^{\infty} \Pr(\Sigma_I = y | H_{\tilde{S}}, I_i) dy \Pr(I_i | H_{\tilde{S}}, I_{\mu k, k-1}) \\ &\leq c\varepsilon \end{aligned} \quad (3.22)$$

where from Eq. 3.18 to Eq. 3.19 and from Eq. 3.19 to Eq. 3.20 we used the law of total probability;[‡] from Eq. 3.20 to Eq. 3.21 we dropped the redundant event $H_{\tilde{S}}$ in the conditioning, due to conditional independence; finally, from Eq. 3.21 to Eq. 3.22 we used Definition 3.2.1 which implies that for any $i \in \{\mu k, \dots, k-1\}$ it holds

$$\Pr(|\Sigma_A - (z - y)| < \varepsilon | \Sigma_I = y, I_i) = \Pr\left(|\Sigma_{[k-i]} - (z - y)| < \varepsilon\right) \leq c\varepsilon.$$

[‡]. For simplicity, we denote the density of Σ_I conditional on $H_{\tilde{S}} \cap I_i$ as $\Pr(\Sigma_I = y | H_{\tilde{S}}, I_i)$.

Proof of Eq. 3.25 Let $A = \tilde{S}' \setminus \tilde{S}$. Analogously to the calculations from Eq. 3.19 to Eq. 3.21, by the law of total probability we have

$$\begin{aligned} & \sum_{i=0}^{\mu k-1} \Pr(I_i) \cdot \Pr(H_{\tilde{S}'} | H_{\tilde{S}}, I_i) \\ &= \sum_{i=0}^{\mu k-1} \Pr(I_i) \cdot \int_{-\infty}^{\infty} \Pr(|\Sigma_A - (z - y)| < \varepsilon | \Sigma_I = y, I_i) \Pr(\Sigma_I = y | H_{\tilde{S}}, I_i) dy \\ &= \sum_{i=0}^{\mu k-1} \Pr(I_i) \cdot \int_{-\infty}^{\infty} \Pr(|\Sigma_{[k-i]} - (z - y)| < \varepsilon) \Pr(\Sigma_I = y | H_{\tilde{S}}, I_i) dy \end{aligned} \quad (3.23)$$

$$\leq c \frac{\varepsilon}{\sqrt{k}} \sum_{i=0}^{\mu k-1} \Pr(I_i) \cdot \int_{-\infty}^{\infty} \Pr(\Sigma_I = y | H_{\tilde{S}}, I_i) dy \quad (3.24)$$

$$\leq c \frac{\varepsilon}{\sqrt{k}} \quad (3.25)$$

where from Eq. 3.23 to Eq. 3.24 we used Definition 3.2.1, which implies that for any $i \in \{0, \dots, \frac{9}{10}k - 1\}$ it holds

$$\Pr(|\Sigma_{[k-i]} - (z - y)| < \varepsilon) \leq c' \frac{\varepsilon}{\sqrt{k-i}} \leq c \frac{\varepsilon}{\sqrt{k}}.$$

The three bounds on the addenda in Eqs. 3.10, 3.11, and 3.12 (Eqs. 3.14, 3.15, and 3.25, respectively), combined with Eq. 3.13, conclude the proof.

3.3 Sparse Strong Lottery Ticket Hypothesis (SSLTH)

We now apply our results on the RFSS problem to the SLTH and obtain guarantees on the sparsity of winning tickets for Dense Neural Networks (DNNs, Theorem 3.3.1) and Equivariant NNs (Theorem 3.3.4).

3.3.1 Sparsity-controlled dense networks

The next theorem essentially interpolates between the two extremes of (Malach et al., 2020, Theorem 2.1) (where $\gamma m = \Theta(m_t)$) and (Pensia et al., 2020, Theorem 1) (where $\gamma m = \Theta(m)$), where we recall that m and m_t represent the number of parameters of the overparameterized and the target networks, respectively, and γ is the density of the winning ticket.

Throughout the network results in this chapter, vectors and matrices are written in boldface, while scalars are not. We use $\sigma(t) = \max\{t, 0\}$ for the ReLU activation and $\|\mathbf{W}\|$ for the spectral norm of a matrix $\mathbf{W}$. Let $\mathcal{F}$ be the family of target ReLU networks $f : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_\ell}$ of depth ℓ such that

$$\mathcal{F} = \{f : f(\mathbf{x}) = \mathbf{W}_\ell \sigma(\mathbf{W}_{\ell-1} \dots \sigma(\mathbf{W}_1 \mathbf{x})), \forall i \ \mathbf{W}_i \in \mathbb{R}^{d_i \times d_{i-1}} \text{ and } \|\mathbf{W}_i\| \leq 1\} \quad (3.26)$$

For a given $f \in \mathcal{F}$, for all $i \in [\ell]$, let $\rho_i = \max\{d_{i-1}/d_i, d_i/d_{i-1}\}$, and $\rho = \max_i \rho_i$. In the next theorem, the same ε bounds both the approximation error and the probability of failure. Recalling that c_{amp} is the constant defined in Corollary 3.2.3, we have the following result.

Theorem 3.3.1 (SSLTH for DNNs). *Let g be a randomly initialized feed-forward 2ℓ -layer neural network, in which each weight is drawn from a $\text{Uniform}[-1, 1]$ distribution, of the following form:*

$$g(\mathbf{x}) = \mathbf{M}_{2\ell} \sigma(\mathbf{M}_{2\ell-1} \dots \sigma(\mathbf{M}_1 \mathbf{x})).$$

Let $\gamma' = \gamma'(\varepsilon) \in (0, 1)$, $\mathbf{M}_{2i} \in \mathbb{R}^{d_i \times 2d_{i-1}n_i^}$ and $\mathbf{M}_{2i-1} \in \mathbb{R}^{2d_{i-1}n_i^* \times d_{i-1}}$, with n_i^* satisfying*

$$n_i^* = c_{\text{amp}} \frac{\log_2^2 \left(\frac{2\ell d_{i-1} d_i \gamma' n_i^*}{\varepsilon} \right)}{H_2(\gamma')}. \quad (3.27)$$

With probability at least $1 - \varepsilon$, for every $f \in \mathcal{F}$, where $\mathcal{F}$ is defined as in Eq. 3.26, g can be pruned to obtain a subnetwork of sparsity at least $\alpha = 1 - \gamma$ that approximates f up to an error ε , having defined $\gamma = \rho\gamma'$.

Proof Idea..

The theorem follows from a slight variation of the same approach detailed in (Pensia et al., 2020), in which we use our Corollary 3.2.3 instead of (Lueker, 1998, Corollary 2.5) when pruning g , allowing us to have control over the size of the pruned network. The complete argument is given in the following subsection, along with the supporting results from Pensia et al. (2020) that we recall for completeness. $\square$

3.3.2 Proof of Theorem 3.3.1

In the proof we will refer to the following results, upon which (Pensia et al., 2020, Theorem 1) relies (the statements below differ slightly, as we correct two small typos in their notation and mixing coefficients). With the understanding that by a mixture D of a distribution D_1 and D_2 with probability p it is meant that the pdf (we adopt the convention that this term includes generalized functions, such as Dirac deltas for point masses) of D can be written as a convex combination of the pdf of D_1 and that of D_2 , that is $f_D = pf_{D_1} + (1 - p)f_{D_2}$. For the unfamiliar reader, we note that in the literature this is often stated in short as $D = pD_1 + (1 - p)D_2$.

Lemma 3.3.2 (Pensia et al. (2020, Corollary 1)). *Let $X \sim \text{Uniform}[0, 1]$ (or $X \sim \text{Uniform}[-1, 0]$) and $Y \sim \text{Uniform}[-1, 1]$ be independent random variables. Let P be the distribution of the XY and δ_0 the Dirac delta at 0. Let D be the distribution obtained as mixture of δ_0 and P with probability $1/2$. Then D is the mixture of a $\text{Uniform}[-1/2, 1/2]$ and some distribution Q with probability $\ln(2)/4$.*

Corollary 3.3.3 (Pensia et al. (2020, Corollary 2)). *Let $X_1, \dots, X_n$ be iid with distribution D as defined in Lemma 3.3.2, where $n \geq C \ln(2/\varepsilon)$ for some universal constant C . Then*

$$\Pr \left(\forall z \in [-1, 1], \exists S \subset [n] : |z - \sum_{i \in S} X_i| \leq \varepsilon \right) \geq 1 - \varepsilon.$$

Proof of Theorem 3.3.1.

The key idea is exploiting Corollary 3.2.3 at each step of the pruning strategy established in (Pensia et al., 2020, Theorem 1), where Corollary 3.3.3 is used instead. Without loss of generality, we replace their $\min\{\varepsilon, \delta\}$ with ε . For the sake of easily following the approach adopted in (Pensia et al., 2020), let us define $n^*(x)$ as the function

$$n^*(x) = c_{\text{amp}} \frac{\log_2^2(kx)}{H_2\left(\frac{k}{n^*(x)}\right)} \quad (3.28)$$

where $k = \gamma' n^*(x)$. In the following, we use n^* as short for $n^*(1/\varepsilon)$, and we will only explicitly provide an argument for n^* when it is different than $1/\varepsilon$. For instance, in the last step of the proof, we will use $n^*(2\ell d_i d_{i-1}/\varepsilon)$, which matches the definition of n_i^* given in Eq. 3.27.

Consider Pensia et al. (2020, Lemma 1). When approximating a single link (that is, a weight), after the overparameterization (which creates an additional layer of width $2n^*$ in between the input and the output node) via $4n^*$ links, instead of pruning via Corollary 3.3.3, we prune via Corollary 3.2.3 twice in the second layer, that is we ensure that only $k = \gamma' n^*$ edges yield the desired approximation, both in the edges corresponding to the positive part of the input weights and in those corresponding to the negative part. Thus we obtain at most $4k$ surviving edges, after the preprocessing step and the pruning mask is applied. This yields a sparsity of at least $\alpha' = 1 - \gamma'$. Note that it is because of the preprocessing step that we go from distributions $\text{Uniform}[-1, 1]$ to distributions D , as defined in Lemma 3.3.2, which are shown to be a mixture with $\text{Uniform}[-1, 1]$ and therefore can be also handled via Corollary 3.2.3.

Consider Pensia et al. (2020, Lemma 2). When approximating a real-valued multivariate linear function, after the overparameterization (which creates an additional layer of width $2dn^*(d/\varepsilon)$ in between the d input nodes and the output node) one simply iterates the ideas of the previous case d times. For each input node, the overparameterization surviving the preprocessing step on the weights of the input layer is $4n^*(d/\varepsilon)$. Pruning the second layer of the overparameterized link for each input via Corollary 3.2.3 with $k = \gamma' n^*(d/\varepsilon)$ (again, performing this both on the edges corresponding to the positive part of the input weights and in those corresponding to the negative part), instead of exploiting Corollary 3.3.3, yields that at most $4dk$ edges survive after the pruning mask is applied. This yields a sparsity of at least $\alpha' = 1 - \gamma'$.

Finally, consider Pensia et al. (2020, Lemma 3). When approximating a layer with input dimension d_1 and output dimension d_2 , after the overparameterization (which creates an additional layer of width $2d_1 n^*(d_1 d_2/\varepsilon)$ in between the input nodes and the output nodes) one iterates the ideas of the previous case d_1 times in the input layer through the same preprocessing step, and d_2 times in the output layer, one for each of the d_1 blocks created by the preprocessing (essentially the weights in the input layer are *re-used* d_2 times). For each input node, the overparameterization surviving the preprocessing step is at most $2(d_2 + 1)n^*(d_1 d_2/\varepsilon)$. Overall, after the preprocessing step, we have at most $2d_1(d_2 + 1)n^*(d_1 d_2/\varepsilon)$ parameters. We then use Corollary 3.2.3 (with $k = \gamma' n^*(d_1 d_2/\varepsilon)$) to prune the number of parameters between the introduced additional layer and the d_2 outputs down to $2d_1 d_2 \gamma' n^*(d_1 d_2/\varepsilon)$. As for the edges between the d_1 inputs and the additional layer, only those that reach a neuron in the additional layer, from which there is at least one outgoing edge towards the d_2 outputs, are used; since for each of the d_1 blocks of $2n^*(d_1 d_2/\varepsilon)$ neurons in the additional layer we only kept $2\gamma' n^*(d_1 d_2/\varepsilon)$ outgoing edges to each of the d_2 output neurons, in the worst case (all the nodes involved in the subset-sums are disjoint) we keep $2d_2 \gamma' n^*(d_1 d_2/\varepsilon)$ of them for each of the d_1 neurons. Globally, we are left with a total of at most $2d_1 d_2 \gamma' n^*(d_1 d_2/\varepsilon)$ edges both in the input layer and in the output

layer, thus a total of $4d_1d_2\gamma'n^*(d_1d_2/\varepsilon)$ edges survive the pruning. The density of the surviving edges is then less than

$$\frac{4d_1d_2\gamma'n^*(d_1d_2/\varepsilon)}{2d_1^2n^*(d_1d_2/\varepsilon) + 2d_1d_2n^*(d_1d_2/\varepsilon)} = \frac{2d_2\gamma'}{d_1 + d_2} = \frac{(d_1\frac{d_2}{d_1} + d_2)\gamma'}{d_1 + d_2} \leq \rho_1\gamma',$$

where $\rho_1 = \max\{d_1/d_2, d_2/d_1\}$ and in the last inequality we used that $d_1d_2/d_1 + d_2 \leq \rho_1(d_1 + d_2)$ since $\rho_1 \geq 1$. This ensures a sparsity $\alpha' \geq 1 - \rho_1\gamma'$.

Pensia et al. (2020, Theorem 1) consists of performing, for every $i \in [\ell]$, the previous step on layer i with input dimension d_{i-1} and output dimension d_i . The overparameterization creates an additional layer of nodes of width $2d_{i-1}n^*(2\ell d_{i-1}d_i/\varepsilon)$ in between the d_{i-1} input nodes and the d_i output nodes. Since the construction is stacked ℓ times, this generates 2ℓ layers for the overparameterized network, which will therefore have a starting number of parameters

$$m = \sum_{i=1}^{\ell} 2d_{i-1}^2n^*(2\ell d_{i-1}d_i/\varepsilon) + 2d_{i-1}d_in^*(2\ell d_{i-1}d_i/\varepsilon).$$

Corollary 3.2.3 applied to each stacked overparameterized layer instead of Corollary 3.3.3 as in the previous step yields the following bound on the number m_S of parameters left after pruning:

$$m_S \leq \sum_{i=1}^{\ell} 4d_{i-1}d_ik_i,$$

where $k_i = \gamma'n^*(2\ell d_{i-1}d_i/\varepsilon)$. Recall that $\rho = \max_i \rho_i$, where $\rho_i = \max\{d_i/d_{i-1}, d_{i-1}/d_i\} \geq 1$. Recall that $\gamma = \rho\gamma'$. We obtain that

$$\begin{aligned} m_S &\leq \sum_{i=1}^{\ell} 2d_{i-1} \frac{d_i}{d_{i-1}} d_{i-1}k_i + 2d_{i-1}d_ik_i \\ &\leq \sum_{i=1}^{\ell} 2d_{i-1}\rho_i d_{i-1}k_i + 2d_{i-1}d_ik_i \\ &\leq \rho \sum_{i=1}^{\ell} 2d_{i-1}^2k_i + 2d_{i-1}d_ik_i \\ &= \rho\gamma' \sum_{i=1}^{\ell} 2d_{i-1}^2n^*(2\ell d_{i-1}d_i/\varepsilon) + 2d_{i-1}d_in^*(2\ell d_{i-1}d_i/\varepsilon) = \gamma m \end{aligned}$$

We then get that the density of the edges surviving the pruning is $m_S/m \leq \gamma$, which implies a sparsity of at least $\alpha = 1 - \gamma$. □

3.3.3 Comparison with previous SLTH results

To illustrate a simple example of how Theorem 3.3.1 addresses the main question asked in the introduction, consider the case where we want to approximate a target network with m_t parameters and ℓ layers, each of width d (so $\rho = 1$ and $\gamma' = \gamma$), by pruning an overparameterized network

to achieve a desired sparsity level of $\alpha = 1 - \gamma$. The condition expressed by Equation 3.27 in Theorem 3.3.1 comes from the use of Corollary 3.2.3 when pruning network g , as shown in the proof. If, instead of Corollary 3.2.3, we use its simplified variant Corollary 3.2.4, it is easy to observe that Equation 3.27 would become

$$n_i^* = c_{\text{amp}} \frac{\log_2^2 \left(\frac{2\ell d_{i-1} d_i}{\varepsilon} \right)}{H_2(\gamma')}. \quad (3.29)$$

Using this condition, Theorem 3.3.1 then tells us that we need to prune a randomly initialized network with twice as many layers and a number of parameters of the order of $d^2 \frac{\log^2 \frac{\ell d^2}{\varepsilon}}{H(\gamma)}$.

We will now clarify the connection between Theorem 3.3.1 and the earlier results from (Malach et al., 2020) and (Pensia et al., 2020). Figure 3.3 provides a quick visual comparison.

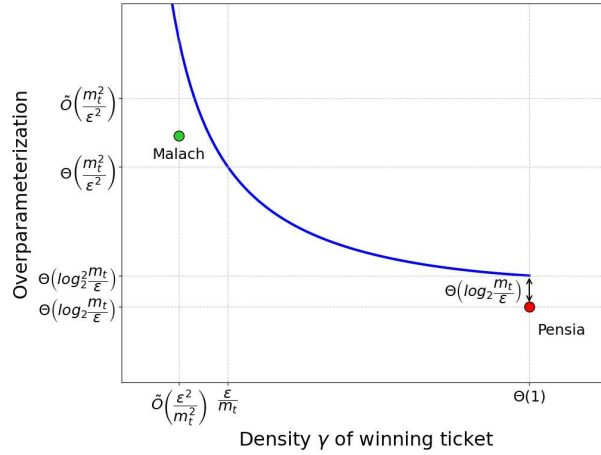


Figure 3.3 – A qualitative plot showing the relationship between the density γ of a winning ticket and the overparameterization required by Theorem 3.3.1 for a target network with m_t parameters. Earlier results from Pensia et al. (2020) and Malach et al. (2020) are shown for comparison.

Malach et al. (2020). When all layers have the same width d , Malach et al. (2020) showed that any target network with ℓ layers and a total of $m_t = d^2 \ell$ parameters can be ε -approximated by pruning a randomly initialized network with 2ℓ layers. The overparameterization of this network, relative to the target network, is $O\left(\frac{m_t^2}{\varepsilon^2} \log_2 \frac{m_t}{\varepsilon}\right) = \tilde{O}\left(\frac{m_t^2}{\varepsilon^2}\right)$. More specifically, the winning ticket found after pruning has a parameter count of the same order as the target network, resulting in a density of $\gamma = \tilde{O}\left(\frac{\varepsilon^2}{m_t^2}\right)$. Notably, this density γ is the inverse of the overparameterization, as the size of the winning ticket matches that of the target network.

Next, we show that Theorem 3.3.1 also yields a density that is polynomial in $\frac{\varepsilon}{m_t}$, when using an overparameterization of $\Theta\left(\frac{m_t^2}{\varepsilon^2}\right)$. Let $z = \left(\frac{m_t}{\varepsilon}\right)$, and note that $\gamma' = \gamma$ in Theorem 3.3.1, since all layers have the same width. As n_i^* in Theorem 3.3.1 represents the overparameterization with respect to the target network, let us set $n_i^* = cz^2$, for some constant c . Equation 3.27 then becomes

$$cz^2 \geq c_{\text{amp}} \frac{\log_2^2(cz^3\gamma)}{H(\gamma)} \quad (3.30)$$

We show that the inequality $cz^2 \geq c_{\text{amp}} \frac{\log_2^2(cz^3\gamma)}{\gamma \log_2(1/\gamma)}$ holds for some big enough constant c when setting $\gamma = \frac{\varepsilon}{m_t} = \frac{1}{z}$, which implies that Equation 3.30 is also satisfied. We get $cz \geq c_{\text{amp}} \frac{\log_2^2(cz^2)}{\log_2(z)}$. The calculation in Appendix A.4 verifies that this inequality holds for a sufficiently large constant c .

Overall, when using an overparameterization $\Theta\left(\frac{m_t^2}{\varepsilon^2}\right)$, we find a winning ticket with density $\frac{\varepsilon}{m_t}$, as shown in Figure 3.3.

Pensia et al. (2020). For simplicity, let us still consider target networks where all layers have the same width d , and we apply Theorem 3.3.1 using the simplified condition from Equation 3.29. When $\gamma m = \Theta(m)$, i.e. the density γ is a constant as in (Pensia et al., 2020) (see Section 3.3.3.1), the entropy term $H_2(\gamma')$ in the right-hand side of Equation 3.29 also becomes a constant. In this setting, we recover the result shown in (Pensia et al., 2020, Theorem 1), up to a logarithmic factor, as illustrated in Figure 3.3.

3.3.3.1 Why near-optimal tickets cannot be highly sparse

The claim is a direct consequence of the proof of (Pensia et al., 2020, Theorem 2) (Appendix B). There, in Step 3, it is shown that

$$|\mathcal{G}| \geq \frac{1}{2} \left(\frac{1}{2\varepsilon} \right)^{d^2},$$

where $\mathcal{G}$ is the set of subnetworks that can be formed. Let m be the number of parameters of the original network. If we consider subnetworks of size at most $k = \lfloor \gamma m \rfloor$, where $0 < \gamma \leq 1$ and $k \geq 1$, we have[§]

$$|\mathcal{G}| \leq \sum_{i=1}^k \binom{m}{i} \leq 2^{\gamma m \log_2(e/\gamma)},$$

which combined with the previous inequality implies

$$\gamma m \log_2 \left(\frac{e}{\gamma} \right) \geq d^2 \log_2 \left(\frac{1}{2\varepsilon} \right) - 1$$

If we have an overparameterized network of size $m = \mathcal{O}(d^2 \log_2(1/(2\varepsilon)))$, as in (Pensia et al., 2020), the last inequality requires γ to be bounded below by a positive constant, or equivalently $\gamma m = \Theta(m)$. Indeed, $\gamma \log_2(e/\gamma)$ tends to zero as γ tends to zero.

3.3.4 Extension to equivariant networks

Quite similarly to Theorem 3.3.1, the next result essentially generalizes (Ferbach et al., 2022) up to a factor $\log_2 \frac{1}{\varepsilon}$. The theorem is stated with the understanding that for G -equivariant networks, in order to preserve G -equivariance, pruning is best done not with respect to the parameters expressing the network in the canonical basis (i.e. directly on the weights of the network), but with respect to the *equivariant parameters*, that is those coefficients expressing the linear layers of the network as a linear combination of the elements of the corresponding equivariant

§. This follows from the upper bound $\sum_{i=1}^k \binom{n}{i} \leq \left(\frac{en}{k} \right)^k$ on the partial sum of binomial coefficients.

basis (Ferbach et al., 2022). For simplicity, due to the technical set-up, we assume all feature spaces being $\mathbb{F} = (\mathbb{R}^d, \sigma)$, with σ the linear representation of the group G , and the same number n of such feature spaces being stacked in each layer. A G -equivariant linear map from the i th feature space to the $i + 1$ st can be decomposed in a corresponding equivariant basis denoted $\mathcal{B}_{i \rightarrow i+1} = \mathcal{B}$. Since all feature spaces are the same, we omit the layers' indices. When stacking n feature spaces in the input and output of the i th layer, the full equivariant basis is denoted $k_{n \rightarrow n}$, and finally the basis of the G -equivariant maps from $\mathbb{F}^n$ to $\mathbb{F}^n$ can be written as the Kronecker product $k_{n \rightarrow n} \otimes \mathcal{B}$. For any basis $\mathcal{B} = \{b_1, \dots, b_p\}$, we denote its cardinality $p = |\mathcal{B}|$ and define $\|\mathcal{B}\| = \max_{\|\beta\|_\infty} \|\sum_{k=1}^p \beta_k b_k\|$, with $\|\cdot\|$ in the r.h.s. being the operator norm inherited from the ℓ_p norm.

Theorem 3.3.4 (SSLTH for Equivariant Networks). *Let h be a random 2ℓ -layer G -equivariant network where all equivariant parameters are drawn from a $\text{Uniform}[-1, 1]$ distribution, every odd layer expressed in the associated equivariant basis $k_{\tilde{n} \rightarrow n} \otimes \mathcal{B}$ and every even layer expressed in the associated equivariant basis $k_{n \rightarrow \tilde{n}} \otimes \mathcal{B}$. Let $\gamma = \gamma(\varepsilon) \in (0, 1)$, with $\tilde{n}$ satisfying*

$$\tilde{n} = c_{\text{amp}} \frac{\log_2^2 \left(\frac{2\ell n^2 \max\{|\mathcal{B}|, \|\mathcal{B}\|\} \gamma \tilde{n}}{\varepsilon} \right)}{H_2(\gamma)}.$$

With probability at least $1 - \varepsilon$, for every ℓ -layer G -equivariant neural network f , with all layers expressed in the associated equivariant basis $k_{n \rightarrow n} \otimes \mathcal{B}$, h can be pruned to obtain a G -equivariant subnetwork of sparsity at least $\alpha = 1 - \gamma$ that approximates f up to an error ε .

The proof, which we omit, is analogous to that of Theorem 3.3.1, since Ferbach et al. (2022, Theorem 1) exploit the exact same pruning strategy as Pensia et al. (2020), except for the fact that it is applied not to the original parameters of the equivariant network, but to the network expressed in terms of its equivariant basis (the sparsity α is here also intended with respect to the equivariant parameters count). This allows the construction to apply without losing the property of equivariance in the pruned approximating subnetwork obtained. The crucial step is when Corollary 3.2.3 is applied in (Ferbach et al., 2022, Lemma 1), instead of (Lueker, 1998, Corollary 2.5). This is done in parallel, multiple times, across non-overlapping coefficients of the equivariant basis. Thanks to the careful preprocessing devised by the authors, this preserves equivariance and at the same time ensures that each application of Corollary 3.2.3 is independent of the others.

To conclude the section, we mention that Theorem 3.3.4 applies in particular to vanilla CNNs, which are a special case of equivariant neural networks where the group is $G = (\mathbb{Z}^2, +)$, recovering previous SLTH results on CNN (Burkholz, 2022a; da Cunha et al., 2022). Furthermore, we remark that Theorem 3.3.1 can be revisited through the improvement upon the 2ℓ -depth overparameterization devised in (Burkholz, 2022b), i.e., it is possible to provide sparsity guarantees also for overparameterizations requiring depth $\ell + 1$ only. The analysis is more technical and we omit it, but the ideas are analogous to what shown in (Burkholz, 2022b). An analogous improvement is suggested as future work in (Ferbach et al., 2022).

3.3.5 Lower bound on the required overparameterization

We now adapt the lower bound of (Pensia et al., 2020) in order to almost match the required overparameterization of our Theorem 3.3.1, considering the simple scenario in which we want to

approximate the family $\mathcal{F}$ of all linear networks with weights forming a matrix having spectral norm less than $\sqrt{k}$; more formally

$$\mathcal{F} := \{h_{\mathbf{W}} : \mathbf{W} \in \mathbb{R}^{d \times d}, \|\mathbf{W}\| \leq \sqrt{k}\}, \quad \text{where } h_{\mathbf{W}}(\mathbf{x}) = \mathbf{W}\mathbf{x}. \quad (3.31)$$

Here $\|\mathbf{x}\|$ is the Euclidean norm of a vector, and $\|\mathbf{W}\|$ is the corresponding spectral norm of a matrix. The formal claim states that, if a network with n parameters can approximate every $h_{\mathbf{W}} \in \mathcal{F}$ with probability at least $1/2$ (after it is pruned down to k parameters), then the hypothesis of Theorem 3.2.1 in Eq. 3.1 must hold. ¶

Theorem 3.3.5. *Let $n, k \in \mathbb{N}$, with $1 \leq k \leq \lambda n$, having set $\lambda = 1 - 1/2\pi \approx 0.84$. Consider a neural network g with n parameters, and let $\mathcal{G}_k$ be the set of neural networks that can be formed by pruning g down to k parameters. Let $\mathcal{F}$ be as defined in Eq. 3.31. If it holds that, for some $\varepsilon < 1/16$,*

$$\forall h_{\mathbf{W}} \in \mathcal{F}, \mathbb{P} \left(\exists g' \in \mathcal{G}_k : \max_{\mathbf{x}: \|\mathbf{x}\| \leq 1} \|h_{\mathbf{W}}(\mathbf{x}) - g'(\mathbf{x})\| < \varepsilon \right) \geq \frac{1}{2}, \quad (3.32)$$

then it holds that

$$n \geq \frac{d^2}{2} \frac{\log_2 \frac{k}{\varepsilon}}{H_2 \left(\frac{k}{n} \right)}.$$

Proof idea..

We consider a family of matrices in the spectral-norm ball of Equation (3.31) whose pairwise distances exceed 2ε . A single pruned network can approximate at most one member of this family on the unit ball. Since each target is approximated with probability at least $1/2$, averaging over the random initialization forces the number of available size- k pruning masks to be at least half the size of this family. The packing argument gives at least $(\sqrt{k}/(2\varepsilon))^{d^2}$ separated matrices, while there are at most $\binom{n}{k}$ pruning masks. Comparing these counts and applying the entropy bound on $\binom{n}{k}$ yields the stated lower bound under the theorem’s assumptions. The details are given in Appendix A.5. □

3.4 Conclusion and Future Work

In this chapter, we have extended previous results on the Strong Lottery Ticket Hypothesis by quantifying the required overparameterization as a function of the sparsity of the subnetworks. Central to our results is a proof of the Random Fixed-size Subset Sum (RFSS) Problem, a refinement of the seminal Random Subset Sum (RSS) Problem in which the subsets have a required fixed size.

These guarantees are existential: they do not provide an efficient way to find a winning ticket of the prescribed sparsity. Although sparse subnetworks can reduce inference costs, the cost of locating them must also be considered when assessing the overall computational benefit.

A challenging open problem is to extend our analysis of RFSS to the multidimensional case, in which the random samples and targets are vectors in $\mathbb{R}^d$. Previous extensions of RSS to the

¶. Equivalently, the hypothesis of Corollary 3.2.3 must hold up to a factor $\Theta(\log_2 \frac{k}{\varepsilon})$.

multidimensional setting have enabled structured-pruning variants of the SLTH (da Cunha et al., 2022). A multidimensional RFSS result would make it possible to quantify, in the structured-pruning setting, how the required overparameterization depends on the sparsity of the structured subnetworks.

Another future direction is to refine the RFSS analysis in Theorem 3.2.1 so that it succeeds with probability $1 - \varepsilon$ rather than with constant probability. This would remove the additional factor $\log_2(1/\varepsilon)$ introduced by the amplification in Corollary 3.2.3 and bring the upper bound closer to the lower bound.

Finally, an important future direction is to improve training-by-pruning methods such as (Fischer and Burkholz, 2022; Fischer, A. Gadhikar, et al., 2022; Otsuka et al., 2024; Ramanujan et al., 2020; Zhou et al., 2019) or to develop new ones, in order to allow to efficiently find strong lottery tickets of a desired sparsity, thus empirically validating our theoretical predictions.

CHAPTER 4

Structured vs. Unstructured Pruning: An Exponential Gap

The Strong Lottery Ticket Hypothesis (SLTH) states that large, randomly initialized neural networks contain sparse subnetworks capable of approximating a target function at initialization without training, suggesting that pruning alone is sufficient. Pruning methods are typically classified as unstructured, where individual weights can be removed from the network, and structured, where parameters are removed according to specific patterns, as in neuron pruning. Existing theoretical results supporting the SLTH rely almost exclusively on unstructured pruning, showing that logarithmic overparameterization suffices to approximate simple target networks. In contrast, neuron pruning has received limited theoretical attention, despite its practical appeal for direct hardware speedups. This chapter considers the problem of approximating a single bias-free ReLU neuron by pruning hidden units of a randomly initialized two-layer ReLU network, effectively isolating the intrinsic limitations of neuron pruning. We show that achieving an ε -approximation requires a starting network size of $\Omega(1/\varepsilon)$ for neuron pruning, whereas weight pruning succeeds with only $O(\log(1/\varepsilon))$ hidden units, revealing an exponential separation between the two approaches.

4.1	Introduction	45
4.2	Problem Formulation and Preliminaries	47
4.3	Main Results and Proof Strategy	48
4.4	Proof of Theorem 4.3.1	50
4.4.1	Union bound over all pruned subnetworks	50
4.4.2	Restriction over simple input families	50
4.4.3	Breakpoints and necessary conditions for approximation	51
4.4.4	Construction of a dominating capped process	53
4.4.5	Construction of a dominating birth-death process	53
4.4.6	Back to the union bound	58
4.5	Conclusion and Future Work	59

The preceding chapter developed positive SLTH guarantees based on unstructured weight pruning and the random subset-sum mechanism. This chapter provides the complementary perspective: it asks whether the more computationally regular strategy of pruning entire neurons can offer comparable approximation guarantees. We show that it cannot, even for the elementary task of approximating a single bias-free ReLU neuron.

4.1 Introduction

Modern neural networks are typically trained in highly overparameterized regimes, often containing orders of magnitude more parameters than are seemingly necessary to represent the target function. Despite this redundancy, empirical evidence suggests that such large models can be pruned aggressively after training, yielding sparse subnetworks that retain comparable performance. Understanding the origin of this phenomenon has become a central topic in the theory of deep learning.

A prominent line of work addressing this question is the *Lottery Ticket Hypothesis* (LTH), introduced by [Frankle and Carbin \(2018\)](#). The LTH states that sufficiently large randomly initialized networks contain sparse subnetworks, called *winning tickets*, which can achieve the performance of the original dense network, when trained in isolation. Subsequent empirical works refined this perspective and proposed pruning strategies that identify such subnetworks efficiently ([Ramanujan et al., 2020](#); [Zhou et al., 2019](#)).

These developments motivated an even stronger conjecture, known as the *Strong Lottery Ticket Hypothesis* (SLTH), which asserts that winning tickets already exist at initialization, without requiring any training of the retained weights. In other words, pruning alone suffices to obtain a performant subnetwork. This formulation removes the need to analyze training dynamics and has enabled a series of rigorous existence proofs for fully connected and convolutional architectures ([Ferbach et al., 2022](#); [Malach et al., 2020](#); [Orseau et al., 2020](#); [Pensia et al., 2020](#)). The broader development of these results is reviewed in Section 2.2.2. Given a target network and $\varepsilon > 0$, a central question is how much overparameterization is needed for a randomly initialized network to contain, with high probability, a pruned subnetwork that ε -approximates the target network (see Definition 4.2.1 for the precise metric). This dependence on ε is critical in deep settings: since approximation errors compound across layers, the required accuracy must shrink with the target network’s width and depth. Consequently, a pruning scheme with linear dependence on $1/\varepsilon$ becomes prohibitively expensive, while the logarithmic dependence achieved by weight pruning remains compatible with existing SLTH constructions ([Pensia et al., 2020](#)).

Most existing theoretical results establishing the SLTH rely on *unstructured pruning*, also known as *weight pruning*, in which individual edges are removed from the network. A key technical insight, first identified by [Malach et al. \(2020\)](#) and later sharpened by [Pensia et al. \(2020\)](#), is that weight pruning allows one to approximate target weights by doing subset sums of randomly initialized weights. By leveraging a classical result of [Lueker \(1998\)](#) on the random subset sum problem, [Pensia et al. \(2020\)](#) showed that a random network with only logarithmic overparameterization—on the order of $O(\log(d/\varepsilon))$ per target weight—contains subnetworks that ε -approximate a broad family of target functions. Matching lower bounds show that this logarithmic dependence is essentially optimal for constant-depth networks ([Pensia et al., 2020](#)). While unstructured pruning provides a powerful theoretical framework for establishing the existence of accurate subnetworks, its implications for computational efficiency are less clear. Indeed, unstructured pruning

typically produces sparsity patterns with zeros in arbitrary locations in the weight matrices. Such sparsity does not, by itself, translate into predictable speedups on standard hardware, where dense linear algebra routines are optimized for contiguous memory access and regular computation.

The aforementioned gap between parameter sparsity and actual speedups motivates the study of *structured pruning* methods, which enforce sparsity at the level of entire blocks, rows, or columns of weight matrices. Such structure directly translates into smaller matrices and fewer arithmetic operations, yielding genuine reductions in memory usage and inference time. Among structured approaches, the simplest and most natural is *neuron pruning*, in which entire hidden units are removed, effectively deleting rows (and corresponding columns) of the network’s weight matrices. Understanding the expressive power and limitations of neuron pruning is therefore essential for bridging theoretical guarantees with practically meaningful efficiency gains. In contrast to weight pruning, the role of neuron pruning has received comparatively little theoretical attention. A broader account of structured pruning is given in Section 2.2.4. Positive SLTH guarantees have been obtained for structured convolutional pruning through multidimensional subset-sum constructions (A. d. Cunha et al., 2023). Those results concern different architectures and grouping constraints, however, and do not establish comparable guarantees for pruning complete neurons in the setting considered here.

The closest prior theoretical evidence comes from Malach et al. (2020), who observed that neuron pruning appears fundamentally weaker than weight pruning and pointed to results on random-feature models. The connection is informative but not exact. In a standard random-feature model, the hidden weights are fixed while the output coefficients are learned. Strong lottery-ticket neuron pruning is more restrictive: both the hidden and output weights are inherited from the random initialization, and the only permitted operation is to retain or discard complete neurons. In particular, Yehudai and Shamir (2019) showed that there exist target ReLU neurons with bias for which approximation by random features is provably hard: unless one allows either an exponential number of random features or exponentially large coefficients, uniform approximation over standard Gaussian inputs is impossible. Subsequent work strengthened this result by showing with a similar proof strategy that even when the magnitude of the coefficients is unrestricted, approximating such target ReLU neurons still requires an exponential number of random features (Kamath et al., 2020). These impossibility results are proved under the assumption that the target ReLU neuron may have a bias as large as $O(d^4)$, where d is the input dimension. They therefore do not settle the bias-free neuron-pruning problem studied here: the difficulty in those results could depend, at least in part, on the large target bias*. In particular, they do not show whether neuron pruning remains inefficient without biases or how the required source-network size depends on ε .

Our Contribution In this chapter, we revisit neuron pruning in a clean and natural setting. We focus on approximating a single ReLU neuron using a larger randomly initialized ReLU network with one hidden layer, where both the target neuron and the hidden neurons have zero bias. This setting isolates the expressive limitations of neuron pruning from confounding effects caused by large biases. Importantly, approximating a single neuron is arguably the simplest nontrivial approximation task, so lower bounds in this regime already indicate fundamental limitations of neuron pruning. We show that neuron pruning fundamentally requires a starting network with $\Omega(d/\varepsilon)$ hidden neurons to successfully ε -approximate a target ReLU neuron, when the output weights are

*. The proof in (Yehudai and Shamir, 2019) relies on the presence of such a bias, and removing this assumption would require a fundamentally different proof strategy, as confirmed by the authors via private communication.

bounded away from zero (Theorem 4.3.1). We further show that in the fully random initialization setting, where the output weights are also random, a weaker lower bound of $\Omega(\sqrt{d} \log(d)/\varepsilon)$ still holds (Theorem 4.3.2). This establishes an exponential gap between weight and neuron pruning in their dependence on the approximation accuracy ε , as weight pruning achieves an ε -approximation with only $O(d \log(d/\varepsilon))$ units (Pensia et al., 2020). This suggests that neuron pruning is not a viable SLTH mechanism in deep networks: in existing constructions, the required approximation accuracy ε must shrink with the target network’s width and depth, making a linear dependence on $1/\varepsilon$ prohibitive, whereas weight pruning only incurs logarithmic dependence on $1/\varepsilon$ (see (Pensia et al., 2020)).

At a technical level, to handle the bias-free setting, we adopt a novel proof strategy in which we track the location of hidden units’ nonlinearities along carefully constructed input families and derive necessary conditions for approximation. We then reformulate these conditions as hitting events of suitable stochastic processes, whose hitting probabilities yield bounds on the success probability of approximation.

4.2 Problem Formulation and Preliminaries

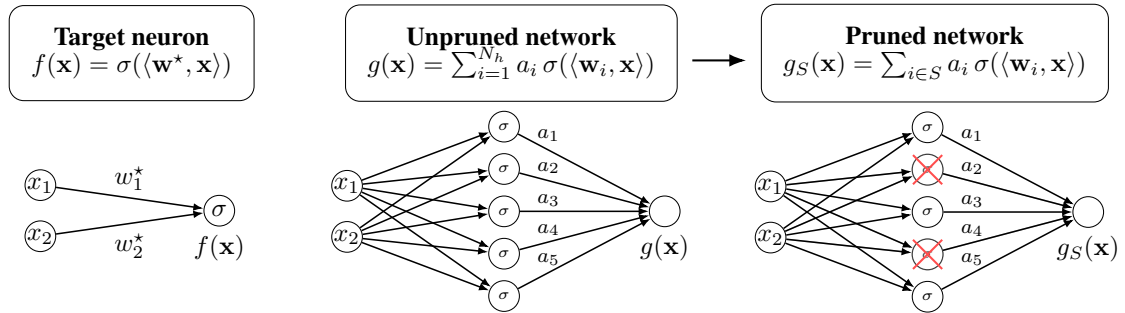


Figure 4.1 – Target ReLU neuron f , random network g ($N_h = 5$), and subnetwork g_S pruned from g by retaining a subset S of hidden units. For input dimension $d = 2$, each unit i has incoming weights $\mathbf{w}_i \in \mathbb{R}^2$ (not shown) and output weight $a_i \in \mathbb{R}$.

As in Chapter 3, vectors are denoted in boldface (e.g., $\mathbf{x}, \mathbf{w}$), while scalars are written in regular font. For vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, we write $\langle \mathbf{u}, \mathbf{v} \rangle$ for their dot product. We use $\sigma(t) := \max\{t, 0\}$ to denote the ReLU activation function. When we say that a function $h : \mathbb{R} \rightarrow \mathbb{R}$ is *linear* on an interval, we mean that it coincides on that interval with an *affine* function of the form $t \mapsto at + b$, for some $a, b \in \mathbb{R}$.

In Section 4.3, we study the problem of approximating a single target ReLU neuron using a larger randomly initialized neural network g , solely by pruning neurons in the hidden layer. Specifically, we consider a bias-free two-layer ReLU network of the form $g(\mathbf{x}) = \sum_{i=1}^{N_h} a_i \sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle)$, where $\mathbf{x} \in \mathbb{R}^d$, with hidden-layer neuron weights $\mathbf{w}_i \in \mathbb{R}^d$ and output weights $a_i \in \mathbb{R}$. Here N_h is the number of available hidden neurons. Unlike the preceding chapter, which counts retained weights to measure sparsity, this chapter asks how large N_h must be when pruning keeps or removes whole neurons. Figure 4.1 provides a visual overview of the neuron pruning setting and the notation we use.

The target function we will consider in our work is a single bias-free ReLU neuron $f(\mathbf{x}) := \sigma(\langle \mathbf{w}^*, \mathbf{x} \rangle)$, with $\|\mathbf{w}^*\|_2 = \sqrt{d}$. A *neuron-pruned subnetwork* g_S is obtained from g by selecting a

subset $S \subseteq [N_h]$ of hidden units and retaining exactly those neurons with all their incident edges, yielding $g_S(\mathbf{x}) = \sum_{i \in S} a_i \sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle)$. In line with the Strong Lottery Ticket Hypothesis, pruning is the only operation allowed: the retained weights are not trained or modified.

We will often use the following notion of ε -approximation.

Definition 4.2.1 (ε -approximation). For $\varepsilon > 0$ and radius $R \geq 1$, we say that a function h ε -approximates another function f on the ball of radius R if $\sup_{\|\mathbf{x}\|_2 \leq R} |h(\mathbf{x}) - f(\mathbf{x})| \leq \varepsilon$.

4.3 Main Results and Proof Strategy

Theorem 4.3.1 shows that, in a one hidden layer random network with unit output weights, neuron pruning alone cannot even ε -approximate a single target ReLU neuron without bias, unless the initial width is $\Omega(d/\varepsilon)$. This section presents a sketch of the proof of the theorem, in which we introduce the main ideas and provide an overview of the argument, referring the reader to Section 4.4 for the complete, detailed proof.

For simplicity, in Theorem 4.3.1 we fix the output weights of network g to be equal to 1. The same proof strategy extends to output-weight distributions that are uniformly bounded away from zero, such as Rademacher; the adaptation is given in Appendix B.1.1. Also, we require input dimension $d \geq 2$; for $d = 1$, neuron pruning reduces to weight pruning, and we can directly use the pruning lower bound proved in (Pensia et al., 2020). The requirement that $\|\mathbf{w}^*\|_2 = \sqrt{d}$ is mainly to simplify the notation in the proof; if, instead, we assume $\|\mathbf{w}^*\|_2 = 1$, then with straightforward modifications one obtains the lower bound $N_h \geq \min\{c_0 \frac{\sqrt{d}}{\varepsilon}, 2^{c_1 d}\}$.

Theorem 4.3.1. *Let $d \geq 2$, $\varepsilon \in (0, 1)$. Consider a one hidden-layer ReLU network without bias and unit output weights of the form $g(\mathbf{x}) = \sum_{i=1}^{N_h} 1 \cdot \sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle)$, where the weights $\{\mathbf{w}_i\}_{i=1}^{N_h}$ are drawn independently from $\mathcal{N}(0, I_d)$, and $\mathbf{x} \in \mathbb{R}^d$. Then, there exist a target neuron with weight vector $\mathbf{w}^* \in \mathbb{R}^d$ with $\|\mathbf{w}^*\|_2 = \sqrt{d}$, and universal constants $c_0, c_1 > 0$ such that, if*

$$N_h < \min\left\{c_0 \frac{d}{\varepsilon}, 2^{c_1 d}\right\},$$

then with probability at least $1 - e^{-\Omega(d)}$ over the draw of $\{\mathbf{w}_i\}_{i=1}^{N_h}$, for every subset $S \subseteq [N_h]$ and every fixed constant $R > 2$,

$$\sup_{\|\mathbf{x}\|_2 \leq R} \left| \sum_{i \in S} \sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle) - \sigma(\langle \mathbf{w}^*, \mathbf{x} \rangle) \right| \geq C\varepsilon,$$

for some universal constant $C > 0$.

Proof Idea. As neuron pruning may retain any number $k \in \{1, \dots, N_h\}$ of hidden units, we begin in Section 4.4.1 by controlling the probability of successful approximation uniformly over all possible pruned network sizes. This is achieved via a union bound over k , which reduces the analysis to bounding, for a fixed k , the probability p_k that a random network with k hidden units ε -approximates the target.

Our analysis is then based on tracking the evolution of *breakpoints*—the locations at which ReLU activations of the hidden neurons change slope—along carefully chosen one-dimensional input families, as described in Section 4.4.2. When a high-dimensional ReLU network is restricted to such paths, both the target function and the approximating network reduce to piecewise-linear functions whose behavior is entirely determined by the number, location, and interaction

of these breakpoints. Crucially, these breakpoints impose necessary conditions for approximation. Definition 4.4.1 formalizes when a breakpoint induces a non-negligible approximation error. Lemma 4.4.1 shows that the presence of a breakpoint away from the target nonlinearity prevents ε -approximation, while Lemma 4.4.2 establishes that at least one breakpoint must be placed within an ε -neighborhood of the target nonlinearity. Together, these results imply that successful neuron pruning requires introducing exactly one suitably aligned breakpoint and canceling all others.

We formalize this constraint by viewing neuron pruning as a stochastic sequential process in which neurons are selected one by one and the number of unresolved breakpoints evolves over time. The resulting process, faithfully captures the breakpoint dynamics induced by pruning. To make this process tractable, we construct a sequence of couplings: first to a capped process that limits the total number of breakpoints, and then to a homogeneous birth–death process (Sections 4.4.4 and 4.4.5). Each coupling is designed so that the dominating process is strictly more favorable to successful approximation, and therefore yields an upper bound on the success probability of the original pruning process.

The above argument allows us to derive an upper bound for the ε -approximation of a single one-dimensional input family (Lemma 4.4.7). To obtain dimension-dependent bounds, we exploit the fact that our construction uses $\lfloor d/2 \rfloor$ input families supported on disjoint coordinate pairs. The induced breakpoint processes are independent across these families, and therefore the overall success probability decays exponentially in the input dimension d (Lemma 4.4.8); see also Figure 4.3 for a visual interpretation.

Finally, Section 4.4.6 combines these bounds on p_k with the initial union bound over all pruned subnetworks, yielding the stated lower bound on the number of hidden neurons required for neuron pruning to achieve ε -approximation with non-negligible probability. $\square$

We also consider a variant of Theorem 4.3.1 in which the output weights are randomly initialized. In this setting, we obtain a lower bound on the required width N_h with the same dependence on $1/\varepsilon$, while the scaling in the input dimension is weaker. The proof strategy is substantially different from that of Theorem 4.3.1, and it relies on a simpler necessary condition for approximation. A proof idea follows the theorem, and the complete proof is given in Appendix B.1.2. In Theorem 4.3.2, if we instead consider a target weight vector with unit norm $\|\mathbf{w}^*\|_2 = 1$, the lower bound on N_h loses a factor of $\sqrt{d}$, scaling instead as $\log d/\varepsilon$.

Theorem 4.3.2. *Let $d \geq 2$, $R > 2$, $\varepsilon \in (0, 1)$, $\delta \in (0, 1)$, and let $\mathbf{w}^* \in \mathbb{R}^d$ with $\|\mathbf{w}^*\|_2 = \sqrt{d}$. Consider $g(\mathbf{x}) = \sum_{j=1}^{N_h} a_j \sigma(\langle \mathbf{w}_j, \mathbf{x} \rangle)$, where $\mathbf{w}_j \sim \mathcal{N}(0, I_d)$ and $a_j \sim \mathcal{N}(0, 1)$ are independent. Then there exist a constant $c_0 = c_0(\delta) > 0$ such that, if*

$$N_h < c_0 \frac{\sqrt{d} \log d}{\varepsilon},$$

then with probability at least $1 - \delta$ over the initialization, for every $S \subseteq [N_h]$,

$$\sup_{\|\mathbf{x}\|_2 \leq R} \left| \sum_{j \in S} a_j \sigma(\langle \mathbf{w}_j, \mathbf{x} \rangle) - \sigma(\langle \mathbf{w}^*, \mathbf{x} \rangle) \right| \geq \varepsilon/4.$$

Proof Idea. By rotational symmetry, take $\mathbf{w}^* = \sqrt{d} \mathbf{e}_1$ and restrict to the $d - 1$ input lines $\mathbf{x}_i(t) = t\mathbf{e}_1 + \mathbf{e}_i$. On each line, the target is $\sigma(\sqrt{d}t)$, whose sole breakpoint is at $t = 0$. If none of the initialized neurons has a breakpoint in $[-\varepsilon/\sqrt{d}, \varepsilon/\sqrt{d}]$, every pruned subnetwork is linear there and incurs error at least $\varepsilon/4$ by Lemma 4.4.2. Thus, error below $\varepsilon/4$ requires a nearby breakpoint on every line, even if all initialized neurons are allowed as candidates; the output weights do not affect this necessary condition.

The breakpoint events share the neurons' first-coordinate weights, but become independent across lines once we condition on those weights. Bounds on the sums of their absolute values and squares limit the chance of a hit on each line. Requiring hits on all $d - 1$ lines then makes successful approximation unlikely below the stated width threshold. $\square$

4.4 Proof of Theorem 4.3.1

The proof follows the sequence of reductions outlined in the proof sketch: we begin with a union bound over candidate pruned subnetworks, use carefully chosen input families to turn approximation into breakpoint conditions, and control those conditions through stochastic-process comparisons and independence across the families before completing the width lower bound.

4.4.1 Union bound over all pruned subnetworks

For any $k \in [N_h]$, pruning g down to k hidden neurons produces $\binom{N_h}{k}$ distinct subnetworks. Let E_k be the event that there is a k -neuron pruned subnetwork that ε -approximates the target, and let $p_k := \Pr(\text{a fixed } k\text{-neuron network } \varepsilon\text{-approximates the target})$. Then, by a union bound,

$$\Pr\left(\bigcup_{k=1}^{N_h} E_k\right) \leq \sum_{k=1}^{N_h} \binom{N_h}{k} p_k. \quad (4.1)$$

The rest of the proof is devoted to quantifying the probability p_k .

4.4.2 Restriction over simple input families

We introduce some simple families of inputs that will be used to derive necessary conditions on the function g for neuron pruning to succeed. For each $i \in \{1, \dots, \lfloor d/2 \rfloor\}$, define the input family $\mathbf{x}_i(t) := t\mathbf{e}_{2i-1} + \mathbf{e}_{2i}$ for $t \in \mathbb{R}$, where $\mathbf{e}_j$ denotes the j -th standard basis vector. This family has exactly two nonzero coordinates: a variable entry t at index $2i - 1$ and a fixed entry 1 at index $2i$. There are $\lfloor d/2 \rfloor$ such families, supported on disjoint pairs of coordinates.

Recall that $\mathbf{w}^* = (w_1^*, \dots, w_d^*) \in \mathbb{R}^d$ denotes the target weight vector. We choose $\mathbf{w}^* = (1, \dots, 1)^\top$. Along the input path $\mathbf{x}_i(t)$, the output of the target neuron is

$$f(\mathbf{x}_i(t)) = \sigma(\langle \mathbf{w}^*, \mathbf{x}_i(t) \rangle) = \sigma(w_{2i-1}^* t + w_{2i}^*),$$

which defines a one-dimensional piecewise-linear function of t with a single *breakpoint* $t_i^* := -w_{2i}^*/w_{2i-1}^* = -1$. On one side of t_i^* the function is affine with slope $w_{2i-1}^* = 1$, and on the other side it is identically zero. Similarly, for the randomly initialized network $g(\mathbf{x})$, for each hidden neuron j with an incoming weight vector $\mathbf{w}_j = (w_{j,1}, \dots, w_{j,d})$, and for each input family $\mathbf{x}_i(t)$, we have

$$\sigma(\langle \mathbf{w}_j, \mathbf{x}_i(t) \rangle) = \sigma(w_{j,2i-1} t + w_{j,2i}).$$

Each hidden neuron j therefore introduces its own breakpoint $t_{i,j} := -w_{j,2i}/w_{j,2i-1}$ for the input family $\mathbf{x}_i(t)$. Consequently, given a subset $S \subseteq [N_h]$ of hidden neurons, the restriction $g_S(\mathbf{x}_i(t))$ of the pruned network g_S on the input family $\mathbf{x}_i(t)$ yields a piecewise-linear function of t , whose breakpoints are induced by the neurons in S .

4.4.3 Breakpoints and necessary conditions for approximation

To analyze the probability p_k , it is convenient to view neuron pruning as a sequential selection process in which the k neurons that are kept in the final subnetwork are chosen one at a time.

Fix an index $i \in \{1, \dots, \lfloor d/2 \rfloor\}$ and consider the restriction of the network to the input family $\mathbf{x}_i(t)$ introduced above, in which each hidden neuron induces a single breakpoint at a location determined by its weights. As neurons are selected sequentially, each newly added neuron can have one of the following effects on the breakpoint structure of the current partial sum of chosen neurons for the input family $\mathbf{x}_i(t)$: (i) it may introduce a new breakpoint, (ii) it may cancel an existing breakpoint through interaction with previously selected neurons, or (iii) it may leave the breakpoint structure unchanged. After all k neurons have been selected, the restriction of the resulting pruned network to $\mathbf{x}_i(t)$ is a piecewise-linear function whose breakpoints are entirely determined by the selected neurons.

Since we are interested in inputs x satisfying $\|x\|_2 \leq R$, for an input family $\mathbf{x}_i(t)$ this restriction implies $|t| \leq \sqrt{R^2 - 1}$. Consequently, only breakpoints lying in the interval $I_R = [-\sqrt{R^2 - 1}, \sqrt{R^2 - 1}]$ can affect the approximation error on bounded inputs. Breakpoints outside this interval have no effect and will therefore be ignored and not counted.

We partition I_R into $\Theta(R/\varepsilon)$ subintervals (bins) of length ε (up to at most one shorter boundary interval), and denote this partition by $\mathcal{P}_\varepsilon$. The mere presence of a breakpoint does not by itself make a bin broken: several small slope changes may still leave the function uniformly close to an affine function at the scale relevant to the approximation guarantee. We therefore use the following quantitative definition. The universal threshold $c_{\text{br}} > 0$ is fixed below in Lemma 4.4.6.

Definition 4.4.1 (Broken bin). Let $I_B \subset \mathbb{R}$ be an interval (bin) of length ε , and let $h : I_B \rightarrow \mathbb{R}$ be a function. We say that I_B is *broken* for h if there exist three points $t_1 < t_2 < t_3$ in I_B such that for every affine function $\ell(t) = at + b$, $\max_{j=1,2,3} |h(t_j) - \ell(t_j)| \geq c_{\text{br}} \varepsilon$.

By Lemma 4.4.1 and Lemma 4.4.2, ε -approximation along the path $\mathbf{x}_i(t)$ requires the breakpoint structure of the pruned network to be aligned with that of the target: after selecting k neurons, the only possible broken bin is the one containing the target breakpoint t_i^* , and no additional broken bins may occur.

Lemma 4.4.1 (Broken bin prevents approximation). *Let $f(t) := \sigma(at + b)$, with $a \neq 0$, and let $I_B \subset \mathbb{R}$ be an interval of length ε such that the breakpoint $t^* := -b/a$ satisfies $t^* \notin I_B$. Let $h : \mathbb{R} \rightarrow \mathbb{R}$ be any function. If I_B is broken for h in the sense of Definition 4.4.1, then $\sup_{t \in I_B} |f(t) - h(t)| \geq c_{\text{br}} \varepsilon$,*

Proof.

Since $t^* \notin I_B$, the function f is linear on I_B . By Definition 4.4.1, for any affine function ℓ , there exist three points in I_B on which h differs from ℓ by at least $c_{\text{br}} \varepsilon$. Taking $\ell = f$ yields the claim. $\square$

Lemma 4.4.2 (A breakpoint is necessary for approximation). *Let $f(t) := \sigma(at + b)$, $a \neq 0$, and let $t^* := -b/a$ be its breakpoint. Let $h : \mathbb{R} \rightarrow \mathbb{R}$ be a function that is linear on an interval I containing $[t^* - \delta, t^* + \delta]$ for some $\delta > 0$. Then, $\sup_{t \in [t^* - \delta, t^* + \delta]} |f(t) - h(t)| \geq |a| \delta / 4$.*

Proof.

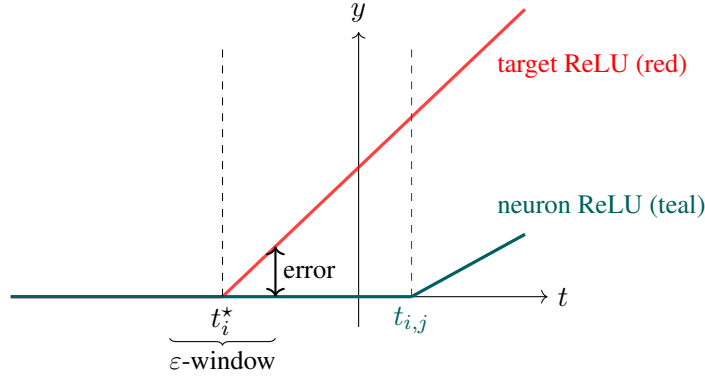


Figure 4.2 – Breakpoint alignment along a one-dimensional input family $\mathbf{x}_i(t)$. Along $\mathbf{x}_i(t)$, a bias-free ReLU neuron $\sigma(\langle \mathbf{w}, \mathbf{x} \rangle)$ reduces to the one-dimensional function $t \mapsto \sigma(w_{2i-1}t + w_{2i})$, whose breakpoint is $t_i(\mathbf{w}) = -w_{2i}/w_{2i-1}$. The target (red) has breakpoint t_i^* , while a randomly drawn neuron (teal) has breakpoint $t_{i,j}$. If $|t_{i,j} - t_i^*| > \varepsilon$, then on the ε -neighborhood of t_i^* the teal function is linear whereas the target changes slope, yielding a nontrivial uniform error, as stated in Lemma 4.4.2.

By translation, we may assume $t^* = 0$, so that $f(t) = \sigma(at)$. Let $h(t) = at + b$ be any affine function. Consider the three points $t = -\delta, 0, \delta$. At these points, $f(-\delta) = 0, f(0) = 0, f(\delta) = a\delta$ if $a > 0$, and the roles of $\pm\delta$ are reversed if $a < 0$. In either case, $|f(\delta) - f(-\delta)| = |a|\delta$. Define the errors $\Delta_- := h(-\delta) - f(-\delta), \Delta_0 := h(0) - f(0), \Delta_+ := h(\delta) - f(\delta)$. Since h is linear, $h(0) = \frac{1}{2}(h(-\delta) + h(\delta))$, which implies $\Delta_0 = \frac{1}{2}(\Delta_- + \Delta_+ + |a|\delta)$ (up to relabeling the endpoints when $a < 0$).

If $|\Delta_-| < \frac{|a|\delta}{4}$ and $|\Delta_+| < \frac{|a|\delta}{4}$, then $\Delta_0 > \frac{1}{2}\left(-\frac{|a|\delta}{4} - \frac{|a|\delta}{4} + |a|\delta\right) = \frac{|a|\delta}{4}$, so $|\Delta_0| \geq \frac{|a|\delta}{4}$.

Therefore, at least one of $|\Delta_-|, |\Delta_0|, |\Delta_+|$ is at least $\frac{|a|\delta}{4}$, and hence

$$\sup_{t \in [-\delta, \delta]} |f(t) - h(t)| \geq \frac{|a|\delta}{4}. \quad \square$$

It is convenient to rephrase the necessary ε -approximation conditions derived from Lemma 4.4.1 and Lemma 4.4.2 in an alternative form that simplifies the following analysis. Rather than viewing the goal as approximating the target ReLU itself, we conceptually start from a mirrored version of the target along $\mathbf{x}_i(t)$, obtained by flipping its output sign while preserving the same breakpoint location t_i^* . This mirrored function induces exactly one broken bin, namely the bin containing t_i^* . A neuron whose breakpoint lies in this same bin and whose slope jump is suitably close to the target's slope will cancel the mirrored breakpoint, so that their sum is linear on the bin and hence the bin becomes unbroken.

From this viewpoint, successful ε -approximation on the input family $\mathbf{x}_i(t)$ implies starting from a configuration with a single broken bin (in which the target breakpoint lies) and requiring that, after selecting k neurons, no broken bins remain. We adopt this reformulation throughout the remainder of the proof: the bin containing t_i^* is treated as initially broken, and we consider approximation to be successful only if after selecting k neurons we have no broken bins.

Using this reformulation, we now describe the sequential selection of k neurons as a stochastic process that tracks the number of broken bins. As we will introduce other stochastic processes later

in the proof, we will refer to this one as the *original pruning process*. At each step $s \in \{1, \dots, k\}$, a new neuron is added and it may increase by one, decrease by one, or leave unchanged the number of broken bins in $\mathcal{P}_\varepsilon$, depending on where its breakpoint falls. Since we treat the bin containing t_i^* as initially broken, the starting number of broken bins at step $s = 0$ will be one.

Definition 4.4.2. We define B_s^{orig} as the number of broken bins after having selected s neurons in the original pruning process, with $B_0^{\text{orig}} = 1$.

As noted above, for a successful ε -approximation on the input family $\mathbf{x}_i(t)$, we want to have $B_k^{\text{orig}} = 0$. Now consider all $\lfloor d/2 \rfloor$ disjoint input families $\mathbf{x}_i(t)$. For the overall approximation to be successful, we need to have $B_k^{\text{orig}} = 0$ for all such families. By construction, the breakpoints $t_{i,j}$ are independent across all $\lfloor d/2 \rfloor$ families $\mathbf{x}_i(t)$, so $p_k \leq \prod_{i=1}^{\lfloor d/2 \rfloor} \Pr(B_k^{\text{orig}} = 0)$, which gives the following.

Lemma 4.4.3. *It holds $p_k \leq (\Pr(B_k^{\text{orig}} = 0))^{\lfloor d/2 \rfloor}$.*

4.4.4 Construction of a dominating capped process

In this section, we introduce a simplified process that stochastically dominates the original pruning process described in Section 4.4.3, in the sense that it produces fewer broken bins. Analyzing this simplified process will yield an upper bound on the probability that the original pruning procedure succeeds. As in the previous section, fix an index $i \in \{1, \dots, \lfloor d/2 \rfloor\}$ and consider the restriction of the network to the input family $\mathbf{x}_i(t)$.

Fix a positive integer T . We define the *capped process* directly from the original process by

$$B_s^{\text{cap}} := \min\{B_s^{\text{orig}}, T\}, \quad s \in \{0, \dots, k\}.$$

Thus, $B_s^{\text{cap}} = T$ means that at least T bins are actually broken; it does not discard the underlying breakpoint configuration. Since adding one neuron introduces only one breakpoint on a fixed input family, both the original and capped counts change by at most one at each step.

By construction, after having selected s shared neurons, if we look at the capped and original processes we have

$$B_s^{\text{cap}} \leq B_s^{\text{orig}}, \quad \forall s \in \{1, \dots, k\} \quad (4.2)$$

The capped process is more favorable to maintaining a small number of broken bins. Moreover, $B_s^{\text{cap}} = 0$ if and only if $B_s^{\text{orig}} = 0$. In particular, we have the following upper bound on the probability of a successful ε -approximation for a random network with k hidden neurons.

Lemma 4.4.4. *It holds $\Pr(B_k^{\text{orig}} = 0) = \Pr(B_k^{\text{cap}} = 0)$.*

4.4.5 Construction of a dominating birth-death process

In the capped process described in Section 4.4.4, the probability that a selected neuron creates a new broken bin depends on the current number and locations of broken bins. For example, this probability is high when few bins are broken and lower when most are already broken. This dependence makes the exact process difficult to analyze. To simplify the analysis, we introduce a homogeneous birth-death process that we will use to upper bound $\Pr(B_k^{\text{cap}} = 0)$.

Definition 4.4.3. A (q, p, T) -chain $(B_s^{\text{bd}})_{s \geq 0}$ is a birth-and-death process on a state space $\{0, 1, \dots, T\}$ such that $B_0^{\text{bd}} = 1$ and

- from any state $b < T$, it transitions from state b to $b + 1$ with probability p ;
- from any state $b > 0$, it transitions from state b to $b - 1$ with probability q ;
- otherwise, it remains at b .

We choose the birth and death probabilities so that the homogeneous chain is more favorable to successful approximation than the capped process. More precisely, we construct a coupling for which

$$B_s^{\text{bd}} \leq B_s^{\text{cap}}, \quad \forall s \in \{1, \dots, k\} \quad (4.3)$$

even though the transition probabilities of the capped process may depend on its complete break-point history rather than only on its current count.

Lemma 4.4.5. Let H_s denote the complete history after s neurons have been selected. Suppose there are constants $p, q > 0$, with $p + q \leq 1$, such that for every possible history,

$$\Pr(B_{s+1}^{\text{cap}} = B_s^{\text{cap}} + 1 \mid H_s) \geq p \quad \text{whenever } B_s^{\text{cap}} < T, \quad (4.4)$$

$$\Pr(B_{s+1}^{\text{cap}} = B_s^{\text{cap}} - 1 \mid H_s) \leq q \quad \text{for every } H_s. \quad (4.5)$$

Then the capped process and the (q, p, T) -chain admit a coupling satisfying Equation (4.3). Consequently, $\Pr(B_k^{\text{cap}} = 0) \leq \Pr(B_k^{\text{bd}} = 0)$.

Proof.

We construct the coupling inductively, conditioning at step s on the complete history H_s and on the two current counts. If the counts agree below T , the increment of the capped process stochastically dominates the increment of the birth–death chain: its probability of $+1$ is at least p , by Equation (4.4), and its probability of -1 is at most q , by Equation (4.5). Hence the two increments can be coupled in increasing order. The same argument applies when both counts equal T , except that the birth–death chain has no upward transition there.

If $B_s^{\text{cap}} - B_s^{\text{bd}} \geq 2$, the ordering cannot be reversed in one step because both counts move by at most one. If the gap is one, the only pair of increments that could reverse it is a birth of the birth–death chain together with a death of the capped process. These events can be made disjoint: the former has probability p , while the latter has conditional probability at most q , and $p + q \leq 1$. This completes the inductive coupling. Since both processes start at one, it yields $B_s^{\text{bd}} \leq B_s^{\text{cap}}$ for every s . The claimed probability inequality follows because $B_k^{\text{cap}} = 0$ implies $B_k^{\text{bd}} = 0$ under this coupling. $\square$

We choose the cap to be a sufficiently small constant fraction of the total number of bins. Namely, for a sufficiently large constant $C_{\text{cap}} > 0$, let

$$T = \max \left\{ 1, \left\lfloor \frac{R}{C_{\text{cap}} \varepsilon} \right\rfloor \right\}. \quad (4.6)$$

The harmless rounding, as well as the regime in which the maximum equals one, can be absorbed into the universal constants below.

The next lemma supplies the geometric fact needed for a history-uniform birth bound. Its hypothesis permits the unbroken bin to contain an arbitrary collection of pre-existing breakpoints.

Lemma 4.4.6 (Adding a sufficiently steep neuron breaks an unbroken bin). *There exist universal constants $\kappa_{\text{br}}, c_{\text{br}} > 0$, with $c_{\text{br}} \leq \kappa_{\text{br}}/32$, such that the following holds. Let $B = [u, u + \varepsilon]$, and let $h : B \rightarrow \mathbb{R}$ be a piecewise-linear function for which B is unbroken. Let $r : B \rightarrow \mathbb{R}$ be piecewise-linear with exactly one breakpoint $t_0 \in [u + \varepsilon/4, u + 3\varepsilon/4]$ and slope jump Δ . If $|\Delta| \geq \kappa_{\text{br}}$, then B is broken for $h + r$ in the sense of Definition 4.4.1.*

Proof.

Set $t_1 = t_0 - \varepsilon/4$, $t_2 = t_0$, and $t_3 = t_0 + \varepsilon/4$. Let m_- and m_+ be the slopes of r to the left and right of t_0 , respectively, so that $\Delta = m_+ - m_-$. Fix an affine function $\tilde{\ell}$ with slope q , and set

$$E := \max_{j=1,2,3} |r(t_j) - \tilde{\ell}(t_j)|.$$

Since r is affine on each side of t_0 and $t_2 - t_1 = t_3 - t_2 = \varepsilon/4$, we have

$$\begin{aligned} |q - m_-| &= \left| \frac{(\tilde{\ell}(t_2) - r(t_2)) - (\tilde{\ell}(t_1) - r(t_1))}{t_2 - t_1} \right| \leq \frac{2E}{\varepsilon/4} = \frac{8E}{\varepsilon}, \\ |q - m_+| &= \left| \frac{(\tilde{\ell}(t_3) - r(t_3)) - (\tilde{\ell}(t_2) - r(t_2))}{t_3 - t_2} \right| \leq \frac{2E}{\varepsilon/4} = \frac{8E}{\varepsilon}. \end{aligned}$$

Therefore,

$$|\Delta| = |m_+ - m_-| \leq |m_+ - q| + |q - m_-| \leq \frac{16E}{\varepsilon},$$

which proves that, for every affine function $\tilde{\ell}$,

$$\max_{j=1,2,3} |r(t_j) - \tilde{\ell}(t_j)| \geq \frac{|\Delta|\varepsilon}{16}. \quad (4.7)$$

Because B is unbroken for h , the negation of Definition 4.4.1, applied to this particular triple, gives an affine function a such that

$$\max_{j=1,2,3} |h(t_j) - a(t_j)| < c_{\text{br}}\varepsilon. \quad (4.8)$$

Now fix any affine function ℓ . Since $\ell - a$ is affine, the reverse triangle inequality together with Equation (4.7) and Equation (4.8) gives

$$\begin{aligned} \max_j |h(t_j) + r(t_j) - \ell(t_j)| &\geq \max_j |r(t_j) - (\ell - a)(t_j)| - \max_j |h(t_j) - a(t_j)| \\ &\geq \left(\frac{\kappa_{\text{br}}}{16} - c_{\text{br}} \right) \varepsilon \geq c_{\text{br}}\varepsilon. \end{aligned}$$

Thus the fixed triple witnesses that B is broken for $h + r$. $\square$

We now verify the conditional bounds in Lemma 4.4.5. Fix any complete history H_s for which $B_s^{\text{cap}} < T$. Then $B_s^{\text{orig}} = B_s^{\text{cap}} < T$, so all but a sufficiently small constant fraction of the bins are genuinely unbroken. Conditioned on H_s , these bins and their internal breakpoint configurations are fixed, whereas the next neuron is independent of the history. Its breakpoint location and slope jump have a joint density that is continuous and strictly positive on the compact range obtained by restricting the jump magnitude, for example, to $[\kappa_{\text{br}}, \kappa_{\text{br}} + 1]$. Hence, with a probability $p_0 > 0$

uniform over H_s , its breakpoint lies in the central half of an unbroken bin and its slope jump has magnitude at least κ_{br} . By Lemma 4.4.6, this bin becomes broken. The new neuron is affine on every other bin, so their broken or unbroken status is unchanged. This proves Equation (4.4) with $p = p_0$. We may decrease p_0 if necessary so that $p_0 \leq 1/3$.

Regarding deaths, let q_0 be the uniform upper bound obtained by considering T broken bins and making the favorable assumption that every new breakpoint landing in one of them repairs that bin. Their total length is at most $T\varepsilon$. Since the breakpoint-location density is bounded above on I_R , choosing C_{cap} sufficiently large gives $q_0 < p_0/2$.

If $B_s^{\text{cap}} < T$, then $B_s^{\text{orig}} = B_s^{\text{cap}} < T$, so there are fewer than T genuinely broken bins and the probability of a decrease is at most q_0 . If $B_s^{\text{cap}} = T$, the capped count can decrease only when $B_s^{\text{orig}} = T$, in which case the same bound applies. If $B_s^{\text{orig}} > T$, repairing one broken bin still leaves at least T broken bins, so the capped count remains equal to T . Thus Equation (4.5) holds with $q = q_0$. In particular, $p + q < 1$, so the hypotheses of Lemma 4.4.5 hold.

The resulting birth–death process is coupled below the capped process and is therefore more favorable to successful approximation. It thus provides an upper bound on the probability that neuron pruning yields a network g_S that ε -approximates the target ReLU.

We can now finally bound the probability $\Pr(B_k^{\text{bd}} = 0)$ of such a (p, q, T) -chain to be in state 0 after k steps, which by Lemma 4.4.5 gives us an upper bound on the probability of a successful ε -approximation.

A crucial part of the proof is now to establish the following bound using the law of total probability.

Lemma 4.4.7. *It holds $\Pr(B_k^{\text{bd}} = 0) \leq e^{-\Omega(\min(k, T))}$.*

Proof.

Let R_k and L_k denote the number of right moves and left moves, respectively, during the k steps of the process. A right move happens when a new broken bin is introduced, and a left move when it is removed. Then $B_k^{\text{bd}} = 1 + R_k - L_k$, which means that in order to have $B_k^{\text{bd}} = 0$ we must have $L_k > R_k$. By the law of total probability, we thus have

$$\begin{aligned}
 \Pr(B_k^{\text{bd}} = 0) &= \Pr(1 + R_k = L_k) \\
 &\leq \Pr(L_k > R_k) \\
 &= \Pr(L_k > R_k \mid R_k < pk/2) \Pr(R_k < pk/2) \\
 &\quad + \Pr(L_k > R_k \mid R_k \geq pk/2) \Pr(R_k \geq pk/2) \\
 &\leq \Pr(R_k < pk/2) + \Pr(L_k > R_k \mid R_k \geq pk/2) \\
 &\leq \Pr(R_k < pk/2) + \Pr(L_k \geq pk/2 \mid R_k \geq pk/2) \\
 &\leq \Pr(R_k < pk/2) + \Pr(L_k \geq pk/2), \tag{4.9}
 \end{aligned}$$

where, in the last inequality, we used that conditioning on performing at least a certain number of right moves can only decrease the probability of performing at least a given number of left moves, since those moves cannot be to the left. More formally, if we define $h(r) := \Pr(L_k \geq pk/2 \mid R_k = r)$, then

$$\Pr\left(L_k \geq \frac{pk}{2}\right) = \sum_{r=0}^k h(r) \Pr(R_k = r) = \mathbb{E}[h(R_k)],$$

and, similarly,

$$\Pr\left(L_k \geq \frac{pk}{2} \mid R_k \geq \frac{pk}{2}\right) = \mathbb{E}\left[h(R_k) \mid R_k \geq \frac{pk}{2}\right].$$

Since $h(r)$ is decreasing in r , it follows that

$$\mathbb{E}\left[h(R_k) \mid R_k \geq \frac{pk}{2}\right] \leq \mathbb{E}[h(R_k)],$$

which justifies the last step in Equation (4.9).

Intuitively, Equation (4.9) states that the probability that the process ends in state $B_k^{\text{bd}} = 0$ can be bounded by the event that the number of right moves is too small, or that the number of left moves is at least the number of right moves.

Case 1: $k \leq 2T/3$. Since $k \leq 2T/3$, the cap at T of the birth-death process is never reached. Each step produces a right move independently with probability p , so $\mathbb{E}[R_k] = pk$, and a Chernoff bound gives $\Pr(R_k < pk/2) \leq e^{-\Omega(k)}$. On the other hand, each step produces a left move with probability at most the constant $q \leq p/2$. Hence $\mathbb{E}[L_k] \leq qk$. Applying a Chernoff bound to L_k , we obtain $\Pr(L_k \geq \frac{pk}{2}) \leq e^{-\Omega(k)}$, and therefore

$$\Pr(R_k < pk/2) + \Pr(L_k \geq pk/2) \leq e^{-\Omega(k)}. \quad (4.10)$$

Case 2: $k > 2T/3$. At step $\hat{k} = k - 2T/3$, there are two possibilities: either $B_{\hat{k}}^{\text{bd}} \leq T/3$ or $B_{\hat{k}}^{\text{bd}} > T/3$. If $B_{\hat{k}}^{\text{bd}} \leq T/3$, then we can use the same argument as in Case 1 to bound the probability of success, as the cap at T will not be reached during the remaining $2T/3$ steps. If $B_{\hat{k}}^{\text{bd}} > T/3$, then reaching state 0 in the remaining $2T/3$ steps requires at least $T/3$ left moves, an event with probability at most $e^{-\Omega(T)}$. Writing $\Delta L = L_k - L_{\hat{k}}$ and $\Delta R = R_k - R_{\hat{k}}$, we have

$$\Pr(B_k^{\text{bd}} = 0) \leq \Pr(\Delta L \geq \Delta R \mid B_{\hat{k}}^{\text{bd}} \leq T/3) + \Pr(\Delta L > T/3 \mid B_{\hat{k}}^{\text{bd}} > T/3). \quad (4.11)$$

The first term is bounded by $e^{-\Omega(T)}$ using the argument of Case 1; starting from a position larger than 1 only makes reaching 0 less likely. The second term is bounded by a standard Chernoff bound.

Putting both cases together gives

$$\Pr(B_k^{\text{bd}} = 0) \leq e^{-\Omega(\min(k, T))}. \quad \square$$

With Lemma 4.4.7, we now have all the ingredients to establish an exponential upper bound for p_k . Combining Lemma 4.4.3, Lemma 4.4.4, Lemma 4.4.5, Lemma 4.4.7 yields the following.

Lemma 4.4.8. *It holds $p_k \leq e^{-\Omega(\min(k, T)d)}$.*

Lemma 4.4.8 gives the desired exponentially small upper bound on the probability of successful ε -approximation, as shown in the next section.

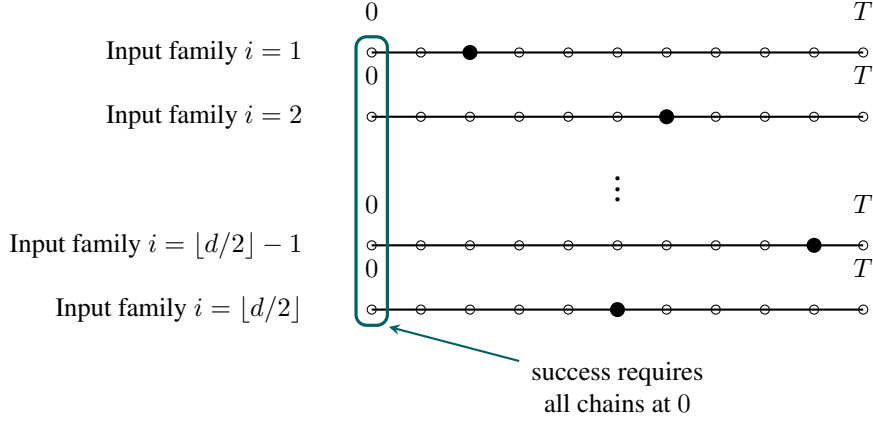


Figure 4.3 – Stacked state-line representation of the birth–death chain $(B_s^{\text{bd}})_{s \geq 0}$. Each horizontal line is the state space $\{0, 1, \dots, T\}$ for one input family, and the filled dot marks the current value of B_s^{bd} . A necessary condition for ε -approximation is that all $\lfloor d/2 \rfloor$ independent chains reach state 0.

4.4.6 Back to the union bound

We now conclude the proof by returning to the union bound of Equation (4.1). We split the sum in two, depending on the value of k :

$$\sum_{k=1}^{N_h} \binom{N_h}{k} p_k = \sum_{k=1}^{T-1} \binom{N_h}{k} p_k + \sum_{k=T}^{N_h} \binom{N_h}{k} p_k \quad (4.12)$$

From Lemma 4.4.8, we can prove the following.

Lemma 4.4.9. *It holds $\sum_{k=1}^{T-1} \binom{N_h}{k} p_k \leq e^{-\Omega(d)}$ and $\sum_{k=T}^{N_h} \binom{N_h}{k} p_k \leq e^{-\Omega(d)}$.*

Proof.

For $k \leq T$, from Lemma 4.4.8 we get $p_k \leq e^{-c_4 k d}$ for some constant $c_4 > 0$. Using the bound $\binom{N_h}{k} \leq (e N_h / k)^k$, we obtain

$$\sum_{k=1}^{T-1} \binom{N_h}{k} p_k \leq \sum_{k=1}^{T-1} e^{-k(c_4 d - \log N_h + O(1))} \leq \sum_{k=1}^{T-1} e^{-\Omega(kd)} \leq e^{-\Omega(d)}, \quad (4.13)$$

where we used $\log N_h \leq cd$ and bounded the resulting geometric series.

For $k > T$, Lemma 4.4.8 and the choice of T in Equation (4.6) give $p_k \leq e^{-c_5 d/\varepsilon}$ for some constant $c_5 > 0$. Using $\sum_{k > k_0} \binom{N_h}{k} \leq 2^{N_h}$, we obtain

$$\sum_{k=T}^{N_h} \binom{N_h}{k} p_k \leq 2^{N_h} e^{-c_5 d/\varepsilon} \leq e^{-\Omega(d/\varepsilon)} \leq e^{-\Omega(d)}, \quad (4.14)$$

where the penultimate inequality uses the hypothesis $N_h \leq cd/\varepsilon$. $\square$

From Lemma 4.4.9, we get that Equation (4.12) is at most $e^{-\Omega(d)}$, proving that with probability at least $1 - e^{-\Omega(d)}$, no neuron-pruned subnetwork ε -approximates the target ReLU uniformly on the ball of radius R . This completes the proof of Theorem 4.3.1.

4.5 Conclusion and Future Work

This chapter has shown that neuron pruning is fundamentally weaker than weight pruning within the Strong Lottery Ticket Hypothesis. Whereas weight pruning can approximate a target ReLU neuron with logarithmic dependence on $1/\varepsilon$, neuron pruning requires $\Omega(1/\varepsilon)$ hidden neurons, even in the clean bias-free setting. Thus, the structured sparsity provided by neuron pruning comes with an intrinsic approximation-theoretic cost, revealing a tension between hardware-friendly pruning patterns and the overparameterization guarantees underlying the SLTH.

The optimal dependence on the input dimension d remains open. Theorem 4.3.1 gives a width lower bound of order $\min\{d/\varepsilon, 2^{cd}\}$ for some constant $c > 0$, while results for random-feature models suggest that exponentially many neurons in d may be necessary. In Appendix B.2, we prove such an exponential lower bound when only one neuron may be retained, suggesting that this trivial case may already be asymptotically optimal in its dependence on d . Proving an exponential lower bound when an arbitrary number of neurons may be retained, and extending the analysis to deeper architectures or activation functions beyond ReLU, are natural directions for future work.

Scheduling Compressible Machine-Learning Tasks

CHAPTER 5

Foundations for Scheduling Compressible Tasks

This chapter introduces the ideas needed for the two scheduling studies. Both consider inference tasks that can receive different amounts of computation: more work generally improves prediction accuracy, but consumes more time and resources. The scheduler must therefore decide where each task runs and how much work it receives. We explain how to model these choices and why a simpler problem that permits splitting a task across machines helps construct and evaluate feasible schedules. We also review the related work relevant to both contributions.

5.1 Background	65
5.1.1 Scheduling compressible inference tasks	65
5.1.2 From inference-time configurations to accuracy functions	65
5.1.3 Optimization as a language for scheduling	66
5.1.4 Continuous, integer, and mixed-integer optimization	67
5.1.5 Fractional relaxations and upper bounds	67
5.1.6 Rounding and approximation guarantees	68
5.1.7 Lagrangian multipliers and KKT conditions	69
5.2 Related Work	69
5.2.1 Model compression and inference-time configuration	70
5.2.2 Placement and scheduling of inference requests	70
5.2.3 Scheduling with controllable processing times	71
5.2.4 Energy-aware inference and scheduling	71
5.2.5 Communication- and deadline-aware edge scheduling	72
5.2.6 Optimization and approximation in inference scheduling	72
5.2.7 Positioning of the two contributions	73

5.1 Background

5.1.1 Scheduling compressible inference tasks

An inference service receives requests for machine-learning predictions. Throughout this part, we call each such request a *task*; *job* is the corresponding term commonly used in scheduling theory. The scheduler decides how to allocate computational resources to these tasks. In a conventional scheduling problem, each task has a fixed processing requirement, and the scheduler primarily decides where and when that fixed amount of work is performed.

The setting considered in this thesis introduces a further decision. A *compressible* machine-learning inference task can be executed at different levels of computational effort by selecting different model configurations. In the contribution chapters, a task is called *fully compressible* when its workload is modeled as continuously adjustable. A configuration, or compression level, is the concrete model choice that realizes a particular workload. A larger workload generally produces a better prediction, whereas a smaller workload reduces the resources required to obtain a result. The scheduler must therefore decide not only which machine receives a task, but also how much computation that task receives.

For example, suppose there is insufficient capacity to run every task at its largest configuration. If each task has a fixed workload, some tasks may have to be left unprocessed. With compressible tasks, the scheduler can instead allocate less computation to several tasks, allowing more of them to be served at lower prediction accuracy. This illustrates why task placement and workload allocation must be considered together.

5.1.2 From inference-time configurations to accuracy functions

Once-For-All networks are an important motivation for both scheduling contributions. A single network is trained so that subnetworks with different depths, widths, kernel sizes, or input resolutions can subsequently be selected without retraining the model for each deployment condition. The Once-For-All paper reports more than 10^{19} possible subnetworks (Cai et al., 2020), each with an associated computational workload and prediction accuracy. This vast but discrete choice motivates the continuous workload–accuracy abstraction used in the fully compressible scheduling model of Barros et al. (2024).

The scheduler does not train the network or construct its subnetworks. Instead, it is given the relationship between computational workload and prediction accuracy for each task, and uses it to decide how much computation to allocate. For each task j , let f_j denote its executed workload and let $\bar{f}_j$ denote its maximum useful workload. The allocation decision satisfies

$$0 \leq f_j \leq \bar{f}_j. \quad (5.1)$$

The utility obtained from this workload is prediction accuracy in both contributions. We therefore represent it by an *accuracy function*

$$a_j : [0, \bar{f}_j] \longrightarrow \mathbb{R}_+, \quad f_j \longmapsto a_j(f_j). \quad (5.2)$$

Thus, *accuracy* names the particular objective studied in this thesis, while *utility* refers to the more general abstraction. The same model applies to any nonnegative utility function that depends on allocated computation. At zero allocated workload, the function gives $a_j(0)$. If this value is positive, it represents baseline utility available without the workload modeled here, for example from

a random prediction. If an unprocessed task yields no useful output, the appropriate assumption is $a_j(0) = 0$.

The function a_j is assumed to be nondecreasing: assigning additional useful computation should not reduce the task's prediction accuracy. It is also modeled as concave, expressing diminishing returns. Early units of computation provide a larger marginal gain than later ones. Concavity therefore captures a basic allocation principle: under resource scarcity, distributing computation among several tasks can be more valuable than fully executing one while leaving another unserved.

For the scheduling algorithms, we represent this accuracy function by straight-line segments, using measurements or estimates for selected configurations to construct a piecewise-linear approximation.

5.1.3 Optimization as a language for scheduling

An optimization problem consists of three ingredients: *decision variables*, an *objective*, and *constraints*. Decision variables encode the choices available to the scheduler. The objective assigns a numerical value to those choices. Constraints exclude allocations that the platform cannot implement. A choice satisfying every constraint is a *feasible solution*; an *optimal solution* is a feasible solution with the best objective value.

Let J be the set of tasks and M the set of machines. The shared objective of the two scheduling contributions is to maximize aggregate accuracy,

$$\max \sum_{j \in J} a_j(f_j). \quad (5.3)$$

The formulation must connect each workload f_j with a placement decision. Introduce a binary variable

$$x_{j\mu} \in \{0, 1\}, \quad x_{j\mu} = 1 \iff \text{task } j \text{ is assigned to machine } \mu, \quad (5.4)$$

and a continuous variable $y_{j\mu} \geq 0$ for the workload of task j executed on machine μ . The generic assignment constraints are

$$f_j = \sum_{\mu \in M} y_{j\mu}, \quad (5.5)$$

$$0 \leq y_{j\mu} \leq \bar{f}_j x_{j\mu}, \quad (5.6)$$

$$\sum_{\mu \in M} x_{j\mu} \leq 1. \quad (5.7)$$

Together, these constraints state that an executed task receives a bounded workload on at most one machine. They describe the common placement structure without prescribing how machine capacity is represented. For example, if one machine can perform at most ten units of work before a deadline, assigning two six-unit tasks to it satisfies the placement constraints but exceeds its capacity. A feasible schedule must move one task to another machine or reduce the tasks' workloads. The remaining feasibility constraints depend on the scheduling environment and are introduced only in the corresponding contribution chapters.

At the shared level, both problems use the same class of concave workload–accuracy objectives, require an implementable single-machine placement, and seek to allocate limited computation. Their differences lie in what makes an allocation feasible and in how the fractional model is rounded, not in the overall optimization pipeline.

5.1.4 Continuous, integer, and mixed-integer optimization

The model combines two types of decisions: how much computation to allocate to a task, and which machine receives it. The workloads f_j and $y_{j\mu}$ are continuous variables: they can take any value in an interval. Placement is an integer decision represented here by the binary variable $x_{j\mu}$. It must be either 0 or 1, because in an implementable schedule a task is assigned to machine μ or it is not.

A *linear program* (LP) has continuous variables, a linear objective, and linear constraints, and can be solved in polynomial time. More generally, maximizing a concave objective over a convex feasible region is a convex optimization problem (Boyd and Vandenberghe, 2004). Under standard representation and accuracy assumptions, such problems admit efficient solution methods. This theoretical tractability is complemented by mature solvers that handle large practical instances effectively.

A model containing both integer and continuous variables is a *mixed-integer program*. The binary placement variables make its feasible set nonconvex: averaging two solutions that place a task on different machines produces a fractional placement rather than another implementable one. Mixed-integer programs are NP-hard in general. Exact solvers can be effective on moderate instances, but they do not provide a polynomial-time solution method for arbitrary instances.

Earlier, we modeled each task's workload–accuracy trade-off by a concave piecewise-linear function. This form lets us express accuracy in the optimization problem using linear constraints. Imagine extending each straight segment of a concave curve across the full workload interval. At every workload, the curve follows the lowest of these extended lines. Writing L_j for the segments of task j , this gives

$$a_j(f) = \min_{\ell \in L_j} \{\alpha_{j\ell} f + \beta_{j\ell}\}. \quad (5.8)$$

Introducing a variable z_j for the achieved accuracy of task j and imposing

$$z_j \leq \alpha_{j\ell} f_j + \beta_{j\ell}, \quad \ell \in L_j, \quad (5.9)$$

places z_j below every affine segment. Maximizing $\sum_j z_j$ raises it to $a_j(f_j)$. Consequently, when the platform constraints are linear, the exact formulation is a *mixed-integer linear program* (MILP).

5.1.5 Fractional relaxations and upper bounds

A *relaxation* removes or weakens a requirement of an optimization problem. It enlarges the feasible set and normally produces a problem that is easier to solve. The shared conceptual step in both contributions is to relax the requirement that each task use a single machine. In the generic formulation above, this can be illustrated by replacing

$$x_{j\mu} \in \{0, 1\} \quad \text{with} \quad 0 \leq x_{j\mu} \leq 1. \quad (5.10)$$

For example, if $\bar{f}_j = 10$, the relaxed values $x_{j1} = x_{j2} = 1/2$ and $y_{j1} = y_{j2} = 5$ satisfy the generic assignment constraints: the task's ten units of work are split equally between two machines. An integral placement cannot do this.

Chapter 6 expresses this idea through fractional allocations, whereas Chapter 7 removes the assignment variables and their linking constraints to obtain an assignment-free relaxation. In either case, a task's workload may be distributed among machines. For the piecewise-linear accuracy functions used here, removing the integral placement restriction leaves an LP when the remaining

constraints are linear. Its fractional solution may not itself be executable, but it can be computed more efficiently and used to guide the construction of a feasible, single-machine schedule.

The relaxation also provides an upper bound on what any feasible schedule could achieve. Let OPT denote the optimum of the exact maximization problem and OPT_f the optimum of its fractional relaxation. Every integral solution remains feasible after relaxation, while the converse need not hold. Therefore

$$OPT \leq OPT_f. \quad (5.11)$$

The fractional optimum is therefore an optimistic benchmark: it may be unattainable by a real schedule, but no feasible integral schedule can have a larger objective value. The next subsection explains how rounding uses the fractional allocation to choose one machine for each task, potentially reducing workloads to recover feasibility, and how the upper bound helps assess the resulting schedule.

5.1.6 Rounding and approximation guarantees

A *rounding algorithm* converts a fractional solution into a feasible integral one. In the present setting, it must resolve fractional placements so that every executed task ultimately uses a single machine. This conversion may require reducing some workloads to restore the feasibility constraints of the original problem.

Let SOL be the objective value of the schedule returned by the rounding algorithm. Since that schedule is feasible for the exact problem,

$$SOL \leq OPT \leq OPT_f. \quad (5.12)$$

An *approximation algorithm* is an efficient algorithm accompanied by a worst-case guarantee on how its output compares with the optimum. The fractional upper bound is useful because a guarantee relative to OPT_f immediately implies one relative to the unknown OPT . For example, if the rounding analysis proves

$$SOL \geq \rho OPT_f, \quad 0 < \rho \leq 1, \quad (5.13)$$

then Equation (5.11) gives $SOL \geq \rho OPT$. Similarly, an additive bound $OPT_f - SOL \leq G$ implies $OPT - SOL \leq G$.

Concavity allows a guarantee about retained workload to be translated into a guarantee about retained accuracy. Suppose that $a_j(0) = 0$ and that rounding reduces task j 's workload from f_j to λf_j , where $\lambda \in [0, 1]$. Concavity gives

$$a_j(\lambda f_j) \geq \lambda a_j(f_j). \quad (5.14)$$

Geometrically, the graph of a_j lies above the line segment joining $(0, 0)$ and $(f_j, a_j(f_j))$. Consequently, retaining a fraction λ of the workload preserves at least the same fraction of the accuracy. Equivalently, if rounding discards at most a fraction $1 - \lambda$ of the workload, it loses at most that fraction of the original accuracy. When $a_j(0) > 0$, the same statement applies to the accuracy gained relative to the baseline $a_j(0)$.

The full optimization pipeline shared by the two contributions can therefore be summarized as

$$\begin{array}{ccc} \text{exact mixed-integer model} & \xrightarrow{\text{relax placement}} & \text{fractional continuous model} \\ & \searrow \text{round and restore feasibility} & \\ & \text{implementable approximate schedule.} & \end{array} \quad (5.15)$$

The contribution chapters instantiate this pipeline with different feasibility constraints and rounding arguments. Those distinctions are introduced locally, where they are needed; the conceptual roles of the exact model, relaxation, fractional upper bound, and performance analysis remain the same in both.

5.1.7 Lagrangian multipliers and KKT conditions

The Karush–Kuhn–Tucker (KKT) conditions are a set of optimality conditions for constrained continuous problems. Chapter 6 uses them to characterize an optimal solution of its fractional scheduling problem. These conditions apply to the continuous relaxation, not to the original problem with binary machine assignments.

The fractional LP in Chapter 6 minimizes negative aggregate accuracy; this selects the same solution as maximizing aggregate accuracy, the objective used so far. To explain the Lagrangian, consider an LP with continuous decision variables collected in $\mathbf{u}$, an objective $F(\mathbf{u})$ to minimize, and constraints $g_i(\mathbf{u}) \leq 0$ for $i = 1, \dots, p$, with all terms moved to the left-hand side. The functions F and g_i are affine. We associate a nonnegative number η_i with each constraint and form the *Lagrangian*

$$L(\mathbf{u}, \boldsymbol{\eta}) = F(\mathbf{u}) + \sum_{i=1}^p \eta_i g_i(\mathbf{u}), \quad \eta_i \geq 0. \quad (5.16)$$

The Lagrangian combines the objective with a weighted sum of the constraint expressions. The weights η_i are called *dual variables* or *Lagrangian multipliers*. They are distinct from the fractional allocation variables introduced by relaxing the machine-assignment requirement.

The KKT conditions combine four requirements at an optimum $(\mathbf{u}^*, \boldsymbol{\eta}^*)$:

$$\begin{aligned} g_i(\mathbf{u}^*) &\leq 0, & \eta_i^* &\geq 0 \quad (i = 1, \dots, p), \\ \eta_i^* g_i(\mathbf{u}^*) &= 0 \quad (i = 1, \dots, p), \\ \nabla_{\mathbf{u}} L(\mathbf{u}^*, \boldsymbol{\eta}^*) &= \mathbf{0}. \end{aligned} \quad (5.17)$$

The first line requires the LP constraints to be satisfied and the multipliers to be nonnegative. The second is *complementary slackness*: if a constraint has room to spare, its multiplier must be zero; a tight constraint may have a positive multiplier. The last line is *stationarity*: the gradient of the Lagrangian with respect to the decision variables vanishes. Here any nonnegativity bounds on variables are included among the constraints $g_i \leq 0$, so stationarity takes this form. For a feasible LP with a finite optimum, these conditions characterize optimality (Boyd and Vandenberghe, 2004).

In Chapter 6, multipliers are attached to the accuracy-segment, deadline, maximum-workload, energy-budget, and nonnegativity constraints of the fractional LP. For example, if the energy budget is not fully used, complementary slackness makes its multiplier zero. The chapter uses these conditions to derive properties of an optimal fractional schedule and in its algorithmic analysis. Chapter 7 also solves an LP relaxation, but its analysis does not use KKT conditions.

5.2 Related Work

The two scheduling contributions connect model compression with resource scheduling. Compressible models offer different workload–accuracy choices, while inference-serving systems study how tasks are assigned and executed. Scheduling with controllable processing times

treats the amount of work itself as a decision, and approximation algorithms help handle the resulting machine-assignment choices. We first review these connections, then the energy- and communication-aware work most closely related to the two contributions.

5.2.1 Model compression and inference-time configuration

Model compression aims to reduce the computational, memory, or communication cost of neural-network inference. The literature includes pruning, quantization, knowledge distillation, and architecture-search techniques; [Cheng et al. \(2018\)](#) provide a broad survey of this area. Many approaches produce one fixed model after training or adaptation. Such a model does not give a scheduler a choice of computational workload for each inference task.

Slimmable and once-for-all networks instead train shared weights that support multiple sub-networks. Universally Slimmable Networks execute at different widths within a single trained model ([Yu and T. S. Huang, 2019](#)), while AutoSlim searches over channel counts to meet a resource constraint ([Yu and T. Huang, 2019](#)). Once-For-All extends this idea to a large family of subnetworks varying in depth, width, kernel size, and input resolution, and specializes the trained network for a target deployment scenario ([Cai et al., 2020](#)). Murmuration selects model configurations and partitioning strategies online as edge conditions and service-level objectives change ([Lin et al., 2024](#)).

These works show how one trained model or adaptable framework can offer several choices of accuracy and computational cost. They focus on producing or selecting configurations. The scheduling contributions in this thesis take workload–accuracy profiles as input and study how to allocate limited computation and place multiple tasks on machines.

5.2.2 Placement and scheduling of inference requests

Classical scheduling studies how jobs share one or more machines under constraints such as release times and deadlines. Its foundations include the real-time scheduling analysis of [C. L. Liu and Layland \(1973\)](#) and broader treatments of complexity and approximation ([B. Chen et al., 1998](#)). Inference-serving systems may also need to account for heterogeneous hardware, distributed execution, prediction quality, and changing demand. [H. Ma et al. \(2023\)](#), for example, study online inference scheduling under uncertain edge-server availability.

Several systems focus on distributing inference or analytics requests across devices, edge nodes, and cloud resources. Examples include large-scale approximate video analytics ([H. Zhang et al., 2017](#)), inference delivery networks with allocation guarantees ([Salem et al., 2021](#)), online provisioning of edge inference services ([Jin et al., 2020](#)), and mobile deep inference supported by edge computing ([Ogden and Guo, 2018](#)). They address placement and resource management in distributed inference. The contributions here study the specific joint choice of each task’s workload and execution machine under scheduling constraints.

A closer line of work jointly considers inference quality and resource use. Scylla selects DNN and hardware configurations for quality-aware mobile vision ([Jiang et al., 2020](#)), whereas NestDNN serves multiple applications by choosing among resource–accuracy trade-offs offered by nested models ([Fang et al., 2018](#)). Jellyfish maps requests to models in dynamic edge networks while accounting for end-to-end latency and accuracy ([Nigade et al., 2022](#)). [She et al. \(2023\)](#) study on-demand edge inference with accuracy and deadline guarantees and select among early-exit configurations.

These systems generally select from a finite menu of models, exits, or profiled configurations. The fully compressible model used here instead treats workload as continuously adjustable. This lets us analyze workload allocation and machine placement together, including the loss incurred when a fractional allocation is rounded to a feasible schedule. As discussed in the background section, a system with discrete configurations would also have to realize the selected workloads using its available variants.

5.2.3 Scheduling with controllable processing times

The idea that a job’s processing requirement can be chosen is not unique to machine learning. In scheduling theory it appears under terms such as *controllable*, *compressible*, or *resource-dependent* processing times. [Vickson \(1980\)](#) studies single-machine sequencing with controllable processing times, while [Alidaee and Ahmadian \(1993\)](#) consider related decisions on parallel machines. [Shabtay and Kaspi \(2006\)](#) study parallel-machine scheduling in which processing times are coupled to a convex resource-consumption function. Online assignment with controllable processing times is considered by [R. Wu et al. \(2023\)](#).

Energy-aware variants also jointly select processing levels and machine assignments. [X. Wu and Che \(2019\)](#), for example, use a memetic differential-evolution method for energy-efficient parallel-machine scheduling. In such formulations, controllability frequently represents expediting, resource investment, or a finite set of processing modes. The machine-learning setting gives the decision a different semantic meaning: reducing processing corresponds to selecting a smaller inference configuration, and the value of the retained computation is measured through prediction accuracy.

[Barros et al. \(2024\)](#) connect these two areas directly by studying fully compressible inference tasks on heterogeneous machines. Their model demonstrates how an adjustable neural network can be viewed as a scheduling job with continuously variable processing. The contributions developed in the following chapters share this perspective while introducing distinct platform constraints and new approximation analyses.

5.2.4 Energy-aware inference and scheduling

Energy-aware scheduling has been studied at several levels of the computing stack. In mobile edge computing, [Xu et al. \(2020\)](#) optimize DNN-inference offloading to reduce energy consumption. At the accelerator level, [Yidi Wang et al. \(2021\)](#) balance energy efficiency and real-time performance when spatially sharing a GPU. Energy-aware resource scheduling has also been investigated for serverless edge platforms ([Aslanpour et al., 2022](#)). These studies show that placement and execution policy affect inference energy. The question in this thesis is how to allocate a variable workload to each task while respecting a shared energy budget.

A related scheduling literature couples energy with controllable processing times. [Yin et al. \(2016\)](#) study tardiness and energy consumption on a single machine using discrete compression levels, while [Tsao et al. \(2020\)](#) optimize controllable processing times under time-varying electricity prices. The parallel-machine method of [X. Wu and Che \(2019\)](#), discussed above, jointly selects processing modes and assignments through an evolutionary metaheuristic. These studies treat processing level as an energy decision. In the present setting, reducing computation also changes prediction accuracy, and the analysis compares a rounded schedule with a fractional upper bound.

Chapter 6 brings this energy-aware perspective to fully compressible inference tasks. It combines heterogeneous machine speeds and energy efficiencies, task deadlines, a global energy budget, and continuously adjustable workloads, and studies both the fractional optimum and a rounded schedule with a provable guarantee.

5.2.5 Communication- and deadline-aware edge scheduling

Distributed scheduling must account not only for computation but also for when and where data are available. Network-aware data-center scheduling explicitly models the interaction between computation and network tasks (Giroire et al., 2019). In cloud workflow scheduling, J. Liu et al. (2019) consider task-specific release times and uncertain execution durations, while X. Ma et al. (2021) schedule deadline-constrained workflows across rented cloud resources. C. Zhang et al. (2020) study online dispatching and scheduling with heterogeneous job utilities in edge environments.

Preemption and communication constraints further shape feasible schedules. Dedas jointly dispatches tasks and schedules network bandwidth and server processing to meet deadlines (Meng et al., 2019). N. Wang et al. (2022) analyze preemptive scheduling for distributed machine-learning jobs in edge–cloud networks. These works focus on dispatching, communication, and timing. The second contribution in this thesis also chooses each task’s workload, with prediction accuracy depending on that choice.

Within inference serving, systems such as Jellyfish (Nigade et al., 2022), on-demand early-exit scheduling (She et al., 2023), and Infer-Edge (Mounesan et al., 2025) combine latency requirements with choices about model configuration or early exits. Chapter 7 treats workload as continuous and makes the communication delay depend on the selected destination. Data sent to a distant machine may arrive later, leaving less time to process the task before its deadline. Each destination therefore gives the task a different execution window; rounding must assign it to one machine and keep its processing inside that window.

5.2.6 Optimization and approximation in inference scheduling

Approximation guarantees complement empirical evaluations of inference schedulers by bounding performance in the worst case. VideoEdge uses an approximation approach to choose processing plans and placements for hierarchical video analytics (Hung et al., 2018). The inference-delivery formulation of Salem et al. (2021) provides optimality guarantees for distributing models across a network. Fresa and Champati (2022) maximize inference accuracy between an edge device and an edge server using an LP relaxation followed by rounding; their guarantee bounds both the achieved accuracy and the resulting makespan. She et al. (2023) derive a greedy approximation method for deadline-constrained early-exit selection.

The LP-relaxation example illustrates how a fractional solution can guide an algorithm and provide a benchmark for its analysis. The two contributions in this thesis follow that broad approach while combining continuously adjustable workloads with single-machine placement. Their feasibility constraints and rounding arguments differ, but both use the fractional optimum to assess the quality of the schedule they construct.

5.2.7 Positioning of the two contributions

The first contribution, presented in Chapter 6, allocates variable task workloads across heterogeneous machines under deadlines and a global energy budget. The second, presented in Chapter 7, studies tasks arriving over time with end-to-end deadlines, destination-dependent communication delays, and preemptive execution. Both solve a fractional relaxation and then round its solution. The first rounding procedure must respect task deadlines and the global energy budget; the second must keep each task's processing within its destination-dependent execution window.

CHAPTER 6

Energy-Aware Scheduling of Compressible Inference Tasks

This chapter studies energy-aware scheduling of compressible inference tasks on machines with different speeds and energy efficiencies. For each task, the scheduler chooses a machine and a computational workload, trading prediction accuracy against processing time and energy consumption while respecting task deadlines and a global energy budget. We formulate the resulting DEADLINE SCHEDULING WITH COMPRESSIBLE TASKS - ENERGY AWARE (DSCT-EA) problem as a mixed-integer program. A fractional relaxation gives an efficiently computable accuracy upper bound and guides an approximation algorithm. Experiments with image-classification workloads compare this schedule with fractional and integral benchmarks and with scheduling baselines using fixed or discrete compression levels.

6.1	Introduction	77
6.2	Problem Formulation	78
6.2.1	Accuracy function	80
6.2.2	Fractional Relaxation	80
6.3	Optimal algorithm for DSCT-EA-FR	84
6.3.1	Algorithm for DSCT-EA-FR, one machine	84
6.3.2	Algorithm for DSCT-EA-FR, multiple machines	85
6.4	Approximation algorithm for DSCT-EA	89
6.5	Results	90
6.6	Conclusion	94

6.1 Introduction

In recent years, there has been a growing usage of Machine Learning (ML) models in cloud computing, contributing to the adoption of Machine Learning as a Service (MLaaS) (Ribeiro et al., 2015), studied in several applications such as image recognition and self-driving cars (Raj et al., 2023).

Cloud and network operators have faced challenges in developing efficient strategies for utilizing computational resources to support machine learning tasks. Among these challenges, scheduling is an important one. A scheduler must determine on which machine each task is executed and its processing order. This becomes especially critical when tasks must adhere to deadline constraints.

Neural-network compression techniques such as pruning (Z. Liu et al., 2018) and quantization (Yang et al., 2019) can reduce inference cost. Many approaches produce a fixed compressed model. The scheduling problem here instead needs a choice among configurations with different computational requirements at inference time, as discussed in Chapter 5.

Configurable networks such as Once-For-All and AutoSlim offer subnetworks of different sizes at inference time (Cai et al., 2020; Yu and T. Huang, 2019). A smaller configuration generally requires fewer floating-point operations (FLOPs) and less processing time on a given machine, but may yield lower prediction accuracy. Processing time also depends on machine speed; end-to-end latency can include waiting and communication in addition to processing. Our earlier work (Da Silva Barros et al., 2024) studied deadline-constrained scheduling of compressible image-classification tasks on heterogeneous machines and gave an approximation algorithm for maximizing average accuracy. Since the task count is fixed, maximizing average accuracy is equivalent to maximizing the aggregate accuracy used here.

Cloud and network operators are compelled to mitigate their cloud carbon footprint, driving researchers and scientists to investigate novel methods for conducting ML inference with greater energy efficiency.

The adoption of MLaaS and the expanding size of neural network models has resulted in increased energy consumption, particularly during the inference stage (McDonald et al., 2022). According to reports from NVIDIA (Freund, 2019), 80-90% of Artificial Intelligence (AI) costs stem from inference. Indeed, processing millions of requests, such as those encountered in social networks, can lead to a significant number of inferences in deep learning models, resulting in elevated energy consumption and a sizable carbon footprint (Desislavov et al., 2023).

Consequently, new energy-aware scheduling solutions are urgently needed, even though the introduction of energy consumption considerations further complicates the scheduling problem. Neural network compression could be used to reduce the energy consumption of a model by reducing its size during inference. Therefore, we propose constraining cloud inference services to operate within an energy budget, which sets a maximum energy threshold for executing a set of inference tasks. The energy consumption of a task is influenced by both its processing time and the specific machine that executes it, given that power consumption can vary across different machines. This chapter extends the deadline-constrained scheduling setting of our previous work by imposing a global energy budget on compressible tasks.

Our objective is to maximize aggregate accuracy while respecting task deadlines and the energy budget. The scheduler chooses a machine and a computational workload for each task; its processing time follows from that workload and the chosen machine's speed. We give a mixed-integer formulation, solve a fractional relaxation, and use its solution to construct an integral

schedule with an additive performance guarantee. We then evaluate the algorithm against fixed-model and discrete-compression baselines.

The contributions of this chapter can be summarized as follows:

- We investigate inference tasks whose workload can be adjusted, trading prediction accuracy against computation and energy consumption under a global energy budget. The formulation also applies to other tasks with diminishing accuracy gains from additional workload.
- We analyze the problem complexity and give (i) an exact algorithm for its fractional relaxation and (ii) an approximation algorithm for the original scheduling problem, with guarantees for machines heterogeneous in speed and energy efficiency.
- Finally, we validate and compare our approach with state-of-the-art solutions.

The common foundations and the broader literature on configurable inference and compressible-task scheduling are reviewed in Chapter 5, particularly Section 5.2. Within this landscape, the present chapter focuses on the combination of heterogeneous machine speeds and energy efficiencies, deadline feasibility, continuously adjustable workloads, and a global energy budget.

The remainder of this chapter begins with the problem formulation in Section 6.2. Sections 6.3 and 6.4 are dedicated to the exact and approximation scheduling algorithms, respectively. In Section 6.5, we assess the algorithm’s performance through experiments. Finally, we delve into the main discussions and conclusions in Section 6.6, alongside potential future works.

6.2 Problem Formulation

In this section, we define the DEADLINE SCHEDULING WITH COMPRESSIBLE TASKS - ENERGY AWARE (DSCT-EA) problem, and formulate it as a Mixed-Integer Program (MIP). Let us consider a set J of n tasks, which can be scheduled on a set M of m machines.

Each machine $r \in M$ is characterized by its speed s_r , quantified in FLOPS (Floating-point Operations per Second), power consumption P_r , measured in Watts (W), and energy efficiency $E_r = s_r/P_r$, computed as the ratio of speed to power consumption. In this regard, the energy efficiency of servers can exhibit significant heterogeneity, largely depending on the hardware architecture specifications. Figure 6.1, derived from the comprehensive evaluation of Desislavov et al. (2023), illustrates the relationship between energy efficiency and speed across various NVIDIA GPUs designed for servers. The general observed trend is that devices exhibit linear improvement in energy efficiency with the advancement of hardware speed. We order the machines by non-decreasing energy efficiency, i.e., $r < r'$ if $E_r < E_{r'}$.

Each task $j \in J$ represents a compressible inference request with maximum useful workload $\bar{f}_j$ floating-point operations and a deadline d_j (measured in seconds). We index tasks by nondecreasing deadlines, so $i < j$ implies $d_i \leq d_j$. An accuracy function $a_j(f)$ specifies the accuracy reached by task j when assigning it workload $0 \leq f \leq \bar{f}_j$. Let $a_j^{\max} = a_j(\bar{f}_j)$ be the maximum accuracy reached by a task. This setting naturally describes scenarios involving slimmable networks, such as OFA and AutoSlim (Cai et al., 2020; Yu and T. Huang, 2019), where the accuracy of a task depends on its level of compression. Further details regarding accuracy functions are provided in Section 6.2.1.

A solution for DSCT-EA specifies, for each task j , (i) the machine on which it is executed and (ii) its computational workload f_j , which determines its compression level.

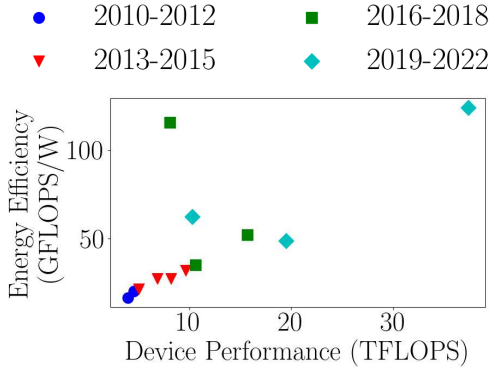


Figure 6.1 – Energy efficiency vs speed for various GPUs (Desislavov et al., 2023).

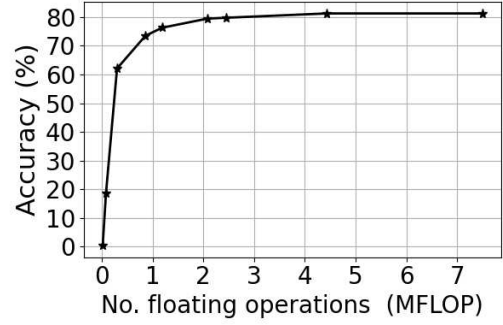


Figure 6.2 – Once-for-All accuracy versus floating-point operations (Da Silva Barros et al., 2024).

We formulate DSCT-EA by minimizing the sum of accuracy losses. Since the number of tasks is fixed, minimizing $\sum_{j \in J} (1 - a_j(f_j)) = n - \sum_{j \in J} a_j(f_j)$ is equivalent to maximizing the aggregate accuracy used in Chapter 5. All optimal-value and approximation bounds below refer to aggregate accuracy, not to this loss objective.

Variables. We have two sets of decision variables. x_{jr} represents a set of binary variables with a value of 1 if task j is processed on machine r . t_{jr} denotes the set of variables indicating the processing time of task j on machine r . The number of operations dedicated to task j is given by $f_j = \sum_{r \in M} s_r \cdot t_{jr}$.

Model Formulation

$$\min \quad \sum_{j=1}^n \left[1 - a_j \left(\sum_{r \in M} s_r \cdot t_{jr} \right) \right] \quad (6.1a)$$

$$\text{s.t.} \quad \sum_{i=1}^j t_{i,r} \leq d_j, \quad j \in J, r \in M \quad (6.1b)$$

$$t_{jr} \cdot s_r \leq \bar{f}_j, \quad j \in J, r \in M \quad (6.1c)$$

$$t_{jr} \leq x_{jr} \cdot d_j, \quad j \in J, r \in M \quad (6.1d)$$

$$\sum_{r \in M} x_{jr} = 1, \quad j \in J \quad (6.1e)$$

$$\sum_{r \in M} \sum_{j \in J} \frac{s_r}{E_r} \cdot t_{jr} \leq B, \quad (6.1f)$$

$$t_{jr} \in \mathbb{R}_{\geq 0}, x_{jr} \in \{0, 1\} \quad j \in J, r \in M \quad (6.1g)$$

Equation (6.1a) minimizes aggregate accuracy loss, equivalently maximizing aggregate accuracy. Constraint (6.1b) ensures that each task finishes by its deadline, accounting for the processing times of earlier tasks on the same machine. Constraint (6.1c) restricts the workload of task j to at most $\bar{f}_j$. Constraints (6.1d) and (6.1e) guarantee that each task is assigned to one machine; its processing time may still be zero. Finally, constraint (6.1f) states that the total energy consumed

by all machines must not exceed the energy budget B for the system. In this model, computation consumes energy only while a machine processes a task.

NP Hardness of the problem. The optimization problem DSCT-EA is NP-hard: it contains the deadline-scheduling problem DSCT (Da Silva Barros et al., 2024) as a special case when the energy budget is nonbinding.

6.2.1 Accuracy function

We consider accuracy functions that stem from slimmable neural networks (Cai et al., 2020; Yu and T. Huang, 2019). Figure 6.2 provides an example of such a function, where the accuracy achieved in a ML inference task depends on the model's size processing it, directly linked to the total number of floating-point operations performed. Such accuracy functions are concave, meaning that the derivative with respect to the number of floating-point operations decreases as the number of operations increases. For a task j where f floating operations have been executed, we define the *marginal gain* as the right-hand derivative of the accuracy function $a'_j^+(f)$, and the *marginal loss* as the left-hand derivative $a'_j^-(f)$. In this work, we will consider piecewise linear functions, where these two derivatives may not always be equal.

Piecewise linear function We consider the special case of linear piecewise functions in the following, as (i) they are a good model for the accuracy function of slimmable models (see Figure 6.2) and (ii) they can be used to approximate any accuracy functions, for a sufficiently large number of pieces. We use the following notation:

$$a(f) = \begin{cases} \alpha_k \cdot f + b_k, & \text{if } p_k \leq f \leq p_{k+1}, \end{cases} \quad \forall k \in \mathcal{K} \quad (6.2)$$

Here, $\mathcal{K} = \{1, \dots, K\}$, where K is the number of segments, and α_k and b_k , for $k \in \mathcal{K}$, represent, respectively, the slope and y -intercept point for a linear segment k . The values of p_k and p_{k+1} , called breakpoints, represent the start and the end point for each linear segment k . We have $a(p_{K+1}) = a^{\max}$, and $a(0) = b_1 = a^{\min}$.

6.2.2 Fractional Relaxation

Here, we introduce a fractional relaxation of the DSCT-EA problem, named DEADLINE SCHEDULING WITH COMPRESSIBLE TASKS - ENERGY AWARE - FRACTIONAL RELAXATION (DSCT-EA-FR), in which a task can be executed on multiple machines. We will utilize a solution to this problem as a starting point for our approximation algorithm for DSCT-EA. We give the formulation for the case in which the accuracy functions are piecewise linear.

Piecewise linear formulation. DSCT-EA-FR could be directly reformulated as a linear program by simply altering the type of the variable x_{jr} in the MIP of Section 6.2 to a real number within the range $[0, 1]$. However, we can further simplify the fractional relaxation by directly omitting

the variables x_{jr} , as shown below:

$$\min \quad \sum_{j=1}^n -z_j \quad (Dual) \quad (6.3a)$$

$$\text{s.t.} \quad z_j \leq \alpha_{jk} \sum_{r \in M} s_r \cdot t_{jr} + b_{jk}, \quad j \in J, k \in \mathcal{K} \quad (\gamma_{jk}) \quad (6.3b)$$

$$\sum_{i=1}^j t_{i,r} \leq d_j, \quad j \in J, r \in M \quad (\mu_{jr}) \quad (6.3c)$$

$$\sum_{r \in M} t_{jr} \cdot s_r \leq \bar{f}_j, \quad j \in J \quad (\lambda_j) \quad (6.3d)$$

$$\sum_{r \in M} \sum_{j \in J} \frac{s_r}{E_r} t_{jr} \leq B, \quad (\beta) \quad (6.3e)$$

$$t_{jr} \in \mathbb{R}_{\geq 0}, \quad j \in J, r \in M \quad (\nu_{jr}) \quad (6.3f)$$

A new variable z_j is introduced for each task $j \in J$. Constraints (6.3b) ensures that it is lower than the value of the accuracy function a_j over each segment $k \in K$ for any values of the execution times $t_{jr}, r \in M$. The minimization makes its equal for an optimal solution.

Karush - Kuhn - Tucker Conditions We first discuss the KKT (Karush-Kuhn-Tucker) conditions for the optimization problem and then, we present an algorithm to solve it with polynomial time complexity.

In our convex problem, the KKT conditions are necessary and sufficient conditions for optimality (Boyd and Vandenberghe, 2004) (Chap. 5 p. 226 and 244). First, we write the Lagrangian of the above formulation:

$$\begin{aligned} L(\mathbf{t}, \mathbf{z}; \boldsymbol{\gamma}, \boldsymbol{\mu}, \boldsymbol{\lambda}, \boldsymbol{\nu}, \beta) = & - \sum_{j \in J} z_j + \boldsymbol{\gamma}^\top \boldsymbol{\ell}(\mathbf{t}, \mathbf{z}) + \boldsymbol{\mu}^\top \mathbf{g}(\mathbf{t}) \\ & + \boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{t}) + \boldsymbol{\nu}^\top \mathbf{i}(\mathbf{t}) + \beta q(\mathbf{t}). \end{aligned}$$

The multiplier vectors are

$$\begin{aligned} \boldsymbol{\gamma} &= (\gamma_{jk})_{j \in J, k \in \mathcal{K}}, \quad \boldsymbol{\mu} = (\mu_{jr})_{j \in J, r \in M}, \\ \boldsymbol{\lambda} &= (\lambda_j)_{j \in J}, \quad \boldsymbol{\nu} = (\nu_{jr})_{j \in J, r \in M}. \end{aligned}$$

Their corresponding constraint residuals are

$$\begin{aligned} \boldsymbol{\ell}(\mathbf{t}, \mathbf{z}) &= \left(z_j - \alpha_{jk} \sum_{r \in M} s_r t_{jr} - b_{jk} \right)_{j \in J, k \in \mathcal{K}}, \\ \mathbf{g}(\mathbf{t}) &= \left(\sum_{i=1}^j t_{i,r} - d_j \right)_{j \in J, r \in M}, \\ \mathbf{h}(\mathbf{t}) &= \left(\sum_{r \in M} s_r t_{jr} - \bar{f}_j \right)_{j \in J}, \\ \mathbf{i}(\mathbf{t}) &= (-t_{jr})_{j \in J, r \in M}, \\ q(\mathbf{t}) &= \sum_{j \in J} \sum_{r \in M} \frac{s_r}{E_r} t_{jr} - B. \end{aligned}$$

At an optimum, stationarity requires the gradient of the Lagrangian with respect to the primal variables $\mathbf{t}$ and $\mathbf{z}$ to be zero. Written componentwise, this gives:

$$\frac{\partial L}{\partial t_{jr}} = - \sum_{k \in \mathcal{K}} \alpha_{jk} s_r \gamma_{jk} + \sum_{i=j}^n \mu_{ir} + \lambda_j s_r - \nu_{jr} + \beta \frac{s_r}{E_r} = 0, \quad \forall j \in J, r \in M. \quad (6.4)$$

$$\frac{\partial L}{\partial z_j} = -1 + \sum_{k \in \mathcal{K}} \gamma_{jk} = 0, \quad \forall j \in J. \quad (6.5)$$

Discussion on Constraints (6.3b). For each task j , we distinguish two cases according to its total allocated workload $f_j = \sum_{r \in M} s_r t_{jr}$. Either the workload f_j lies *between two breakpoints* of its piecewise-linear accuracy function or it lies *at a breakpoint*. In the former case, a single constraint (6.3b) is tight, and its corresponding dual variable is positive ($\gamma_{jl} \geq 0$), while the dual variables of the other segment constraints are zero. Equation (6.5) then implies $\gamma_{jl} = 1$. In the latter case, two constraints 6.3b are tight, and the two consecutive corresponding dual variables are positive ($\gamma_{jl} \geq 0$ and $\gamma_{j,l+1} \geq 0$). The dual variables of all the other constraints are null. Equation 6.5 implies that $\gamma_{jl} + \gamma_{j,l+1} = 1$. Hereafter, we express the equations using a single dual variable γ_{jl} for task j (equal to 1 when f_j is not at a breakpoint).

Marginal-gain conditions for tasks on the same machine r . Consider two consecutive tasks $j < n$ and $j + 1$ on machine r . Applying Equation (6.4) to task j gives:

$$\mu_{jr} + \sum_{i=j+1}^n \mu_{ir} = \alpha_{jl} s_r \gamma_{jl} + \alpha_{j(l+1)} s_r (1 - \gamma_{jl}) - \lambda_j s_r + \nu_{jr} - \beta \frac{s_r}{E_r}$$

For task $j + 1$:

$$\sum_{i=j+1}^n \mu_{ir} = \alpha_{(j+1)l'} s_r \gamma_{j'l'} + \alpha_{(j+1)l'+1} s_r \gamma_{j'(l'+1)} - \lambda_{j+1} s_r + \nu_{(j+1)r} - \beta \frac{s_r}{E_r}$$

Taking the difference, we obtain:

$$\begin{aligned} \mu_{jr} = & (\alpha_{jl} s_r \gamma_{jl} + \alpha_{j(l+1)} s_r (1 - \gamma_{jl})) - (\alpha_{(j+1)l'} s_r \gamma_{j'l'} \\ & + \alpha_{(j+1)l'+1} s_r \gamma_{j'(l'+1)}) - \lambda_j s_r + \lambda_{j+1} s_r + \nu_{jr} - \nu_{(j+1)r}. \end{aligned} \quad (6.6)$$

If both tasks j and $j + 1$ are not constrained (by $\bar{f}$, by 0, and by their deadlines), the complementary slackness conditions state that $\lambda_j = \lambda_{j+1} = \nu_{jr} = \nu_{(j+1)r} = 0$, giving

$$\alpha_{jl} \gamma_{jl} + \alpha_{j(l+1)} (1 - \gamma_{jl}) = \alpha_{(j+1)l'} \gamma_{j'l'} + \alpha_{(j+1)l'+1} \gamma_{j'(l'+1)} \quad (6.7)$$

If both tasks are not at a breakpoints (i.e., $\gamma_{j(l+1)} = \gamma_{(j+1)l'+1} = 0$), we get

$$\alpha_{jl} = \alpha_{(j+1)l'} \quad (6.8)$$

It means that two unconstrained consecutive tasks should have the same marginal gains (same slopes). Otherwise, it would have been better to execute the one with the larger slope till its deadline or next breakpoint.

If j and/or $j + 1$ tasks are at a breakpoint, using that $\alpha_{jl} \geq \alpha_{j(l+1)}$ and $\alpha_{(j+1)l} \geq \alpha_{(j+1)(l+1)}$ (concave accuracy functions), it gives that

$$\alpha_{(j+1)(l')} \geq \alpha_{jl} \geq \alpha_{(j+1)(l'+1)} \quad \text{for } j' \text{ at a breakpoint} \quad (6.9)$$

$$\alpha_{(j+1)(l')} \geq \alpha_{j(l+1)} \text{ and } \alpha_{jl} \geq \alpha_{(j+1)(l'+1)} \text{ when} \quad (6.10)$$

both j and $j+1$ at a breakpoint

The first inequality ensures that it is not better to decrease t_j and increase t_{j+1} , and the second one that is not better to increase t_j and decrease t_{j+1} . We say here that the marginal gains of j and j' are comparable.

If task j is constrained by its deadline, in this case $\mu_{jr} \geq 0$, because of the non-negativity of dual variables. We obtain

$$\alpha_{jl} \geq \alpha_{(j+1)l'} \quad \text{when } j + 1 \text{ is not at a breakpoint and} \quad (6.11)$$

$$\alpha_{jl} \geq \alpha_{(j+1)(l'+1)} \quad \text{otherwise.} \quad (6.12)$$

We thus have decreasing marginal gains (decreasing slopes) of tasks (not constrained by their maximum number of operations) on a given machine in an optimal solution. Indeed, otherwise, it would have been better to increase the execution time of task $j + 1$.

Note that, when $j + 1$ is constrained by its maximum number of operations ($f_{j+1} = \bar{f}_{j+1}$), it may happen that the marginal gain and loss of $j + 1$ are higher than the marginal gain of j . Indeed, in this case, the dual variable $\lambda_{j+1} \geq 0$. Equation (6.6) becomes $\alpha_{(j+1)l'} = \alpha_{jl}s_r + \lambda_{j+1}$, when j and j' are not at breakpoint and j is not constrained by its deadline nor by $\bar{f}_j$. Other cases are dealt with similarly.

The Energy Profiles We now define an energy profile for each machine, which will be used throughout the rest of this chapter. The *energy profile* p_r of machine $r \in M$ is an upper bound on its total processing time, measured in seconds, chosen to respect the energy budget. In particular, the following conditions must be satisfied:

$$\sum_{j \in J} t_{jr} \leq p_r, \forall r \in M \quad \text{and} \quad \sum_{r \in M} p_r \cdot P_r \leq B.$$

These two conditions state that (i) the amount of work on a machine cannot exceed its profile, and (ii) when all machines are fully used up to their profile, the total energy consumption is at most the energy budget. The energy profile for our system is the collection $p = \{p_1, \dots, p_m\}$ of the energy profiles of all machines.

We also define the *energy marginal gain and loss* of a task j executed on machine r , whose number of operation f_j is in segment l , as

$$\text{Energy Marginal Gain} = \frac{E_r}{s_r} \frac{\partial^+ a_j}{\partial t_{jr}} = \begin{cases} E_r \alpha_{j(l+1)} & \text{if } j \text{ at breakpoint} \\ E_r \alpha_{j(l)} & \text{otherwise} \end{cases}$$

$$\text{Energy Marginal Loss} = \frac{E_r}{s_r} \frac{\partial^- a_j}{\partial t_{jr}} = E_r \alpha_{j(l)}.$$

Characteristics of the Energy Profiles. Using again Equation (6.4) and similar techniques, we can show that:

- For machines r and r' such that $E_{r'} \leq E_r$, the *energy profile of the most energy efficient machine*

should be the longest one.

- The last tasks executed on machines with not full energy profile should have *equal energy marginal gains* when between breakpoint or *comparable* gain when at breakpoint, meaning that the energy marginal loss of j' (first slope) should be lower than the one of j with the first slope (otherwise, it would have been better to decrease the execution time of j and increase the one of j'), while its energy marginal gain (second slope) should be higher than the one of j' (otherwise, it would have been better to decrease the execution time of j' and increase the one of j).
- The *energy slope product of the last task should be lower on machines with a non full energy profile than on machines with a full energy profile.*

6.3 Optimal algorithm for DSCT-EA-FR

In this section, we present an optimal algorithmic solution for DSCT-EA-FR, considering piecewise linear accuracy functions. In particular, we first present an algorithm for solving DSCT-EA-FR when a single machine is used, which is then used as a building block to tackle the multiple-machines scenario.

6.3.1 Algorithm for DSCT-EA-FR, one machine

Algorithm 6.1 can be used to optimally solve DSCT-EA-FR on one machine. Its inputs are the deadlines of tasks and a list *listSegments* that stores information about the linear segments of the accuracy functions of our tasks. For each segment we know its *slope*, the *task* to which it is associated, its *position* within the accuracy function (e.g., segment number 1 or segment number 2), the number of operations *totalFlops* needed to fully process it, and *usedFlops*, the number of operations currently dedicated to it by the scheduler.

Note that, although Algorithm 6.1 does not explicitly take energy into consideration, incorporating an energy budget is feasible by treating it as an additional deadline (plus some additional steps). Further insights into this process will be provided Algorithm 6.2, where this transformation is explicitly outlined.

The algorithm starts by sorting the segments in non-increasing order of slopes (line 1). Then, for each segment, the algorithm first computes its total processing time based on the value of *totalFlops* and the speed of the machine (line 5), and then adjusts it according to the deadlines of the following tasks (lines 6 - 7). Indeed, increasing too much the processing time for a segment could cause a following task to violate its deadline, which is prevented by line 7. The process is repeated for all the segments. Note that Algorithm 6.1 always prioritizes segments with higher slope and, for a task $j \in J$, a segment k is considered only if segment $k - 1$ (which has a higher slope) was already processed.

Theorem 6.3.1. *Algorithm 6.1 presents a time complexity $O(n^2)$.*

Proof.

At line 1, the algorithm sorts the list of segments in $O(n \log(n))$. Assuming a constant number of segments for each task, the algorithm loops over all n tasks at line 2 and, for each of them, it loops over the following ones (at most n) at line 6. This gives a complexity of $O(n^2)$. $\square$

The optimality of Algorithm 6.1 is shown in the next section, contextually with the proof of optimality of the algorithm for multiple machines (Theorem 6.3.2).

Require: List of deadlines $[d_1, \dots, d_n]$, speed s of the machine, and a list of segments $listSegments$, storing information about the linear segments of the accuracy functions of tasks.

Ensure: List of task execution times $[t_1, \dots, t_n]$.

```

1:  $listSegments.order()$  ▷ Non-increasing order of slopes
2: for  $seg \in listSegments$  do
3:    $j \leftarrow seg.task$ 
4:    $k \leftarrow seg.position$ 
5:    $contribution \leftarrow \frac{seg.totalFlops}{s}$ 
6:   for  $i > j$  do ▷ For all tasks after
7:      $contribution \leftarrow \min(contribution, d_i - \sum_{k \leq i} t_k)$ 
8:    $t_j \leftarrow t_j + contribution$  ▷ Update processing time of task  $j$ 
9: return  $[t_1, \dots, t_n]$ 

```

Algorithm 6.1 – Exact algorithm for scheduling on one machine using piecewise linear accuracy functions

6.3.2 Algorithm for DSCT-EA-FR, multiple machines

After dealing with a single machine, we now propose an algorithm for solving DSCT-EA-FR in the multiple-machines scenario, where energy budget is also taken into consideration.

Intuitively, more energy efficient machines should be preferred when we must respect some energy budget; indeed, for a unit of energy, an efficient machine can execute more floating operations than a less efficient one.

Following this intuition, let us define the *naive energy profile* p^{naive} of a system as the energy profile in which we use the most efficient machines until the energy budget is met.

Although intuition might suggest otherwise, the naive energy profile does not always lead to an optimal solution.

Algorithm 6.4 computes an optimal energy profile and solution for DSCT-EA-FR on multiple machines. It comprises two main steps:

- *ComputeNaiveSolution*: It computes the naive profile and the optimal solution for this profile. This procedure is described in Algorithm 6.2.
- *RefineProfile*: Check if changing the energy profile could lead to a better solution. If so, update accordingly the energy profile and the solution. Repeat this step until no better solution can be found. This procedure is described in Algorithm 6.3.

ComputeNaiveSolution Algorithm 6.2 computes the optimal solution for the naive energy profile.

Firstly, the naive energy profile is derived from the energy budget B . This is accomplished by sorting the set of machines M in non-increasing order of energy efficiency E and computing, for each of them, the energy profile by selecting the minimum between the maximal execution time for a machine ($d^{\max}$) and the time needed to reach the energy budget B .

Then, the algorithm transforms the problem into an "equivalent" one on a single-machine problem. This is done by checking for each task and for each machine whether the deadline falls inside the machine profile, in which case the machine can be fully used for the task. Otherwise, only a fraction of the machine can be used to process the task. Next, the algorithm uses Algorithm 6.1 to compute an optimal solution for this single-machine scenario.

We then need to go back to the multiple-machines scenario. It iterates through the tasks in non-decreasing order of deadlines and checks if it's possible to evenly distribute the tasks across the machines, scheduling the same amount of time for each machine. If energy profile limitations prevent this, the algorithm schedules the maximum allowed time on the least energy-efficient machine and repeats the process with the remaining machines. This procedure is repeated for all tasks.

Require: List of segments $listSegments$, the naive energy profile $p^{naive} = [p_1, \dots, p_m]$, the speeds $[s_1, \dots, s_m]$ and the energy efficiencies $[E_1, \dots, E_m]$ of the machines. Deadlines $[d_1, \dots, d_n]$ are in non-decreasing order.

```

1:  $M.order()$  ▷ Order M in non-increasing order of energy efficiency  $E$ 
2:  $E_{temp} = 0$ 
3: for  $r \in M$  do ▷ Compute naive energy profile
4:    $p_r \leftarrow \min(\frac{B - E_{temp}}{\frac{s_r}{E_r}}, d^{max})$ 
5:    $E_{temp} \leftarrow E_{temp} + p_r \cdot \frac{s_r}{E_r}$ 
6: for  $j \in J$  do ▷ Compute temporary deadlines
7:   for  $r \in M$  do
8:      $d_j^{temp} \leftarrow d_j^{temp} + \min(s_r, \frac{p_r \cdot s_r}{d_j}) \cdot d_j$ 
9:  $t \leftarrow \text{Algorithm 6.1}(d = d^{temp}, s = 1.0)$  ▷ Computes solution in single machine
10:  $K \leftarrow M$ 
11: for  $j \in J$  do ▷ Distribute the task along machines
12:   while  $t_j \geq 0$  do
13:      $k_{min} \leftarrow \arg \min_{k \in K} (E_k)$ 
14:     if  $\sum_{j' \leq j} t_{j'} + \frac{t_j}{\sum_{k \in K} s_k} > p_{k_{min}}$  then ▷ Schedule until the machine energy profile
15:        $t_j \leftarrow t_j - s_{k_{min}}(p_{k_{min}} - \sum_{j' \leq j} t_{j'})$ 
16:        $t_{jk_{min}} \leftarrow p_{k_{min}} - \sum_{j' \leq j} t_{j'}$ 
17:        $K \leftarrow K \setminus \{k_{min}\}$ 
18:     else ▷ Same time for all machines
19:       for  $k \in K$  do
20:          $t_{jk} \leftarrow \frac{t_j}{\sum_{k \in K} s_k}$ 
21:        $t_j \leftarrow 0$ 
22: return  $[t_{jr} : \forall j \in J, \forall r \in M]$ 

```

Algorithm 6.2 – `ComputeNaiveSolution`: Computes an optimal scheduling for the naive energy profile

RefineProfile Once we have the optimal solution for the naive energy profile, Algorithm 6.3 checks whether a better solution could be found by tweaking the energy profile.

Let us define the *accuracy-per-Joule* $\psi_{seg,r}$ as the accuracy gained per unit of energy by increasing the processing time of a linear segment seg in machine r . Note that this is equivalent to the energy marginal gain discussed in Section 6.2, as accuracy-per-Joule is also computed as $\psi_{seg,r} = seg.slope \cdot E_r$.

Firstly, Algorithm 6.3 generates a list of all pairs $(segment, machine)$, sorted in non-increasing order of accuracy-per-Joule. The pairs at the top of the list offer the highest potential for increasing accuracy.

Next, the algorithm iterates over the list and, for each pair (seg, r) , it computes how much the execution time of seg could be increased on machine r , keeping track of the additional energy E_{add} that it would consume. To do this, it considers the minimum between the energy required for the remaining floating operations of seg and the energy needed to fill the gap between the tasks that were already scheduled and the deadline of $seg.task$. For each pair analyzed, the algorithm iterates over the list of pairs in reverse order, searching for tasks to be reduced. If a pair (seg', r') is found that presents a smaller accuracy-per-Joule value than (seg, r) , it means that it would be beneficial to decrease the execution time of seg' in machine r' and increase that of seg on r . The amount of energy E_{sub} that can be saved by reducing the execution time of seg' is computed, and finally the minimum value between E_{add} and E_{sub} is the actual amount of energy that is "allocated" for seg in machine r , and "deallocated" for seg' in machine r' .

Require: List of Segments $listSegments$; List of machines speed s and energy efficiency E ; List of task deadlines d

```

1:  $P \leftarrow \{\}$  ▷ Set containing the pairs (segment, machine)
2: for  $seg \in listSegments$  do
3:   for  $r \in M$  do
4:      $P \leftarrow P + \{(seg, r)\}$ 
5:  $P.order()$  ▷ Order P in non-increasing order of accuracy per Joule
6: for  $(seg, r) \in P$  do
7:    $j \leftarrow seg.task$ 
8:    $E_{add} \leftarrow \min(\frac{seg.totalFlops - seg.usedFlops}{E_r}, (d_j - \sum_{k \leq j} t_{kr}) \frac{s_r}{E_r})$ 
9:   for  $(seg', r') \in P.reverse()$  do
10:    if  $r > r'$  and  $\psi_{seg,r} > \psi_{seg',r'}$  then ▷ Check accuracy-per-Joule for refining Profile
11:       $E_{sub} \leftarrow \min(\frac{seg'.flops}{E_{r'}}, t_{j'r'} \frac{s_{r'}}{E_{r'}})$ 
12:       $E_{transf} \leftarrow \min(E_{add}, E_{sub})$  ▷ Energy transferred between machines
13:       $t_{jr} \leftarrow t_{jr} + \frac{E_{transf} \cdot E_r}{s_r}$ 
14:       $seg.usedFlops \leftarrow seg.usedFlops + E_r \cdot E_{transf}$ 
15:       $t_{j'r'} \leftarrow t_{j'r'} - \frac{E_{transf} \cdot E_{r'}}{s_{r'}}$ 
16:       $seg'.usedFlops \leftarrow seg'.usedFlops + E_{r'} \cdot E_{transf}$ 
17:       $E_{add} \leftarrow E_{add} - E_{transf}$ 
18: return  $[t_{jr} : \forall j \in J, \forall r \in M]$ 

```

Algorithm 6.3 – RefineProfile: Adapts the naive profile in order to find the optimal solution

Theorem 6.3.2. Algorithm 6.4 is optimal and has a time complexity $O(n^2m^2)$.

Proof.

Optimality. At the end of the first phase, computeNaiveSolution, the execution times on all machines satisfy the first set of KKT conditions given by Equations (6.11), (6.12) (non increasing marginal gains on a machine) and Equations (6.8) and (6.10) (comparable marginal gain for tasks not constrained by their deadline and maximum execution time). Indeed, the procedure

considers the linear pieces of accuracy functions by decreasing slopes. It is thus not possible to have a task with a lower marginal gain and an earlier deadline, which is processed before one with a higher marginal gain and a later deadline.

At the end of the second phase RefineProfile, they satisfy the second set of KKT Conditions stated in the part *Characteristics of the Energy Profiles* of Section 6.2.2: higher energy marginal gains on more energy efficient machines and comparable energy marginal gain for machines with non full energy profiles. We now argue that the conditions of non-increasing marginal gains are maintained following the RefineProfile phase, specifically after each iteration of the for loop starting at line 5 of RefineProfile. Let's first consider machine r' , where we decrease the processing time of a task j' (line 14 of the RefineProfile Algorithm). It's important to note that j' will always be the task on machine r' with the lowest marginal loss. Consider that the number of operations of j' , $f_{j'}$, is in the linear segment $k > 1$ of its accuracy function. If we reduce its number of operations, but not up to the breakpoint $p_{j'k}$, it does not change its marginal gain $\alpha_{j'k}$ and loss $\alpha_{j'k}$, so the conditions are still satisfied. Now, if we pass the breakpoint $p_{j'k}$, the marginal gain increases (by concavity of the accuracy function) and is equal to $\alpha_{j'(k-1)}$. However, we are insured that it won't become higher than the marginal loss of a later task, say α_{il} , with $i > j'$. Indeed, if we decreased the number of floating operations of j' , when it was at $p_{j'k}$, it means that it had the lowest marginal loss $\alpha_{j'(k-1)} \leq \alpha_{il}$. So, the KKT conditions (6.11) and (6.12) are still satisfied. A similar argument shows that, when increasing the number of floating operations of task j on machine r , we cannot decrease its marginal gain lower than the one the marginal loss of a later task (not constrained by its maximum number of floating operations).

The KKT conditions are necessary and sufficient conditions for optimality as the problem is convex. Thus, the algorithm provides an optimal solution.

Complexity. In `computeNaiveSolution` (Algorithm 6.2), from lines 6-8, the temporary deadlines are computed, which is done in $O(nm)$. In line 9, according to Theorem 6.3.1, it is done in $O(n^2)$. In lines 11-21, the algorithm iterates over every task and each machine.

In `refineProfile` (Algorithm 6.3), it iterates over each task and each machine and orders the pair list, which is done in $O(nm \cdot \log(nm))$. Then, in lines 6-17, we iterate for each pair (*segment, machine*) in a nested loop. Subsequently, Algorithm 6.3 has a time complexity $O(n^2m^2)$. Therefore, DSCT-EA-FR-OPT has a time complexity $O(n^2m^2)$. $\square$

Require: List of Segments *listSegments*; List of machines speed *s* and energy efficiency *E*;
List of task deadlines *d*

- 1: $\mathbf{t} \leftarrow \text{computeNaiveSolution}()$ $\triangleright$ Algorithm 6.2
- 2: $\mathbf{t} \leftarrow \text{refineProfile}()$ $\triangleright$ Algorithm 6.3
- 3: **return** $[t_{jr} : \forall j \in J, \forall r \in M]$

Algorithm 6.4 – DSCT-EA-FR-OPT: Computes the optimal solution for DSCT-EA-FR using Piecewise Linear function as accuracy function

6.4 Approximation algorithm for DSCT-EA

Here, we propose an approximation algorithm DEADLINE SCHEDULING WITH COMPRESSIBLE TASKS - ENERGY AWARE - APPROXIMATION ALGORITHM (DSCT-EA-APPROX for short), which addresses the DSCT-EA problem. We adapted the algorithm developed in (Da Silva Barros et al., 2024). For addressing the energy budget constraints added to the problem, we set that, for each machine, the energy profile found in the fractional relaxation solution works as an upper bound of the load scheduled in the machine. If the work in the machine is equal to the energy profile, i.e. $\sum_{j \in J} t_j = p_r$, the machine is no longer considered in the scheduling. The pseudocode is described in Algorithm 6.5.

Input: $[d_1, \dots, d_n], [t_1^{\max}, \dots, t_n^{\max}], [s_1, \dots, s_m], [E_1, \dots, E_m]$

Output: List of task processing times $[t_{jr} : \forall j \in J, r \in M]$

```

1: Sort the segments by non increasing order deadlines.
2:  $\mathbf{t}^f, \mathbf{w}_r^{\max} = \text{DSCT-EA-FR-OPT}([d_1, \dots, d_n], [t_1^{\max}, \dots, t_n^{\max}], [s_1, \dots, s_m], [E_1, \dots, E_m]) \triangleright$ 
   Compute the optimal fractional solution  $\mathbf{t}^f$ , e.g. for piecewise, Algorithm 6.4.
3: for  $r \in M$  do
4:    $S_r \leftarrow []$   $\triangleright$  List of tasks scheduled on machine  $r$ 
5:    $w_r \leftarrow 0$   $\triangleright w_r$  amount of work on  $r$ 
6:    $F \leftarrow \{\}$   $\triangleright$  Set with fully completed machines
7:   for  $j \in J$  do  $\triangleright$  Schedule each task on the machine with the least amount of work,  $r_{\text{best}}$ 
8:      $r_{\text{best}} \leftarrow \arg \min_{r \in M \setminus F} w_r$ 
9:      $t_{j,r_{\text{best}}} \leftarrow \min(\sum_{r=1}^m t_{j,r}^f, w_r^{\max} - w_r)$ 
10:     $S_{r_{\text{best}}} \leftarrow S_{r_{\text{best}}} \cup [j]$ 
11:    if  $w_{r_{\text{best}}} = w_{r_{\text{best}}}^{\max}$  then
12:       $F \leftarrow F \cup [r_{\text{best}}]$ 
13:   for  $r \in M$  do  $\triangleright$  Cut tasks violating their deadlines and shift the following ones.
14:     for  $j \in S_r$  do
15:       if  $t_{j,r}^{(i)} + t_{jr} \geq d_j$  then
16:          $v_j \leftarrow t_{j,r}^{(i)} + t_{jr} - d_j$ 
17:          $t_{j,r} \leftarrow t_{j,r} - v_j$   $\triangleright$  Cut task
18:         for  $i > j \in S_r$  do  $\triangleright$  Shift the following tasks
19:            $t_{i,r}^{(i)} \leftarrow t_{i,r}^{(i)} - v_j$ 
20: return  $[t_{jr} : \forall j \in J, r \in M]$ 

```

Algorithm 6.5 – DSCT-EA-APPROX: Approximation algorithm for the scheduling problem on several machines

Let OPT be the maximum aggregate accuracy of an integral DSCT-EA schedule, OPT_f the maximum aggregate accuracy of its fractional relaxation, and SOL the accuracy returned by DSCT-EA-APPROX. The fractional solution is an upper bound, so $SOL \leq OPT \leq OPT_f$. Applying the additive rounding analysis of Theorem 3 in (Da Silva Barros et al., 2024) gives

$$OPT_f - G \leq SOL \leq OPT \leq OPT_f. \quad (6.13)$$

Here G bounds the accuracy lost when the fractional allocation is rounded. The bound is

$$G = m \int_0^\infty \max_{j,r} \frac{\partial a_j(s_r t)}{\partial t} dt.$$

For the piecewise-linear accuracy functions considered here, it takes the form

$$G = m(a^{\max} - a^{\min}) \left(1 + \ln \left(\frac{\theta_{\max}}{\theta_{\min}} \right) \right). \quad (6.14)$$

Here $\theta_{\min} = \min_{j \in J} \alpha_{j0}$ and $\theta_{\max} = \max_{j \in J} \alpha_{j|K|}$ denote the slope bounds used in this guarantee.

6.5 Results

In this section, we evaluate the approximation algorithm DSCT-EA-APPROX (Algorithm 6.5). We first compare its achieved accuracy with the fractional upper bound in Equation (6.13); the observed gap is typically much smaller than the worst-case additive bound G . We then discuss the running time of our algorithm and demonstrate that DSCT-EA-APPROX provides results within a reasonable time frame. Next, we compare the performance of our algorithm with state-of-the-art methods that either do not employ compression or utilize only limited discrete levels of compression. We show that it can achieve large energy gains with only a small impact on the task accuracy. Lastly, we consider a scenario with 2 heterogeneous machines to investigate the energy profile and the computational contribution of each machine, i.e., the number of floating operations performed. **Hardware Settings.** For the experiments, we used a laptop equipped with an Intel Core i9-12900H CPU and an NVIDIA RTX A2000 GPU.

Experiments. We generate synthetic sets of tasks using the Once-For-All (Cai et al., 2020) approach applied to ResNet (K. He et al., 2016), with 1000 classification classes.

We set the minimum task accuracy to $a^{\min} = 1/1000$, the accuracy of a random guess, and the maximum accuracy to $a^{\max} = 0.82$. These values were chosen based on our tests of `ofa-resnet` on the ImageNet-1k (Deng et al., 2009) dataset. We denote as θ_j the "task efficiency" of a task $j \in J$, representing the slope of its first segment. We fix the minimum value of task efficiency to $\theta_{\min} = 0.1$. In our experiments, we modeled the accuracy function of a task j as a piecewise-linear function, constructed by performing a linear regression with 5 segments over an exponential accuracy function of parameter θ_j (Figure 6.2 provides an example, although with more than 5 segments).

We considered machine speeds that are uniformly distributed between 1 TFLOPS and 20 TFLOPS, and energy efficiencies uniformly distributed between 5 GFLOPS/W and 60 GFLOPS/W. These values were selected based on research findings presented in (Desislavov et al., 2023).

We define the *deadline tolerance level*, denoted by ρ , as $\rho = \frac{m^2 \cdot d_{\max}}{\sum_{j \in J} (f_j \cdot \sum_{r \in M} s_r)}$. The higher the value of ρ , the more time is allocated for the tasks.

We build scenarios with varying (i) task heterogeneity and (ii) energy budget ratio levels. To this end, we define the *task heterogeneity ratio* μ as $\mu \stackrel{\text{def}}{=} \frac{\theta_{\max}}{\theta_{\min}}$, which reflects the similarity between the accuracy functions of the different tasks. Task efficiencies are uniformly distributed between $\theta_{\min}$ and $\theta_{\max}$, and the value $\bar{f}_j$ for each task is computed so to have $a_j(\bar{f}_j) = a^{\max}$. We also define an energy budget ratio β to describe how restrictive the energy budget is. We normalize

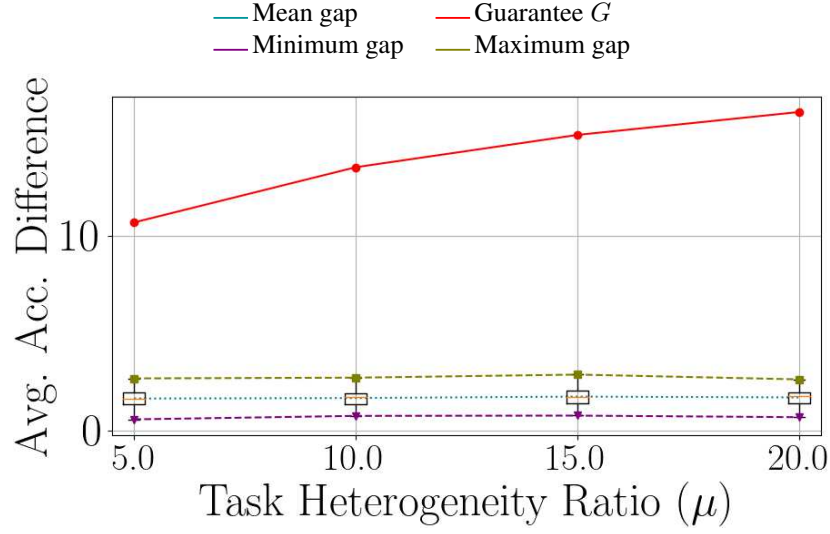


Figure 6.3 – Gap to the fractional upper bound (average accuracy difference between DSCT-EA-UB and DSCT-EA-APPROX) over 100 experiments with varying task heterogeneity.

B by the energy that all machines would consume if they ran continuously until the latest deadline: $\beta = \frac{B}{\sum_{r \in M} d^{\max} \cdot s_r / E_r}$, where $d^{\max} = \max_{j \in J} (d_j)$ and s_r / E_r is the power consumption of machine r . Smaller values of β correspond to stricter energy budgets.

Baselines. We benchmark the performance of DSCT-EA-APPROX against 3 different solutions:

- DSCT-EA-UB: the fractional upper bound OPT_f , computed by DSCT-EA-FR-OPT for DSCT-EA-FR. This is a benchmark, not an implementable schedule.
- EDF-NOCOMPRESSION: here, no compression is applied, i.e., each processed task j receives its maximum useful workload $\tilde{f}_j$. We use the EDF (Earliest Deadline First) strategy combined with scheduling on the machine with the least amount of work (Y.-W. Zhang et al., 2023) to determine where tasks should be scheduled. Scheduling is performed until the energy budget is reached, at which point no further tasks are scheduled.

- EDF-3COMPRESSIONLEVELS: this algorithm considers a discrete number of compression levels for neural networks. Three compression levels are used, corresponding to accuracy levels of 27%, 55%, and 82%. A strategy based on prior research (Lee and Song, 2021) is implemented. Like the previous approach, tasks are scheduled until the energy budget is reached.

Performance guarantees To assess the performance of our algorithm, we measure its gap to the fractional upper bound OPT_f . This is an upper bound on, not necessarily the value of, the best integral schedule. We consider a set of $n = 100$ tasks and $m = 5$ machines, with deadline tolerance $\rho = 0.35$ and energy budget ratio $\beta = 0.5$.

We vary the task heterogeneity ratio μ between 5.0 and 20.0, conducting 100 experiments for each value of μ , and collect the mean, maximum, and minimum accuracy values.

Figure 6.3 shows that the observed accuracy loss relative to OPT_f is usually much smaller than the worst-case allowance G in Equation (6.13).

Running-time analysis We compare the running time of DSCT-EA-APPROX with that of the exact integral solver DSCT-EA-Opt, implemented with cvx-MOSEK (ApS, 2022). The solver returns an optimal solution when it certifies one within the time limit. We considered 2 distinct

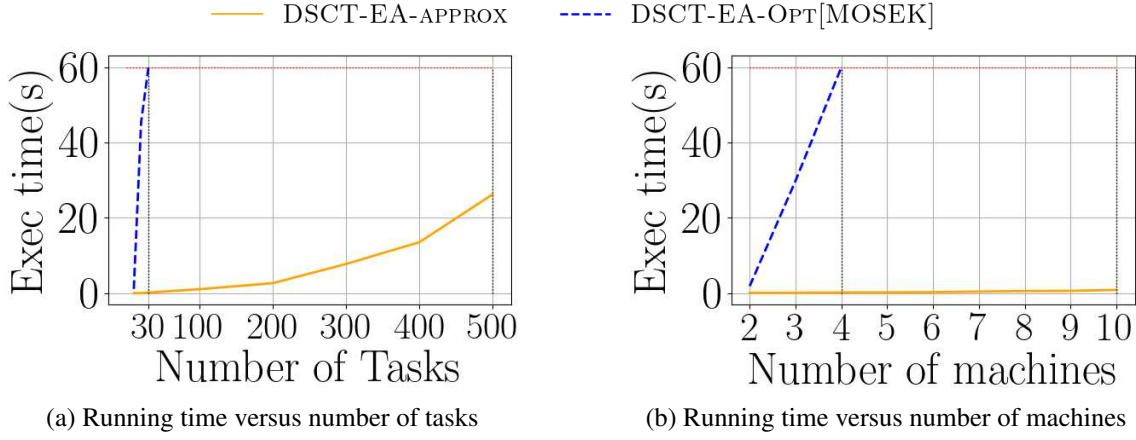


Figure 6.4 – Running times of DSCT-EA-APPROX versus the integral solver DSCT-EA-Opt (ApS, 2022) for instances with increasing (a) numbers of tasks, with $m = 5$, and (b) numbers of machines, with $n = 50$.

scenarios: (i) when increasing the number of tasks from $n = 10$ to $n = 500$, keeping fixed $m = 5$ machines, and (ii) when increasing the number of machines from $m = 2$ to $m = 10$, fixing $n = 50$ tasks. We considered an average over 10 instances for each experiment analyzed and a time limit of 60s.

The results are shown in Figures 6.4a and 6.4b, respectively. The integral solver (DSCT-EA-Opt), not the fractional upper bound DSCT-EA-UB, reaches the time limit beyond the smaller instances (around $n = 30$ and $m = 4$ in the two experiments). By contrast, DSCT-EA-APPROX handles instances with hundreds of tasks and several machines.

We also evaluated the execution time of DSCT-EA-FR-OPT against the Mosek solver applied to DSCT-EA-FR, varying the number the tasks from 100 to 500, fixing $m = 5$. The results are described in Table 6.1. We observe that the developed algorithm DSCT-EA-FR-OPT provides faster results for all instances tested, even with a non-optimized python implementation.

Table 6.1 – Comparison of Execution Times for DSCT-EA-FR-OPT and DSCT-EA-FR [Mosek]

Number of tasks	100	200	300	400	500
DSCT-EA-FR-OPT (s)	1.05	2.66	7.75	13.52	26.2
DSCT-EA-FR [Mosek] (s)	1.11	4.26	11.01	21.23	38.07

Comparison with the State-of-the-Art In this section, we compare DSCT-EA-APPROX with the fixed-model baseline EDF-NOCOMPRESSION, the discrete-compression baseline EDF-3COMPRESSIONLEVELS, and the fractional upper bound DSCT-EA-UB under different energy constraints.

We varied the energy budget ratio from $\beta = 0.1$ to $\beta = 1.0$ to explore a wide range of energy constraints. Additionally, we considered $n = 100$ tasks, $m = 2$ machines, and a deadline tolerance value $\rho = 1.0$. All tasks were assumed to be uniform with $\theta = 0.1$.

Figure 6.5 shows average accuracy across the energy-budget sweep. Accuracy is low when β is close to zero, reflecting the tight budget. Across these instances, DSCT-EA-APPROX stays

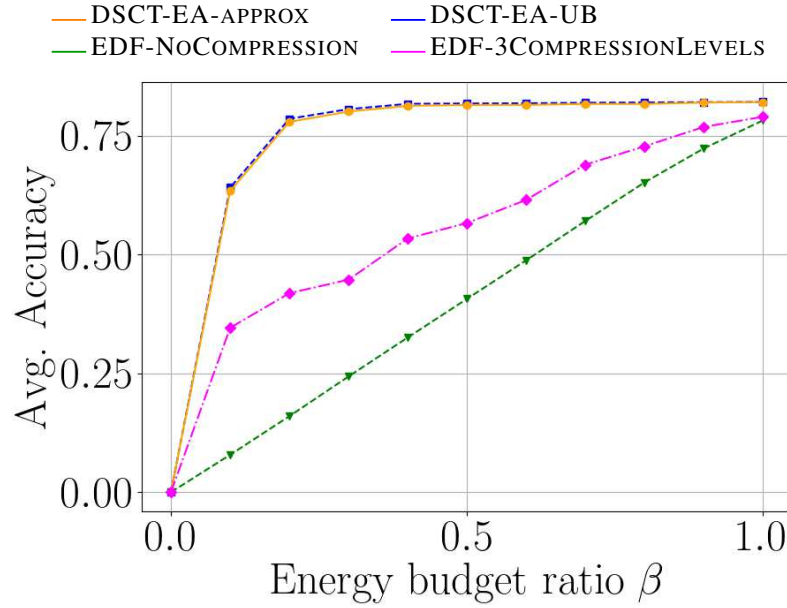


Figure 6.5 – Average task accuracy for DSCT-EA-APPROX and the baselines over energy budget ratio β , for $m = 2$ and $n = 100$.

close to the fractional upper bound DSCT-EA-UB and outperforms the two scheduling baselines. At $\beta = 1.0$, all methods reach approximately $a^{\max}$ because the budget permits full processing.

Energy Gain. Note, that, using our solution, DSCT-EA-APPROX, 70% of the energy can be saved up while only reducing by 2% the average task accuracy, compared to a scenario without compression, achieving a high energy sobriety.

Workload Balancing with Heterogeneous Machines and Tasks. Lastly, we explore how DSCT-EA-APPROX balanced the workload across machines with different energy efficiencies and speeds. We consider a simple scenario with $m = 2$, where machine 1 has speed $s_1 = 2$ TFLOPS and energy efficiency $E_1 = 80$ GFLOPS/W, while for machine 2 we have $s_2 = 5$ TFLOPS and $E_2 = 70$ GFLOPS/W. These values were chosen according to (Desislavov et al., 2023). Essentially, machine 1 is slower but more energy efficient than machine 2.

We used $n = 100$ tasks and looked at two distinct scenarios. In the first one (Uniform Tasks), we used a batch of sets with efficiency value θ uniformly distributed between 0.1 and 4.9. In the second scenario (Earliest High Efficient Tasks), we divide tasks into two sets: the earliest 30% of tasks (according to their deadline) have high efficiency ($4.0 \leq \theta \leq 4.9$), while the remaining ones are not very efficient ($0.1 \leq \theta \leq 1.0$). All the experiments used a deadline tolerance value $\rho = 0.01$, indicating very strict deadlines.

Figs. 6.6a and 6.6b display the final energy profile computed by DSCT-EA-APPROX for both machines, with varying energy budget ratios β , for scenarios Uniform Tasks and Earliest High Efficient Tasks, respectively. Recall that by energy profile we mean how the workload (in seconds) is distributed across the two machines by DSCT-EA-APPROX, which considers energy constraints during scheduling. Fig. 6.6a shows that, unsurprisingly, the computed profile for scenario Uniform Tasks is very close to the naive one. The same cannot be said about the Earliest High Efficient Tasks scenario, as Fig. 6.6b shows a behavior that sensibly deviates from the naive profile.

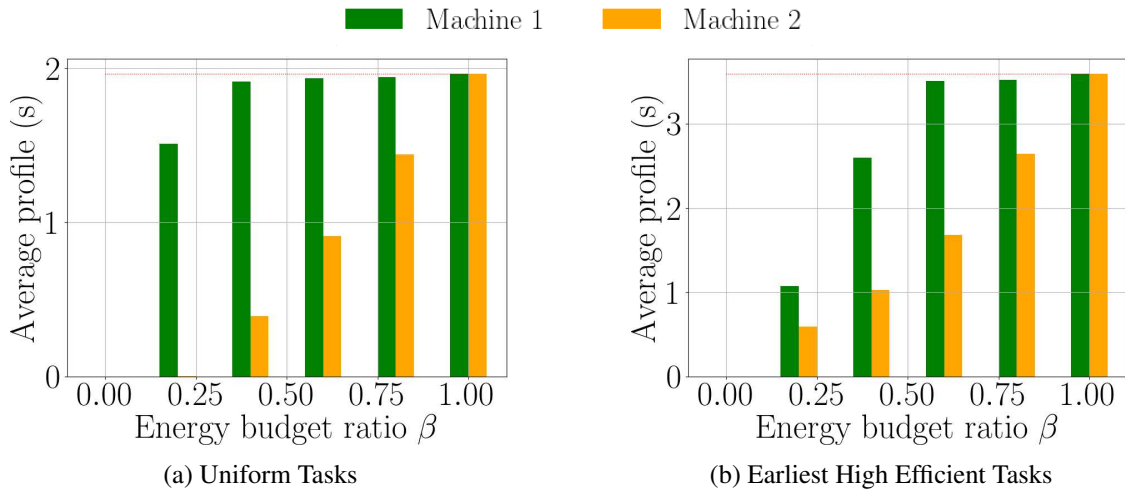


Figure 6.6 – Energy profile of 2 machines when varying the energy budget ratio. Machine 1 is more energy efficient than machine 2.

For $\beta \leq 0.4$, there's a notable difference between the energy profiles of machines 1 and 2 compared to the last deadline $d^{\max}$. This difference arises because there's a significant refinement in the profile caused by tasks with high efficiency values θ being constrained by the deadline on machine 1. Consequently, the refinement in the profile increases the workload on machine 2 to accommodate these tasks. For example, for $\beta = 0.4$, the naive profile indicates $p_1^{\text{naive}} = 1.96\text{s}$ and $p_2^{\text{naive}} = 0.0$. However, the final profile yields $p_1 = 1.26\text{s}$ and $p_2 = 0.25\text{s}$. Thus, the profile was refined during the optimal algorithm, leading to an increase of the workload of machine 2.

6.6 Conclusion

This chapter studied the scheduling of compressible inference tasks under deadlines and a global energy budget. We formulated the integral problem as a mixed-integer program, gave an exact algorithm for its fractional relaxation, and used the fractional solution to construct an approximate integral schedule.

In the evaluated instances, the algorithm outperforms fixed-model and discrete-compression baselines and achieves accuracy close to the fractional upper bound. The experiments also illustrate how machine speed and energy efficiency affect placement as the energy budget changes.

Looking ahead, we identify the integration of renewable power sources into the scheduling problem as promising avenues for future research. Moreover, we intend to consider in the problem model the energy consumption resulted from communication of devices.

Communication-Aware Scheduling across the Edge–Cloud Continuum

Machine-learning services increasingly execute across heterogeneous edge and cloud resources. Model compression and adaptive inference make it possible to vary the computation assigned to a task, trading processing cost for accuracy. Exploiting this flexibility requires jointly deciding where, when, and for how long each task executes, while communication delays induce destination-dependent availability windows.

We study the scheduling of fully compressible tasks on heterogeneous machines with destination-dependent release times and deadlines. We give an exact integral formulation and propose an eligibility-aware relax-and-round algorithm that solves a fractional relaxation before assigning each task to one machine while limiting discarded workload. The relaxation also admits exact min-cost-flow formulations in several important special cases. We establish general workload-retention and accuracy certificates, together with stronger approximation guarantees under homogeneous speeds and common availability windows. Experiments show that the algorithm remains close to the fractional upper bound, adapts placement across edge and cloud resources, and scales to instances for which the integral formulation is impractical.

7.1	Introduction	97
7.2	Problem Model and Exact Formulation	98
7.2.1	Communication-Aware Scheduling Model	98
7.2.2	Exact Integral Formulation	99
7.3	Fractional Relaxation and Rounding	100
7.3.1	Assignment-Free Fractional Relaxation	100
7.3.2	Eligibility-Aware Minimum-Discard Rounding	101
7.4	Min-Cost-Flow Formulations of the Fractional Relaxation	102
7.5	Theoretical Analysis	104
7.5.1	From the Rounding Rule to Workload Loss	105
7.5.2	Capacity Overflow in One Interval	106
7.5.3	General Destination-Dependent Certificates	111
7.5.4	From Retained Workload to Accuracy	113
7.5.5	Specialization to Common Windows	114
7.5.5.1	Fastest-machine guarantee	114
7.5.5.2	Identical speeds	115
7.5.5.3	Heterogeneous speeds and full capacity	116
7.6	Experimental Evaluation	117
7.6.1	Methodology	117
7.6.2	Communication-Aware Scheduling	118
7.6.2.1	Solution quality	118
7.6.2.2	Placement adaptation	118
7.6.3	Quality Under Common Windows	120
7.6.4	Machine-Speed Heterogeneity	121
7.6.5	Running-Time Scalability	121
7.7	Conclusion	122

7.1 Introduction

Machine Learning (ML) models are now fundamental components of computing systems, with applications ranging from self-driving cars (J. Zhang et al., 2022) and unmanned aerial vehicles (Raj et al., 2023) to Mobile Edge Computing (MEC) (Shakarami et al., 2020). The rapid development of foundation models (Epoch AI, 2024), including Large Language Models, has further accelerated AI adoption in academia and industry (Perrault and Clark, 2024).

Cloud providers support this demand through Machine Learning as a Service (MLaaS) (Ribeiro et al., 2015), giving users access to ML models without managing the underlying infrastructure. Meanwhile, edge computing, 5G/6G networks, and accelerators such as GPUs and FPGAs enable inference throughout the edge-cloud continuum (Drainakis et al., 2021). Reducing inference latency in these distributed environments remains a central challenge.

One promising approach uses *compressible neural networks*, which allow inference-time configuration without retraining (Cai et al., 2020; Lin et al., 2024; Yu and T. Huang, 2019). In particular, Once-For-All (Cai et al., 2020) trains a single network from which specialized subnetworks can be selected for different hardware platforms and latency constraints, exposing a trade-off between accuracy, computational demand, and inference latency.

Efficiently scheduling compressible inference tasks nevertheless remains an open problem. Edge-cloud resources have limited computation, memory, storage, and network capacity. Furthermore, inference tasks have different release times and deadlines, and assigning a task to a remote machine introduces communication delay. These delays depend on the selected execution machine and therefore change the effective execution window of each task. As a result, a faster machine may become less attractive if communication leaves insufficient time for computation. Thus, scheduling decisions must jointly consider the quality-processing-time trade-off and system constraints, including heterogeneity among communication links. Existing algorithms for compressible inference (Barros et al., 2024; Silva Barros et al., 2024) use simpler system models that do not consider task-specific release times and communication latency.

We study communication-aware scheduling of compressible ML tasks on heterogeneous machines across the edge-cloud continuum. Each task has a release time, an end-to-end deadline, and a concave accuracy function of its executed Floating Point Operations (FLOPs), capturing diminishing returns from additional computation. Machine-dependent communication delays induce a different execution window at each destination. We maximize total accuracy by jointly choosing each task’s machine and workload, and thus its compression level. Our relax-and-round algorithm first solves a fractional relaxation and then rounds it into a feasible schedule. Experiments on synthetic and Google-cluster-inspired workloads (Reiss et al., 2011) show near-optimal accuracy and consistent improvements over the considered scheduling baselines.

Our contributions can be summarized as:

- We give an exact integral formulation for scheduling compressible ML tasks with release times, deadlines, destination-dependent delays, and heterogeneous machines.
- We develop eligibility-aware relax-and-round scheduling, with workload-retention and accuracy guarantees, including stronger bounds in structured settings.
- For common execution windows, we formulate the fractional relaxation exactly as a min-cost flow and discuss extensions to destination-dependent windows.
- Simulations show accuracy close to the optimum and substantial scalability gains over the exact integral formulation.

The common optimization framework and the broader literature on configurable inference and compressible-task scheduling are reviewed in Chapter 5, with the complete literature survey in Section 5.2. Relative to that landscape, this chapter focuses on the interaction between continuously adjustable workloads and destination-dependent feasibility: task-specific release times, end-to-end deadlines, and communication delays determine when each task can execute on each machine. Its contribution is therefore not only a richer system model, but also a rounding analysis that preserves the temporal eligibility of fractionally allocated work.

7.2 Problem Model and Exact Formulation

We consider a scheduler that assigns compressible ML inference tasks to computing resources distributed across the edge–cloud continuum. Each task must be assigned to one machine while respecting communication-dependent execution windows and machine capacities. We next define this general communication-aware scheduling problem and give an exact integral formulation.

7.2.1 Communication-Aware Scheduling Model

Let $J = \{1, \dots, n\}$ be the set of tasks and $M = \{1, \dots, m\}$ the set of available machines. Machine $\mu \in M$ has processing speed $s_\mu > 0$, measured in FLOPs per unit of time. We make no assumption relating processing speed and communication delay: a faster machine may be either closer to or farther from a task’s data.

Task j becomes available at its source at time r_j , and its result must reach its consumer by the end-to-end deadline d_j . The source and consumer need not coincide with the scheduler and may be located anywhere in the edge–cloud continuum. For every task–machine pair, let $\tau_{j\mu}^{\text{in}}$ be the time needed to make the input available at machine μ , and let $\tau_{j\mu}^{\text{out}}$ be the time needed to return the result to its consumer. Execution on machine μ must therefore lie in the effective window

$$r_{j\mu} = r_j + \tau_{j\mu}^{\text{in}}, \quad d_{j\mu} = d_j - \tau_{j\mu}^{\text{out}}. \quad (7.1)$$

If the output size is negligible, $\tau_{j\mu}^{\text{out}}$ may contain only propagation and fixed protocol delays; it can be set to zero when return delay is negligible. A pair with $r_{j\mu} \geq d_{j\mu}$ is ineligible.

The delays in (7.1) are fixed or predicted input parameters. Thus, the model captures the effect of communication on computation placement, but it does not schedule transfers or model contention among them. It also directly accommodates effective release times and deadlines obtained by other means: the algorithm only requires the windows $[r_{j\mu}, d_{j\mu}]$.

Let $\mathcal{B} = \{r_{j\mu}, d_{j\mu} : j \in J, \mu \in M\}$ be the set of all effective window endpoints. Sorting the distinct points in $\mathcal{B}$ partitions the time horizon into consecutive elementary intervals Γ . We denote the duration of $I \in \Gamma$ by D_I and define

$$\Gamma_{j\mu} = \{I \in \Gamma : I \subseteq [r_{j\mu}, d_{j\mu}]\}.$$

Thus, $\Gamma_{j\mu}$ is the set of elementary intervals in which task j may execute on machine μ . There are at most $2nm$ effective endpoints, and hence $|\Gamma| \leq 2nm - 1$. When every task has the same window on all machines, this improves to $|\Gamma| \leq 2n - 1$. Because no window endpoint lies inside an elementary interval, task j is either eligible on machine μ throughout I , or ineligible throughout it.

Each task represents an inference request for a compressible model, such as Once-For-All (Cai et al., 2020). The computation devoted to task j is a workload $f_j \in [0, \bar{f}_j]$, where $\bar{f}_j$ is the maximum useful workload, and its accuracy is $a_j(f_j)$. Following prior work (Barros et al., 2024; Silva Barros et al., 2024), we model this workload–accuracy trade-off as a nonnegative, nondecreasing, concave piecewise-linear function, capturing diminishing returns of additional computation for compressible models. Equivalently, for a finite set L_j of affine pieces, $a_j(f) = \min_{\ell \in L_j} \{\alpha_{j\ell}f + \beta_{j\ell}\}$ on $[0, \bar{f}_j]$.

A feasible (integral) schedule assigns each task receiving positive processing to one machine. Processing may be preempted and resumed on that machine, but it may neither migrate between machines nor execute on several machines simultaneously. The workload completed in t units of processing time on machine μ is $s_\mu t$. The objective is to maximize the sum of the tasks’ achieved accuracies.

7.2.2 Exact Integral Formulation

Binary variable $x_{j\mu}$ indicates whether task j is assigned to machine μ . Continuous variable $t_{j\mu I}$ is its processing time on that machine during interval I , f_j is its total completed workload, and z_j is its achieved accuracy. The exact mixed-integer linear formulation is

$$\max \sum_{j \in J} z_j \tag{7.2a}$$

$$\text{s.t. } f_j = \sum_{\mu \in M} \sum_{I \in \Gamma} s_\mu t_{j\mu I}, \quad j \in J, \tag{7.2b}$$

$$z_j \leq \alpha_{j\ell} f_j + \beta_{j\ell}, \quad j \in J, \ell \in L_j, \tag{7.2c}$$

$$0 \leq f_j \leq \bar{f}_j, \quad j \in J, \tag{7.2d}$$

$$t_{j\mu I} = 0, \quad j \in J, \mu \in M, I \notin \Gamma_{j\mu}, \tag{7.2e}$$

$$t_{j\mu I} \leq D_I x_{j\mu}, \quad j \in J, \mu \in M, I \in \Gamma, \tag{7.2f}$$

$$\sum_{j \in J} t_{j\mu I} \leq D_I, \quad \mu \in M, I \in \Gamma, \tag{7.2g}$$

$$\sum_{\mu \in M} x_{j\mu} \leq 1, \quad j \in J, \tag{7.2h}$$

$$x_{j\mu} \in \{0, 1\}, t_{j\mu I}, z_j \geq 0. \tag{7.2i}$$

Constraint (7.2b) converts processing time into FLOPs, while (7.2c) represents the hypograph of the concave piecewise-linear accuracy function. Constraint (7.2d) limits useful computation. The effective windows are enforced by (7.2e), while (7.2g) limits the workload assigned to machine μ during interval I to its capacity $s_\mu D_I$, equivalently requiring its total processing time not to exceed D_I . Constraints (7.2f) and (7.2h) together ensure that every processed task uses a single machine. We use an inequality in (7.2h) because a task receiving no processing need not select a machine; requiring equality would not change the optimum.

Proposition 7.2.1. *Program (7.2) exactly represents the feasible preemptive schedules described above.*

Proof.

A feasible schedule induces a solution by setting $x_{j\mu} = 1$ for the machine used by task j and $t_{j\mu I}$ to its execution time in interval I , and by setting f_j and z_j to its workload and accuracy; all constraints then follow from schedule feasibility. Conversely, (7.2f)–(7.2h) confine

every task’s positive processing variables to one machine, (7.2e) ensures eligibility, and (7.2g) allows the processing assigned to each machine–interval pair to be sequenced within that interval. Concatenating these interval schedules gives a feasible preemptive schedule with the same workloads and objective value. $\square$

7.3 Fractional Relaxation and Rounding

Although (7.2) is exact, it is a mixed-integer linear program. Solving such programs is NP-hard in general, which can make the exact solution impractical at scale. We therefore propose a relax-and-round algorithm. It first removes the single-machine requirement and solves the resulting assignment-free linear program, which is polynomial-time solvable and may split a task across machines. The rounding stage then selects one machine for each task and, in every interval, retains as much of its fractional workload as possible.

7.3.1 Assignment-Free Fractional Relaxation

We completely remove the assignment variables $x_{j\mu}$, together with the linking and assignment Constraints (7.2f) and (7.2h). Let $t_{j\mu I}^f$ denote the processing time allocated to task j on machine μ during interval I by this fractional relaxation. The relaxation is

$$\max \sum_{j \in J} z_j \tag{7.3a}$$

$$\text{s.t. } f_j = \sum_{\mu \in M} \sum_{I \in \Gamma} s_{\mu} t_{j\mu I}^f, \quad j \in J, \tag{7.3b}$$

$$z_j \leq \alpha_{j\ell} f_j + \beta_{j\ell}, \quad j \in J, \ell \in L_j, \tag{7.3c}$$

$$0 \leq f_j \leq \bar{f}_j, \quad j \in J, \tag{7.3d}$$

$$t_{j\mu I}^f = 0, \quad j \in J, \mu \in M, I \notin \Gamma_{j\mu}, \tag{7.3e}$$

$$\sum_{j \in J} t_{j\mu I}^f \leq D_I, \quad \mu \in M, I \in \Gamma, \tag{7.3f}$$

$$\sum_{\mu \in M} t_{j\mu I}^f \leq D_I, \quad j \in J, I \in \Gamma, \tag{7.3g}$$

$$t_{j\mu I}^f \geq 0, \quad z_j \geq 0. \tag{7.3h}$$

Constraint (7.3g) is redundant in the integral formulation: it follows from (7.2f) and (7.2h). It must be stated explicitly after those constraints are removed. Without it, the relaxation could allocate a full interval of processing time to the same task on every machine simultaneously. The fractional model still relaxes the integral one: it allows migration between machines, including using different machines in different intervals, but it does not allow a task to accumulate more than one interval’s worth of processing time during a single interval.

Every feasible integral solution yields a feasible solution of (7.3) by setting $t^f = t$ and discarding x . Consequently, the fractional optimum OPT_f is an upper bound on the integral optimum OPT . Program (7.3) is a linear program and can be solved directly. Section 7.4 gives equivalent min-cost-flow formulation under common execution windows and briefly discusses possible extensions. This specialized representation is not required by the rounding procedure.

Require: Fractional solution of (7.3)

Ensure: Feasible single-machine workload vectors

- 1: Compute f_{jI}, f_j, δ_j ; sort tasks by nonincreasing δ_j
- 2: $c_{\mu I} \leftarrow s_\mu D_I$ for every μ, I
- 3: **for** each task j in density order **do**
- 4: $e_j^\mu \leftarrow \sum_I (f_{jI} - \chi_{j\mu I} c_{\mu I})_+^\mu \quad (\mu \in M)$
- 5: Choose $\mu_j \in \arg \min_{\mu \in M} e_j^\mu$
- 6: **for** each $I \in \Gamma$ **do**
- 7: $\hat{f}_{jI} \leftarrow \chi_{j\mu_j I} \min\{f_{jI}, c_{\mu_j I}\}$
- 8: $c_{\mu_j I} \leftarrow c_{\mu_j I} - \hat{f}_{jI}$
- 9: **return** μ_j and $\hat{t}_{j\mu_j I} = \hat{f}_{jI}/s_{\mu_j}$

Algorithm 7.1 – Eligibility-aware minimum-discard rounding

7.3.2 Eligibility-Aware Minimum-Discard Rounding

For task j , let $f_{jI} = \sum_{\mu \in M} s_\mu t_{j\mu I}^f$ be the workload assigned by the fractional solution in interval I , and let $f_j = \sum_{I \in \Gamma} f_{jI}$ be its total workload. Thus, the fractional solution associates task j with a workload vector $(f_{jI})_{I \in \Gamma}$. This vector records when its computation is performed, but not the fractional machine decomposition used to obtain it. The rounding procedure assigns this entire vector to one candidate machine and clips it against that machine's eligibility and residual capacity.

The tasks are considered in nonincreasing average accuracy density $\delta_j = (a_j(f_j) - a_j(0))/f_j$ for $f_j > 0$; we set $\delta_j = 0$ when $f_j = 0$. When the accuracy functions are normalized by $a_j(0) = 0$, this is simply $a_j(f_j)/f_j$. This order prioritizes tasks whose fractional allocation provides the largest average accuracy improvement per FLOP, so that scarce machine capacity is first offered to the most valuable workload.

Machine μ has FLOP capacity $C_{\mu I} = s_\mu D_I$ in interval I . Let $\chi_{j\mu I}$ be one when $I \in \Gamma_{j\mu}$, and zero otherwise. Algorithm 7.1 summarizes the rounding procedure. It constructs the workload vectors, orders the tasks, and initializes every machine–interval capacity. For each task, it computes how much of its vector would be discarded on every candidate machine, selects a minimizing machine, retains the part that is eligible and fits, and updates the corresponding residual capacities.

For the analysis, let $c_{\mu I}^{(j-1)}$ denote the residual capacity just before task j , with $c_{\mu I}^{(0)} = C_{\mu I}$; the pseudocode updates this quantity in place and omits the superscript. For each candidate machine, e_j^μ is the workload that would be discarded if j were assigned there. Since f_j is fixed, minimizing e_j^μ is equivalent to maximizing retained workload; ties may be broken arbitrarily.

After choosing μ_j , only the retained workload $\hat{f}_{jI}$ computed by the update loop reduces residual capacity. Thus, an ineligible interval retains nothing and consumes no capacity, while capacities on unselected machines are unchanged.

The running time is $O(n \log n + nm|\Gamma|)$. By the bounds in Section 7.2.1, this is $O(n^2 m^2)$ in general and $O(n^2 m)$ for machine-independent windows, and can be smaller when window endpoints coincide.

Feasibility follows directly: every task uses one machine, eligibility removes processing outside its window, and clipping enforces residual capacities. Thus, $\hat{t}_{j\mu_j I} = \hat{f}_{jI}/s_{\mu_j}$ satisfies the assignment, window, and capacity constraints of (7.2).

7.4 Min-Cost-Flow Formulations of the Fractional Relaxation

The general fractional relaxation (7.3) can always be solved as a linear program. This section gives an equivalent min-cost-flow formulation for heterogeneous machines with common execution windows, and then describes two extensions: arbitrary destination-dependent windows on identical-speed machines, and speed-aligned nested eligibility on heterogeneous machines.

Figure 7.1 provides a running example of the complete construction for two tasks, two intervals, and two heterogeneous machines. The network can be read from left to right: flow passes from the source through a task’s accuracy gadget, then through an eligible interval–layer node, and finally to the sink. We introduce these components in that order.

All flows are measured in FLOPs. Let q and t be the source and sink, respectively, and let $\bar{F} = \sum_{j \in J} \bar{f}_j$. We prescribe a flow of value $\bar{F}$ that must be routed from q to t . A direct bypass arc (q, t) , with capacity $\bar{F}$ and cost zero, represents potential workload that is not scheduled.

Accuracy and interval networks For every task j , introduce nodes u_j and v_j , and add an arc (q, u_j) of capacity $\bar{f}_j$. Write the breakpoints and nonincreasing slopes of its piecewise-linear concave accuracy function as $0 = p_{j,0} < \dots < p_{j,k_j} = \bar{f}_j$, with $\alpha_{j,1} \geq \dots \geq \alpha_{j,k_j} \geq 0$. For each segment ℓ , let $\Delta p_{j,\ell} = p_{j,\ell} - p_{j,\ell-1}$ and add a parallel arc from u_j to v_j with capacity $\Delta p_{j,\ell}$ and cost $-\alpha_{j,\ell}$. The nodes u_j, v_j and these parallel arcs form task j ’s accuracy gadget. The negative cost turns accuracy maximization into cost minimization. Earlier segments have larger slopes and therefore more negative, lower-cost arcs. A minimum-cost flow consequently fills them before later segments. Sending f_j units through the gadget then incurs cost $-(a_j(f_j) - a_j(0))$.

We first assume that execution windows are machine-independent, so $\Gamma_{j\mu} = \Gamma_j$ for every task j and machine μ . Relabel the machines in nonincreasing speed order, so that $s_1 \geq \dots \geq s_m > 0$, set $s_{m+1} = 0$, and define $\delta_r = s_r - s_{r+1}$ for $r = 1, \dots, m$. To visualize this decomposition, draw one vertical bar of height s_μ for each machine, ordered from fastest to slowest, and cut the bars horizontally at the distinct speed levels. With two machines, the lower strip has height s_2 and spans both bars, while the upper strip has height $s_1 - s_2$ and appears only in the first bar. Equivalently, $\delta_1 = s_1 - s_2$, $\delta_2 = s_2$, and hence $s_1 = \delta_1 + \delta_2$ and $s_2 = \delta_2$. We call the upper strip *layer 1* and the lower strip *layer 2*. Thus the layer index records how many machines contain that rate increment; a layer is not a separate physical processor.

In general, layer r is the strip between heights s_{r+1} and s_r . It has height δ_r and appears on exactly the r fastest machines. Machine μ is obtained by stacking layers $\mu, \dots, m$, and hence $s_\mu = \sum_{r=\mu}^m \delta_r$, as illustrated by the two-machine decomposition above. During an interval of duration D_I , each of the r copies of layer r contributes $\delta_r D_I$ FLOPs, for a total layer capacity of $r\delta_r D_I$.

For every interval I and every r with $\delta_r > 0$, introduce a layer node w_{rI} . For each task j available throughout I , add (v_j, w_{rI}) with capacity $\delta_r D_I$, and add (w_{rI}, t) with capacity $r\delta_r D_I$. All these arcs have cost zero. In the running example, task 1 is available in both intervals and task 2 only in I_2 . For clarity, the figure takes $k_1 = k_2 = 2$; the general construction contains one parallel accuracy arc for every segment.

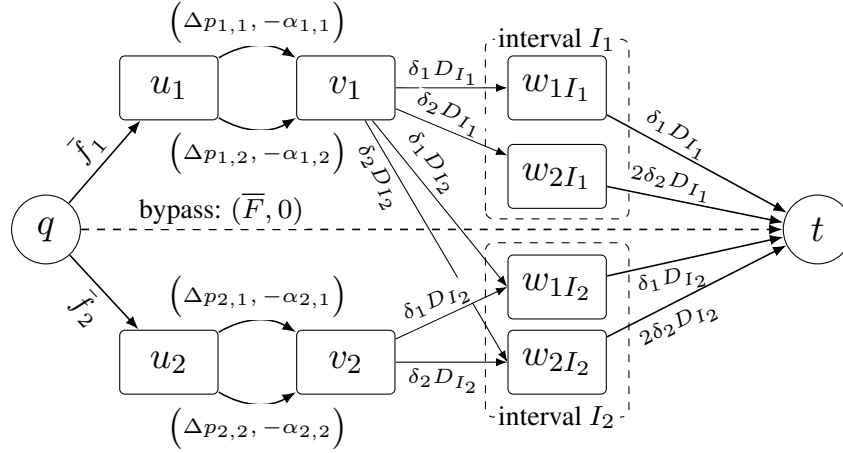


Figure 7.1 – Complete min-cost-flow network for two tasks with a two-segment accuracy function, two intervals, and two heterogeneous-speed machines. Labels are of the form $(\text{capacity}, \text{cost})$, and cost is 0 when only one value is specified. Task 1 is available in both intervals, task 2 only in I_2 .

Why the layer capacities are exact Let x_{jrI} denote the flow on (v_j, w_{rI}) . The workload assigned to task j in interval I is $f_{jI} = \sum_r x_{jrI}$, and the two arc capacities impose $x_{jrI} \leq \delta_r D_I$ and $\sum_j x_{jrI} \leq r\delta_r D_I$. The first bound prevents one task from using several copies of a speed increment simultaneously; the second is the total amount of that increment on the r fastest machines. Consequently, for every set K of k tasks,

$$\sum_{j \in K} f_{jI} \leq D_I \sum_{r=1}^m \delta_r \min\{k, r\} = D_I \sum_{\mu=1}^{\min\{k, m\}} s_\mu. \quad (7.4)$$

These are exactly the interval majorization constraints induced by machine capacity and the prohibition on parallel execution of one task. Conversely, any workload vector satisfying them can be decomposed into the layer flows above and, interval by interval, into fractional machine times. Thus the network has exactly the feasible workload vectors of the fractional relaxation.

Special cases With one machine, $\delta_1 = s_1$, and the construction reduces to an interval capacity $s_1 D_I$; it therefore solves the preemptive single-machine problem itself. If all m machines have speed s , then $\delta_1 = \dots = \delta_{m-1} = 0$ and $\delta_m = s$. The only remaining layer has per-task capacity $s D_I$ and total capacity $ms D_I$, giving the identical-speed fractional relaxation.

Arbitrary windows and identical speeds When all machines have speed s , the min-cost-flow representation can also accommodate arbitrary destination-dependent windows. For each task–interval pair with nonempty eligibility, introduce a node y_{jI} and add a zero-cost arc (v_j, y_{jI}) of capacity $s D_I$. For every machine–interval pair, introduce a node $w_{\mu I}$, connect it to t with capacity $s D_I$, and add $(y_{jI}, w_{\mu I})$ with capacity $s D_I$ whenever $I \in \Gamma_{j\mu}$. The arc entering y_{jI} enforces the per-task Constraint (7.3g), the arc leaving $w_{\mu I}$ enforces the machine-capacity Constraint (7.3f), and the intermediate arcs encode eligibility. Flow $x_{j\mu I}$ on $(y_{jI}, w_{\mu I})$ corresponds to processing time $t_{j\mu I}^f = x_{j\mu I}/s$. Thus this network is equivalent to (7.3) for arbitrary destination-dependent release times and deadlines.

Nested destination-dependent windows For heterogeneous speeds, the layer construction extends to destination-dependent windows when eligibility is nested consistently with the speed order. For a task j and interval I , let M_{jI} denote the task’s *eligibility set*, namely the set of machines on which j may execute during I . Assume that every nonempty eligibility set is a suffix $M_{jI} = \{h_{jI}, h_{jI} + 1, \dots, m\}$. For example, with machine-independent effective deadlines, the release-time order $r_{j1} \geq r_{j2} \geq \dots \geq r_{jm}$ is sufficient. The suffix condition is more general because it is stated directly in terms of the effective release times and deadlines. Partition the tasks into blocks $B_{hI} = \{j : M_{jI} = \{h, h + 1, \dots, m\}\}$. Thus B_{hI} groups the tasks eligible in I exactly on machines $h, \dots, m$. For intuition, fix I and layer r , whose r copies lie on machines $1, \dots, r$. A task in B_{hI} can use precisely the copies on machines $h, \dots, r$ when $h \leq r$. Thus B_{rI} can use one copy, $B_{r-1,I}$ two, and so on, while B_{1I} can use all r .

Create w_{hrI} for every $h \leq r$, connect v_j to w_{hrI} for every $j \in B_{hI}$ with capacity $\delta_r D_I$, and chain

$$w_{rrI} \longrightarrow w_{r-1,r,I} \longrightarrow \dots \longrightarrow w_{1rI} \longrightarrow t$$

for each r . Give the arc leaving w_{hrI} capacity $(r - h + 1)\delta_r D_I$, where for $h = 1$ this is the arc to t . If X_{hrI} is the layer- r flow entering from block h , this arc carries the flow of blocks $h, \dots, r$ and hence enforces

$$\sum_{g=h}^r X_{grI} \leq (r - h + 1)\delta_r D_I. \quad (7.5)$$

This bounds their demand by the capacity of the $r - h + 1$ copies on machines $h, \dots, r$. Since the accessible copy sets are nested, these cumulative bounds give exactly the required fractional feasibility conditions. This alignment is essential: arbitrary machine-dependent eligibility cannot in general be represented by independent speed layers. Such instances remain covered by the linear program (7.3).

Algorithmic consequence The networks are acyclic and have a prescribed flow value, so negative costs do not make the objective unbounded; a polynomial-time min-cost-flow algorithm computes an optimal fractional solution. The basic network has one node per interval–layer pair and one arc per admissible task–interval–layer triple. The identical-speed extension has $O((n+m)|\Gamma|)$ task–interval and machine–interval nodes. The nested extension adds at most $m(m+1)/2$ block–layer nodes and the same order of chain arcs per interval.

7.5 Theoretical Analysis

The analysis begins with the minimum-discard rule. For every density prefix, the workload discarded by the algorithm is bounded by the average candidate discarded workload. We decompose this average into *capacity overflow*, which does not fit on a candidate machine, and *eligibility loss*, which would fit but cannot be retained because the task is ineligible on that machine. The main technical step bounds the aggregate capacity overflow within each interval by a majorization potential. Summing these interval bounds gives retained-workload guarantees for every density prefix; density ordering then converts them into accuracy guarantees.

The general analysis allows destination-dependent execution windows. We later specialize it to common windows, where the eligibility term vanishes and stronger speed-dependent guarantees are possible. Throughout the section, machines are indexed by nonincreasing speed: $s_1 \geq s_2 \geq \dots \geq s_m > 0$. We write $s_\Sigma = \sum_{\mu=1}^m s_\mu$ and $C_{\mu I} = s_\mu D_I$.

7.5.1 From the Rounding Rule to Workload Loss

Recall that tasks are processed in nonincreasing density order, and reindex them as $1, \dots, n$ in that order. For the first k tasks, define their fractional workload in interval I , their total fractional workload, and their retained workload as

$$F_{kI} = \sum_{j=1}^k f_{jI}, \quad F_k = \sum_{I \in \Gamma} F_{kI}, \quad L_k = \sum_{j=1}^k \ell_j,$$

where $\ell_j = \sum_I \hat{f}_{jI}$. The prefix workload discarded by rounding is therefore $E_k = F_k - L_k$.

For candidate machine μ , Algorithm 7.1 defines

$$e_j^\mu = \sum_{I \in \Gamma} e_{jI}^\mu, \quad e_{jI}^\mu = (f_{jI} - \chi_{j\mu I} c_{\mu I}^{(j-1)})_+.$$

This interval-wise candidate discarded workload has a useful exact decomposition. Here $c_{\mu I}^{(j-1)}$ is the actual residual capacity produced by the algorithm: after selecting μ_j , only $c_{\mu_j I}$ is reduced, and it is reduced by the retained workload $\hat{f}_{jI}$. For a task j , candidate machine μ , and interval I , let

$$o_{j\mu I} = (f_{jI} - c_{\mu I}^{(j-1)})_+$$

be the *capacity overflow*: the part that would not fit even if the task were eligible. Let

$$\pi_{j\mu I} = (1 - \chi_{j\mu I}) \min\{f_{jI}, c_{\mu I}^{(j-1)}\}$$

be the *eligibility loss*: workload that would fit but cannot be retained because $I \notin \Gamma_{j\mu}$. Then

$$e_{jI}^\mu = o_{j\mu I} + \pi_{j\mu I}. \quad (7.6)$$

Indeed, when the task is eligible, only the overflow is discarded; when it is ineligible, the overflow and the otherwise fitting part sum to f_{jI} .

For a prefix, aggregate these quantities within each interval:

$$O_{I,k} = \sum_{j=1}^k \sum_{\mu=1}^m o_{j\mu I}, \quad \Pi_{I,k} = \sum_{j=1}^k \sum_{\mu=1}^m \pi_{j\mu I}.$$

The algorithm chooses a machine minimizing the total candidate discarded workload. Consequently,

$$\begin{aligned} E_k &= \sum_{j=1}^k \min_{\mu} e_j^\mu \\ &\leq \frac{1}{m} \sum_{j=1}^k \sum_{\mu=1}^m e_j^\mu = \frac{1}{m} \sum_{I \in \Gamma} (O_{I,k} + \Pi_{I,k}). \end{aligned} \quad (7.7)$$

Equivalently,

$$L_k \geq F_k - \frac{1}{m} \sum_{I \in \Gamma} (O_{I,k} + \Pi_{I,k}). \quad (7.8)$$

This is the basic connection between the rounding rule and the theoretical analysis. Everything in the next subsection is devoted to upper-bounding the aggregate capacity-overflow term $O_{I,k}$. Under common windows, $\Pi_{I,k} = 0$, so capacity overflow is the only source of candidate discarded workload.

We first record the structural property inherited from the fractional relaxation that makes such a bound possible.

Lemma 7.5.1 (Interval majorization). *For every interval I and every set K of $h \leq m$ tasks,*

$$\sum_{j \in K} f_{jI} \leq D_I \sum_{\mu=1}^h s_{\mu}.$$

Moreover, $F_{kI} \leq s_{\Sigma} D_I$. Thus, in each interval, the largest task workloads are no larger in aggregate than the corresponding largest machine capacities.

Proof.

Set $x_{j\mu I} = t_{j\mu I}^f / D_I$. By (7.3f), each machine receives total weight at most one from the tasks in K ; by (7.3g), their combined weight over all machines is at most h . Hence

$$\sum_{j \in K} f_{jI} = D_I \sum_{\mu=1}^m s_{\mu} \sum_{j \in K} x_{j\mu I} \leq D_I \sum_{\mu=1}^h s_{\mu},$$

because the weighted sum is maximized by assigning unit weight to the h fastest machines. Summing (7.3f) over all machines gives $F_{kI} \leq s_{\Sigma} D_I$. $\square$

7.5.2 Capacity Overflow in One Interval

We now introduce a separate residual-capacity process used only for the analysis. Fix one interval and suppress its index. Let $C = (C_1, \dots, C_m)$, with $C_1 \geq \dots \geq C_m$, be its capacity vector, and let $\mathbf{f} = (f_1, \dots, f_h)$ be the task workloads in that interval. Consider any sequence of selected machines $\boldsymbol{\mu} = (\mu_1, \dots, \mu_h)$. Starting from $v^{(0)} = C$, define the virtual residual capacities by

$$v_{\mu_j}^{(j)} = (v_{\mu_j}^{(j-1)} - f_j)_+, \quad v_{\mu}^{(j)} = v_{\mu}^{(j-1)} \quad (\mu \neq \mu_j).$$

Unlike the actual update of Algorithm 7.1, the virtual process ignores eligibility. It applies the clipped update above even when the task is ineligible on the selected machine; in that case, the actual process retains no workload and leaves the corresponding residual capacity unchanged. When the task is eligible, the actual and virtual updates have the same clipped form.

The aggregate capacity overflow of this process is

$$\mathcal{O}(\mathbf{f}, C; \boldsymbol{\mu}) = \sum_{j=1}^h \sum_{\mu=1}^m (f_j - v_{\mu}^{(j-1)})_+. \quad (7.9)$$

At each step, $\mathcal{O}$ sums the overflow that the current task would incur on *every* candidate machine, although only μ_j is updated.

To connect this one-interval notation with $O_{I,k}$, fix an interval I and prefix k , and set $\mathbf{f}_{I,k} = (f_{1I}, \dots, f_{kI})$ and $\boldsymbol{\mu}_k = (\mu_1, \dots, \mu_k)$. If every task is eligible on its selected machine during I , then $\hat{f}_{jI} = \min\{f_{jI}, c_{\mu_{jI}}^{(j-1)}\}$. The actual and virtual updates therefore coincide, giving

$$O_{I,k} = \mathcal{O}(\mathbf{f}_{I,k}, C_I; \boldsymbol{\mu}_k).$$

This equality holds, in particular, under common windows. With destination-dependent windows, an ineligible placement leaves the actual residual unchanged but reduces the virtual residual. Hence the virtual residuals are no larger than the actual ones, and $O_{I,k} \leq \mathcal{O}(\mathbf{f}_{I,k}, C_I; \boldsymbol{\mu}_k)$.

Consequently, an upper bound on $\mathcal{O}$ also bounds $O_{I,k}$. Through (7.7), it therefore controls the capacity-overflow component of the workload discarded by the rounding algorithm.

Deriving such a bound requires more information than the total residual capacity. Two placements can leave the same total capacity distributed very differently across machines and therefore produce different overflow for subsequent tasks. We capture this distribution, together with the sizes of the remaining tasks, through largest-prefix sums. We first define these prefix sums and a trimming operation, and then use them to construct the quantity that bounds the accumulated overflow.

Prefix curves and trimming For a nonnegative vector or multiset z , let $P_r(z)$ be the sum of its r largest entries, with $P_0(z) = 0$ and missing entries treated as zero. For finite nonnegative vectors or multisets x and y with the same total, we say that x majorizes y , written $x \succcurlyeq y$, if $P_r(x) \geq P_r(y)$ for every r , padding the shorter object with zeros when their lengths differ.

For a vector x satisfying $\sum_{\mu} x_{\mu} \geq M$, define

$$T_M(x)_{\mu} = \min\{x_{\mu}, \lambda\}, \quad \sum_{\mu=1}^m \min\{x_{\mu}, \lambda\} = M.$$

Thus, $T_M(x)$ caps the largest coordinates at a common level until its total becomes M . When only M workload remains, capacity beyond that total cannot help place the remaining tasks. Trimming removes this irrelevant spare capacity while preserving the shape that matters for future overflow.

Lemma 7.5.2 (Monotonicity of trimming). *If $x \leq y$ coordinatewise and $\sum_{\mu} x_{\mu} \geq M$, then $T_M(x) \succcurlyeq T_M(y)$.*

Proof.

Write $x' = T_M(x)$ and $y' = T_M(y)$, and let λ_x, λ_y be their capping levels. Since $x \leq y$, the smaller vector needs a weakly higher capping level to retain the same total: $\lambda_x \geq \lambda_y$.

For equal-total nonnegative vectors, $x' \succcurlyeq y'$ is equivalent to

$$\sum_{\mu} (x'_{\mu} - z)_{+} \geq \sum_{\mu} (y'_{\mu} - z)_{+} \quad \text{for every } z \geq 0.$$

The inequality is immediate when $z \geq \lambda_y$. If $z < \lambda_y$, then z lies below both capping levels and

$$\sum_{\mu} \min\{x'_{\mu}, z\} = \sum_{\mu} \min\{x_{\mu}, z\}$$

$$\begin{aligned}
&\leq \sum_{\mu} \min\{y_{\mu}, z\} \\
&= \sum_{\mu} \min\{y'_{\mu}, z\}.
\end{aligned}$$

Subtracting both sides from the common total M proves the claim. $\square$

The next lemma describes what happens when one task is removed from a trimmed residual vector. Its role is to compare the new residual-prefix curve with the old one without introducing unrelated auxiliary vectors.

Lemma 7.5.3 (Residual insertion). *Let $u, b \in \mathbb{R}_+^m$ have total S , where b is nonincreasing, $b \succcurlyeq u$, and $b_1 \geq f$. Choose a coordinate μ_0 , and let u' be obtained by placing a task of size f on that coordinate:*

$$u'_{\mu_0} = (u_{\mu_0} - f)_+, \quad u'_{\mu} = u_{\mu} \quad (\mu \neq \mu_0).$$

After retrimming, let $u'' = T_{S-f}(u')$. Then $(b, 0) \succcurlyeq (f, u'')$.

Proof.

Since $u'' \leq u' \leq u$, we already have $P_r(u'') \leq P_r(b)$. It remains to prove

$$f + P_r(u'') \leq P_{r+1}(b), \quad 0 \leq r \leq m-1. \quad (7.10)$$

If $u_{\mu_0} \geq f$, no second trimming is needed: $u'' = u'$. The vector (f, u') is obtained by splitting the coordinate u_{μ_0} into f and $u_{\mu_0} - f$. Splitting one coordinate cannot increase a largest-prefix sum, so (7.10) follows from $b \succcurlyeq u$.

Suppose instead that $u_{\mu_0} < f$, and set $\eta = f - u_{\mu_0}$. Retrimming removes mass η from u' . Let λ be the new capping level and $q = \#\{\mu : u'_{\mu} > \lambda\}$. Then $\eta = P_q(u') - q\lambda$, and therefore

$$f + q\lambda = P_q(u') + u_{\mu_0} \leq P_{q+1}(u) \leq P_{q+1}(b).$$

Together with $P_1(b) \geq f$, concavity of the prefix curve of b implies $P_{r+1}(b) \geq f + r\lambda$ for $0 \leq r \leq q$. In this range, the r largest coordinates of u'' equal λ , proving (7.10).

For $r > q$, all removed mass lies among the r largest coordinates of u' . Hence

$$\begin{aligned}
f + P_r(u'') &= f + P_r(u') - \eta \\
&= u_{\mu_0} + P_r(u') \\
&\leq P_{r+1}(u) \leq P_{r+1}(b).
\end{aligned}$$

$\square$

The least-concave-majorant potential Consider a state of the virtual process with residual vector v and multiset G of remaining task sizes. Let $S = \sum_{g \in G} g$. The invariant $S \leq \sum_{\mu} v_{\mu}$ holds initially and remains true after every clipped placement: the remaining workload decreases by f , while total residual capacity decreases by only $\min\{f, v_{\mu_0}\}$. Define the useful residual vector $u = T_S(v)$.

The remaining-task curve is

$$J_r(G) = \begin{cases} P_r(G), & 0 \leq r < m, \\ S, & r = m. \end{cases}$$

The value at m is the total remaining workload, even when G contains more than m tasks. For $0 \leq r \leq m$, set $H_r = \max\{P_r(u), J_r(G)\}$, and let

$$B = (B_0, \dots, B_m)$$

be the *least concave majorant* of H : the smallest sequence lying above H whose increments are nonincreasing. Its endpoints are $B_0 = 0$ and $B_m = S$. Define its increment vector $b = (b_1, \dots, b_m)$ by $b_r = B_r - B_{r-1}$. Concavity gives $b_1 \geq \dots \geq b_m \geq 0$. Hence $B_r = \sum_{q=1}^r b_q = P_r(b)$: B is exactly the prefix-sum curve of b .

We define the potential as the sum of the interior values of this common majorant:

$$\Phi(v, G) = \sum_{r=1}^{m-1} B_r = \sum_{r=1}^{m-1} P_r(b). \quad (7.11)$$

The endpoint B_0 is zero, while $B_m = S$ records only the total remaining workload. The interior values capture how that workload is distributed across the prefix ranks. The proof below shows that their sum decreases enough to pay for the aggregate candidate overflow.

Because $B_r \geq H_r$, the prefix sums of b dominate both curves: $P_r(b) \geq P_r(u)$ and $P_r(b) \geq J_r(G)$. Hence $b \succcurlyeq u$. The definition $J_m(G) = S = P_m(b)$ also completes the prefix comparison with the remaining tasks, so $b \succcurlyeq G$ under the zero-padding convention, even when G contains more than m tasks. The purpose of the least concave majorant is precisely to construct a single fictitious vector b that majorizes both objects relevant to future overflow.

Example Let $m = 4$, $v = (10, 1, 1, 1)$, and $G = \{3, 3, 2\}$. Here $S = 8$ and $u = T_8(v) = (5, 1, 1, 1)$. The curves defining the potential are

r	0	1	2	3	4
$P_r(u)$	0	5	6	7	8
$J_r(G)$	0	3	6	8	8
H_r	0	5	6	8	8
B_r	0	5	6.5	8	8

The pointwise maximum H is not concave because its increments are $(5, 1, 2, 0)$. Its least concave majorant has increments $(5, 1.5, 1.5, 0)$, and $\Phi(v, G) = 19.5$. If a task of size 3 is placed on the first machine, its aggregate candidate overflow is $(3 - 10)_+ + 3(3 - 1)_+ = 6$. The new potential is 13, so the drop 6.5 is sufficient to pay for this overflow. The theorem formalizes this behavior for every state and every selected machine.

Theorem 7.5.4 (Capacity-overflow bound). *Suppose the task sizes $f_1, \dots, f_h$ satisfy the interval-majorization inequalities of Lemma 7.5.1 with respect to C , and let $F = \sum_j f_j \leq \sum_\mu C_\mu$. For every task order and every sequence of selected machines,*

$$\mathcal{O}(\mathbf{f}, C; \boldsymbol{\mu}) \leq \Phi_0(\mathbf{f}, C), \quad (7.12)$$

where $\Phi_0(\mathbf{f}, C) = \Phi(C, \{f_1, \dots, f_h\})$.

Proof.

Consider one state (v, G) , let S be the remaining workload, and let $f \in G$ be the next task. Write $G' = G \setminus \{f\}$ and $u = T_S(v)$. Apply f to the selected coordinate of u , obtaining u' , and retrim to $u'' = T_{S-f}(u')$.

Let B be the least concave majorant defining $\Phi(v, G)$, and let $b_r = B_r - B_{r-1}$. Since B dominates the remaining-task curve, $b_1 = B_1 \geq f$. By Lemma 7.5.3, $(b, 0) \succcurlyeq (f, u'')$.

We now construct an explicit concave upper bound for the two curves in the auxiliary next state (u'', G') . Set $\tilde{B}_0 = 0$, $\tilde{B}_m = S - f$, and, for $r < m$,

$$\tilde{B}_r = \min\{B_r, B_{r+1} - f\}. \quad (7.13)$$

For the remaining tasks, removing f gives both $J_r(G') \leq B_r$ and $f + J_r(G') \leq B_{r+1}$. The residual-insertion lemma gives the same two inequalities with $J_r(G')$ replaced by $P_r(u'')$. Therefore, $\tilde{B}$ dominates both new curves.

To verify concavity, let $q = \max\{r : b_r \geq f\}$, which exists because $b_1 \geq f$. Equation (7.13) becomes

$$\tilde{B}_r = \begin{cases} B_r, & r < q, \\ B_{r+1} - f, & q \leq r < m. \end{cases}$$

Away from the transition, the increments are inherited from B . At the transition the increment is $b_q + b_{q+1} - f$, which lies below b_q and above b_{q+1} because $b_q \geq f > b_{q+1}$. If $q = m$, the last increment is $b_m - f \geq 0$. Thus, in every case, the increments of $\tilde{B}$ are nonincreasing.

Let B^{new} be the least concave majorant for (u'', G') . By its minimality, $B_r^{\text{new}} \leq \tilde{B}_r$, and hence

$$\begin{aligned} \Phi(v, G) - \Phi(u'', G') &\geq \sum_{r=q}^{m-1} (B_r - B_{r+1} + f) \\ &= \sum_{r=q+1}^m (f - b_r) = \sum_{r=1}^m (f - b_r)_+. \end{aligned} \quad (7.14)$$

Since $b \succcurlyeq u$ and $z \mapsto (f - z)_+$ is convex, Karamata's inequality yields

$$\Phi(v, G) - \Phi(u'', G') \geq \sum_{\mu=1}^m (f - u_\mu)_+.$$

It remains to compare the auxiliary update of u with the corresponding update of the untrimmed virtual residual v . Let v' be obtained by applying the same task to the same coordinate of v . Since $u \leq v$, we have $u' \leq v'$. By Lemma 7.5.2,

$$u'' = T_{S-f}(u') \succcurlyeq T_{S-f}(v').$$

The auxiliary residual curve therefore lies above the next virtual residual curve, which implies $\Phi(u'', G') \geq \Phi(v', G')$. Moreover, $\sum_{\mu} (f - v_\mu)_+ \leq \sum_{\mu} (f - u_\mu)_+$. Combining these facts gives

$$\sum_{\mu=1}^m (f - v_\mu)_+ \leq \Phi(v, G) - \Phi(v', G').$$

Apply this one-step inequality successively. The potential drops telescope, and the final potential is zero because no task remains:

$$\mathcal{O} \leq \sum_{j=1}^h (\Phi(v^{(j-1)}, G^{(j-1)}) - \Phi(v^{(j)}, G^{(j)})) = \Phi_0(\mathbf{f}, C). \quad \square$$

The potential is the strongest direct bound on aggregate capacity overflow furnished by this proof. It also admits simpler upper bounds that depend only on the total workload or only on the capacities. For $0 \leq F < \sum_{\mu} C_{\mu}$, uniquely write

$$F = \sum_{\mu=1}^q C_{\mu} + R, \quad 0 \leq R < C_{q+1},$$

where $q \in \{0, \dots, m-1\}$. When $F = \sum_{\mu} C_{\mu}$, set $q = m$ and $R = 0$. Define

$$\Psi_C(F) = \sum_{\mu=1}^q (m - \mu)C_{\mu} + (m - q - 1)R. \quad (7.15)$$

Corollary 7.5.5 (Closed capacity-overflow bounds). *Under the assumptions of Theorem 7.5.4,*

$$\mathcal{O}(\mathbf{f}, C; \boldsymbol{\mu}) \leq \Phi_0(\mathbf{f}, C) \leq \Psi_C(F) \leq \sum_{\mu=1}^m (m - \mu)C_{\mu}. \quad (7.16)$$

The last term is equivalently $\sum_{\mu < \nu} \max\{C_{\mu}, C_{\nu}\}$.

Proof.

Let

$$C^F = (C_1, \dots, C_q, R, 0, \dots, 0).$$

Its prefix curve is $P_r(C^F) = \min\{F, P_r(C)\}$. The interval-majorization inequalities and the total bound F imply that this curve dominates the initial remaining-task curve. It also dominates the prefix curve of $T_F(C)$. Since C^F is nonincreasing, its prefix curve is concave and is therefore an admissible majorant in the definition of Φ_0 . Consequently,

$$\Phi_0(\mathbf{f}, C) \leq \sum_{r=1}^{m-1} P_r(C^F) = \Psi_C(F).$$

Finally, $P_r(C^F) \leq P_r(C)$. Summing the prefix values of C counts C_{μ} exactly $m - \mu$ times, so

$$\sum_{r=1}^{m-1} P_r(C) = \sum_{\mu=1}^m (m - \mu)C_{\mu}. \quad \square$$

The three levels in (7.16) serve different purposes. The potential Φ_0 uses the actual task sizes and can be the sharpest. The function $\Psi_C(F)$ retains only the total workload F . The final capacity-only expression is the maximum value reached at full capacity.

7.5.3 General Destination-Dependent Certificates

We now apply the one-interval result to the actual rounding process. Fix an interval I and a prefix k , and instantiate the virtual process with $\mathbf{f}_{I,k}$, C_I , and $\boldsymbol{\mu}_k$. The actual residuals $c_{\mu I}^{(j)}$ and virtual residuals $v_{\mu I}^{(j)}$ both start from $C_{\mu I}$. On an eligible placement, both apply the clipped update

$x \mapsto (x - f_{jI})_+$; on an ineligible placement, the actual residual is unchanged while the virtual residual applies that update. Induction therefore gives

$$v_{\mu I}^{(j)} \leq c_{\mu I}^{(j)} \quad \text{for every } \mu, j.$$

It follows that the actual overflow in (7.7) is no larger than the aggregate capacity overflow of the virtual process:

$$O_{I,k} \leq \sum_{j=1}^k \sum_{\mu=1}^m (f_{jI} - v_{\mu I}^{(j-1)})_+.$$

The right-hand side is $\mathcal{O}(\mathbf{f}_{I,k}, C_I; \boldsymbol{\mu}_k)$. By interval majorization and Theorem 7.5.4, it is at most the initial potential obtained from C_I and $(f_{1I}, \dots, f_{kI})$. Denote this potential by $\Phi_{I,k}$.

Theorem 7.5.6 (General retained-workload certificate). *For every density prefix k ,*

$$L_k \geq F_k - \frac{1}{m} \sum_{I \in \Gamma} (\Phi_{I,k} + \Pi_{I,k}). \quad (7.17)$$

Proof.

Equation (7.8) gives

$$L_k \geq F_k - \frac{1}{m} \sum_I (O_{I,k} + \Pi_{I,k}).$$

The virtual-process argument above gives $O_{I,k} \leq \Phi_{I,k}$ for every interval. Substituting these interval bounds proves the theorem. $\square$

The eligibility term $\Pi_{I,k}$ uses the residual capacities observed during rounding, so (7.17) is an ex-post, instance-dependent certificate. A weaker term computable directly from the fractional profile is

$$\bar{\Pi}_{I,k} = \sum_{j=1}^k \sum_{\mu=1}^m (1 - \chi_{j\mu I}) f_{jI}.$$

Since $\pi_{j\mu I} \leq (1 - \chi_{j\mu I}) f_{jI}$, we have $\Pi_{I,k} \leq \bar{\Pi}_{I,k}$.

Corollary 7.5.7 (Explicit retained-workload certificate). *For every density prefix k ,*

$$L_k \geq F_k - \frac{1}{m} \sum_{I \in \Gamma} (\Psi_{C_I}(F_{kI}) + \bar{\Pi}_{I,k}).$$

Proof.

Apply $\Phi_{I,k} \leq \Psi_{C_I}(F_{kI})$ from Corollary 7.5.5 and $\Pi_{I,k} \leq \bar{\Pi}_{I,k}$ to (7.17). $\square$

7.5.4 From Retained Workload to Accuracy

Prefix workload bounds are useful because tasks were sorted by nonincreasing average accuracy density. Let

$$A_{\text{base}} = \sum_{j=1}^n a_j(0), \quad SOL = \sum_{j=1}^n a_j(\ell_j),$$

and recall that $\delta_j = (a_j(f_j) - a_j(0))/f_j$ for $f_j > 0$. Thus,

$$OPT_f = A_{\text{base}} + \sum_{j=1}^n \delta_j f_j. \quad (7.18)$$

Concavity of a_j implies $a_j(\ell_j) - a_j(0) \geq \delta_j \ell_j$ for $0 \leq \ell_j \leq f_j$. Therefore,

$$SOL \geq OPT_f - \sum_{j=1}^n \delta_j (f_j - \ell_j). \quad (7.19)$$

Set $\delta_{n+1} = 0$. Since $E_k = \sum_{j=1}^k (f_j - \ell_j)$, summation by parts gives

$$\sum_{j=1}^n \delta_j (f_j - \ell_j) = \sum_{k=1}^n (\delta_k - \delta_{k+1}) E_k. \quad (7.20)$$

All coefficients on the right are nonnegative. Consequently, any upper bound on every prefix loss E_k can be substituted directly into (7.20).

Theorem 7.5.8 (General accuracy certificate). *The rounded solution satisfies*

$$SOL \geq OPT_f - \frac{1}{m} \sum_{k=1}^n (\delta_k - \delta_{k+1}) \sum_{I \in \Gamma} (\Phi_{I,k} + \Pi_{I,k}). \quad (7.21)$$

The same statement holds with $\Phi_{I,k} + \Pi_{I,k}$ replaced by $\Psi_{C_I}(F_{kI}) + \bar{\Pi}_{I,k}$.

Proof.

Rearranging (7.17) gives the explicit prefix-loss bound

$$E_k = F_k - L_k \leq \frac{1}{m} \sum_I (\Phi_{I,k} + \Pi_{I,k}).$$

Substitute this inequality into (7.20), using $\delta_k - \delta_{k+1} \geq 0$, and then substitute the result into (7.19). This proves (7.21). The explicit version follows in the same way from Corollary 7.5.7. $\square$

One multiplicative guarantee is available even with destination-dependent windows when some machine is eligible whenever any other machine is.

Proposition 7.5.9 (Fallback-machine guarantee). *Suppose there is a machine μ_0 such that $\Gamma_{j\nu} \subseteq \Gamma_{j\mu_0}$ for every task j and machine ν . Let $\rho_0 = s_{\mu_0}/s_\Sigma$. Then*

$$L_k \geq \rho_0 F_k \quad \text{for every density prefix } k,$$

and $SOL \geq \rho_0 OPT_f$.

Proof.

Whenever $f_{jI} > 0$, the fractional solution uses at least one eligible machine in interval I ; by assumption, task j is then also eligible on μ_0 . The algorithm retains at least the candidate workload on that machine.

Fix I , and let $C_0 = s_{\mu_0} D_I$. Before task j , the workload already retained on μ_0 is at most $F_{j-1,I}$. Its candidate retained workload is therefore at least

$$\min\{f_{jI}, (C_0 - F_{j-1,I})_+\} = \min\{C_0, F_{jI}\} - \min\{C_0, F_{j-1,I}\}.$$

Summing over $j \leq k$ telescopes to $\min\{C_0, F_{kI}\}$. Since $F_{kI} \leq s_{\Sigma} D_I$,

$$\min\{s_{\mu_0} D_I, F_{kI}\} \geq \rho_0 F_{kI}.$$

Summing over intervals gives $L_k \geq \rho_0 F_k$.

Finally, summation by parts also gives $\sum_j \delta_j \ell_j = \sum_k (\delta_k - \delta_{k+1}) L_k \geq \rho_0 \sum_k (\delta_k - \delta_{k+1}) F_k = \rho_0 (OPT_f - A_{\text{base}})$. Together with $SOL \geq A_{\text{base}} + \sum_j \delta_j \ell_j$ and $A_{\text{base}} \geq 0$, this implies $SOL \geq \rho_0 OPT_f$. $\square$

7.5.5 Specialization to Common Windows

Suppose for the remainder of this subsection that task windows are machine-independent: $\Gamma_{j\mu} = \Gamma_j$ for every j, μ . Whenever $f_{jI} > 0$, task j is then eligible on every machine. Thus, $\Pi_{I,k} = 0$, the actual and virtual residual processes coincide, and the general retained-workload certificates simplify to

$$L_k \geq F_k - \frac{1}{m} \sum_I \Phi_{I,k}, \quad (7.22)$$

$$L_k \geq F_k - \frac{1}{m} \sum_I \Psi_{C_I}(F_{kI}). \quad (7.23)$$

The first bound uses the actual interval workload profiles and is the strongest direct consequence of the majorization potential. The second is a closed form depending only on the total prefix workload in each interval.

7.5.5.1 Fastest-machine guarantee

Under common windows, the fastest machine is an eligible fallback machine.

Corollary 7.5.10 (Fastest-machine guarantee). *Every density prefix satisfies*

$$L_k \geq \frac{s_1}{s_{\Sigma}} F_k,$$

and the rounded solution satisfies

$$SOL \geq \frac{s_1}{s_{\Sigma}} OPT_f.$$

Proof.

Apply Proposition 7.5.9 with $\mu_0 = 1$. $\square$

7.5.5.2 Identical speeds

When all machines have the same speed, averaging candidate retention gives a stronger ratio. The key is that, in any interval, the first unit of fractional workload can fit on all m candidate machines, the next unit can be guaranteed on $m - 1$, and so on. The following staircase function records these decreasing multiplicities. For $0 \leq p \leq m$, let

$$Q(p) = \sum_{r=1}^{\lfloor p \rfloor} (m - r + 1) + (m - \lfloor p \rfloor)(p - \lfloor p \rfloor).$$

Thus, Q is piecewise linear with successive slopes $m, m - 1, \dots, 1$; in particular, $Q(0) = 0$ and $Q(m) = m(m + 1)/2$.

Theorem 7.5.11 (Homogeneous-speed approximation). *If all m machines have the same speed and task windows are machine-independent, then*

$$SOL \geq \frac{m + 1}{2m} OPT_f.$$

Proof.

We first establish the prefix retained-workload bound needed for the accuracy argument. Fix an interval I , let $\bar{C}_I = sD_I$ be the capacity of one machine, and normalize all workloads and residual capacities by $\bar{C}_I$. Write

$$x_j = \frac{f_{jI}}{\bar{C}_I}, \quad p_j = \sum_{h=1}^j x_h.$$

Fractional feasibility gives $0 \leq x_j \leq 1$ and $p_k \leq m$.

Immediately before task j , the total normalized residual capacity is at least $m - p_{j-1}$, because previous tasks have retained at most their fractional workload. For fixed x_j , the aggregate candidate retained workload is $\sum_{\mu} \min\{x_j, c_{\mu}^{(j-1)}\}$. This concave symmetric function is minimized, subject to $0 \leq c_{\mu}^{(j-1)} \leq 1$ and the given total residual, when the residual mass is concentrated in as few machines as possible.

Write $p_{j-1} = q + r$, where $q = \lfloor p_{j-1} \rfloor$ and $0 \leq r < 1$. The worst residual vector therefore has $m - q - 1$ entries equal to one, one entry equal to $1 - r$, and q zero entries. Hence

$$\begin{aligned} \sum_{\mu=1}^m \min\{x_j, c_{\mu}^{(j-1)}\} &\geq (m - q - 1)x_j + \min\{x_j, 1 - r\} \\ &= Q(p_j) - Q(p_{j-1}). \end{aligned} \tag{7.24}$$

Summing over $j \leq k$ telescopes to $Q(p_k)$. Since Q is concave, it lies above the chord joining its endpoints:

$$Q(p_k) \geq \frac{Q(m)}{m} p_k = \frac{m + 1}{2} p_k.$$

Returning to the original scale and summing over intervals gives

$$\sum_{j=1}^k \sum_{\mu=1}^m g_j^{\mu} \geq \frac{m + 1}{2} F_k.$$

The algorithm retains $\ell_j = \max_{\mu} g_j^{\mu} \geq m^{-1} \sum_{\mu} g_j^{\mu}$. Therefore,

$$L_k \geq \frac{m+1}{2m} F_k \quad \text{for every density prefix } k.$$

Applying summation by parts exactly as in the fallback-machine proof yields $SOL \geq A_{\text{base}} + \frac{m+1}{2m}(OPT_f - A_{\text{base}})$, which implies the stated bound. $\square$

For comparison, with identical speeds but destination-dependent windows, the staircase argument still applies before eligibility masking. The masked candidate retention is smaller by exactly the eligibility term, so

$$L_k \geq \frac{m+1}{2m} F_k - \frac{1}{m} \sum_I \Pi_{I,k}. \quad (7.25)$$

Thus, arbitrary destination-dependent eligibility does not preserve the pure homogeneous ratio unless its penalty can also be controlled.

7.5.5.3 Heterogeneous speeds and full capacity

This final part continues to assume common windows. We derive from (7.23) a closed bound that depends only on the total fractional workload, rather than on its distribution over intervals. Let

$$\mathcal{H} = \sum_{I \in \Gamma} D_I.$$

For a fixed interval, $\Psi_{C_I}(x)$ has marginal slope $m-1$ over its first capacity block of length $s_1 D_I$, slope $m-2$ over the next block of length $s_2 D_I$, and so on, ending with slope zero over the block of length $s_m D_I$.

To upper-bound $\sum_I \Psi_{C_I}(F_{kI})$ for a fixed total $\sum_I F_{kI} = F_k$, fill the blocks with the largest marginal slopes first. Across all intervals, the slope- $(m-\mu)$ blocks have combined length $s_{\mu} \mathcal{H}$. Therefore, if

$$F_k = \mathcal{H} \sum_{\mu=1}^q s_{\mu} + R, \quad 0 \leq R < \mathcal{H} s_{q+1},$$

with $q = m$, $R = 0$ at total capacity, then

$$\sum_I \Psi_{C_I}(F_{kI}) \leq \mathcal{H} \sum_{\mu=1}^q (m-\mu) s_{\mu} + (m-q-1)R. \quad (7.26)$$

Substituting (7.26) into (7.23) and collecting the coefficients of F_k gives

$$L_k \geq \frac{1}{m} \left(\mathcal{H} \sum_{\mu=1}^q \mu s_{\mu} + (q+1)R \right). \quad (7.27)$$

For example, if $F_k < \mathcal{H} s_1$, then $q = 0$, $R = F_k$, and this bound reduces to $L_k \geq F_k/m$. This explains why the closed largest-capacities-first estimate can be weak at small workload.

Corollary 7.5.12 (Full-capacity retained workload). *Suppose the fractional solution uses all aggregate capacity: $F_n = s_{\Sigma} \mathcal{H}$. Then every interval is saturated and*

$$\frac{L_n}{F_n} \geq \alpha(s) := \frac{\sum_{\mu=1}^m \mu s_{\mu}}{m \sum_{\mu=1}^m s_{\mu}}.$$

Proof.

Since $F_{nI} \leq s_\Sigma D_I$ for every interval, equality after summing over all intervals implies $F_{nI} = s_\Sigma D_I$ for each I . Set $q = m$ and $R = 0$ in (7.27), and divide the resulting inequality by $F_n = s_\Sigma \mathcal{H}$. $\square$

For homogeneous speeds, $\alpha(s) = (m+1)/(2m)$. For heterogeneous speeds, it depends on the complete speed vector and approaches $1/m$ when the fastest machine dominates. In that regime, the independent fastest-machine guarantee s_1/s_Σ can be much stronger.

The corollary is only a retained-workload statement for the fully saturated final prefix. It is not an approximation guarantee by itself: the density argument requires the same multiplicative retained-workload bound for every prefix. Whether the full-capacity coefficient remains valid below full capacity is open.

Conjecture 7.5.13 (Full capacity is the worst retained-workload ratio). *For common windows and heterogeneous speeds, every density prefix satisfies*

$$L_k \geq \alpha(s) F_k.$$

If the conjecture holds, summation by parts immediately gives $SOL \geq \alpha(s) OPT_f$. The homogeneous theorem proves this statement when all speeds are equal. For general heterogeneous speeds, the current analysis proves the full-capacity endpoint, the instance-dependent potential certificates, and the separate fastest-machine guarantee, but not the conjectured interpolation between zero and full capacity.

7.6 Experimental Evaluation

We evaluate the algorithm’s quality under destination-dependent windows, its gap to the fractional and integral optima, the value of adaptive compression, the effect of machine-speed heterogeneity, and its scalability against the exact model. Code and detailed instructions for reproducing all experimental results, including full reruns, are available in the companion artifact ([Communication-Aware Scheduling Artifact 2026](#)).

7.6.1 Methodology

Accuracy functions Following (Barros et al., 2024), task j ’s accuracy after workload f is $a_j(f) = 0.82(1 - \exp(-\theta_j f))$, which approaches 82% with diminishing gains at a rate controlled by θ_j . We define the maximum useful workload $\bar{f}_j$ through $a_j(\bar{f}_j) = 0.99999 \cdot 0.82$ and use a concave five-segment piecewise-linear approximation on $[0, \bar{f}_j]$. To choose θ_j , we consider two workload models. In the *standard* workload model, θ_j is sampled with replacement from $\{0.1, 0.2, \dots, 1.9\}$. In the *lognormal* model, we fit a lognormal distribution to task processing times from Google Cluster traces (Reiss et al., 2011), yielding a trace-inspired heavy-tailed workload. At unit speed, each maximum workload is sampled as $\log \bar{f}_j \sim \mathcal{N}(4.98, 2.03^2)$; we set $\theta_j = -\log(1 - 0.99999)/\bar{f}_j$, so all tasks reach the same truncation accuracy.

Temporal tightness The window-slack factor $\sigma \in \{0.1, 0.2, \dots, 1\}$ controls the temporal flexibility of an instance. The generator sets the release span to $H_\sigma = \sigma(\sum_j \bar{f}_j)/(m\bar{s})$, where $\bar{s}$ is the mean machine speed. Release times are sampled independently from $U[0, H_\sigma]$ and indexed in nondecreasing order. Original window lengths are sampled as $d_j - r_j \sim U[\sigma \bar{f}_j/\bar{s}, 1.25\sigma \bar{f}_j/\bar{s}]$. Hence, σ scales both release dispersion and window length, while larger tasks receive proportionally longer windows. The values 0.1 and 1 give the tightest and loosest instances, respectively; deadlines may extend beyond H_σ .

Methods and metrics The fractional optimum OPT_f , obtained from Equation (7.3), upper-bounds the integral optimum OPT ; the rounded value SOL follows by applying Algorithm 7.1. We compute OPT with Gurobi when it can be certified and report $R_f = SOL/OPT_f$ and, when available, $R_{\text{int}} = SOL/OPT$. Accuracy is normalized by the task count. Curves average independent instances, with common seeds across parameter sweeps. Variability indicators are omitted because dispersion across replicates is consistently small and visually uninformative. All optimization models use Gurobi.

7.6.2 Communication-Aware Scheduling

We first evaluate the destination-dependent setting that motivates the general model. The experiment contains a local, fog, and cloud machine with speeds 1, 4, and 16, respectively. Tasks follow the lognormal workload model. Input transfer changes the release time at each destination, while return latency reduces its effective deadline. The experiment therefore exercises the full destination-dependent window model of Section 7.2.1.

To scale communication delays relative to the scheduling constraints, let W be the median original window length. Task j 's input size u_j is drawn from a lognormal distribution, truncated to $[0.25, 4]$, and normalized to have median one. Local communication delay is zero. At the fog, we set $\tau_{j,\text{fog}}^{\text{in}} = \frac{1}{2}\lambda W((1-q) + qu_j)$ and $\tau_{j,\text{fog}}^{\text{out}} = \frac{1}{2}\lambda W$; cloud delays are twice as large, reflecting its more remote location. We use $q = 0.5$, so fixed and data-dependent components contribute equally to the input delay of a median-size task, and sweep $\lambda \in \{0, 0.05, 0.1, 0.2, 0.4, 0.8, 1.2, 2, 4\}$. Hence, λW is the median total fog communication penalty. Normalizing by W makes the sweep comparable across instances and ranges from negligible delays to regimes in which communication consumes a typical window and makes remote execution infeasible for many tasks.

Each communication-severity sweep reuses the task accuracy functions, original windows, and input sizes of the same base instance; only the delays and resulting execution windows vary.

7.6.2.1 Solution quality

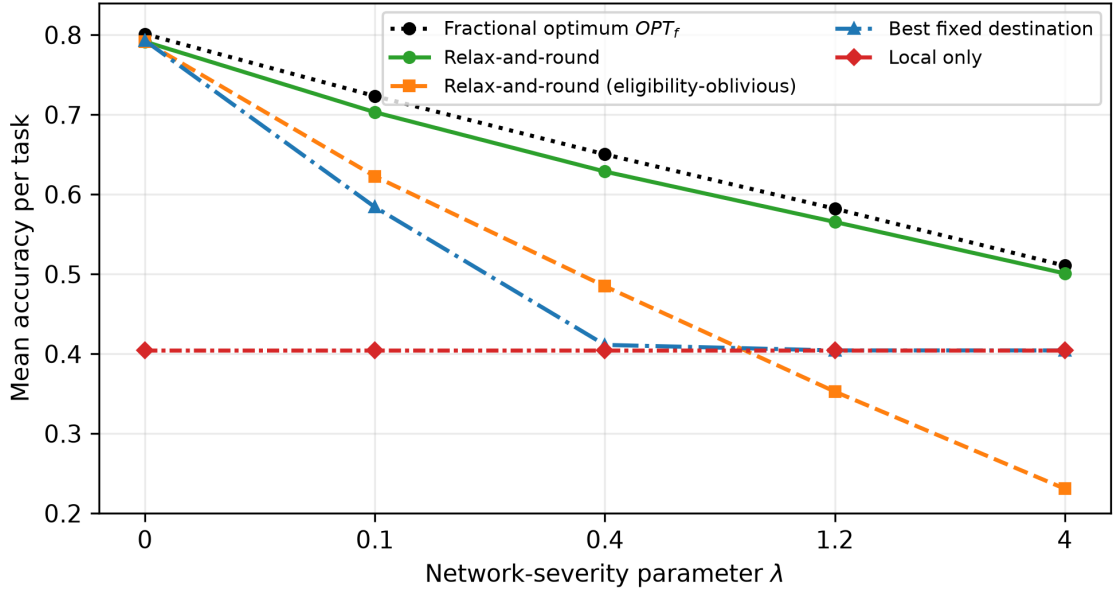
We first use $n = 10$, for which every integral solve is certified optimal. For each $\sigma \in \{0.1, 0.5, 1\}$, five base instances are reused at all nine λ values. Table 7.1 shows that SOL remains close to OPT . At $\sigma = 0.1$, the mean ratios SOL/OPT and OPT/OPT_f are 0.984 and 0.954, indicating that most of the remaining gap is due to relaxation rather than rounding. Across the full sweep, the minimum SOL/OPT and SOL/OPT_f are 0.911 and 0.851.

7.6.2.2 Placement adaptation

We next use $n = 100$, $\sigma = 0.5$, and ten base instances, omitting the integral solve. The *eligibility-oblivious* ablation starts from the same fractional solution but chooses a destination

Table 7.1 – Mean quality for destination-dependent $n = 10$ instances, aggregated over λ .

Metric	$\sigma = 0.1$	$\sigma = 0.5$	$\sigma = 1.0$
Mean SOL/OPT	0.984	0.995	0.993
Mean SOL/OPT_f	0.939	0.976	0.987
Mean OPT/OPT_f	0.954	0.981	0.994

Figure 7.2 – Mean accuracy for destination-dependent $n = 100$ instances at five representative λ values.

without considering $\chi_{j\mu I}$; workload outside the selected destination’s window is then discarded. Three fixed-destination baselines send every task to the local, fog, or cloud machine and solve the resulting single-machine instance; we report their pointwise best.

In Figure 7.2, our algorithm maintains mean $R_f \geq 0.967$. At $\lambda = 4$, its mean accuracy is 0.501, versus 0.231 for eligibility-oblivious rounding and 0.404 for local-only execution. Its gain over the best fixed destination is 52.9% at $\lambda = 0.4$ and 23.9% at $\lambda = 4$, showing the value of adaptive placement even when fixed remote policies are inferior to local execution.

Figure 7.3 explains this advantage. At $\lambda = 4$, 78.0% of active tasks run locally, while the 12.3% sent to the cloud carry 93.4% of the retained workload. Panel (b) groups tasks by $\bar{f}_j/u_j$: larger inputs cost more to transfer, whereas larger workloads benefit more from remote speed. Offloading therefore increases with the compute-to-data ratio, especially under severe communication delay.

The global partition contains 199 intervals when $\lambda = 0$ and 599 otherwise. For $n = 100$, the end-to-end time for the fractional solve, both rounding variants, and all baselines rises from 3.03 to about 9.5 seconds.

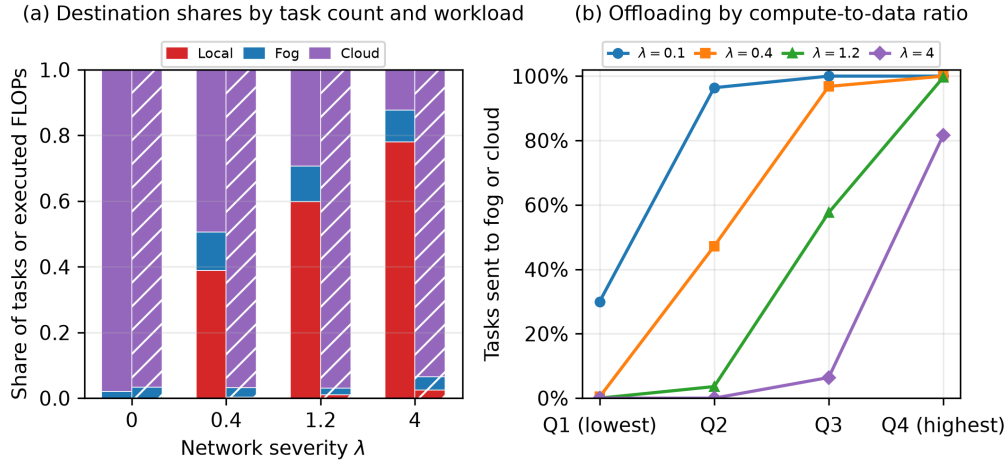


Figure 7.3 – Placement for communication-aware $n = 100$ instances: (a) task and workload shares; (b) remote execution by compute-to-data quartile.

7.6.3 Quality Under Common Windows

We next consider a common-window setting, in which each task has the same availability window on every machine. This can model a colocated cloud cluster with heterogeneous accelerators but shared, pre-staged task data, where machine-dependent communication delays are negligible relative to task deadlines. We compare our relax-and-round algorithm with two greedy limited-compression baselines from the literature, adapted to support task-dependent release times. *Three-level* compression (Lee and Song, 2021) restricts each task to the smallest workloads attaining one third, two thirds, or all of $a_j(\bar{f}_j)$. It first attempts the one-third choices, with ties in release-time order, and then considers upgrades by nonincreasing accuracy gain; an upgrade may move the task. *No compression* (Y.-W. Zhang et al., 2023) visits tasks in release-time order and assigns $\bar{f}_j$ to the machine with the most residual window capacity, dropping the task if it does not fit.

We test $(n, m) \in \{(10, 2), (10, 3), (20, 4), (100, 10)\}$, both workload models, and all ten σ values, reusing five seeded base instances within each workload-size family. Speeds are drawn independently from $U[1, 20]$ and windows are common across machines, giving 400 runs. Across the eight families, mean R_f ranges from 0.977 to 0.995, with individual minimum 0.885; for the three smaller configurations, mean R_{int} ranges from 0.983 to 0.998.

For the representative $(100, 10)$ case, Figure 7.4 shows that the rounding gap is concentrated in the tightest windows: as σ increases from 0.1 to 1, mean R_f rises from 0.905 to 0.999 for standard workloads and from 0.971 to 0.992 for lognormal workloads.

Adaptive compression is important. Averaged over all σ , the ratios for standard (lognormal) workloads are 0.978 (0.987) for our algorithm, 0.759 (0.789) for three-level compression, and 0.202 (0.242) without compression. At the smallest slack factors, the no-compression baseline schedules no task for $\sigma \leq 0.3$. Results are even stronger for trace-inspired lognormal than standard synthetic workloads.

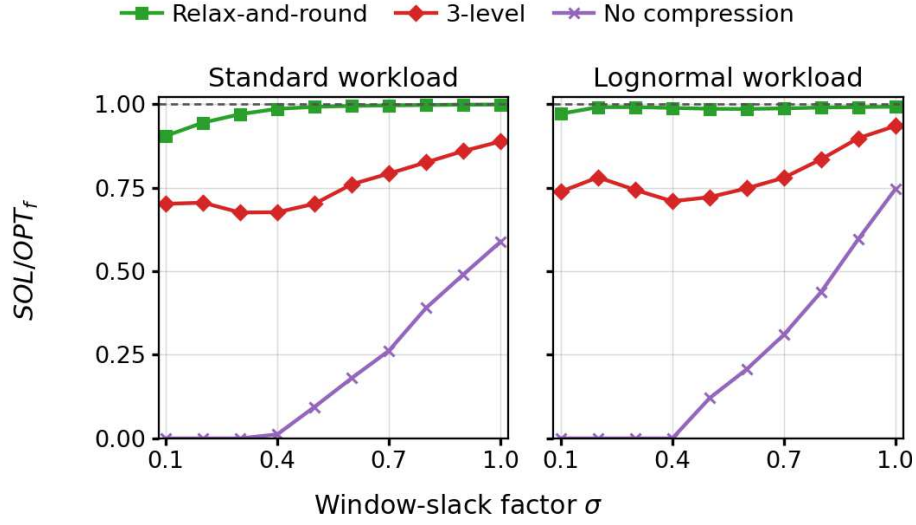


Figure 7.4 – Mean R_f of our algorithm over five common-window (100, 10) instances, compared with the three-level and no-compression baselines.

7.6.4 Machine-Speed Heterogeneity

For standard (100, 10) workloads, homogeneous speeds give mean $R_f = 0.991$ and minimum 0.962, versus 0.978 and 0.890 for independent $U[1, 20]$ speeds. The homogeneous results exceed the proved $(m + 1)/(2m) = 0.55$ guarantee. For heterogeneous speeds, we compare against the proved fastest-machine guarantee and the coefficient $\alpha(s)$, which would guarantee $SOL \geq \alpha(s)OPT_f$ if Conjecture 7.5.13 holds.

We use speeds $(1, \dots, 1, \kappa)$, with 12 values of $\kappa = s_{\max}/s_{\min}$ between 1 and 100, three slack factors, and five instances per point. Reusing seeded jobs and normalized temporal draws, and scaling time by $1/\bar{s}$, keeps aggregate capacity invariant. Across the resulting 180 runs, mean R_f is 0.971 and the minimum is 0.874, attained at $\sigma = 0.1$, $\kappa = 8$.

Every mean in Figure 7.5 exceeds both references, substantially so at low κ , where the conjectured curve is stronger. As κ grows, the proved fastest-machine guarantee approaches the observed performance.

7.6.5 Running-Time Scalability

Table 7.2 uses standard workloads, independent $U[1, 20]$ speeds, and $\sigma = 0.1$. Runtimes include preprocessing, model construction, optimization, and rounding, using Gurobi 13.0.2 on an Intel Core i9-13950HX with 32 GB RAM. Fractional entries average five runs through $n = 100$ and two thereafter; exact entries average five runs through $n = 20$. The exact model averages 0.727 s at (20, 4), versus 0.037 s for our algorithm, and reaches the one-hour limit at (40, 8). Our algorithm takes 0.164 s on that same instance (the table reports the five-instance mean, 0.138 s), a speedup of at least 2.2×10^4 . Larger exact runs are omitted; the fractional solve dominates total runtime, while rounding remains negligible.

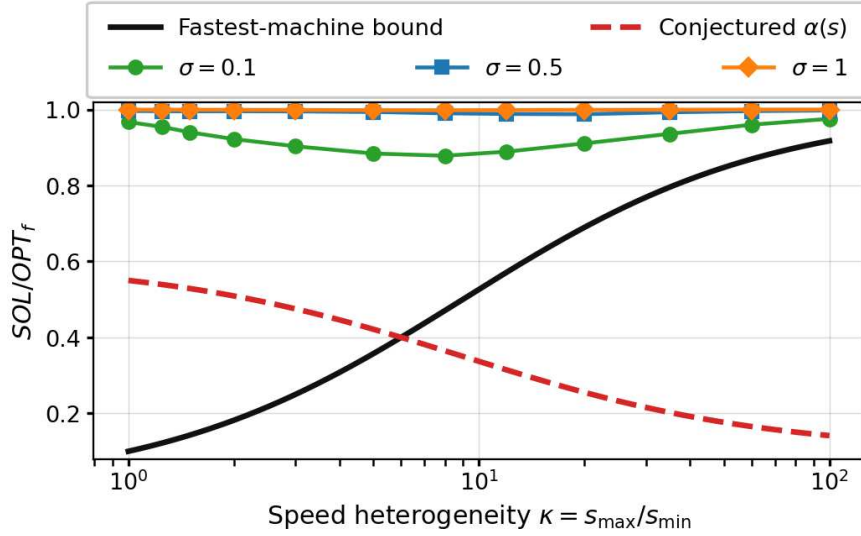


Figure 7.5 – Dominant-machine sweep for common-window (100, 10) instances: mean R_f , the proved fastest-machine bound, and conjectured $\alpha(s)$.

Table 7.2 – Mean runtime (s) at $\sigma = 0.1$; TL denotes the one-hour limit.

(n, m)	Exact	Fractional	Rounding	Total
(10, 2)	0.024	0.013	0.005	0.017
(20, 4)	0.727	0.033	0.004	0.037
(40, 8)	TL (1h)	0.133	0.005	0.138
(60, 10)	TL (1h)	0.358	0.007	0.365
(100, 10)	–	0.877	0.013	0.890
(200, 20)	–	10.798	0.041	10.839
(400, 40)	–	123.267	0.343	123.611

7.7 Conclusion

We studied communication-aware scheduling of compressible ML inference tasks across heterogeneous edge-cloud resources and developed a relax-and-round algorithm with workload-retention and accuracy guarantees. For common execution windows, we also derived an exact min-cost-flow formulation of the relaxation. Experiments show accuracy close to the fractional upper bound, communication-aware placement decisions, and substantially better scalability than the exact integral formulation.

Future work could use the min-cost-flow structure to design combinatorial algorithms and test whether they outperform a general-purpose LP solver when the relaxation dominates runtime. Other directions include online variants with uncertain arrivals or communication delays; memory, energy, and bandwidth constraints; extending the flow formulation to more general destination-dependent execution windows; and tighter guarantees for heterogeneous machine speeds.

CHAPTER 8

Conclusion and Perspectives

8.1 Summary of Contributions

This thesis has studied how to use less computation for machine-learning inference, and how to allocate the computation that remains. The first part examined which subnetworks can be selected from random neural networks without changing their weights. The second part examined how to distribute adjustable inference workloads across constrained computing resources.

On the model side, Chapter 3 introduced the Random Fixed-Size Subset Sum problem to prescribe how many parameters a strong lottery ticket retains. It led to sparsity guarantees for dense and equivariant networks, with near-matching lower bounds on the required size of the random source network. Chapter 4 then examined what happens when pruning must remove entire neurons. Even for the approximation of one bias-free ReLU neuron, this restriction requires width proportional to the inverse approximation error, whereas weight pruning achieves logarithmic dependence on that error. The greater regularity of neuron pruning therefore comes with a provable cost in this setting.

On the scheduling side, Chapter 6 developed a fractional solution method and an approximation algorithm that jointly choose task workloads and machines under deadlines and a global energy budget. Chapter 7 incorporated release times and destination-dependent communication delays, obtaining an eligibility-aware relax-and-round algorithm and exact min-cost-flow formulations of the relaxation in important cases. The experiments in both settings indicate that adapting the workload of each task can preserve high aggregate accuracy while respecting resource constraints, and that the resulting algorithms scale beyond the range in which exact integral optimization is practical.

The additional contribution in Appendix C addresses sparse execution itself: a sparse-matrix-multiplication path for graph neural network message passing in Julia reduces the intermediate-memory costs of gather–scatter implementations. It reinforces a theme of the thesis: a reduction in mathematical work becomes useful in deployment only when it can be expressed and scheduled efficiently on the available software and hardware.

8.2 Limitations and Open Questions

The strong lottery ticket results are existence guarantees. They do not show how to find a ticket of a prescribed sparsity efficiently, nor do they establish that its irregular sparsity yields proportional inference savings on actual hardware. The neuron-pruning separation is proved for a

deliberately simple, bias-free two-layer setting; the optimal dependence on input dimension when many neurons may be retained, and the extent to which the result carries over to deeper networks, remain open.

The scheduling guarantees likewise rely on explicit models of task utility, machine capabilities, and timing. The algorithms use known task information, while real deployments may face uncertain arrivals, variable communication delays, and changing energy availability. The experiments evaluate the proposed formulations and algorithms under controlled workloads; they do not by themselves establish end-to-end gains in a live inference service.

8.3 Research Perspectives

The pruning results suggest two related theoretical questions. The fixed-size subset-sum analysis currently concerns scalar targets; extending it to vectors could reveal whether sparsity can be controlled as precisely when groups of parameters must be retained together. Improving its success-probability bounds would also reduce the extra source size introduced by probability amplification. At the other end of the spectrum, the neuron-pruning lower bound leaves open how the required width depends on input dimension when the ticket may retain many neurons. Resolving this would give a clearer account of the price of structured pruning beyond the single-neuron regime.

The next challenge is to turn these existence results into a usable form of compression. An efficient procedure for finding tickets with a chosen density would allow the theoretical trade-offs to be tested empirically. Those tests should measure more than parameter counts: memory use and inference time depend on whether the retained connections can be executed efficiently. This is especially relevant to the comparison between irregular weight masks and whole-neuron pruning, where the approximation and implementation advantages point in different directions.

A further direction is to test how far the scheduling approach extends beyond the image-classification models that motivate its workload–accuracy trade-offs. Self-slimmable transformers trained with sparse mixture-of-experts dropout provide one concrete example: their number of active experts can be varied at inference time (T. Chen et al., 2023). Applying such configurable models to tasks such as reasoning or text generation would require measurements of task-specific quality, latency, and energy at different inference configurations. It is not clear in advance whether these measurements would follow the nondecreasing concave utility curves assumed in the present algorithms: the available configurations may be discrete, and their gains may depend on the input. If so, the scheduling problem would need to account for the actual operating points and possibly the time spent selecting or switching configurations. Studying which guarantees survive under these conditions would show whether the current relax-and-round approach applies more broadly or needs a different formulation.

For scheduling, the priority is to retain useful guarantees as the models become closer to a live service. In an online setting, the scheduler would have to decide how much computation to allocate to each task and where to execute it as soon as the task arrives, without knowing which tasks will arrive later and possibly with only estimates of future network delays. Energy drawn by communication, renewable supply, memory capacity, and network bandwidth would add constraints that can change which placements are feasible. These extensions should be introduced selectively, so that the effect of each one on solution quality and computational cost can be understood. In

parallel, the min-cost-flow formulations offer a route to specialized algorithms for the fractional problem, which is a significant part of the running time on larger instances.

Finally, a joint evaluation could use measured accuracy and resource profiles of real model configurations on heterogeneous devices as inputs to the scheduler. It would test whether the theoretical and algorithmic savings described in this thesis translate into lower end-to-end inference costs, and which forms of model compression remain useful once placement, communication, and hardware behavior are taken into account.

A broader perspective concerns the rebound effect: reducing the resources required by each inference can make AI services cheaper and easier to use, thereby increasing demand enough to offset part, or even all, of the initial savings (Luccioni et al., 2025). Technical improvements in pruning, model configuration, and scheduling therefore do not guarantee a reduction in total resource consumption on their own. Achieving such a reduction also requires attention to how these efficiencies are used. Users need information that helps them choose a level of model quality proportionate to their needs and avoid unnecessary computation, while service providers can support these choices through transparent resource indicators and resource-conscious default configurations. Efficiency must therefore be paired with sufficiency. Lower resource use depends on making each inference more efficient while also avoiding unnecessary computation.

References

- Alidaee, Bahram and Ahmad Ahmadian (1993). « Two parallel machine sequencing problems involving controllable job processing times ». In: *European Journal of Operational Research* 70.3, pp. 335–341.
- Allasia, Giampietro (1981). « Approximation of the normal distribution functions by means of a spline function ». In: *Statistica* 41.2, pp. 325–332.
- ApS, MOSEK (2022). *The MOSEK optimization toolbox for Python manual. Version 10.0*. URL: <https://docs.mosek.com/latest/pythonapi/index.html>.
- Aslanpour, Mohammad Sadegh, Adel N Toosi, Muhammad Aamir Cheema, and Raj Gaire (2022). « Energy-aware resource scheduling for serverless edge computing ». In: *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE.
- Barros, Tiago Da Silva, Frédéric Giroire, Ramon Aparicio-Pardo, Stephane Perennes, and Emanuele Natale (2024). « Scheduling with fully compressible tasks: Application to deep learning inference with neural network compression ». In: *Proc. IEEE/ACM CCGrid*. IEEE, pp. 327–336.
- Borst, Sander, Daniel Dadush, Sophie Huiberts, and Samarth Tiwari (Feb. 2023). « On the Integrality Gap of Binary Integer Programs with Gaussian Data ». In: *Mathematical Programming* 197.2, pp. 1221–1263. ISSN: 1436-4646. DOI: [10.1007/s10107-022-01828-1](https://doi.org/10.1007/s10107-022-01828-1). (Visited on 06/05/2023).
- Boyd, Stephen P and Lieven Vandenberghe (2004). *Convex optimization*. Cambridge university press.
- Burkholz, Rebekka (July 2022a). « Convolutional and Residual Networks Provably Contain Lottery Tickets ». In: *Proceedings of the 39th International Conference on Machine Learning*. Baltimore: PMLR, pp. 2414–2433. (Visited on 02/27/2023).
- (Dec. 2022b). « Most Activation Functions Can Win the Lottery Without Excessive Depth ». In: *Thirty-Sixth Conference on Neural Information Processing Systems*. (Visited on 02/27/2023).
- Burkholz, Rebekka, Nilanjana Laha, Rajarshi Mukherjee, and Alkis Gotovos (Apr. 2022). « On the Existence of Universal Lottery Tickets ». In: *International Conference on Learning Representations*. virtual. (Visited on 02/27/2023).
- Cai, Han, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han (2020). « Once-for-All: Train One Network and Specialize it for Efficient Deployment ». In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=HylxE1HKwS>.
- Chen, Bo, Chris N Potts, and Gerhard J Woeginger (1998). « A review of machine scheduling: Complexity, algorithms and approximability ». In: *Handbook of Combinatorial Optimization: Volume 1–3*, pp. 1493–1641.
- Chen, Tianlong, Zhenyu Zhang, Ajay Jaiswal, Shiwei Liu, and Zhangyang Wang (2023). « Sparse MoE as the New Dropout: Scaling Dense and Self-Slimmable Transformers ». In: *arXiv preprint arXiv:2303.01610*. arXiv: [2303.01610](https://arxiv.org/abs/2303.01610). URL: <https://arxiv.org/abs/2303.01610>.

- Cheng, Yu, Duo Wang, Pan Zhou, and Tao Zhang (2018). « Model compression and acceleration for deep neural networks: The principles, progress, and challenges ». In: *IEEE Signal Processing Magazine* 35.1.
- Cunha, Arthur da, Francesco D’Amore, and Emanuele Natale (Nov. 2023). « Polynomially Over-Parameterized Convolutional Neural Networks Contain Structured Strong Winning Lottery Tickets ». In: *Thirty-Seventh Conference on Neural Information Processing Systems*. (Visited on 04/23/2024).
- Cunha, Arthur Carvalho Walraven da, Francesco D’Amore, Frédéric Giroire, Hicham Lesfari, Emanuele Natale, and Laurent Viennot (2023). « Revisiting the Random Subset Sum Problem ». In: *31st Annual European Symposium on Algorithms, ESA 2023, September 4-6, 2023, Amsterdam, The Netherlands*. Ed. by Inge Li Gørtz, Martin Farach-Colton, Simon J. Puglisi, and Grzegorz Herman. Vol. 274. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 37:1–37:11. DOI: [10.4230/LIPIcs.ESA.2023.37](https://doi.org/10.4230/LIPIcs.ESA.2023.37). URL: <https://doi.org/10.4230/LIPIcs.ESA.2023.37>.
- da Cunha, Arthur, Emanuele Natale, and Laurent Viennot (Apr. 2022). « Proving the Strong Lottery Ticket Hypothesis for Convolutional Neural Networks ». In: *ICLR 2022 - 10th International Conference on Learning Representations*. Virtual, France. (Visited on 08/04/2022).
- Da Silva Barros, Tiago, Frederic Giroire, Ramon Aparicio-Pardo, Stephane Perennes, and Emanuele Natale (2024). « Scheduling with Fully Compressible Tasks: Application to Deep Learning Inference with Neural Network Compression ». In: *IEEE/ACM International Symposium on Cluster, Cloud, and Internet Computing*.
- Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei (2009). « ImageNet: A large-scale hierarchical image database ». In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255. DOI: [10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- Desislavov, Radosvet, Fernando Martínez-Plumed, and José Hernández-Orallo (2023). « Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning ». In: *Sustainable Computing: Informatics and Systems* 38, p. 100857.
- Drainakis, Georgios, Panagiotis Pantazopoulos, Konstantinos V Katsaros, Vasilis Sourlas, and Angelos Amditis (2021). « On the Distribution of ML Workloads to the Network Edge and Beyond ». In: *IEEE Conference on Computer Communications Workshops (INFOCOM WK-SHPS)*.
- Epoch AI (2024). *Data on Notable AI Models*. Accessed: 2025-01-16. URL: <https://epoch.ai/data/notable-ai-models>.
- Fang, Biyi, Xiao Zeng, and Mi Zhang (2018). « Nstdnn: Resource-aware multi-tenant on-device deep learning for continuous mobile vision ». In: *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking*.
- Ferbach, Damien, Christos Tsirigotis, Gauthier Gidel, and Joey Bose (Sept. 2022). « A General Framework For Proving The Equivariant Strong Lottery Ticket Hypothesis ». In: *The Eleventh International Conference on Learning Representations*. (Visited on 03/04/2024).
- Ferre, Davide (2025). *Improving the Performance of GNNs Using Sparse Linear Algebra: Google Summer of Code 2025 Final Report*. URL: <https://gist.github.com/dferre97/cd3834b9125d3059b03ffb2ce61bdb1a> (visited on 08/24/2026).
- Ferré, Davide, Tiago Da Silva Barros, Frédéric Giroire, and Stéphane Pérennes (2026). « Communication-Aware Scheduling of Compressible ML Tasks across the Edge–Cloud Continuum ». Under submission.

- Ferré, Davide, Frédéric Giroire, Emanuele Natale, and Frederik Mallmann-Trenn (2026). « Structured vs. Unstructured Pruning: An Exponential Gap ». Under submission.
- Fey, Matthias and Jan Eric Lenssen (2019). « Fast Graph Representation Learning with PyTorch Geometric ». In: *ICLR Workshop on Representation Learning on Graphs and Manifolds*. arXiv: 1903.02428. URL: <https://arxiv.org/abs/1903.02428>.
- Fischer, Jonas and Rebekka Burkholz (Apr. 2022). « Plant 'n' Seek: Can You Find the Winning Ticket? ». In: *International Conference on Learning Representations*. (Visited on 02/27/2023).
- Fischer, Jonas, Advait Gadhikar, and Rebekka Burkholz (June 2022). *Lottery Tickets with Nonzero Biases*. arXiv: 2110.11150 [cs]. (Visited on 05/13/2024).
- Frankle, Jonathan and Michael Carbin (Sept. 2018). « The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks ». In: *International Conference on Learning Representations*. (Visited on 10/20/2023).
- Fresa, Andrea and Jaya Prakash Varma Champati (2022). « An Offloading Algorithm for Maximizing Inference Accuracy on Edge Device in an Edge Intelligence System ». In: *Proceedings of the 25th International ACM Conference on Modeling Analysis and Simulation of Wireless and Mobile Systems*. MSWiM '22. Montreal, Quebec, Canada: Association for Computing Machinery, pp. 15–23. ISBN: 9781450394826. DOI: 10.1145/3551659.3559044. URL: <https://doi.org/10.1145/3551659.3559044>.
- Freund, Karl (2019). « Google cloud doubles down on nvidia gpus for inference, 2019 ». In: URL <https://www.forbes.com/sites/moorinsights/2019/05/09/google-cloud-doubles-down-on-nvidia-gpus-for-inference>.
- Gadhikar, Advait Harshal, Sohom Mukherjee, and Rebekka Burkholz (2023). « Why Random Pruning Is All We Need to Start Sparse ». In: *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*. Ed. by Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett. Vol. 202. Proceedings of Machine Learning Research. PMLR, pp. 10542–10570. URL: <https://proceedings.mlr.press/v202/gadhikar23a.html>.
- Gilmer, Justin, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl (2017). « Neural Message Passing for Quantum Chemistry ». In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, pp. 1263–1272. URL: <https://proceedings.mlr.press/v70/gilmer17a.html>.
- Giroire, Frédéric, Nicolas Huin, Andrea Tomassilli, and Stéphane Pérennes (2019). « When network matters: Data center scheduling with network tasks ». In: *IEEE Conference on Computer Communications (INFOCOM)*. IEEE, pp. 2278–2286.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2016). « Deep residual learning for image recognition ». In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778.
- He, Yang and Lingao Xiao (2023). « Structured Pruning for Deep Convolutional Neural Networks: A Survey ». In: *CoRR* abs/2303.00566. DOI: 10.48550/arXiv.2303.00566. URL: <https://doi.org/10.48550/arXiv.2303.00566>.
- Hoefler, Torsten, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste (2021). « Sparsity in Deep Learning: Pruning and Growth for Efficient Inference and Training in Neural Networks ». In: *Journal of Machine Learning Research* 22.241, pp. 1–124. URL: <http://jmlr.org/papers/v22/21-0366.html>.

- Hsu, Daniel, Clayton Sanford, Rocco A. Servedio, and Emmanouil V. Vlatakis-Gkaragkounis (2021). « On the Approximation Power of Two-Layer Networks of Random ReLUs ». In: *Conference on Learning Theory*. Ed. by Mikhail Belkin and Samory Kpotufe. Vol. 134. Proceedings of Machine Learning Research. PMLR, pp. 2423–2461. URL: <http://proceedings.mlr.press/v134/hsu21a.html>.
- Hung, Chien-Chun, Ganesh Ananthanarayanan, Peter Bodik, Leana Golubchik, Minlan Yu, Paramvir Bahl, and Matthai Philipose (2018). « Videoedge: Processing camera streams using hierarchical clusters ». In: *2018 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, pp. 115–131.
- Isik, Berivan, Francesco Pase, Deniz Gunduz, Sanmi Koyejo, Tsachy Weissman, and Michele Zorzi (Apr. 2024). « Adaptive Compression in Federated Learning via Side Information ». In: *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*. PMLR, pp. 487–495. (Visited on 05/16/2024).
- Isik, Berivan, Francesco Pase, Deniz Gunduz, Tsachy Weissman, and Zorzi Michele (Sept. 2022). « Sparse Random Networks for Communication-Efficient Federated Learning ». In: *The Eleventh International Conference on Learning Representations*. (Visited on 01/18/2024).
- Jiang, Shuang, Zhiyao Ma, Xiao Zeng, Chenren Xu, Mi Zhang, Chen Zhang, and Yunxin Liu (2020). « Scylla: Qoe-aware continuous mobile vision with fpga-based dynamic deep neural network reconfiguration ». In: *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*.
- Jin, Yibo, Lei Jiao, Zhuzhong Qian, Sheng Zhang, Ning Chen, Sanglu Lu, and Xiaoliang Wang (2020). « Provisioning edge inference as a service via online learning ». In: *2020 17th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*.
- Kamath, Pritish, Omar Montasser, and Nathan Srebro (2020). « Approximate Is Good Enough: Probabilistic Variants of Dimensional and Margin Complexity ». In: *Conference on Learning Theory*. PMLR, pp. 2236–2262.
- Lee, Dayoung and Minseok Song (2021). « Quality-Oriented Task Allocation and Scheduling in Transcoding Servers With Heterogeneous Processors ». In: *IEEE Transactions on Circuits and Systems for Video Technology* 32.3, pp. 1667–1680.
- Lin, Jieyu, Minghao Li, Sai Qian Zhang, and Alberto Leon-Garcia (2024). « Murmuration: On-the-fly dnn adaptation for slo-aware distributed inference in dynamic edge environments ». In: *Proc. Int. Conf. Parallel Process. (ICPP)*, pp. 792–801.
- Liu, Chung Laung and James W Layland (1973). « Scheduling algorithms for multiprogramming in a hard-real-time environment ». In: *Journal of the ACM (JACM)* 20.1, pp. 46–61.
- Liu, Jiagang, Ju Ren, Wei Dai, Deyu Zhang, Pude Zhou, Yaoyue Zhang, Geyong Min, and Noushin Najjari (2019). « Online multi-workflow scheduling under uncertain task execution time in IaaS clouds ». In: *IEEE Trans. Cloud Comput.* 9.3, pp. 1180–1194.
- Liu, Zhuang, Mingjie Sun, Tinghui Zhou, Gao Huang, and Trevor Darrell (2018). « Rethinking the value of network pruning ». In: *arXiv preprint arXiv:1810.05270*.
- Luccioni, Alexandra Sasha, Emma Strubell, and Kate Crawford (2025). « From Efficiency Gains to Rebound Effects: The Problem of Jevons’ Paradox in AI’s Polarized Environmental Debate ». In: *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency*, pp. 76–88.
- Lucibello, Carlo and Aurora Rossi (2025). « GraphNeuralNetworks.jl: Deep Learning on Graphs with Julia ». In: *Journal of Machine Learning Research* 26.80, pp. 1–6. URL: <https://www.jmlr.org/papers/v26/24-2130.html>.

- Lueker, George S. (1982). « On the average difference between the solutions to linear and integer knapsack problems ». In: *Applied Probability - Computer Science, The Interface*. Vol. 1. Birkhäuser.
- (1998). « Exponentially small bounds on the expected optimum of the partition and subset sum problems ». In: *Random Structures & Algorithms* 12, pp. 51–62.
- Ma, Huirong, Rui Li, Xiaoxi Zhang, Zhi Zhou, and Xu Chen (2023). « Reliability-aware online scheduling for dnn inference tasks in mobile edge computing ». In: *IEEE Internet of Things Journal*.
- Ma, Xiaojin, Huahu Xu, Honghao Gao, and Minjie Bian (2021). « Real-time multiple-workflow scheduling in cloud environments ». In: *IEEE Trans. Netw. Service Manag.* 18.4, pp. 4002–4018.
- MacWilliams, Florence Jessie and Neil James Alexander Sloane (1977). *The Theory of Error-Correcting Codes*. Vol. 16. North-Holland Mathematical Library. North-Holland Publishing Company.
- Malach, Eran, Gilad Yehudai, Shai Shalev-shwartz, and Ohad Shamir (July 2020). « Proving the Lottery Ticket Hypothesis: Pruning Is All You Need ». In: *Proceedings of the 37th International Conference on Machine Learning*. ICML’20. JMLR.org, pp. 6682–6691. (Visited on 03/26/2023).
- Marengo, James E, David L Farnsworth, Lucas Stefanic, et al. (2017). « A geometric derivation of the Irwin-Hall distribution ». In: *International Journal of Mathematics and Mathematical Sciences* 2017.
- McDonald, Joseph, Baolin Li, Nathan Frey, Devesh Tiwari, Vijay Gadepally, and Siddharth Samsi (2022). « Great power, great responsibility: Recommendations for reducing energy for training language models ». In: *arXiv preprint arXiv:2205.09646*.
- Meng, Jiaying, Haisheng Tan, Chao Xu, Wanli Cao, Liuyan Liu, and Bojie Li (2019). « Dedas: Online task dispatching and scheduling with bandwidth constraint in edge computing ». In: *Proc. IEEE INFOCOM*. IEEE, pp. 2287–2295.
- Mounesan, Motahare, Xiaojie Zhang, and Saptarshi Debroy (2025). « Infer-edge: Dynamic dnn inference optimization in just-in-time edge-ai implementations ». In: *Proc. IEEE NOMS*. IEEE, pp. 1–9.
- Mozer, Michael C. and Paul Smolensky (1988). « Skeletonization: A Technique for Trimming the Fat from a Network via Relevance Assessment ». In: *Advances in Neural Information Processing Systems I*. MIT Press, pp. 107–115.
- (1989). « Using Relevance to Reduce Network Size Automatically ». In: *Connection Science* 1.1, pp. 3–16.
- Natale, Emanuele, Davide Ferré, Giordano Giambartolomei, Frédéric Giroire, and Frederik Mallmann-Trenn (2024). « On the Sparsity of the Strong Lottery Ticket Hypothesis ». In: *Advances in Neural Information Processing Systems*. Vol. 37, pp. 40565–40592. DOI: [10 . 52202 / 079017 - 1283](https://doi.org/10.52202/079017-1283). URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/47908cab4e5b696d7af5c7de69f3b7d2-Abstract-Conference.html.
- Nigade, Vinod, Pablo Bauszat, Henri Bal, and Lin Wang (2022). « Jellyfish: Timely Inference Serving for Dynamic Edge Networks ». In: *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, pp. 277–290.
- NVIDIA Corporation (2026). *cuSPARSE Documentation*. URL: <https://docs.nvidia.com/cuda/cusparse/> (visited on 08/24/2026).

- Ogden, Samuel S and Tian Guo (2018). « {MODI}: Mobile deep inference made efficient by edge computing ». In: *USENIX Workshop on Hot Topics in Edge Computing*.
- Orseau, Laurent, Marcus Hutter, and Omar Rivasplata (Dec. 2020). « Logarithmic Pruning Is All You Need ». In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS'20. Red Hook, NY, USA: Curran Associates Inc., pp. 2925–2934. ISBN: 978-1-71382-954-6. (Visited on 03/26/2023).
- Otsuka, Hikari, Daiki Chijiwa, Ángel López García-Arias, Yasuyuki Okoshi, Kazushi Kawamura, Thiem Van Chu, Daichi Fujiki, Susumu Takeuchi, and Masato Motomura (Feb. 2024). *Partial Search in a Frozen Network Is Enough to Find a Strong Lottery Ticket*. DOI: [10.48550/arXiv.2402.14029](https://doi.org/10.48550/arXiv.2402.14029). arXiv: [2402.14029](https://arxiv.org/abs/2402.14029) [cs, stat]. (Visited on 05/15/2024).
- Patterson, David A., Joseph Gonzalez, Urs Hölzle, Quoc V. Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David R. So, Maud Texier, and Jeff Dean (2022). « The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink ». In: *Computer* 55.7, pp. 18–28. DOI: [10.1109/MC.2022.3148714](https://doi.org/10.1109/MC.2022.3148714).
- Pensia, Ankit, Shashank Rajput, Alliot Nagle, Harit Vishwakarma, and Dimitris Papailiopoulos (2020). « Optimal lottery tickets via SUBSETSUM: logarithmic over-parameterization is sufficient ». In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS '20. , Vancouver, BC, Canada, Curran Associates Inc. ISBN: 9781713829546.
- Perrault, Ray and Jack Clark (2024). *Artificial Intelligence Index Report 2024*.
- Petrov, Valentin V. (1975). *Sums of Independent Random Variables*. Ergebnisse der Mathematik und ihrer Grenzgebiete. 2. Folge. Springer Berlin, Heidelberg, 348 pages. DOI: <https://doi.org/10.1007/978-3-642-65809-9>.
- Raj, Suman, Harshil Gupta, and Yogesh Simmhan (2023). « Scheduling dnn inferencing on edge and cloud for personalized uav fleets ». In: *IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*.
- Ramanujan, Vivek, Mitchell Wortsman, Aniruddha Kembhavi, Ali Farhadi, and Mohammad Rastegari (June 2020). « What's Hidden in a Randomly Weighted Neural Network? » In: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11890–11899. DOI: [10.1109/CVPR42600.2020.01191](https://doi.org/10.1109/CVPR42600.2020.01191).
- Reiss, Charles, John Wilkes, and Joseph L Hellerstein (2011). « Google cluster-usage traces: format+ schema ». In: *Google Inc., White Paper* 1.1-14, p. 83.
- Communication-Aware Scheduling Artifact (2026). *Reproducibility Artifact for Communication-Aware Scheduling of Compressible ML Tasks across the Edge-Cloud Continuum*. URL: <https://anonymous.4open.science/r/scheduling-release-times-artifact-5CAF>.
- Ribeiro, Mauro, Katarina Grolinger, and Miriam AM Capretz (2015). « Mlaas: Machine learning as a service ». In: *2015 IEEE 14th international conference on machine learning and applications (ICMLA)*. IEEE, pp. 896–902.
- Salem, Tareq Si, Gabriele Castellano, Giovanni Neglia, Fabio Pianese, and Andrea Araldo (2021). « Towards inference delivery networks: Distributing machine learning with optimality guarantees ». In: *2021 19th Mediterranean Communication and Computer Networking Conference (MedComNet)*. IEEE, pp. 1–8.
- Shabtay, Dvir and Moshe Kaspi (2006). « Parallel machine scheduling with a convex resource consumption function ». In: *European Journal of Operational Research* 173.1.

- Shakarami, Ali, Mostafa Ghobaei-Arani, and Ali Shahidinejad (2020). « A survey on the computation offloading approaches in mobile edge computing: A machine learning-based perspective ». In: *Computer Networks* 182, p. 107496.
- Shakhaidarova, N. (1966). « Uniform local and global theorems for densities ». In: *Izv. Akad. Nauk UzSSR Ser. Fiz-Mat. Nauk* 5, pp. 90–91.
- She, Yechao, Minming Li, Yang Jin, Meng Xu, Jianping Wang, and Bin Liu (2023). « On-demand edge inference scheduling with accuracy and deadline guarantee ». In: *Proc. IEEE/ACM IWQoS*. IEEE, pp. 1–10.
- Shevtsova, Irina (2011). *On the absolute constants in the Berry Esseen type inequalities for identically distributed summands*. arXiv: [1111.6554 \[math.PR\]](https://arxiv.org/abs/1111.6554).
- Silva Barros, Tiago da, Davide Ferre, Frederic Giroire, Ramon Aparicio-Pardo, and Stephane Perennes (2024). « Scheduling machine learning compressible inference tasks with limited energy budget ». In: *Proc. Int. Conf. Parallel Process. (ICPP)*, pp. 961–970.
- Tsao, Yu-Chung, Vo-Van Thanh, and Feng-Jang Hwang (2020). « Energy-efficient single-machine scheduling problem with controllable job processing times under differential electricity pricing ». In: *Resources, Conservation and Recycling* 161, p. 104902.
- Vickson, RG (1980). « Two single machine sequencing problems involving controllable job processing times ». In: *AIIE transactions*.
- Wang, Minjie, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang (2019). « Deep Graph Library: A Graph-Centric, Highly-Performant Package for Graph Neural Networks ». In: *arXiv preprint arXiv:1909.01315*. arXiv: [1909.01315](https://arxiv.org/abs/1909.01315). URL: <https://arxiv.org/abs/1909.01315>.
- Wang, Ne, Ruiting Zhou, Lei Jiao, Renli Zhang, Bo Li, and Zongpeng Li (2022). « Preemptive scheduling for distributed machine learning jobs in edge-cloud networks ». In: *IEEE J. Sel. Areas Commun.* 40.8, pp. 2411–2425.
- Wang, Yidi, Mohsen Karimi, Yecheng Xiang, and Hyoseung Kim (2021). « Balancing energy efficiency and real-time performance in GPU scheduling ». In: *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, pp. 110–122.
- Wang, Yulong, Xiaolu Zhang, Lingxi Xie, Jun Zhou, Hang Su, Bo Zhang, and Xiaolin Hu (2020). « Pruning from Scratch ». In: *Proceedings of the AAAI Conference on Artificial Intelligence* 34.7, pp. 12273–12280. DOI: [10.1609/aaai.v34i07.6910](https://doi.org/10.1609/aaai.v34i07.6910). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/6910>.
- Wu, Ruoyu, Wei Bao, and Liming Ge (2023). « Online Task Assignment with Controllable Processing Time ». In: *arXiv preprint arXiv:2305.04453*.
- Wu, Xueqi and Ada Che (2019). « A memetic differential evolution algorithm for energy-efficient parallel machine scheduling ». In: *Omega* 82, pp. 155–165.
- Xu, Zichuan, Liqian Zhao, Weifa Liang, Omer F Rana, Pan Zhou, Qiufen Xia, Wenzheng Xu, and Guowei Wu (2020). « Energy-aware inference offloading for DNN-driven applications in mobile edge clouds ». In: *IEEE Transactions on Parallel and Distributed Systems* 32.4.
- Yang, Jiwei, Xu Shen, Jun Xing, Xinmei Tian, Houqiang Li, Bing Deng, Jianqiang Huang, and Xian-sheng Hua (2019). « Quantization networks ». In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 7308–7316.
- Yehudai, Gilad and Ohad Shamir (2019). « On the Power and Limitations of Random Features for Understanding Neural Networks ». In: *CoRR* abs/1904.00687. URL: <http://arxiv.org/abs/1904.00687>.

- Yin, Lvjiang, Xinyu Li, Chao Lu, and Liang Gao (2016). « Energy-efficient scheduling problem using an effective hybrid multi-objective evolutionary algorithm ». In: *Sustainability* 8.12, p. 1268.
- Yu, Jiahui and Thomas Huang (2019). *AutoSlim: Towards One-Shot Architecture Search for Channel Numbers*. DOI: [10.48550/ARXIV.1903.11728](https://doi.org/10.48550/ARXIV.1903.11728). URL: <https://arxiv.org/abs/1903.11728>.
- Yu, Jiahui and Thomas S Huang (2019). « Universally slimmable networks and improved training techniques ». In: *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 1803–1811.
- Zhang, Chi, Haisheng Tan, Haoqiang Huang, Zhenhua Han, Shaofeng H-C Jiang, Nikolaos Freris, and Xiang-Yang Li (2020). « Online dispatching and scheduling of jobs with heterogeneous utilities in edge computing ». In: *Proc. ACM MobiHoc*, pp. 101–110.
- Zhang, Haoyu, Ganesh Ananthanarayanan, Peter Bodik, Matthai Philipose, Paramvir Bahl, and Michael J Freedman (2017). « Live video analytics at scale with approximation and {Delay-Tolerance} ». In: *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pp. 377–392.
- Zhang, Jimuyang, Ruizhao Zhu, and Eshed Ohn-Bar (2022). « SelfD: Self-learning large-scale driving policies from the web ». In: *Proc. IEEE/CVF CVPR*, pp. 17316–17326.
- Zhang, Yi-Wen, Rong-Kun Chen, and Zonghua Gu (2023). « Energy-aware partitioned scheduling of imprecise mixed-criticality systems ». In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*.
- Zhou, Hattie, Janice Lan, Rosanne Liu, and Jason Yosinski (2019). « Deconstructing Lottery Tickets: Zeros, Signs, and the Supermask ». In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019 (NIPS 2019)*, pp. 3592–3602.

List of Figures

1.1	The Strong Lottery Ticket setting. A larger randomly initialized source network G is pruned, without training its retained weights, to produce a ticket G_m that approximates a prescribed target model f . Solid magenta edges are retained and dashed gray edges are removed. Individual-weight pruning removes connections independently, whereas whole-neuron pruning removes a hidden unit together with all its incident weights. The two SLTH contributions in this thesis study how the required source size, the sparsity of the resulting ticket, and the granularity of the admissible mask affect approximation.	5
1.2	Scheduling compressible inference tasks. Each task has a release time, a deadline, and a continuous workload–accuracy trade-off: extra computation improves accuracy, with diminishing returns. The scheduler jointly chooses the workload and execution machine for every task, maximizing aggregate accuracy subject to feasibility. The energy-aware setting adds a global processing-energy budget and heterogeneous machine energy efficiencies; the communication-aware setting adds destination-dependent communication delays and effective execution windows.	6
3.1	Simplified representation of the procedure for finding Lottery Tickets (LTH). A large random neural network (step 1) is trained by iterative pruning with rewind: when the loss reaches a local minimum (step 2), some weights with smallest absolute value are pruned (step 3) and the value of the remaining edges is then reset to that of the initialization (step 4); finally, training is resumed and the final network is obtained (step 5). Remarkably, the sparser subnetwork is consistently able to reach a loss not larger than that right after pruning.	27
3.2	Simplified representation of the procedure for finding Strong Lottery Tickets (SLTH), or training by pruning. Previous work has shown that it is possible to sparsify a large random neural network in order to obtain subnetworks that achieve good performance for a task under consideration, motivating the <i>Strong Lottery Ticket Hypothesis</i> . No training is required.	27
3.3	A qualitative plot showing the relationship between the density γ of a winning ticket and the overparameterization required by Theorem 3.3.1 for a target network with m_t parameters. Earlier results from Pensia et al. (2020) and Malach et al. (2020) are shown for comparison.	38
4.1	Target ReLU neuron f , random network g ($N_h = 5$), and subnetwork g_S pruned from g by retaining a subset S of hidden units. For input dimension $d = 2$, each unit i has incoming weights $\mathbf{w}_i \in \mathbb{R}^2$ (not shown) and output weight $a_i \in \mathbb{R}$	47

4.2	Breakpoint alignment along a one-dimensional input family $\mathbf{x}_i(t)$. Along $\mathbf{x}_i(t)$, a bias-free ReLU neuron $\sigma(\langle \mathbf{w}, \mathbf{x} \rangle)$ reduces to the one-dimensional function $t \mapsto \sigma(w_{2i-1}t + w_{2i})$, whose breakpoint is $t_i(\mathbf{w}) = -w_{2i}/w_{2i-1}$. The target (red) has breakpoint t_i^* , while a randomly drawn neuron (teal) has breakpoint $t_{i,j}$. If $ t_{i,j} - t_i^* > \varepsilon$, then on the ε -neighborhood of t_i^* the teal function is linear whereas the target changes slope, yielding a nontrivial uniform error, as stated in Lemma 4.4.2.	52
4.3	Stacked state-line representation of the birth–death chain $(B_s^{\text{bd}})_{s \geq 0}$. Each horizontal line is the state space $\{0, 1, \dots, T\}$ for one input family, and the filled dot marks the current value of B_s^{bd} . A necessary condition for ε -approximation is that all $\lfloor d/2 \rfloor$ independent chains reach state 0.	58
6.1	Energy efficiency vs speed for various GPUs (Desislavov et al., 2023).	79
6.2	Once-for-All accuracy versus floating-point operations (Da Silva Barros et al., 2024).	79
6.3	Gap to the fractional upper bound (average accuracy difference between DSCT-EA-UB and DSCT-EA-APPROX) over 100 experiments with varying task heterogeneity.	91
6.4	Running times of DSCT-EA-APPROX versus the integral solver DSCT-EA-Opt (ApS, 2022) for instances with increasing (a) numbers of tasks, with $m = 5$, and (b) numbers of machines, with $n = 50$.	92
6.5	Average task accuracy for DSCT-EA-APPROX and the baselines over energy budget ratio β , for $m = 2$ and $n = 100$.	93
6.6	Energy profile of 2 machines when varying the energy budget ratio. Machine 1 is more energy efficient than machine 2.	94
7.1	Complete min-cost-flow network for two tasks with a two-segment accuracy function, two intervals, and two heterogeneous-speed machines. Labels are of the form $(\text{capacity}, \text{cost})$, and cost is 0 when only one value is specified. Task 1 is available in both intervals, task 2 only in I_2 .	103
7.2	Mean accuracy for destination-dependent $n = 100$ instances at five representative λ values.	119
7.3	Placement for communication-aware $n = 100$ instances: (a) task and workload shares; (b) remote execution by compute-to-data quartile.	120
7.4	Mean R_f of our algorithm over five common-window $(100, 10)$ instances, compared with the three-level and no-compression baselines.	121
7.5	Dominant-machine sweep for common-window $(100, 10)$ instances: mean R_f , the proved fastest-machine bound, and conjectured $\alpha(s)$.	122
C.1	Gather–scatter and SpMM message-passing paths	159
C.2	Coalescing a coordinate graph	160

Appendix

APPENDIX A

Additional Results and Proofs for Chapter 3

This appendix contains supporting proofs for the fixed-size subset-sum results, the detailed asymptotic calculation used in the comparison with earlier SLTH bounds, and the proof of the lower bound on overparameterization. The proof of the main fixed-size subset-sum theorem remains in Section 3.2.1.

A.1 Uniform sums and the Sum-Bounded property

In this section we provide a detailed proof of Lemma 3.2.2, which states that the uniform distribution in $[-1, 1]$ is sum-bounded, as stated in Definition 3.2.1. We remark that, while the proof is written for uniform random variables, it should be possible to extend it to a family of densities which are unimodal, with bounded variance, and bounded third moment.

Proof of Lemma 3.2.2.

Note first that the distribution of the sum of n i.i.d. variables in $[0, 1]$ is known as the Irwin–Hall distribution I_n .^{*} We will use that $\text{Var}[I_n] = \frac{n}{12}$, where $\text{Var}[X]$ denotes the variance of the random variable X .

For $n \geq 2$, $f(x, n)$ can be defined as the convolution of $f(x) = f(x, 1)$ and $f(x, n - 1)$, i.e.,

$$f(x, n) = \int_{-\infty}^{+\infty} f(x - \tau, n - 1) f(\tau) d\tau.$$

It is straightforward to show, by induction and an elementary substitution in the integral above, which is relied upon in the inductive step, that $f(x, n)$ is symmetric about 0, that is $f(x, n) = f(-x, n)$.

Let us now prove by induction that $f(x, n)$ is nondecreasing on the interval $[-n, 0]$ and nonincreasing over $[0, n]$ (for simplicity, since it vanishes outside $[-n, n]$, we can consider directly the negative half and positive half of the real line, respectively, in the argument that follows).

The claims hold trivially for $f(x)$; also note that

$$f(\tau) = \begin{cases} \frac{1}{2} & \text{if } -1 \leq \tau \leq 1 \\ 0 & \text{otherwise} \end{cases} \implies f(x, n) = \frac{1}{2} \int_{-1}^{+1} f(x - \tau, n - 1) d\tau.$$

^{*} It should be known that I_n is unimodal with a mode in $n/2$, but we were not able to find a reference. It is instructive to note, assuming that I_n is unimodal with a mode in $n/2$, it directly follows that its probability density function is increasing on the interval $[0, n/2]$, and then decreasing over $[n/2, n]$. This implies that $f(x, n)$ (the density of $\Sigma_{[n]}^{\mathcal{U}_n}$), is non decreasing in the interval $[-n, 0]$, has maximum at 0, and non increasing $[0, n]$ for all $n \geq 2$.

If $x \leq x' \leq -1$. Since $x - \tau \leq x' - \tau \leq 0$, by inductive hypothesis we have that $f(x - \tau, n - 1) \leq f(x' - \tau, n - 1)$ over the whole interval $\tau \in [-1, 1]$. Taking integrals yields $f(x, n) \leq f(x', n)$. Now, consider the case when $x \leq -1 \leq x' \leq 0$. If $x + 1 \leq -x' - 1$, $x - \tau \leq x + 1 \leq -x' - 1 \leq -x' + \tau \leq -x + \tau$. By the symmetry about the origin, the inductive hypothesis is $f(x - \tau, n - 1) = f(-x + \tau, n - 1) \leq f(-x' + \tau, n - 1) = f(x' - \tau, n - 1)$ over the whole interval $\tau \in [-1, 1]$, since $-1 \leq -x' + \tau \leq -x + \tau$. Taking integrals yields $f(x, n) \leq f(x', n)$. Otherwise, there exists τ_0 such that $x - \tau_0 = -x' - 1$, $x - \tau > -x' - 1$ for all $\tau \in [-1, \tau_0]$ and $x - \tau < -x' - 1$ for all $\tau \in (\tau_0, 1]$. By symmetry, using $-x = x' + 1 - \tau_0$, $f(x - \tau, n - 1) = f(-x + \tau, n - 1) = f(x' + 1 + \tau - \tau_0, n - 1)$. Thus, for all $\tau \in [-1, \tau_0]$, via the change of variable $\sigma = -(1 + \tau - \tau_0)$ in the middle integral below, we obtain that

$$\int_{-1}^{\tau_0} f(x - \tau, n - 1) d\tau = \int_{-1}^{\tau_0} f(x' + 1 + \tau - \tau_0, n - 1) d\tau = \int_{-1}^{\tau_0} f(x' - \sigma, n - 1) d\sigma. \quad (\text{A.1})$$

For all $\tau \in (\tau_0, 1]$, $x - \tau < -x' - 1 \leq -x' + \tau \leq -x + \tau$, by symmetry about the origin we have that $f(x - \tau, n - 1) \leq f(x' - \tau, n - 1)$ by the inductive hypothesis with the same reasoning of the case $x + 1 \leq -x' - 1$. Taking integrals over the range $[\tau_0, 1]$ for each term of the inductive hypothesis yields

$$\int_{\tau_0}^1 f(x - \tau, n - 1) d\tau \leq \int_{\tau_0}^1 f(x' - \tau, n - 1) d\tau \quad (\text{A.2})$$

Eqs. A.1 and A.2 imply that $f(x, n) \leq f(x', n)$.

Trivially, if $-1 \leq x \leq x' \leq 0$, analogous ideas are put in place as for the previous case, therefore we omit the details. We have thus shown the nondecreasing monotonicity of $f(x, n)$ on the negative half of the real line. By the symmetry of $f(x, n)$ about the origin, on the positive half of the real line the nondecreasing monotonicity turns into nonincreasing monotonicity, and the proof is complete.

Lower bound (first inequality in Eq. 3.2). The variance of $\Sigma_{[n]}^{\mathcal{U}_n}$ is $n/3$ since $\Sigma_{[n]}^{\mathcal{U}_n} = 2(I_n(n) - n/2)$ and $\text{Var}[I_n(n)] = n/12$. We define $Z_n^u = \frac{\Sigma_{[n]}^{\mathcal{U}_n}}{\sqrt{n/3}}$ and we note with F_n its cumulative distribution function. Z_n^u has expectation 0 and standard deviation 1. Consider the probability

$$P_L(n) = \Pr(\sqrt{n} \leq \Sigma_{[n]}^{\mathcal{U}_n} \leq 2\sqrt{n}) = \Pr(\sqrt{3} \leq Z_n^u \leq 2\sqrt{3}).$$

Now, we use the following form of Berry–Esseen inequality, discussed in (Marengo et al., 2017, p. 2).[†]

Theorem A.1.1 (Allasia (Allasia, 1981)). *For all $n \geq 1$,*

$$|F_n(z) - \Phi(z)| \leq \frac{\sqrt{3}}{20\sqrt{n}},$$

where $\Phi(z)$ is the cumulative distribution function of the standard normal distribution.

[†]. It is also possible to obtain our result via classical Berry–Esseen inequality, due to the improved upper bound of 0.4748 on the absolute constant, provided in (Shevtsova, 2011). This would require replacing with 900 the cut-off value for n , which is 18 in the current version of the argument.

Theorem A.1.1 implies

$$P_L(n) \geq \Phi(2\sqrt{3}) - \Phi(\sqrt{3}) - 2 \cdot \frac{\sqrt{3}}{20\sqrt{n}}.$$

When $n \geq 18$,

$$\Phi(2\sqrt{3}) - \Phi(\sqrt{3}) - 2 \cdot \frac{\sqrt{3}}{20\sqrt{n}} \geq \Phi(2\sqrt{3}) - \Phi(\sqrt{3}) - 2 \cdot \frac{\sqrt{3}}{20\sqrt{18}} = C_{18} > 0.$$

That is $P_L(n) \geq C_{18} > 0$. When $2 \leq n < 18$, $P_L(n) = F_n(2\sqrt{3}) - F_n(\sqrt{3}) = c_n > 0$. We thus have

$$P_L(n) \geq \min\{C_i, \text{ for } 2 \leq i \leq 18\} = c'_l > 0.$$

Recall that $P_L(n) = \Pr(\sqrt{n} \leq \Sigma_{[n]}^{\mathcal{U}_n} \leq 2\sqrt{n})$. As the density $f(x, n)$ is decreasing on $\mathbb{R}^+$, we have

$$P_L(n) \leq f(\sqrt{n}, n)\sqrt{n}.$$

Thus,

$$f(\sqrt{n}, n) \geq \frac{P_L(n)}{\sqrt{n}}.$$

Since $P_L(n) \geq c'_l$ then for all $n \geq 2$

$$f(\sqrt{n}, n) \geq \frac{c'_l}{\sqrt{n}}.$$

When $n = 1$, the density $f(1, 1) = \frac{1}{2}$. So, by setting $c_l = \min(c'_l, \frac{1}{2})$, we get that, for all $n \geq 1$, for all $0 \leq x \leq \sqrt{n}$:

$$f(x, n) \geq f(\sqrt{n}, n) \geq \frac{c_l}{\sqrt{n}}.$$

By a symmetric argument, we also have for all $n \geq 1$, for all $-\sqrt{n} \leq x \leq 0$:

$$f(x, n) \geq f(-\sqrt{n}, n) \geq \frac{c_l}{\sqrt{n}}.$$

Upper bound (second inequality in Eq. 3.2). Here, we bound the probability distribution function $f(x, n)$ of $\Sigma_{[n]}^{\mathcal{U}_n} = \sqrt{n/3}Z_n$, where we recall that $Z_n^u = \frac{\Sigma_{[n]}^{\mathcal{U}_n}}{\sqrt{n/3}}$. Denoting f_z the probability distribution function of Z_n^u , we have

$$f_z(x, n) = f\left(\sqrt{\frac{n}{3}}x, n\right) \sqrt{\frac{n}{3}}.$$

We use the following local limit theorem, discussed in (Petrov, 1975, p. 214).

Theorem A.1.2 (Sahaidarova (Shakhaidarova, 1966)). *Let $\{X_n\}$ be a sequence of independent random variables with a common density $p(x)$, such that $E[|X_1|^3] < \infty$, $E[X_1] = 0$, $E[X_1^2] = 1$ and $\sup p(x) \leq C$. Let $p_n(x)$ be the density of the random variable $\frac{1}{\sqrt{n}} \sum_{j=1}^n X_j$. Then*

$$\sup_x |p_n(x) - \phi(x)| \leq \frac{A\beta_3}{\sqrt{n}} \max(1, C^3),$$

where ϕ is the probability distribution function of a standard gaussian, A is an absolute constant, and $\beta_3 = E[|X_1|^3]$.

The theorem can be applied to a uniform continuous distribution with density $p^u(x) = \frac{1}{2\sqrt{3}}$ in the interval $[-\sqrt{3}, \sqrt{3}]$, which has mean 0 and variance 1. We thus get, for every $x \in \mathbb{R}$,

$$f_z(x, n) = p_n^u(x) \leq \phi(0) + \frac{A\beta_3}{\sqrt{n}} = \frac{1}{2\pi} + \frac{A\beta_3}{\sqrt{n}} \leq \frac{1}{2\pi} + A\frac{3\sqrt{3}}{4} = c'_u.$$

In conclusion, setting $c_u = \sqrt{3}c'_u$, for every $x \in \mathbb{R}$ it holds that

$$f(x, n) = \sqrt{\frac{3}{n}} f_z\left(\sqrt{\frac{3}{n}}x, n\right) \leq \frac{\sqrt{3}c'_u}{\sqrt{n}} = \frac{c_u}{\sqrt{n}}. \quad \square$$

A.2 Probability amplification

Proof of Corollary 3.2.3.

As anticipated, we proceed in three steps.

Step 1: Hoeffding bound. We start by showing, following the idea at the base of (Lueker, 1998, Corollary 3.3), that if n' is large enough, a standard Hoeffding bound ensures that with high probability a constant fraction of the sample follows a $\text{Uniform}[-1, 1]$ distribution. Since we assumed that every X_i is a mixture of a $\text{Uniform}[-1, 1]$ distribution with probability p , and another distribution with density g (given by the factors G_i), we can rewrite $X_i = B_i \cdot U_i + (1 - B_i) \cdot G_i$, with U_i being the uniform random variable, G_i being the random variable with density g , B_i being independent Bernoulli random variables with probability p .

Fix $\alpha = \alpha(p) \neq p$, and assume, for now, that n' satisfies Eq. 3.1, and therefore, since $\varepsilon < 1/2$, choosing $c_{\text{hyp}} = c_{\text{hyp}}(p) \geq (\alpha - p)^{-2}$, ensures that, defining $\varepsilon' = \varepsilon/2$,

$$n' \geq c_{\text{hyp}} \log_2 \frac{1}{\varepsilon} \geq \frac{1}{2(\alpha - p)^2} \ln \frac{1}{\varepsilon'}$$

and therefore

$$\Pr\left(\sum_i^{n'} B_i \leq \alpha n'\right) \leq e^{-2(\alpha-p)^2 n'} \leq e^{-\ln \frac{1}{\varepsilon'}} = \varepsilon'.$$

Thus

$$\Pr\left(\sum_i^{n'} B_i > \alpha n'\right) \geq 1 - \varepsilon',$$

that is, with high probability, there is a set of indices $I \subseteq [n']$ of size $|I| \geq \alpha n'$, such that for each $i \in I$ it holds $B_i = 1$, i.e. X_i is uniformly distributed.

Step 2: Application of Theorem 3.2.1 via rejection-sampling. Lemma 3.2.2 ensures that the uniform distribution of the $|I|$ random variables selected in *Step 1* is sum-bounded. Conditionally on the event $\{\sum_i^{n'} B_i > \alpha n'\}$, we can discard all random variables indexed outside I and apply directly Theorem 3.2.1 to $\alpha n'$ of the remaining ones, for any fixed k and $z \in [-\sqrt{k}, \sqrt{k}]$,

since $\alpha c_{\text{hyp}} \geq 1$ by construction. This guarantees a success probability of c'_{thm} for approximating the given target z ; thus,

$$\begin{aligned} & \Pr(\exists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon') \geq \\ & \Pr\left(\exists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon' \mid \sum_i^{n'} B_i > \alpha n'\right) \Pr\left(\sum_i^{n'} B_i > \alpha n'\right) \geq \\ & \Pr\left(\exists S_z \subset I, |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon' \mid |I| > \alpha n'\right) (1 - \varepsilon') \geq c'_{\text{thm}}(1 - \varepsilon') \geq \frac{3}{4}c'_{\text{thm}} = c_{\text{thm}}. \end{aligned}$$

Step 3: Amplification. We now amplify the constant success probability from Step 2 and extend the guarantee to all targets.

Recall that $\varepsilon' = \frac{\varepsilon}{2}$, and let $c_{\text{amp}} = c_{\text{amp}}(p) = 8 \frac{c_{\text{hyp}}}{c_{\text{thm}}}$ and $r = \frac{4}{c_{\text{thm}}} \ln \frac{k}{\varepsilon}$. By assumption,

$$n \geq c_{\text{amp}} \frac{\log_2^2 \frac{k}{\varepsilon}}{H_2\left(\frac{k}{n}\right)} \geq 2rc_{\text{hyp}} \frac{\log_2 \frac{k}{\varepsilon}}{H_2\left(\frac{k}{n}\right)} \geq rc_{\text{hyp}} \frac{\log_2 \frac{k}{\varepsilon'}}{H_2\left(\frac{k}{n}\right)},$$

where the last inequality is ensured by $\varepsilon < 1/2$. By Step 2, we can apply Theorem 3.2.1, with ε' and $n' \geq c_{\text{hyp}} \frac{\log_2 \frac{k}{\varepsilon'}}{H_2\left(\frac{k}{n}\right)} = n^*$, allowing us to prove that we can ε' -approximate any target z with probability at least c_{thm} . The probability of failing to approximate some given z is then at most $1 - c_{\text{thm}}$. From the sample Ω of sum-bounded random variables take r subsamples (without replacement) of cardinality n^* each, $\Omega_1, \dots, \Omega_r$. The probability of failing to approximate some given z with subsetsums from Ω is less than that of failing to approximate it with subsetsums from within every Ω_i 's, and the latter probability is at most $(1 - c_{\text{thm}})^r$; thus, for every $z \in [-\sqrt{k}, \sqrt{k}]$,

$$\Pr(\nexists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon') \leq (1 - c_{\text{thm}})^r.$$

By an union bound, we also have that

$$\begin{aligned} & \Pr\left(\forall z \in [-\sqrt{k}, \sqrt{k}], \exists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon\right) \\ & \geq \Pr\left(\forall z \in \left\{-\sqrt{k} + i\varepsilon' : i \in \left[\frac{2}{\varepsilon'}\sqrt{k}\right]\right\}, \exists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon',\right) \\ & = 1 - \Pr\left(\exists z \in \left\{-\sqrt{k} + i\varepsilon' : i \in \left[\frac{2}{\varepsilon'}\sqrt{k}\right]\right\}, \nexists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon'\right) \\ & \geq 1 - \sum_{z \in \left\{-\sqrt{k} + i\varepsilon' : i \in \left[\frac{2}{\varepsilon'}\sqrt{k}\right]\right\}} \Pr(\nexists S_z \subset [n], |S_z| = k : |\Sigma_{S_z} - z| < \varepsilon') \\ & \geq 1 - \frac{2}{\varepsilon'}\sqrt{k} (1 - c_{\text{thm}})^r = 1 - \frac{2}{\varepsilon'}\sqrt{k} \exp\left(\frac{4}{c_{\text{thm}}} \ln\left(\frac{k}{\varepsilon}\right) \cdot \ln(1 - c_{\text{thm}})\right) \\ & \geq 1 - \frac{2}{\varepsilon'}\sqrt{k} \exp\left(-4 \ln \frac{k}{\varepsilon}\right) = 1 - \frac{2}{\varepsilon'}\sqrt{k} \frac{\varepsilon^4}{k^4} \geq 1 - 4\varepsilon^3 \geq 1 - \varepsilon, \end{aligned}$$

where the last inequality is ensured by $\varepsilon < 1/2$. This completes the proof. $\square$

A.3 Simplified asymptotic condition

Proof of Corollary 3.2.4.

By definition of binary entropy, we have

$$H_2\left(\frac{k}{n}\right) = \frac{k}{n} \log_2\left(\frac{n}{k}\right) + \left(1 - \frac{k}{n}\right) \log_2 \frac{n}{n-k} \quad (\text{A.3})$$

In particular, since both terms in the previous equation are positive, we get

$$H_2\left(\frac{k}{n}\right) \geq \frac{k}{n} \log_2\left(\frac{n}{k}\right) \quad (\text{A.4})$$

We now use Equation (A.4) to derive an upper bound for the quantity $\frac{c_{\text{amp}}}{H_2(\frac{k}{n})} \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{n}$, which will be used later:

$$\begin{aligned} \frac{c_{\text{amp}}}{H_2\left(\frac{k}{n}\right)} \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{n} &\leq \frac{c_{\text{amp}}}{\frac{k}{n} \log_2\left(\frac{n}{k}\right)} \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{n} \\ &= c_{\text{amp}} \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{k} \frac{1}{\log_2\left(\frac{n}{k}\right)} \end{aligned} \quad (\text{A.5})$$

$$\leq c_{\text{amp}} \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{k} \quad (\text{A.6})$$

$$\leq \frac{1}{2}, \quad (\text{A.7})$$

where from Equation (A.5) to Equation (A.6) we used that $\log_2 n/k \geq 1$ for $k \leq n/2$, and then the hypothesis $k \geq 2c_{\text{amp}} \left(\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}\right)$ directly gives Equation (A.7). Let us now rewrite Equation (3.3) in a more convenient form:

$$\begin{aligned} n \frac{H_2\left(\frac{k}{n}\right)}{c_{\text{amp}}} &\geq \log_2^2 \frac{k}{\varepsilon} \\ n \frac{H_2\left(\frac{k}{n}\right)}{c_{\text{amp}}} &\geq \log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon} + \log_2^2 \frac{1}{\varepsilon} \\ n \left(\frac{H_2\left(\frac{k}{n}\right)}{c_{\text{amp}}} - \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{n} \right) &\geq \log_2^2 \frac{1}{\varepsilon} \\ n \left(1 - \frac{c_{\text{amp}}}{H_2\left(\frac{k}{n}\right)} \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{n} \right) &\geq \frac{c_{\text{amp}}}{H_2\left(\frac{k}{n}\right)} \log_2^2 \frac{1}{\varepsilon} \end{aligned} \quad (\text{A.8})$$

$$n \geq \frac{c_{\text{amp}}}{\left(1 - \frac{c_{\text{amp}}}{H_2\left(\frac{k}{n}\right)} \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{n}\right)} \log_2^2 \frac{1}{\varepsilon} \quad (\text{A.9})$$

Using Equation (A.7) we get

$$\frac{c_{\text{amp}}}{\left(1 - \frac{c_{\text{amp}}}{H_2\left(\frac{k}{n}\right)} \frac{\log_2^2 k + 2 \log_2 k \cdot \log_2 \frac{1}{\varepsilon}}{n}\right)} \leq 2c_{\text{amp}} \quad (\text{A.10})$$

To satisfy Equation (A.9), we can then choose n such that

$$n \geq 2c_{\text{amp}} \frac{\log_2^2 \frac{1}{\varepsilon}}{H_2\left(\frac{k}{n}\right)}, \quad (\text{A.11})$$

and then apply Corollary 3.2.3 to end the proof. $\square$

A.4 Details of the comparison with Malach et al.

We verify the inequality used in the comparison with Malach et al. (2020). We show that $cz \geq c_{\text{amp}} \frac{\log_2^2(cz^2)}{\log_2(z)}$ holds for a sufficiently large constant c . Recall that $z = \frac{m_t}{\varepsilon}$, so we can assume $\log_2(z) \geq 1$. We have

$$\log_2^2(cz^2) = (\log_2(c) + 2\log_2(z))^2 \quad (\text{A.12})$$

$$= \log_2^2(c) + 4\log_2(c)\log_2(z) + 4\log_2^2(z) \quad (\text{A.13})$$

$$\stackrel{(a)}{\leq} 6(\log_2^2(c) + \log_2^2(z)) \quad (\text{A.14})$$

$$\stackrel{(b)}{\leq} 12\log_2^2(c)\log_2^2(z), \quad (\text{A.15})$$

where in (a) we used that $2ab \leq a^2 + b^2$, and in (b) that $a + b \leq 2ab$ for a and b greater than 1.

We can then focus on showing that there is a big enough constant c such that $cz \geq 12c_{\text{amp}} \log_2^2(c)\log_2^2(z)$. We get $c \geq 12c_{\text{amp}} \log_2^2(c) \frac{\log_2^2(z)}{z}$, and we have

$$\log_2^2(c) \frac{\log_2^2(z)}{z} \leq \log_2^2(c) \quad (\text{A.16})$$

$$\leq \sqrt{c}. \quad (\text{A.17})$$

We can then focus on $c \geq 12c_{\text{amp}}\sqrt{c}$, which is satisfied for $c \geq 144c_{\text{amp}}^2$.

A.5 Proof of Theorem 3.3.5

Proof of Theorem 3.3.5.

Consider the space $\mathcal{W}_k = \{\mathbf{W} \in \mathbb{R}^{d \times d} : \|\mathbf{W}\| \leq \sqrt{k}\}$, and let $\mathcal{P}_k$ be a 2ε -separated set of $\mathcal{W}_k$, i.e. a subset $\mathcal{P}_k \subset \mathcal{W}_k$ such that for all distinct $\mathbf{W}, \mathbf{W}' \in \mathcal{P}_k$ it holds $\|\mathbf{W} - \mathbf{W}'\| > 2\varepsilon$. We denote $\mathcal{W} = \mathcal{W}_1$, $\mathcal{P} = \mathcal{P}_1$, and the set of all possible subnetworks of g as $\mathcal{G}$ (note that this does not denote $\mathcal{G}_1$, the set of all subnetworks of size 1).

Step 1: Packing argument. In (Pensia et al., 2020, Theorem 2, Step 1), it is shown that any function g' can only approximate at most one member of $\mathcal{P}$ for bounded input $\mathbf{x}$ (say, $\|\mathbf{x}\| \leq 1$). In particular, this also applies to functions g' representing the elements of $\mathcal{G}_k$.

Step 2: Relation between $|\mathcal{G}_k|$ and $|\mathcal{P}_k|$. By *Step 1*, in (Pensia et al., 2020, Theorem 2, Step 2) it is shown that $|\mathcal{P}| \leq 2|\mathcal{G}|$, under the assumption of Eq. 3.32, with $\mathcal{G}_k$ replaced by $\mathcal{G}$. Therefore, also by *Step 1*, replacing $\mathcal{P}$ with $\mathcal{P}_k$ and $\mathcal{G}$ with $\mathcal{G}_k$ in (Pensia et al., 2020, Theorem 2, Step 2), it holds that $|\mathcal{P}_k| \leq 2|\mathcal{G}_k|$. Note that $|\mathcal{G}_k| = \binom{n}{k}$, the number of different ways in which we can select k parameters out of n , so we actually get

$$\binom{n}{k} > \frac{|\mathcal{P}_k|}{2}. \quad (\text{A.18})$$

Step 3: Lower bound on $|\mathcal{P}_k|$. Let us now consider a 2ε -separated set $\mathcal{P}_k^{\max}$ of maximal cardinality. In (Pensia et al., 2020, Theorem 2, Step 3) it is shown that

$$|\mathcal{P}^{\max}| \geq \frac{\text{Vol}(\mathcal{W})}{\text{Vol}(\{\mathbf{W} \in \mathcal{W} : \|\mathbf{W}\| \leq 2\varepsilon\})} = \left(\frac{1}{2\varepsilon}\right)^{d^2}.$$

Here Vol is the Lebesgue measure in $\mathbb{R}^{d \times d}$ identified with $\mathbb{R}^{d^2}$. By the exact same argument, replacing $\mathcal{W}$ with $\mathcal{W}_k$ and thus $\mathcal{P}^{\max}$ with $\mathcal{P}_k^{\max}$, it holds that

$$|\mathcal{P}_k^{\max}| \geq \frac{\text{Vol}(\mathcal{W}_k)}{\text{Vol}(\{\mathbf{W} \in \mathcal{W}_k : \|\mathbf{W}\| \leq 2\varepsilon\})} = \left(\frac{\sqrt{k}}{2\varepsilon}\right)^{d^2}.$$

Combining this fact with Eq. A.18 applied to $\mathcal{P}_k^{\max}$ implies that

$$\binom{n}{k} > \frac{1}{2} \left(\frac{\sqrt{k}}{2\varepsilon}\right)^{d^2}. \quad (\text{A.19})$$

Step 4: Lower bound on n . Consider the standard bound found in (MacWilliams and Sloane, 1977)

$$\binom{n}{k} \leq \sqrt{\frac{n}{2\pi k(n-k)}} 2^{nH_2(k/n)}.$$

and combine it with Eq. A.19. It follows that

$$2^{nH_2(k/n)} \geq \frac{1}{2} \sqrt{\frac{2\pi k(n-k)}{n}} \left(\frac{\sqrt{k}}{2\varepsilon}\right)^{d^2}$$

and taking the logarithm of both sides yields the sought lower bound on n :

$$nH_2\left(\frac{k}{n}\right) \geq \frac{1}{2} \log_2 \left(\frac{2\pi k(n-k)}{n}\right) + d^2 \log_2 \frac{\sqrt{k}}{2\varepsilon} - 1 \quad (\text{A.20})$$

$$\geq d^2 \left(\frac{1}{2} \log_2 k + \log_2 \frac{1}{\varepsilon} - 1\right) - 1 \quad (\text{A.21})$$

$$\geq \frac{d^2}{2} \log_2 \frac{k}{\varepsilon}, \quad (\text{A.22})$$

where from Eq. A.20 to Eq. A.21 we exploited the definition of λ , which ensures that the first term in the r.h.s. of Eq. A.20 is nonnegative;[‡] from Eq. A.21 to Eq. A.22 we used that for all $\varepsilon < 1/16$ it holds that

$$d^2 \left(\log_2 \frac{1}{\varepsilon} - 1 \right) \geq 1. \quad \square$$

[‡]. This term being nonnegative is equivalent to $k(1 - k/n) \geq 1/2\pi$, and since $1 \leq k \leq \lambda n$, any $\lambda \leq 1 - 1/2\pi$ ensures it.

APPENDIX B

Additional Results and Proofs for Chapter 4

This appendix collects the output-weight variants and the restricted single-neuron result associated with Chapter 4. The proof of the chapter’s main lower bound remains in Section 4.4.

B.1 Extensions and Variants

The main lower bound is stated for unit output weights. We now show that the same mechanism applies more broadly and then consider the distinct case in which the output weights are also randomly initialized.

B.1.1 Output weights bounded away from zero

We explain how the proof of Theorem 4.3.1 extends to the case where the output weights are not fixed to 1, but have magnitude bounded away from 0. Consider the network $g(\mathbf{x}) = \sum_{j=1}^{N_h} a_j \sigma(\langle \mathbf{w}_j, \mathbf{x} \rangle)$, where the hidden weights $\mathbf{w}_j \sim \mathcal{N}(0, I_d)$ are independent, and where the coefficients a_j are either deterministic or random and independent of the hidden weights. Assume that $|a_j| \geq a_-$ for some constant $a_- > 0$ and all j .

The proof proceeds exactly as in the unit-weight case after observing that, along any input family $\mathbf{x}_i(t)$, the restriction of the network is $g(\mathbf{x}_i(t)) = \sum_{j \in S} a_j \sigma(w_{j,2i-1}t + w_{j,2i})$. The breakpoint locations remain $t_{i,j} = -w_{j,2i}/w_{j,2i-1}$, so they do not depend on the coefficients a_j . Writing the slope jump as the right derivative minus the left derivative, we have $\Delta_{i,j} = a_j |w_{j,2i-1}|$, $|\Delta_{i,j}| = |a_j| |w_{j,2i-1}|$. Since $|a_j| \geq a_-$, the event

$$|w_{j,2i-1}| \geq \kappa_{\text{br}}/a_-$$

guarantees $|\Delta_{i,j}| \geq \kappa_{\text{br}}$. The sign of a_j is irrelevant for the birth estimate: the strengthened Lemma 4.4.6 depends only on the magnitude of the new slope jump and applies to any previous unbroken configuration. Consequently, a sufficiently steep, well-placed breakpoint creates a birth with a history-uniform probability $p_0(a_-) > 0$.

The sign is also irrelevant for the death upper bound. The capped count can decrease only if the new breakpoint lies in a currently broken bin, and the comparison already makes the favorable assumption that every such hit repairs that bin. Different signs can change whether cancellation actually occurs, but they cannot increase its probability beyond this location-based bound. Thus the conditional transition inequalities Equation (4.4)–Equation (4.5) continue to hold, with only the birth constant depending on a_- .

If the coefficients are random, we condition on their complete realization. The input-family processes are then independent because they involve disjoint coordinates of the hidden weights, and all the preceding bounds are uniform over coefficient realizations satisfying $|a_j| \geq a_-$. Integrating over the coefficients therefore preserves the product estimate across input families and the final counting argument. The same width lower bound follows, with constants depending on a_- .

B.1.2 Randomly initialized output weights

We next prove Theorem 4.3.2. Unlike the preceding extension, this result requires a different argument based on a simpler necessary condition for approximation.

Proof of Theorem 4.3.2.

Because the Gaussian distribution $\mathcal{N}(0, I_d)$ and the input domain $\{\mathbf{x} : \|\mathbf{x}\|_2 \leq R\}$ are perfectly symmetric, the problem is invariant to orthogonal transformations. Therefore, we can rotate the coordinate system so that the vector $\mathbf{w}^*$ aligns with the first standard basis vector. Thus, without loss of generality, we assume $\mathbf{w}^* = \sqrt{d}\mathbf{e}_1$.

For each $i \in \{2, \dots, d\}$, define the one-dimensional input family $\mathbf{x}_i(t) := t\mathbf{e}_1 + \mathbf{e}_i, t \in \mathbb{R}$. Along this path, the output of the target neuron is $f(\mathbf{x}_i(t)) = \sigma(\langle \sqrt{d}\mathbf{e}_1, \mathbf{x}_i(t) \rangle) = \sigma(\sqrt{d}t)$, so its breakpoint is at $t^* = 0$. Note that despite the shared notation, these are distinct from the input families considered in the proof of Theorem 4.3.1. Let $\varepsilon' = \varepsilon/\sqrt{d}$, and let $I := [-\varepsilon', \varepsilon']$. By Lemma 4.4.2, applied to $f(t) = \sigma(\sqrt{d}t)$ and I , any function that is linear on an interval containing I cannot approximate $\sigma(\sqrt{d}t)$ on I with error smaller than $\varepsilon/4$.

Now fix a subset $S \subseteq [N_h]$ and consider the pruned network $g_S(\mathbf{x}) = \sum_{j \in S} a_j \sigma(\langle \mathbf{w}_j, \mathbf{x} \rangle)$, where each hidden neuron j has an incoming weight vector $\mathbf{w}_j = (w_{j,1}, \dots, w_{j,d})$. Its restriction to the i -th family $\mathbf{x}_i(t)$ is

$$g_{S,i}(t) := g_S(\mathbf{x}_i(t)) = \sum_{j \in S} a_j \sigma(w_{j,1}t + w_{j,i}),$$

whose breakpoints are exactly $t_{i,j} := -w_{j,i}/w_{j,1}$, $j \in S$. If none of these breakpoints falls in I , then $g_{S,i}$ is linear on I and cannot $(\varepsilon/4)$ -approximate $\sigma(\sqrt{d}t)$ there. Hence, for each input family $i \in \{2, \dots, d\}$, successful approximation implies the event

$$E_i := \left\{ \exists j \in [N_h] \text{ such that } t_{i,j} \in I \right\}.$$

In particular, overall success implies $\bigcap_{i=2}^d E_i$. We now bound the probability of these events. Let $Z_j := w_{j,1}$ and write

$$L_1 := \sum_{j=1}^{N_h} |Z_j| \quad \text{and} \quad L_2 := \sum_{j=1}^{N_h} Z_j^2.$$

Intuitively, by controlling the sums $L_1 = \sum_j |Z_j|$ and $L_2 = \sum_j Z_j^2$, we rule out the possibility that the shared denominators are collectively large enough to make breakpoint hits likely across all input families.

For fixed i and j , conditional on $Z_j = z$, the breakpoint condition $t_{i,j} \in I$ is equivalent to $|w_{j,i}| \leq \varepsilon'|z|$. Because $w_{j,i} \sim \mathcal{N}(0, 1)$, the probability of the complementary event $\Pr(|w_{j,i}| >$

x) equals 1 at $x = 0$ and decays as roughly $e^{-x^2/2}$ for large x . Consequently, there exist universal constants $a, b > 0$ such that

$$\Pr(|w_{j,i}| > x) \geq \exp(-ax - bx^2) \quad \text{for all } x \geq 0.$$

Thus, the conditional probability that a single unit has no breakpoint in I is bounded below by

$$\Pr(t_{i,j} \notin I \mid Z_j = z) \geq \exp(-a\varepsilon'|z| - b\varepsilon'^2 z^2).$$

Consequently, conditioning on $Z := (Z_1, \dots, Z_{N_h})$ and using independence across neurons, we can bound the probability of E_i by:

$$\Pr(E_i \mid Z) = 1 - \prod_{j=1}^{N_h} \Pr(t_{i,j} \notin I \mid Z_j) \leq 1 - \exp(-a\varepsilon' L_1 - b\varepsilon'^2 L_2).$$

Moreover, conditioned on Z , the events $E_2, \dots, E_d$ are independent, because they depend on disjoint coordinate families $\{w_{j,i}\}_{j=1}^{N_h}$ for different $i \geq 2$. Therefore,

$$\Pr(\text{success} \mid Z) \leq \left(1 - \exp(-a\varepsilon' L_1 - b\varepsilon'^2 L_2)\right)^{d-1}.$$

It remains to control L_1 and L_2 . Since $Z_j \sim \mathcal{N}(0, 1)$, we have $\mathbb{E}[|Z_j|] = \sqrt{2/\pi} < 1$ and $\mathbb{E}[Z_j^2] = 1$, so $\mathbb{E}[L_1] < N_h$ and $\mathbb{E}[L_2] = N_h$. By Markov's inequality,

$$\Pr\left(L_1 > \frac{4N_h}{\delta}\right) \leq \frac{\mathbb{E}[L_1]}{4N_h/\delta} < \frac{\delta}{4}, \quad \text{and} \quad \Pr\left(L_2 > \frac{4N_h}{\delta}\right) \leq \frac{\mathbb{E}[L_2]}{4N_h/\delta} = \frac{\delta}{4}.$$

By a union bound on these two events, with probability at least $1 - \delta/2$, both $L_1 \leq KN_h$ and $L_2 \leq KN_h$ hold, where $K = 4/\delta$. On this high-probability event, and using $\varepsilon' \in (0, 1)$, we have

$$a\varepsilon' L_1 + b\varepsilon'^2 L_2 \leq a\varepsilon' KN_h + b\varepsilon' KN_h = (a + b)KN_h\varepsilon'.$$

Let $c_1 = a + b$. We then have

$$\Pr(\text{success} \mid Z) \leq \left(1 - \exp(-c_1 KN_h\varepsilon')\right)^{d-1}.$$

Now choose $c_0 > 0$ small enough so that $c_1 K c_0 \leq 1/2$. If $N_h < c_0 \frac{\log d}{\varepsilon'}$, then $\exp(-c_1 KN_h\varepsilon') \geq d^{-1/2}$, and hence for $d \geq 2$,

$$\Pr(\text{success} \mid Z) \leq (1 - d^{-1/2})^{d-1} \leq (e^{-d^{-1/2}})^{d-1} \leq e^{-\sqrt{d}/2}.$$

For all sufficiently large d , this is at most $\delta/2$; the finitely many remaining dimensions can be absorbed by shrinking c_0 once more if needed. Therefore, using the total law of probability

$$\Pr(\text{success}) \leq 1 \cdot \Pr(L_1 > KN_h \cup L_2 > KN_h) + \Pr(\text{success} \mid L_1, L_2 \leq KN_h) \cdot 1 \leq \delta.$$

This proves the theorem. $\square$

B.2 The Single-Neuron Regime

The preceding lower bounds leave open the optimal dependence on the input dimension d . To isolate this dependence, we consider the restricted setting in which the pruned network may retain only one neuron. In this regime, successful approximation requires a strong geometric alignment between the retained neuron and the target, leading to an exponential width requirement.

Lemma B.2.1 (Single-neuron approximation requires exponential width). *Let $d \geq 2$. Given $\mathbf{w}^* \in \mathbb{R}^d$, with $\|\mathbf{w}^*\|_2 = 1$, let $f(\mathbf{x}) := \sigma(\langle \mathbf{w}^*, \mathbf{x} \rangle)$ be a target ReLU neuron without bias. Let $\mathbf{w}$ be sampled uniformly at random from the unit sphere $\mathbb{S}^{d-1}$, and consider approximations of the form*

$$g(\mathbf{x}) := a \sigma(\langle \mathbf{w}, \mathbf{x} \rangle), \quad a \in \mathbb{R}.$$

There exist universal constants $c, C > 0$ such that for any $\varepsilon \in (0, 1)$,

$$\Pr \left(\inf_{a \in \mathbb{R}} \sup_{\|\mathbf{x}\|_2 \leq 1} |f(\mathbf{x}) - g(\mathbf{x})| \leq \varepsilon \right) \leq (C\varepsilon)^{cd}.$$

Consequently, if a width- N_h random network is initialized with i.i.d. weights $\mathbf{w}_1, \dots, \mathbf{w}_{N_h} \sim \text{Unif}(\mathbb{S}^{d-1})$ and only a single neuron may be retained, then, for every fixed sufficiently small ε , achieving uniform ε -approximation of f with probability bounded below by a positive constant requires $N_h \geq e^{\Omega(d)}$.

Proof.

Fix $\varepsilon \in (0, 1)$. The function of a inside the infimum is continuous and tends to infinity as $|a| \rightarrow \infty$ (as can be seen by evaluating at $\mathbf{x} = \mathbf{w}$), so the infimum is attained. Suppose that for some $a \in \mathbb{R}$,

$$\sup_{\|\mathbf{x}\|_2 \leq 1} |\sigma(\langle \mathbf{w}^*, \mathbf{x} \rangle) - a \sigma(\langle \mathbf{w}, \mathbf{x} \rangle)| \leq \varepsilon. \quad (\text{B.1})$$

Let $\theta \in [0, \pi]$ denote the angle between $\mathbf{w}$ and $\mathbf{w}^*$. Evaluating (B.1) at $\mathbf{x} = \mathbf{w}^*$ yields

$$|1 - a \sigma(\cos \theta)| \leq \varepsilon.$$

Because $\varepsilon < 1$, we must have $a \sigma(\cos \theta) \geq 1 - \varepsilon > 0$. In particular, $\cos \theta > 0$, so $\theta \in [0, \pi/2)$. If $\theta = 0$, the desired angular bound is immediate. Otherwise, set

$$\mathbf{x} := \frac{\mathbf{w}^* - \cos \theta \mathbf{w}}{\sin \theta}.$$

Then $\|\mathbf{x}\|_2 = 1$, $\langle \mathbf{w}, \mathbf{x} \rangle = 0$, and $\langle \mathbf{w}^*, \mathbf{x} \rangle = \sin \theta$. For this $\mathbf{x}$ we have $\sigma(\langle \mathbf{w}^*, \mathbf{x} \rangle) = \sin \theta$ and $\sigma(\langle \mathbf{w}, \mathbf{x} \rangle) = 0$, and therefore

$$|f(\mathbf{x}) - g(\mathbf{x})| = \sin \theta.$$

Combining with (B.1) yields

$$\sin \theta \leq \varepsilon.$$

Since $\theta \in [0, \pi/2)$ and $\sin \theta \geq 2\theta/\pi$ on this interval, it follows that $\theta \leq (\pi/2)\varepsilon < 2\varepsilon$.

Since $\mathbf{w}$ is uniformly distributed on $\mathbb{S}^{d-1}$, the probability that $\angle(\mathbf{w}, \mathbf{w}^*) \leq 2\varepsilon$ equals the normalized surface area of a spherical cap of half-angle 2ε , where by $\angle(\mathbf{w}, \mathbf{w}^*)$ we denote the angle

between the two vectors $\mathbf{w}$ and $\mathbf{w}^*$. It is standard that there exist universal constants $c, C > 0$ such that for all $\varepsilon \in (0, 1)$,

$$\Pr(\angle(\mathbf{w}, \mathbf{w}^*) \leq 2\varepsilon) \leq (C\varepsilon)^{c(d-1)}.$$

Since $d \geq 2$, decreasing c if necessary gives the claimed exponent cd .

If N_h neurons are sampled independently, a union bound shows that the probability that *any* single neuron can ε -approximate f is at most $N_h(C\varepsilon)^{cd}$. Hence, for every fixed ε such that $C\varepsilon < 1$, making this probability bounded below by a positive constant requires $N_h \geq (C\varepsilon)^{-cd} = e^{\Omega(d)}$. $\square$

Sparse GPU Message Passing for Graph Neural Networks in Julia

This appendix describes sparse-matrix-based message passing for graph neural networks in Julia. A general gather–scatter implementation constructs edge-indexed messages before aggregating them at destination nodes, potentially consuming substantial GPU memory. When messages consist of source features multiplied by scalar edge weights and aggregation is a sum, the same operation can be expressed as sparse matrix multiplication without materializing the edge-message array. We implemented this execution path on CUDA devices in `GraphNeuralNetworks.jl` for compressed sparse graphs and coalesced coordinate graphs. The implementation also required graph-conversion and coalescing utilities and a workaround for a defect in the CUDA sparse-matrix stack. In the tested configurations, measured memory use for message aggregation was reduced by factors of approximately 10 to 123 relative to gather–scatter.

C.1 Context and Motivation	157
C.2 From Gather–Scatter to Sparse Matrix Multiplication	157
C.2.1 The gather–scatter execution path	158
C.2.2 The SpMM execution path	158
C.3 Implementation in <code>GraphNeuralNetworks.jl</code>	158
C.3.1 Compressed sparse graphs (<code>:sparse</code>)	159
C.3.2 Coordinate graphs (<code>:coo</code>) and coalescing	159
C.3.3 Interaction with the CUDA sparse stack	160
C.4 Memory Evaluation	160
C.5 Discussion and Outlook	161

C.1 Context and Motivation

Graph neural networks (GNNs) process data whose entities and relations are represented by a graph. Rather than applying the same dense transformation independently to every entity, a GNN repeatedly exchanges information along graph edges. The resulting *message-passing* abstraction encompasses many graph convolutional architectures and separates three operations: constructing a message on every edge, aggregating incoming messages at each node, and updating the node representation (Gilmer et al., 2017).

Software support is particularly important for this model class. The number of possible node pairs grows quadratically with the number of nodes, whereas many graphs of practical interest contain only a small fraction of those edges. A framework must therefore preserve the sparse graph structure while still exposing computations that can be executed efficiently on an accelerator. Libraries such as the Deep Graph Library and PyTorch Geometric accordingly treat sparse graph operations and GPU execution as central design concerns (Fey and Lenssen, 2019; M. Wang et al., 2019).

`GraphNeuralNetworks.jl` provides graph-learning components in the Julia ecosystem. It supports attributed, heterogeneous, temporal, batched, dense, and sparse graphs, and offers both generic message-passing primitives and optimized operations (Lucibello and Rossi, 2025). Before the sparse CUDA path was added, GPU message passing used gather and scatter operations even when the aggregation could be expressed as sparse matrix multiplication. Although gather–scatter is flexible, it constructs an intermediate feature array indexed by edges. For a graph with many edges or wide node features, this array can become much larger than the node representations themselves.

For this linear-aggregation case, we added a GPU execution path that uses the graph topology directly as a sparse matrix. Message passing then uses the Sparse-Matrix Multiplication (SpMM) routines exposed by `CUDA.jl`, which provide access to NVIDIA’s `cuSPARSE` library (NVIDIA Corporation, 2026). The implementation was developed during Google Summer of Code 2025 and contributed upstream to `GraphNeuralNetworks.jl` (Ferre, 2025).

C.2 From Gather–Scatter to Sparse Matrix Multiplication

Let $G = (V, E)$ be a directed graph with $n = |V|$ nodes and $m = |E|$ edges. Node i has a feature vector $x_i \in \mathbb{R}^d$. For an edge $j \rightarrow i$ with optional edge features $e_{j \rightarrow i}$, a generic message-passing layer computes

$$m_{j \rightarrow i} = \phi(x_i, x_j, e_{j \rightarrow i}), \quad (\text{C.1})$$

$$\bar{m}_i = \text{AGG}_{j \in \mathcal{N}(i)} m_{j \rightarrow i}, \quad (\text{C.2})$$

$$x'_i = \gamma(x_i, \bar{m}_i), \quad (\text{C.3})$$

where ϕ is the message function, AGG is a permutation-invariant reduction such as sum, mean, or maximum, and γ is the node-update function. This formulation is deliberately broad: messages may depend nonlinearly on their source, destination, and edge attributes.

C.2.1 The gather–scatter execution path

In a gather–scatter implementation, the source and destination indices of the m edges are stored in vectors $s, t \in \{1, \dots, n\}^m$. If the node features are arranged as a matrix $X \in \mathbb{R}^{d \times n}$, gathering forms the edge-indexed arrays $X_{:,s}$ and, when required by ϕ , $X_{:,t}$. The message function is evaluated on these arrays, after which messages with the same destination index are reduced by a scatter operation.

This route can express Equations (C.1)–(C.3) with few restrictions. Its principal cost is the materialized message array. Even the simple operation $m_{j \rightarrow i} = x_j$ produces an intermediate array in $\mathbb{R}^{d \times m}$. Its storage is therefore $\Theta(dm)$, in addition to the $\Theta(dn)$ node features and the graph indices. On a GPU this array must be allocated, written, and read again before aggregation. The arithmetic complexity may remain linear in the number of edges, but the additional memory traffic and peak allocation can determine whether the layer fits on the device.

C.2.2 The SpMM execution path

Consider the common weighted-sum message

$$m_{j \rightarrow i} = w_{j \rightarrow i} x_j, \quad \bar{m}_i = \sum_{j \in \mathcal{N}(i)} w_{j \rightarrow i} x_j. \quad (\text{C.4})$$

Under the outgoing-adjacency convention used here, let $A \in \mathbb{R}^{n \times n}$ satisfy

$$A_{ji} = w_{j \rightarrow i}, \quad (\text{C.5})$$

with $A_{ji} = 0$ when the edge is absent. Equation (C.4) then becomes

$$\bar{M} = XA, \quad \bar{M}_{:,i} = \sum_{j=1}^n X_{:,j} A_{ji}. \quad (\text{C.6})$$

Since A contains only the graph edges, it is stored in a sparse format and Equation (C.6) is an SpMM operation. The result has size $d \times n$ and no $d \times m$ edge-feature tensor is required. Figure C.1 contrasts the two data flows.

The two paths are not interchangeable for every GNN layer. SpMM applies directly when the message and reduction can be represented by a sparse linear operator, including copying source features followed by summation and multiplying source features by scalar edge weights. A message that depends nonlinearly on both endpoints, a vector-valued edge feature, or a maximum reduction generally still requires the gather–scatter path or a more specialized fused kernel. The implementation therefore adds optimized dispatches rather than replacing the generic abstraction.

C.3 Implementation in `GraphNeuralNetworks.jl`

The fundamental graph container is `GNNGraph`. Its topology may be represented as a dense adjacency matrix (`:dense`), a compressed sparse adjacency matrix (`:sparse`), or source, target, and optional weight vectors in coordinate form (`:c oo`). Coordinate storage is convenient for graph manipulation and is the default representation for many constructors; compressed sparse storage maps more directly to established sparse linear-algebra routines.

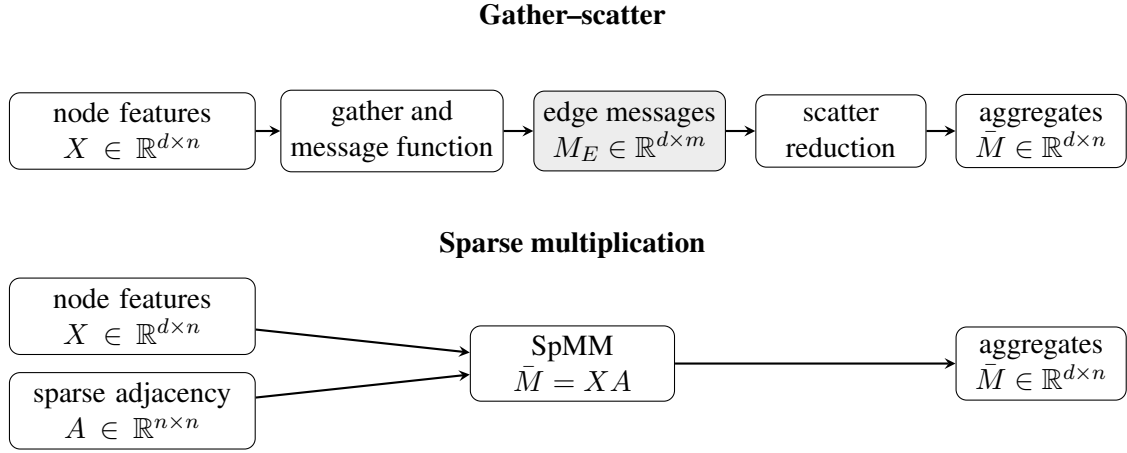


Figure C.1 – Gather-scatter materializes a $d \times m$ edge-message array. For weighted-sum messages, SpMM applies the sparse adjacency matrix directly to the node features and produces only the $d \times n$ aggregate.

The new SpMM execution path preserves the existing message-passing interface. We first added the CUDA sparse-matrix operations needed to construct unweighted adjacency operators, then specialized propagation for messages that either copy x_j or multiply it by a scalar edge weight. Unsupported message functions continue to use gather-scatter. These changes were incorporated through pull requests [#601](#), [#605](#), [#610](#), and [#611](#).

C.3.1 Compressed sparse graphs (:sparse)

For a :sparse graph transferred to an NVIDIA GPU, the adjacency structure is represented by a `CuSparseMatrixCSC`. Before this work, propagation converted the sparse topology back into edge indices and followed the generic gather-scatter route. The new dispatch instead evaluates Equation (C.6) using the sparse multiplication operations exposed by `CUDA.jl`. This change is local to the execution of the supported messages: the public graph and propagation interfaces do not require users to invoke `cuSPARSE` directly.

This specialization is important because merely storing a graph sparsely is insufficient. If the subsequent operator expands the topology into a $d \times m$ message tensor, the largest allocation still scales with the product of feature width and edge count. Retaining the sparse representation through the aggregation kernel is what produces the memory benefit.

C.3.2 Coordinate graphs (:coo) and coalescing

In the :coo representation, `GNNGraph` stores a source index, a target index, and optionally a value for every edge. The same ordered pair may occur more than once. To construct the sparse operator efficiently, we added a `coalesce` operation that orders coordinates and combines duplicate entries, recorded whether a `GNNGraph` was already coalesced, and provided utilities for constructing a `CuSparseMatrixCOO` without redundant sorting. These changes were incorporated through pull requests [#613](#) and [#617](#).

Figure C.2 gives a small weighted example. The two copies of edge $1 \rightarrow 2$ are replaced by a single coordinate whose weight is their sum. The remaining edges are reordered but unchanged.

Consequently, the coalesced coordinate structure represents the same sparse linear operator as the input.

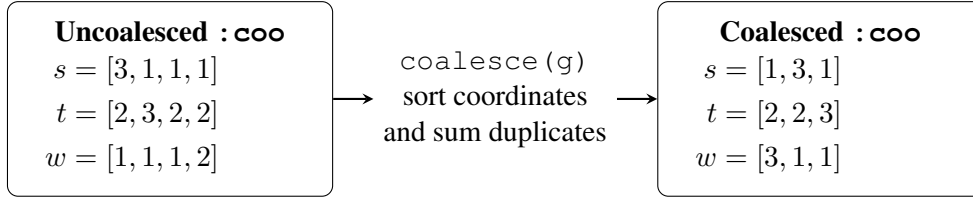


Figure C.2 – Coalescing orders the coordinates and combines parallel edges. Here the weights 1 and 2 on two copies of $1 \rightarrow 2$ become the single weight 3; the represented aggregation is unchanged.

C.3.3 Interaction with the CUDA sparse stack

During development, dense–sparse multiplication using a `CuSparseMatrixCOO` produced incorrect results, whereas the corresponding compressed-sparse operation and an algebraically equivalent sparse–dense operation were correct. A minimal reproducer was reported upstream as [CUDA.jl issue #2820](#). The coordinate path temporarily avoided the defective ordering through a different edge sort, allowing development and testing to continue without silently accepting incorrect results.

The discrepancy arose in the sparse CUDA multiplication path, not in the message-passing formula. It was subsequently fixed by [CUDA.jl pull request #3005](#), which implemented dense–COO multiplication without requiring the prior sort. The alternative edge ordering was therefore a workaround for the affected versions of `CUDA.jl`, rather than a requirement of coalesced COO aggregation.

C.4 Memory Evaluation

Before measuring memory use, we checked that the sparse CUDA paths matched generic propagation for the supported message functions ([Ferre, 2025](#)); these comparisons exposed the `CUDA.jl` defect discussed above. For coordinate graphs, coalescing must preserve the weighted adjacency operator: sorting may reorder edges, but summing the weights of duplicate edges leaves the aggregate XA unchanged for every compatible feature matrix X .

Table [C.1](#) compares measured memory use for sum aggregation by gather–scatter and SpMM with feature width $d = 128$. The benchmarks use graphs with 1024 and 2048 nodes at three densities. The reduction factors are computed from the displayed measurements and are approximate because those measurements are rounded.

Table C.1 – Memory attributed to message aggregation for gather–scatter and SpMM with feature width $d = 128$. Ratios are approximate because the displayed measurements are rounded.

Nodes	Density	Gather–scatter	SpMM	Reduction
1024	0.01	5.6 MiB	550 KiB	10.4×
1024	0.10	52 MiB	923 KiB	57.7×
1024	0.90	461 MiB	4.1 MiB	112.4×
2048	0.01	21 MiB	1.1 MiB	19.1×
2048	0.10	205 MiB	2.6 MiB	78.8×
2048	0.90	1.8 GiB	15 MiB	122.9×

The trend follows the storage analysis in Section C.2. Gather–scatter materializes one d -dimensional value per edge, so its principal intermediate grows as dm . SpMM stores the sparse graph and a node-indexed output, with leading storage terms proportional to m and dn rather than dm . As density and node count increase, the avoided edge-feature tensor accounts for a progressively larger fraction of memory. In the largest tested configuration, measured memory use falls from 1.8 GiB to 15 MiB, a reduction of approximately $123\times$.

The density-0.90 cases should be read as stress tests of intermediate allocation rather than as representative sparse graphs. At sufficiently high density, a dense matrix kernel may become preferable to any sparse representation. Conversely, many real graph datasets are better described by an average degree that stays approximately constant as n grows. Evaluating that regime on standard graph datasets would provide a more application-oriented complement to the synthetic density sweep.

These measurements concern memory use during aggregation; they do not by themselves quantify end-to-end runtime. Timing benchmarks were also added to the upstream package, but execution time depends on the graph representation, the cost of any required coalescing, the CUDA environment, and the workload. We therefore do not infer a numerical speedup from Table C.1. A systematic runtime comparison remains separate from the memory results presented here.

C.5 Discussion and Outlook

For supported linear sum aggregations, `GraphNeuralNetworks.jl` can now use sparse CUDA multiplication instead of constructing an edge-feature tensor. In the tested configurations, this change reduces aggregation memory by factors of approximately 10 to 123, while the existing gather–scatter path remains available for other message functions.

The main limitation is expressiveness. A sparse matrix multiplication implements a linear, sum-based neighborhood aggregation. It does not replace generic message passing for attention coefficients that must first be computed from endpoint features, vector-valued edge transformations, nonlinear messages, or reductions such as maximum. A practical library should preserve both routes and select SpMM only when the message semantics match an optimized sparse operator.

There is also an input-preparation trade-off. A coordinate graph must be coalesced before it can use the efficient path. Sorting and combining coordinates costs time and temporary storage. This cost can be amortized when the same topology is reused across many layers, batches, or optimization steps, as is common for transductive learning on a fixed graph. It may be less attractive

when graph structure changes on every iteration. Recording the coalesced state avoids repeating the conversion unnecessarily.

The measurements in Table C.1 demonstrate lower memory use in the tested configurations, but they do not constitute a complete performance study. A broader evaluation should include standard sparse graph datasets, end-to-end forward and backward passes, and it should compare against frameworks such as PyTorch Geometric. Equivalent sparse support on non-NVIDIA accelerators would also be valuable; the implementation described here specifically targets the CUDA backend.

Inférence efficace en apprentissage automatique : sous-réseaux élagués et ordonnancement de tâches d'inférence compressibles

Davide FERRE'

Résumé

La croissance des réseaux de neurones et l'essor du Machine Learning as a Service entraînent d'importants coûts de calcul et d'énergie lors de l'inférence. Cette thèse étudie la réduction et l'allocation de ce calcul à deux échelles complémentaires : la structure des modèles et l'ordonnancement des ressources.

Nous étudions d'abord l'élagage dans le cadre de la Strong Lottery Ticket Hypothesis (SLTH), selon laquelle un grand réseau aléatoire contient, dès son initialisation, un sous-réseau approchant un modèle cible sans entraînement. Les preuves fondées sur le problème Random Subset Sum ne contrôlaient pas la taille du ticket obtenu. Grâce au problème Random Fixed-Size Subset Sum, nous établissons des garanties de parcimonie pour les réseaux denses et équivariants, avec des bornes inférieures presque correspondantes. Nous montrons ensuite une limite de l'élagage structuré : approcher un neurone ReLU par élagage de neurones requiert une largeur en $\Omega(d/\varepsilon)$, contre $O(d \log(1/\varepsilon))$ par élagage de poids, soit une séparation exponentielle en fonction de la précision.

Nous abordons ensuite l'ordonnancement de tâches d'inférence compressibles. Un premier modèle traite des échéances, de machines hétérogènes et d'un budget énergétique global par relaxation fractionnaire et approximation. Un second intègre les dates d'arrivée, la préemption et des fenêtres d'exécution dépendant des communications dans le continuum edge-cloud. Nous proposons une méthode de relaxation et d'arrondi, des formulations par flot de coût minimal et des garanties sur la charge conservée et la précision.

En complément, une contribution présentée en annexe permet le passage de messages des réseaux de neurones sur graphes par multiplication de matrices creuses dans l'écosystème Julia. Elle réduit la mémoire intermédiaire utilisée par les opérations gather/scatter sur GPU dans les configurations étudiées.

Mots-clés : Apprentissage automatique, réseaux de neurones, parcimonie, élagage, hypothèse du ticket gagnant, ordonnancement, efficacité énergétique, calcul creux.

Efficient machine learning inference: sparse subnetworks and scheduling of compressible tasks

Davide FERRE'

Abstract

The growing scale of neural networks and the adoption of Machine Learning as a Service create substantial computational and energy costs during inference. This thesis studies how inference computation can be reduced and allocated at two complementary scales: model structure and resource scheduling.

First, we study neural-network pruning through the Strong Lottery Ticket Hypothesis (SLTH), according to which a sufficiently large random network contains, at initialization, a subnetwork that approximates a target model without training. Earlier proofs based on the Random Subset Sum problem did not control the size of the resulting ticket. Using the Random Fixed-Size Subset Sum problem, we establish sparsity guarantees for dense and equivariant networks, together with near-matching lower bounds. We then identify an intrinsic limitation of structured pruning: approximating a single ReLU neuron by neuron pruning requires width $\Omega(d/\varepsilon)$, whereas weight pruning needs only $O(d \log(1/\varepsilon))$, yielding an exponential separation in the accuracy dependence.

Second, we address the scheduling of compressible inference tasks. One setting combines deadlines, heterogeneous machines, and a global energy budget, for which we develop an exact fractional algorithm and an approximation algorithm. A second setting incorporates release times, preemption, and destination-dependent execution windows induced by communication delays across the edge–cloud continuum. We provide a relax-and-round method, exact min-cost-flow formulations in important cases, and workload-retention and accuracy guarantees.

An additional contribution, presented in an appendix, enables message passing based on sparse matrix multiplication for graph neural networks in the Julia ecosystem. In the tested configurations, it reduces the intermediate memory used by gather–scatter operations on GPUs.

Keywords: Machine learning, neural networks, sparsity, pruning, strong lottery ticket hypothesis, scheduling, energy efficiency, sparse computing.